VECTOR

MICROSAR Classic vSecPrim

Technical Reference

CRYPTO LIBRARY

Version 7.0.0

Authors	vismwe, viseag
Status	Released

Document Information

History

Author	Date	Version	Remarks
vismwe	2019-01-22	1.00.00	Initial creation of Technical Reference
vismwe			Rework 2.1.1.1
viseag	2019-11-14	2.00.00	API description for KDF2 HMAC SHA 256
viseag	2020-03-06	2.01.00	Rework API descriptions for RSA PSS SHA2-256. Removed limitation for data length.
viseag	2020-04-14	3.00.00	Minor modification of API description for ECDSA and AES
vismwe, viseag	2020-06-05	3.01.00	Rework API descriptions for KDF X9.63 SHA 512, Add API descriptions for CTR DRBG AES128, Add 2.2
viseag	2020-07-10	3.02.00	Add API descriptions for Hash DRBG SHA2-512, Reworked 2.2, Add API descriptions for SHA3, updated Chapter 3.2, updated Figure 1-1, Resolved ESCAN00106983, Rework APIs for CTR DRBG
viseag	2020-08-25	4.00.00	Added API descriptions for CMAC AES256, AES 128 CCM, HKDF HMAC SHA256 and SPAKE2+ Updated Chapter 4.1
viseag	2020-09-09	4.01.00	Reworked API descriptions for HKDF, HMAC SHA256 and AES CCM Updated Chapter 2.1.1 Updated abbreviation list
viseag	2020-10-28	4.02.00	Reworked API descriptions for SPAKE2+ Added Chapter 2.2.2.2 Updated Chapter 2.1.1 Reworked APIs for cipher algorithms to add information about in-place encryption/decryption Added Chapter 2.1.2
viseag	2020-12-10	4.03.00	Added Chapter 4.2.2.10 Corrected values in Table 2-1 Updated API signatures in Table 4-395, Table 4-396, Table 4-400 and Table 4-401 Corrected RFC number in Table 4-383 For generic Hash API: Reworked formatting of Chapter 2.1.1 Updated Chapter 2.5 Updated Table 3-1 Added Chapter 3.3

			Reworked Chapter 4.1, introduced sub-chapter Added Chapter 4.2.1, restructured Chapter 4.2 Removed lowest level from Table of Contents
viseag	2021-04-22	4.04.00	Minor corrections: Changed KDF X9.63 HMAC SHA to KDF X9.63 SHA in Chapter 2.1.1.7 Added return values to RSA algorithms for incorrect encoding or modulus sizes
viseag	2021-11-10	5.00.00	Add HKDF SHA2-256 in Chapter 2.1.1.7 Added HDKF API descriptions in Chapter 4.2.2.7 Introduced Subchapters 4.2.2.4.1, 4.2.2.4.2, 4.2.2.4.3 and 4.2.2.4.4 in Chapter 4.2.2.4 Added Chapter 2.2.3
viseag	2022-01-14	5.01.00	Updated API description for esl_verifyDSAEcP_prim, Added generated MemMap file to Table 3-2
alefarth	2022-05-19	5.02.00	Product name updated to MICROSAR Classic
viseag	2022-07-04	7.00.00	Remove component history, correct referenced document for FIPS 186-2 RNG Added HDKF HMAC SHA384 in Chapters 2.1.1.7 (Provided Algorithms) and 4.2.2.7 (API descriptions) Added particularity in API description of vSecPrim_initHash

Reference Documents

No.	Source	Title	Version
[1]		FIPS PUB 197: Advanced Encryption Standard (AES)	November 2001
[2]		FIPS PUB 46-3: Data Encryption Standard (DES)	October 1999
[3]		R. Rivest; RFC 2268: A Description of the RC2 Encryption Algorithm	March 1998
[4]		Nir; Langley; RFC 7539: ChaCha20 and Poly1305 for IETF Protocols	May 2015
[5]		FIPS PUB 180-4: Secure Hash Standard (SHS)	August 2015
[6]		R. Rivest; RFC 1321: The MD5 Message-Digest Algorithm	April 1992
[7]		ISO/IEC 10118-3; Information technology - Security techniques - Hash-functions - Part 3: Dedicated hash-functions	2004
[8]		FIPS PUB 198-1; The Keyed-Hash Message Authentication Code (HMAC)	July 2008
[9]		M. Dworkin; NIST SP 800-38A: Recommendation for Block Cipher Modes of Operation: the CMAC Mode for Authentication	May 2005
[10]		M. Dworkin; NIST SP 800-38D: Recommendation for Block Cipher Modes of Operation: Galois/Counter Mode (GCM) and GMAC	November 2007
[11]		Aumasson; Bernstein; SipHash: a fast short-input PRF	2012
[12]		B. Kaliski; PKCS #1: RSA Encryption Version 1.5	March 1998
[13]		PKCS #1 V2.2: RSA Cryptography Standard	October 2012
[14]		ANSI X9.62-2005: Public Key Cryptography for the Financial Service Industry - The Elliptic Curve Digital Signature Algorithm (ECDSA)	November 2005
[15]		Josefsson; Liusvaara; RFC 8032: Edwards-Curve Digital Signature Algorithm (EdDSA)	January 2017
[16]		ANSI X9.63-2001: Public Key Cryptography for the Financial Service Industry: Key Agreement and Key Transport Using Elliptic Curve Cryptography	November 2001
[17]		Langley; Hamburg; Turner; RFC 7748: Elliptic Curves for Security (2016)	January 2016
[18]		Kaliski; PKCS #5: Password-Based Cryptography Specification Version 2.0	September 2000
[19]		FIPS PUB 186-2: Digital Signature Standard (DSS)	January 2000
[20]		ISO/IEC 9797-1; Information technology - Security techniques - Message Authentication Codes (MACs) – Part 1: Mechanisms using a block cipher.	2011
[21]		B. Kaliski; PKCS #7: Cryptographic Message Syntax Version 1.5	March 1998

[22]		E. Barker; NIST Special Publication 800-90Ar1: Recommandation for Random Number Generation Using Deterministic Random Bit Generators.	June 2015
[23]		FIPS PUB 202: SHA-3 Standard: Permutation-Based Hash and Extendable-Output Functions	August 2015
[24]	Vector	TechnicalReference_Crypto_30_LibCv.pdf	
[25]		E. Barker, L. Chen, R. Davis; NIST SP 800-56C: Recommendation for Key-Derivation Methods in Key-Establishment Schemes	April 2018
[26]		M. Dworkin; NIST SP 800-38C: Recommendation for Block Cipher Modes of Operation: The CCM Mode for Authentication and Confidentiality	May 2004
[27]		T. Taubert, C. Wood: SPAKE2+, an Augmented PAKE, Version draft-bar-cfrg-spake2plus-00	March 2020



Caution
We have configured the programs in accordance with your specifications in the questionnaire. Whereas the programs do support other configurations than the one specified in your questionnaire, Vector´s release of the programs delivered to your company is expressly restricted to the configuration you have specified in the questionnaire.

Contents

1 Introduction 17
1.1 Architecture Overview 17

2 Functional Description 18
2.1 Features 18
2.1.1 Provided Algorithms 18
2.1.1.1 Symmetric Encryption Algorithms 18
2.1.1.2 Hash Algorithms 19
2.1.1.3 Message Authentication Codes (MAC) 20
2.1.1.4 Asymmetric Signature Algorithms 20
2.1.1.5 Asymmetric Encryption Algorithms 21
2.1.1.6 Key Exchange Algorithms 21
2.1.1.7 Key Derivation Functions (KDF) 22
2.1.1.8 Authenticated Encryption with Associated Data (AEAD) 22
2.1.1.9 Random Number Generation (PRNG) 23
2.1.2 In-place encryption and decryption 23
2.1.3 Limitations 24
2.1.3.1 Parameter Checking 24
2.1.3.2 Module Usage 24
2.2 Additional Algorithm Specific Information 25
2.2.1 Random Number Generation with DRBG 25
2.2.2 SPAKE2+ 26
2.2.2.1 Cipher Suites 26
2.2.2.2 Order of API Calls 27
2.2.3 EC Based Algorithms over GF(p) Curves 27
2.3 Workspace and Call Concept 28
2.4 Clear Workspace 29
2.5 Module Concept 29

3 Integration 31
3.1 Scope of Delivery 31
3.1.1 Static Files 31
3.1.2 Dynamic Files 31
3.2 Entropy Callout 31
3.3 Watchdog Callout 32

4 API Description 33
4.1 Type Definitions 33
4.1.1 General Types 33

4.1.2 Return Types ... 34
4.1.3 Types for Elliptic Curve Domain Parameters ... 37
4.1.4 Algorithm Specific Types ... 40
4.2 Services provided by vSecPrim ... 42
4.2.1 Generalized Service Functions ... 42
4.2.1.1 Generalized Hash Service Functions ... 42
4.2.2 Service Functions Sorted by Algorithms ... 45
4.2.2.1 Symmetric Encryption Algorithms ... 46
4.2.2.2 Hash Algorithms ... 98
4.2.2.3 Message Authentication Codes (MAC) ... 121
4.2.2.4 Asymmetric Signature Algorithms ... 152
4.2.2.5 Asymmetric Encryption Algorithms ... 250
4.2.2.6 Key Exchange Algorithms ... 281
4.2.2.7 Key Derivation Functions (KDF) ... 302
4.2.2.8 Authenticated Encryption with Associated Data (AEAD) ... 314
4.2.2.9 Random Number Generation (PRNG) ... 330
4.2.2.10 Service Functions for Initialization ... 341
4.3 Services used by vSecPrim ... 343
4.4 Configurable Interfaces ... 343
4.4.1 Callout Functions ... 343
4.4.1.1 esl_getBytesRNG ... 343

5 Configuration ... 345
5.1 Configuration Variants ... 345

6 Glossary and Abbreviations ... 346
6.1 Glossary ... 346
6.2 Abbreviations ... 346

7 Contact ... 348

Illustrations

Figure 1-1 Interfaces to adjacent modules of the vSecPrim 17
Figure 2-1 Module concept of vSecPrim 30

Tables

Table 2-1 Parameter classification for CTR DRBG and Hash DRBG operations 25
Table 2-2 DRBG algorithms, security strength and input sizes 26
Table 2-3 Possible values of ‘ciphersuite’ in esl_initSPAKE2P() 27
Table 2-4 Accessible algorithms in module layer vSecPrim 30
Table 3-1 Static files 31
Table 3-2 Generated files 31
Table 4-1 Type definitions for hash function identifiers 33
Table 4-2 Type definitions for output buffer flags 34
Table 4-3 Type definitions for error codes 37
Table 4-4 Type definitions: Elliptic curve domain parameters 38
Table 4-5 Type definitions: Elliptic curve domain extension parameters 39
Table 4-6 Type definitions: Elliptic curve speed up parameters 40
Table 4-7 Type definitions: Padding modes for block ciphers 40
Table 4-8 Type definitions: operation modes for block ciphers 40
Table 4-9 Type definitions: Mode identifiers for DRBG 41
Table 4-10 Type definitions: DRBG reseed counter type 41
Table 4-11 Type definitions: DRBG seed status type 41
Table 4-12 Type definitions: HKDF variant type 41
Table 4-13 Type definitions: SPAKE2+ mode type 41
Table 4-14 vSecPrim_initHash 42
Table 4-15 vSecPrim_updateHash 43
Table 4-16 vSecPrim_finalizeHash 44
Table 4-17 vSecPrim_digestHash 45
Table 4-18 esl_initEncryptAES128 46
Table 4-19 esl_encryptAES128 47
Table 4-20 esl_finalizeEncryptAES128 48
Table 4-21 esl_initDecryptAES128 49
Table 4-22 esl_decryptAES128 50
Table 4-23 esl_finalizeDecryptAES128 50
Table 4-24 esl_initEncryptAES128Block 51
Table 4-25 esl_encryptAES128Block 52
Table 4-26 esl_initDecryptAES128Block 53
Table 4-27 esl_decryptAES128Block 53
Table 4-28 esl_initEncryptAES192 54
Table 4-29 esl_encryptAES192 55
Table 4-30 esl_finalizeEncryptAES192 56
Table 4-31 esl_initDecryptAES192 57
Table 4-32 esl_decryptAES192 57
Table 4-33 esl_finalizeDecryptAES192 58
Table 4-34 esl_initEncryptAES192Block 59
Table 4-35 esl_encryptAES192Block 60
Table 4-36 esl_initDecryptAES192Block 60
Table 4-37 esl_decryptAES192Block 61
Table 4-38 esl_initEncryptAES256 62
Table 4-39 esl_encryptAES256 63
Table 4-40 esl_finalizeEncryptAES256 63

Table 4-41 esl_initDecryptAES256 64
Table 4-42 esl_decryptAES256 65
Table 4-43 esl_finalizeDecryptAES256 66
Table 4-44 esl_initEncryptAES256Block 66
Table 4-45 esl_encryptAES256Block 67
Table 4-46 esl_initDecryptAES256Block 68
Table 4-47 esl_decryptAES256Block 69
Table 4-48 esl_generateKeyDES 69
Table 4-49 esl_initEncryptDES 70
Table 4-50 esl_encryptDES 71
Table 4-51 esl_finalizeEncryptDES 71
Table 4-52 esl_initDecryptDES 72
Table 4-53 esl_finalizeDecryptDES 73
Table 4-54 esl_decryptDES 74
Table 4-55 esl_initEncryptDESBlock 74
Table 4-56 esl_encryptDESBlock 75
Table 4-57 esl_initDecryptDESBlock 76
Table 4-58 esl_decryptDESBlock 76
Table 4-59 esl_generateKey3TDES 77
Table 4-60 esl_generateKey2TDES 77
Table 4-61 esl_initEncryptTDES 78
Table 4-62 esl_encryptTDES 79
Table 4-63 esl_finalizeEncryptTDES 80
Table 4-64 esl_initDecryptTDES 81
Table 4-65 esl_decryptTDES 81
Table 4-66 esl_finalizeDecryptTDES 82
Table 4-67 esl_initEncryptTDESBlock 83
Table 4-68 esl_encryptTDESBlock 84
Table 4-69 esl_initDecryptTDESBlock 84
Table 4-70 esl_decryptTDESBlock 85
Table 4-71 esl_generateKeyRC2 85
Table 4-72 esl_initEncryptRC2 86
Table 4-73 esl_encryptRC2 87
Table 4-74 esl_finalizeEncryptRC2 88
Table 4-75 esl_initDecryptRC2 89
Table 4-76 esl_decryptRC2 90
Table 4-77 esl_finalizeDecryptRC2 90
Table 4-78 esl_initEncryptRC2Block 91
Table 4-79 esl_encryptRC2Block 92
Table 4-80 esl_initDecryptRC2Block 92
Table 4-81 esl_decryptRC2Block 93
Table 4-82 esl_initEncryptChaCha20 94
Table 4-83 esl_encryptChaCha20 95
Table 4-84 esl_finalizeEncryptChaCha20 95
Table 4-85 esl_initDecryptChaCha20 96
Table 4-86 esl_decryptChaCha20 97
Table 4-87 esl_finalizeDecryptChaCha20 98
Table 4-88 esl_initSHA1 98
Table 4-89 esl_updateSHA1 99
Table 4-90 esl_finalizeSHA1 100
Table 4-91 esl_initSHA224 100
Table 4-92 esl_updateSHA224 101
Table 4-93 esl_finalizeSHA224 102
Table 4-94 esl_initSHA256 102

Table 4-95 esl_updateSHA256 ... 103
Table 4-96 esl_finalizeSHA256 ... 104
Table 4-97 esl_initSHA384 ... 104
Table 4-98 esl_updateSHA384 ... 105
Table 4-99 esl_finalizeSHA384 ... 106
Table 4-100 esl_initSHA512 ... 106
Table 4-101 esl_updateSHA512 ... 107
Table 4-102 esl_finalizeSHA512 ... 108
Table 4-103 esl_initSHA512_224 ... 108
Table 4-104 esl_updateSHA512_224 ... 109
Table 4-105 esl_finalizeSHA512_224 ... 110
Table 4-106 esl_initSHA512_256 ... 110
Table 4-107 esl_updateSHA512_256 ... 111
Table 4-108 esl_finalizeSHA512_256 ... 112
Table 4-109 esl_initSHA3 ... 112
Table 4-110 esl_updateSHA3 ... 113
Table 4-111 esl_finalizeSHA3 ... 114
Table 4-112 esl_initSHAKE ... 114
Table 4-113 esl_updateSHAKE ... 115
Table 4-114 esl_getXOFDigestSHAKE ... 116
Table 4-115 esl_finalizeSHAKE ... 117
Table 4-116 esl_initMD5 ... 117
Table 4-117 esl_updateMD5 ... 118
Table 4-118 esl_finalizeMD5 ... 119
Table 4-119 esl_initRIPEMD160 ... 119
Table 4-120 esl_updateRIPEMD160 ... 120
Table 4-121 esl_finalizeRIPEMD160 ... 120
Table 4-122 esl_generateKeyHashMACSHA1 ... 121
Table 4-123 esl_initHashMACSHA1 ... 122
Table 4-124 esl_updateHashMACSHA1 ... 122
Table 4-125 esl_finalizeHashMACSHA1 ... 123
Table 4-126 esl_verifyHashMACSHA1 ... 124
Table 4-127 esl_generateKeyHashMACSHA256 ... 124
Table 4-128 esl_initHashMACSHA256 ... 125
Table 4-129 esl_updateHashMACSHA256 ... 126
Table 4-130 esl_finalizeHashMACSHA256 ... 127
Table 4-131 esl_verifyHashMACSHA256 ... 127
Table 4-132 esl_generateKeyHashMACSHA384 ... 128
Table 4-133 esl_initHashMACSHA384 ... 128
Table 4-134 esl_updateHashMACSHA384 ... 129
Table 4-135 esl_finalizeHashMACSHA384 ... 130
Table 4-136 esl_verifyHashMACSHA384 ... 131
Table 4-137 esl_generateKeyHashMACRIPEMD160 ... 131
Table 4-138 esl_initHashMACRIPEMD160 ... 132
Table 4-139 esl_updateHashMACRIPEMD160 ... 133
Table 4-140 esl_finalizeHashMACRIPEMD160 ... 133
Table 4-141 esl_verifyHashMACRIPEMD160 ... 134
Table 4-142 esl_getExpandedKeyCMACAES128 ... 135
Table 4-143 esl_initExpandedCMACAES128 ... 135
Table 4-144 esl_initCMACAES128 ... 136
Table 4-145 esl_updateCMACAES128 ... 137
Table 4-146 esl_finalizeCMACAES128 ... 137
Table 4-147 esl_verifyCMACAES128 ... 138
Table 4-148 esl_initExpandedCMACAES128 ... 139

Table 4-149 esl_getExpandedKeyCMACAES128 139
Table 4-150 esl_initCMACAES256 140
Table 4-151 esl_updateCMACAES256 141
Table 4-152 esl_finalizeCMACAES256 141
Table 4-153 esl_verifyCMACAES256 142
Table 4-154 esl_initExpandedCMACAES256 143
Table 4-155 esl_getExpandedKeyCMACAES256 144
Table 4-156 esl_initGMAC 144
Table 4-157 esl_updateGMAC 145
Table 4-158 esl_finalizeGMAC 146
Table 4-159 esl_verifyGMAC 146
Table 4-160 esl_initPoly1305 147
Table 4-161 esl_updatePoly1305 147
Table 4-162 esl_finalizePoly1305 148
Table 4-163 esl_verifyPoly1305 149
Table 4-164 esl_initSipHash 149
Table 4-165 esl_updateSipHash 150
Table 4-166 esl_finalizeSipHash 151
Table 4-167 esl_verifySipHash 151
Table 4-168 esl_initSignRSASHA1_V15 152
Table 4-169 esl_updateSignRSASHA1_V15 153
Table 4-170 esl_finalizeSignRSASHA1_V15 154
Table 4-171 esl_initSignRSACRTSHA1_V15 155
Table 4-172 esl_updateSignRSACRTSHA1_V15 156
Table 4-173 esl_finalizeSignRSACRTSHA1_V15 156
Table 4-174 esl_initVerifyRSASHA1_V15 157
Table 4-175 esl_updateVerifyRSASHA1_V15 158
Table 4-176 esl_finalizeVerifyRSASHA1_V15 158
Table 4-177 esl_finalizeHashVerifyRSASHA1_V15 159
Table 4-178 esl_verifySigVerifyRSASHA1_V15 160
Table 4-179 esl_initSignRSASHA256_V15 161
Table 4-180 esl_updateSignRSASHA256_V15 161
Table 4-181 esl_finalizeSignRSASHA256_V15 162
Table 4-182 esl_initSignRSACRTSHA256_V15 163
Table 4-183 esl_updateSignRSACRTSHA256_V15 164
Table 4-184 esl_finalizeSignRSACRTSHA256_V15 165
Table 4-185 esl_initVerifyRSASHA256_V15 165
Table 4-186 esl_updateVerifyRSASHA256_V15 166
Table 4-187 esl_finalizeHashVerifyRSASHA256_V15 167
Table 4-188 esl_verifySigVerifyRSASHA256_V15 168
Table 4-189 esl_finalizeVerifyRSASHA256_V15 168
Table 4-190 esl_initSignRSARIPEMD160_V15 169
Table 4-191 esl_updateSignRSARIPEMD160_V15 170
Table 4-192 esl_finalizeSignRSARIPEMD160_V15 171
Table 4-193 esl_initSignRSACRTRIPEMD160_V15 172
Table 4-194 esl_updateSignRSACRTRIPEMD160_V15 172
Table 4-195 esl_finalizeSignRSACRTRIPEMD160_V15 173
Table 4-196 esl_initVerifyRSARIPEMD160_V15 174
Table 4-197 esl_updateVerifyRSARIPEMD160_V15 175
Table 4-198 esl_finalizeHashVerifyRSARIPEMD160_V15 175
Table 4-199 esl_verifySigVerifyRSARIPEMD160_V15 176
Table 4-200 esl_finalizeVerifyRSARIPEMD160_V15 177
Table 4-201 esl_verifyPaddingRSAEM_V15 178
Table 4-202 esl_initSignRSASHA1_PSS 178

Table 4-203 esl_updateSignRSASHA1_PSS 179
Table 4-204 esl_finalizeHashSignRSASHA1_PSS 180
Table 4-205 esl_finalizeSigSignRSASHA1_PSS 181
Table 4-206 esl_finalizeSigSignRSASHA1_PSS_NoSalt 182
Table 4-207 esl_finalizeSigSignRSASHA1_PSS_DigestLengthSalt 182
Table 4-208 esl_finalizeSignRSASHA1_PSS 183
Table 4-209 esl_finalizeSignRSASHA1_PSS_NoSalt 184
Table 4-210 esl_finalizeSignRSASHA1_PSS_DigestLengthSalt 185
Table 4-211 esl_initVerifyRSASHA1_PSS 185
Table 4-212 esl_updateVerifyRSASHA1_PSS 186
Table 4-213 esl_finalizeHashVerifyRSASHA1_PSS 187
Table 4-214 esl_verifySigVerifyRSASHA1_PSS 188
Table 4-215 esl_verifySigVerifyRSASHA1_PSS_AutoSalt 189
Table 4-216 esl_verifySigVerifyRSASHA1_PSS_NoSalt 189
Table 4-217 esl_verifySigVerifyRSASHA1_PSS_DigestLengthSalt 190
Table 4-218 esl_finalizeVerifyRSASHA1_PSS 191
Table 4-219 esl_finalizeVerifyRSASHA1_PSS_AutoSalt 191
Table 4-220 esl_finalizeVerifyRSASHA1_PSS_NoSalt 192
Table 4-221 esl_finalizeVerifyRSASHA1_PSS_DigestLengthSalt 193
Table 4-222 esl_calcSaltedHashRSASHA1_PSS 193
Table 4-223 esl_generateMaskMGF1RSASHA1_PSS 194
Table 4-224 esl_initSignRSASHA256_PSS 195
Table 4-225 esl_updateSignRSASHA256_PSS 196
Table 4-226 esl_finalizeHashSignRSASHA256_PSS 197
Table 4-227 esl_finalizeSigSignRSASHA256_PSS 198
Table 4-228 esl_finalizeSigSignRSASHA256_PSS_NoSalt 198
Table 4-229 esl_finalizeSigSignRSASHA256_PSS_DigestLengthSalt 199
Table 4-230 esl_finalizeSignRSASHA256_PSS 200
Table 4-231 esl_finalizeSignRSASHA256_PSS_NoSalt 201
Table 4-232 esl_finalizeSignRSASHA256_PSS_DigestLengthSalt 201
Table 4-233 esl_initVerifyRSASHA256_PSS 202
Table 4-234 esl_updateVerifyRSASHA256_PSS 203
Table 4-235 esl_finalizeHashVerifyRSASHA256_PSS 204
Table 4-236 esl_verifySigVerifyRSASHA256_PSS 205
Table 4-237 esl_verifySigVerifyRSASHA256_PSS_AutoSalt 206
Table 4-238 esl_verifySigVerifyRSASHA256_PSS_NoSalt 206
Table 4-239 esl_verifySigVerifyRSASHA256_PSS_DigestLengthSalt 207
Table 4-240 esl_finalizeVerifyRSASHA256_PSS 208
Table 4-241 esl_finalizeVerifyRSASHA256_PSS_AutoSalt 209
Table 4-242 esl_finalizeVerifyRSASHA256_PSS_NoSalt 209
Table 4-243 esl_finalizeVerifyRSASHA256_PSS_DigestLengthSalt 210
Table 4-244 esl_generateMaskMGF1RSASHA256_PSS 211
Table 4-245 esl_calcSaltedHashRSASHA256_PSS 212
Table 4-246 esl_initSignRSARIPEMD160_PSS 213
Table 4-247 esl_updateSignRSARIPEMD160_PSS 213
Table 4-248 esl_finalizeHashSignRSARIPEMD160_PSS 214
Table 4-249 esl_finalizeSigSignRSARIPEMD160_PSS 215
Table 4-250 esl_finalizeSigSignRSARIPEMD160_PSS_NoSalt 216
Table 4-251 esl_finalizeSigSignRSARIPEMD160_PSS_DigestLengthSalt 217
Table 4-252 esl_finalizeSignRSARIPEMD160_PSS 218
Table 4-253 esl_finalizeSignRSARIPEMD160_PSS_NoSalt 218
Table 4-254 esl_finalizeSignRSARIPEMD160_PSS_DigestLengthSalt 219
Table 4-255 esl_initVerifyRSARIPEMD160_PSS 220
Table 4-256 esl_updateVerifyRSARIPEMD160_PSS 221

Table 4-257 esl_finalizeHashVerifyRSARIPEMD160_PSS ... 221
Table 4-258 esl_verifySigVerifyRSARIPEMD160_PSS ... 223
Table 4-259 esl_verifySigVerifyRSARIPEMD160_PSS_AutoSalt ... 223
Table 4-260 esl_verifySigVerifyRSARIPEMD160_PSS_NoSalt ... 224
Table 4-261 esl_verifySigVerifyRSARIPEMD160_PSS_DigestLengthSalt ... 225
Table 4-262 esl_finalizeVerifyRSARIPEMD160_PSS ... 225
Table 4-263 esl_finalizeVerifyRSARIPEMD160_PSS_AutoSalt ... 226
Table 4-264 esl_finalizeVerifyRSARIPEMD160_PSS_NoSalt ... 227
Table 4-265 esl_finalizeVerifyRSARIPEMD160_PSS_DigestLengthSalt ... 227
Table 4-266 esl_calcSaltedHashRSARIPEMD160_PSS ... 228
Table 4-267 esl_generateMaskMGF1RSARIPEMD160_PSS ... 229
Table 4-268 esl_calcSaltedHashRSASHA256_PSS ... 230
Table 4-269 esl_initSignRSA_prim ... 231
Table 4-270 esl_signRSA_prim ... 232
Table 4-271 esl_initVerifyRSA_prim ... 232
Table 4-272 esl_verifyRSA_prim ... 233
Table 4-273 esl_initSignRSACRT_prim ... 234
Table 4-274 esl_signRSACRT_prim ... 235
Table 4-275 esl_initSignDSAEcP_prim ... 236
Table 4-276 esl_signDSAEcP_prim ... 237
Table 4-277 esl_initVerifyDSAEcP_prim ... 238
Table 4-278 esl_verifyDSAEcP_prim ... 239
Table 4-279 esl_initSignEdDSA ... 240
Table 4-280 esl_signEdDSA ... 241
Table 4-281 esl_initVerifyEdDSA ... 242
Table 4-282 esl_verifyEdDSA ... 243
Table 4-283 esl_updateEdDSA ... 243
Table 4-284 esl_initGenerateKeyPairEdDSA ... 244
Table 4-285 esl_generateKeyPairEdDSA ... 245
Table 4-286 esl_initGenerateKeyEcP_prim ... 246
Table 4-287 esl_generateKeyEcP_prim ... 247
Table 4-288 esl_getMaxLengthOfEcPmessage ... 247
Table 4-289 esl_getLengthOfEcPpublicKey_comp ... 248
Table 4-290 esl_getLengthOfEcPprivateKey ... 249
Table 4-291 esl_getLengthOfEcPsignature_comp ... 249
Table 4-292 esl_getLengthOfEcPsecret_comp ... 250
Table 4-293 esl_initEncryptRSA_prim ... 251
Table 4-294 esl_encryptRSA_prim ... 251
Table 4-295 esl_initDecryptRSA_prim ... 252
Table 4-296 esl_decryptRSA_prim ... 253
Table 4-297 esl_initDecryptRSACRT_prim ... 254
Table 4-298 esl_decryptRSACRT_prim ... 255
Table 4-299 esl_initEncryptRSA_V15 ... 256
Table 4-300 esl_encryptRSA_V15 ... 256
Table 4-301 esl_initDecryptRSA_V15 ... 257
Table 4-302 esl_decryptRSA_V15 ... 258
Table 4-303 esl_decryptPubRSA_V15 ... 259
Table 4-304 esl_decryptPrivRSA_V15 ... 260
Table 4-305 esl_initDecryptRSACRT_V15 ... 261
Table 4-306 esl_decryptRSACRT_V15 ... 261
Table 4-307 esl_initEncryptRSASHA1_OAEP ... 262
Table 4-308 esl_encryptRSASHA1_OAEP_Label ... 263
Table 4-309 esl_encryptRSASHA1_OAEP_Seed ... 264
Table 4-310 esl_encryptRSASHA1_OAEP_Label_Seed ... 265

Table 4-311 esl_encryptRSASHA1_OAEP ... 266
Table 4-312 esl_initDecryptRSASHA1_OAEP ... 266
Table 4-313 esl_decryptRSASHA1_OAEP_Label ... 267
Table 4-314 esl_decryptRSASHA1_OAEP ... 268
Table 4-315 esl_decodeRSASHA1_OAEP ... 269
Table 4-316 esl_initDecryptRSACRTSHA1_OAEP ... 270
Table 4-317 esl_decryptRSACRTSHA1_OAEP_Label ... 271
Table 4-318 esl_decryptRSACRTSHA1_OAEP ... 271
Table 4-319 esl_initEncryptRSASHA256_OAEP ... 272
Table 4-320 esl_encryptRSASHA256_OAEP_Label ... 273
Table 4-321 esl_encryptRSASHA256_OAEP_Seed ... 274
Table 4-322 esl_encryptRSASHA256_OAEP_Label_Seed ... 275
Table 4-323 esl_encryptRSASHA256_OAEP ... 275
Table 4-324 esl_initDecryptRSASHA256_OAEP ... 276
Table 4-325 esl_decryptRSASHA256_OAEP_Label ... 277
Table 4-326 esl_decryptRSASHA256_OAEP ... 278
Table 4-327 esl_decodeRSASHA256_OAEP ... 279
Table 4-328 esl_initDecryptRSACRTSHA256_OAEP ... 280
Table 4-329 esl_decryptRSACRTSHA256_OAEP_Label ... 280
Table 4-330 esl_decryptRSACRTSHA256_OAEP ... 281
Table 4-331 esl_initECDH ... 282
Table 4-332 esl_initExchangeKeyDHEcP_key ... 283
Table 4-333 esl_exchangeKeyDHEcP_key ... 284
Table 4-334 esl_generateEphemeralKeyPairECDH ... 284
Table 4-335 esl_importStaticPrivateKeyECDH ... 285
Table 4-336 esl_generateSharedSecretECDH ... 286
Table 4-337 esl_initGenerateSharedSecretDHEcP_prim ... 287
Table 4-338 esl_generateSharedSecretDHEcP_prim ... 288
Table 4-339 esl_initECBD ... 289
Table 4-340 esl_generateEphemeralKeyPairECBD ... 289
Table 4-341 esl_calculateIntermediateECBD ... 290
Table 4-342 esl_initSharedSecretECBD ... 291
Table 4-343 esl_updateSharedSecretECBD ... 292
Table 4-344 esl_retrieveSharedSecretECBD ... 292
Table 4-345 esl_initSPAKE2PPreamble ... 293
Table 4-346 esl_calcSPAKE2PPreamble ... 295
Table 4-347 esl_initSPAKE2P ... 296
Table 4-348 esl_setPreambleDataPartyASPAKE2P ... 297
Table 4-349 esl_setPreambleDataPartyBSPAKE2P ... 298
Table 4-350 esl_calcPubValSPAKE2P ... 299
Table 4-351 esl_setAdditionalInformationSPAKE2P ... 300
Table 4-352 esl_calcSharedSecretSPAKE2P ... 301
Table 4-353 esl_confirmKeySPAKE2P ... 302
Table 4-354 esl_initKDFX963SHA1 ... 302
Table 4-355 esl_deriveKeyKDFX963SHA1 ... 303
Table 4-356 esl_initKDFX963SHA256 ... 304
Table 4-357 esl_deriveKeyKDFX963SHA256 ... 305
Table 4-358 esl_initKDFX963SHA512 ... 305
Table 4-359 esl_deriveKeyKDFX963SHA512 ... 306
Table 4-360 esl_initKDF2HMACSHA1 ... 307
Table 4-361 esl_deriveKeyKDF2HMACSHA1 ... 308
Table 4-362 esl_initKDF2HMACSHA256 ... 308
Table 4-363 esl_deriveKeyKDF2HMACSHA256 ... 309
Table 4-364 esl_initHKDF_Hash_OneStep ... 310

Table 4-365 esl_deriveKeyHKDF_Hash_OneStep 311
Table 4-366 esl_initHKDF_HMAC_SHA256 311
Table 4-367 esl_deriveKeyHKDF_HMAC_SHA256 312
Table 4-368 esl_initHDKF_HMAC_SHA384 313
Table 4-369 esl_deriveKeyHKDF_HMAC_SHA384 314
Table 4-370 esl_initEncryptGCM 315
Table 4-371 esl_updateAuthDataEncryptGCM 315
Table 4-372 esl_updatePlaintextEncryptGCM 316
Table 4-373 esl_finalizeEncryptGCM 317
Table 4-374 esl_initDecryptGCM 318
Table 4-375 esl_updateAuthDataDecryptGCM 319
Table 4-376 esl_updateCiphertextDecryptGCM 320
Table 4-377 esl_finalizeDecryptGCM 320
Table 4-378 esl_initAESCCM 321
Table 4-379 esl_updateAESCCMEncrypt 323
Table 4-380 esl_finalizeAESCCMEncrypt 324
Table 4-381 esl_updateAESCCMDecrypt 325
Table 4-382 esl_finalizeAESCCMDecrypt 326
Table 4-383 esl_initEncryptChaChaPoly 327
Table 4-384 esl_updateDataChaChaPoly 327
Table 4-385 esl_updateAADChaChaPoly 328
Table 4-386 esl_finalizeChaChaPoly 329
Table 4-387 esl_initDecryptChaChaPoly 329
Table 4-388 esl_verifyChaChaPoly 330
Table 4-389 esl_initFIPS186 331
Table 4-390 esl_stirFIPS186 332
Table 4-391 esl_getBytesFIPS186 332
Table 4-392 esl_initCTRDRBG 333
Table 4-393 esl_seedCTRDRBG 334
Table 4-394 esl_getBytesCTRDRBG 335
Table 4-395 esl_getStateCTRDRBG 336
Table 4-396 esl_restoreStateCTRDRBG 337
Table 4-397 esl_initHASHDRBG 338
Table 4-398 esl_seedHASHDRBG 339
Table 4-399 esl_getBytesHASHDRBG 340
Table 4-400 esl_getStateHASHDRBG 341
Table 4-401 esl_restoreStateHASHDRBG 341
Table 4-402 esl_initESCryptoLib 342
Table 4-403 esl_initWorkSpaceHeader 342
Table 4-404 Services used by vSecPrim 343
Table 4-405 esl_getBytesRNG 344
Table 6-1 Glossary 346
Table 6-2 Abbreviations 347

1 Introduction

This document describes the functionality, APIs and configuration of the BSW module vSecPrim. The vSecPrim is only used by AUTOSAR modules, but vSecPrim is no AUTOSAR module itself. The module provides cryptographic algorithms which are used by Vector modules.

1.1 Architecture Overview

The next figure shows the interfaces to adjacent modules of the vSecPrim. These interfaces are described in Chapter 4.

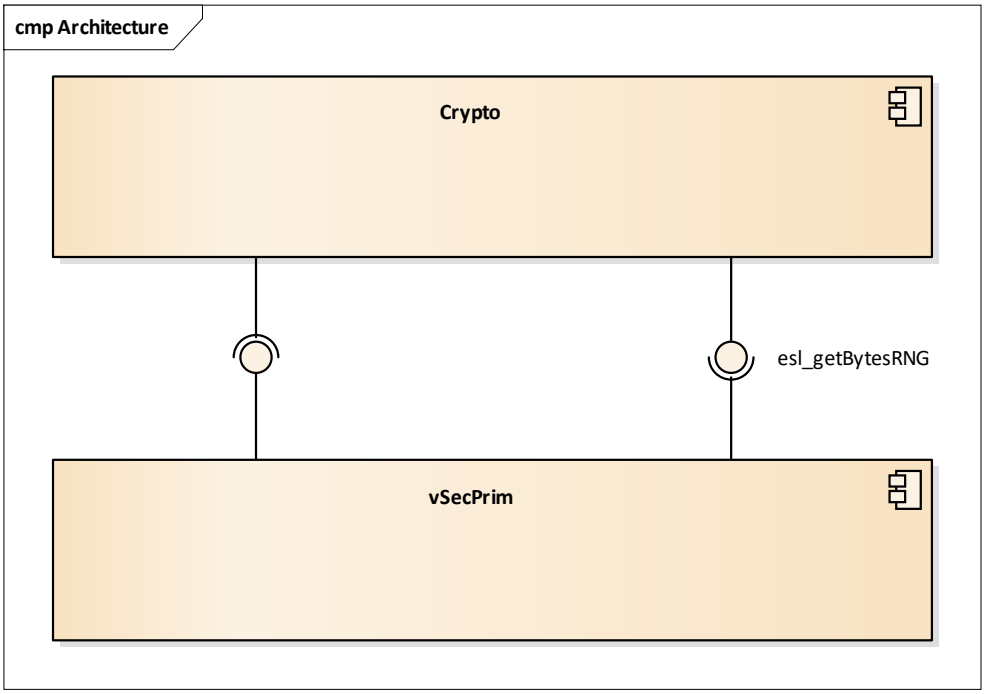


Figure 1-1 Interfaces to adjacent modules of the vSecPrim

2 Functional Description

2.1 Features

2.1.1 Provided Algorithms

The listed features cover the complete functionality specified for the vSecPrim.

2.1.1.1 Symmetric Encryption Algorithms

Symmetric encryption means that sender and recipient of an encrypted message share a private key, which is used both for encrypting and decrypting the message.

Block ciphers are symmetric encryption methods that process the message block-wisely. If block cipher algorithms shall be applied to messages that are longer than the algorithm's block length, these algorithms may require additional input parameters like the desired block streaming mode or the padding mode.

The vSecPrim provides the following symmetric encryption algorithms:

- Advanced Encryption Standard (AES) encryption and decryption in various modes of operation and key lengths of 128, 192 and 256 bit according to FIPS 197 [1]. The modes of operation are according to NIST SP 800-38A [9].

  The operation and padding modes can be combined with each other for the key sizes 128, 192 and 256 bit. Note that the electronic codebook (ECB) and the cipher block chain (CBC) mode require padding, if the initial length of the data to be encrypted or decrypted is not a multiple of the AES block size. For the counter (CTR) mode padding is not required, see [9].

  Primitive variants:

  - AES-128 (192/256) in electronic codebook (ECB) mode without padding
  - AES-128 (192/256) in electronic codebook (ECB) mode with PKCS#7 padding [21]
  - AES-128 (192/256) in electronic codebook (ECB) mode with ISO 9797-1 padding [20] (sometimes also referred to as ISO 7816-4 padding, bit padding or as padding ‘one with zeroes’)
  - AES-128 (192/256) in cipher block chaining (CBC) mode without padding
  - AES-128 (192/256) in cipher block chaining (CBC) mode with PKCS#7 padding [21]

    AES-128 (192/256) in cipher block chaining (CBC) mode with ISO 9797-1 padding [20] (sometimes also referred to as ISO 7816-4 padding, bit padding or as padding ‘one with zeroes’)
  - AES-128 (192/256) in counter mode (CTR) without padding
  - AES-128 (192/256) in counter mode (CTR) with PKCS#7 padding [21]

  - AES-128 (192/256) in counter mode (CTR) with ISO 9797-1 padding [20] (sometimes also referred to as ISO 7816-4 padding, bit padding or as padding ‘one with zeroes’)
- Data Encryption Standard (DES) encryption and decryption in various modes of operation and a key length of 56 bit according to FIPS 46-3 [2].
- Triple Data Encryption Algorithm (TDEA) encryption and decryption in various modes of operation and key lengths of 112 or 168 bit according to FIPS 46-3 [2].
- Rivest Cipher 2 (RC2) encryption and decryption in various modes of operation according to RFC 2268 [3].
- ChaCha20 (a variant of Salsa20) encryption and decryption according to RFC 7539 [4].

Please refer to Chapter 2.1.2 or to the API descriptions in Chapter 4.2 for information about whether in-place encryption and decryption is possible.

2.1.1.2 Hash Algorithms

A hash function maps given input data to a so-called message digest of fixed length. That is, the information in the possibly very long input string is "condensed" into a shorter output string. In addition, cryptographic hash functions shall be collision resistant, i.e., for a given hash value it shall be hard to find an input string that results in exactly this digest. Therefore, if two input strings only vary slightly from one another, their hash values will be completely different. This holds for all the hash function supported by vSecPrim:

- Secure Hash Algorithm SHA-1 based hash according to FIPS 180-4 [5].
- Secure Hash Algorithm SHA-224 based hash according to FIPS 180-4 [5].
- Secure Hash Algorithm SHA-256 based hash according to FIPS 180-4 [5].
- Secure Hash Algorithm SHA-384 based hash according to FIPS 180-4 [5].
- Secure Hash Algorithm SHA-512/512-256/512-224 based hash according to FIPS 180-4 [5].
- Secure Hash Algorithm SHA3 according to FIPS PUB 202 [23], including
  - SHA3-224,
  - SHA3-256,
  - SHA3-384,
  - SHA3-512,
  - SHAKE128 and
  - SHAKE256.
- Message-Digest Algorithm 5 based hash according to RFC 1321 [6].

- RIPEMD-160 based hash according to ISO/IEC 10118-3 [7].

2.1.1.3 Message Authentication Codes (MAC)

Message authentication codes ensure that the authenticity of a message is intact, i.e., it ensures that the message is sent by the person who claims to have sent it - and not by somebody else - and that the message content is unchanged.

Generating a MAC for a message requires a secret key, which the sender and the recipient must share. Upon receipt of the message and the MAC, the recipient can re-compute the MAC using the secret key and compare it to the obtained MAC. This allows him to detect modifications in the message.

The vSecPrim provides the following MAC algorithms:

- HMAC SHA-1 hash-based MAC according to FIPS 198-1 [8].
- HMAC SHA-256 hash-based MAC according to FIPS 198-1 [8].
- HMAC SHA-384 hash-based MAC according to FIPS 198-1 [8].
- HMAC RIPEMD-160 hash-based MAC according to FIPS 198-1 [8].
- CMAC AES cipher-based MAC according to NIST SP 800 38B [9] for AES key sizes 128 and 256 bit.
- GMAC AES GCM-based MAC according to NIST SP 800 38D [10].
- Poly1305 Authenticator based MAC according to RFC 7539 [4].
- SipHash MAC according to SipHash [11].

2.1.1.4 Asymmetric Signature Algorithms

Like a MAC, digital signatures shall ensure the authenticity of the auhor's identity and the integrity of the content of the signed message.

In contrast to MACs, digital signature protocols rely on the usage of a public and a private key. While the key used for signature generation must remain private, the key used for signature verification can be publicly known. Since only the sender knows his private key, no other party can generate a signature using his private key and later claim that the message was signed by the sender.

The vSecPrim provides the following signature algorithms:

- RSASSA-PKCS1-v1.5 signature algorithms based digital signatures with key length of 512-4096 bit according to PKCS #1 V1.5 [12].
  - PKCS1-v1.5 encoding based on RIPEMD160
  - PKCS1-v1.5 encoding based on SHA1

  - PKCS1-v1.5 encoding based on SHA256
- RSASSA-PSS signature algorithms based digital signatures with key length of 512-4096 bit according to PKCS #1 V2.2 [13].
  - EMSA-PSS encoding based on RIPEMD160
  - EMSA-PSS encoding based on SHA1
  - EMSA-PSS encoding based on SHA256

  RSASSA-PSS is sometimes also referred to as RSA PSS.
- Elliptic Curve Digital Signature Algorithm (EC-DSA) based digital signatures with key lengths of 128-521 bit according to ANSI X9.62 [14].

  vSecPrim also supports key generation of ECDSA keying material, i.e., private and public key, according to [14].
- Edwards-Curve Digital Signature Algorithm (EdDSA) based digital signatures over Curve25519 according to RFC 8032 [15]. Instances of the curve are
  - Ed25519 (PureEdDSA)
  - Ed25519ph (HashEdDSA, i.e., the input is hashed with SHA512 before signing with Ed25519)
  - Ed25519ctx (contextualized extension of the Ed25519 scheme)

  vSecPrim also supports key generation of EdDSA keying material, i.e., private and public key, according to [15].

2.1.1.5 Asymmetric Encryption Algorithms

Asymmetric encryption means that the sender uses a different key to encrypt a message than the recipient uses to decrypt a message. While the decryption key must remain private, the encryption key can be publicly known.

The vSecPrim provides the following asymmetric encryption algorithms:

- RSAES-PKCS1-v1.5 process based encryption (and decryption) with key length of 512 - 4096 bit according to PKCS #1 V1.5 [12].
- RSAES-OAEP process based encryption (and decryption) with key length of 512 - 4096 bit according to PKCS #1 V2.2 [13].

2.1.1.6 Key Exchange Algorithms

Key exchange algorithms allow for two or more parties to define a shared secret via public communication channels. The parties interchange values along an open communication channel, which can be publicly known. Internally, each party links its own value to the other parties' values to derive a shared secret key.

The vSecPrim provides the following key exchange algorithms:

- Elliptic Curve Diffie-Hellman (EC-D/H) key agreement over GF(p) curves with key lengths of 128-521 bit according to ANSI X9.63 [16].
- Elliptic Curve Diffie-Hellman (EC-D/H) key agreement over Curve25519 according to RFC 7748 [17].
- Elliptic Curve Burmester-Desmedt (EC-B/D) multi-party key agreement over GF(p) curves with key lengths of 128-521 bit.
- SPAKE2+ password-authenticated key agreement over curve SECP256 based on the hash function SHA256, the key derivation function HKDF HMAC SHA256 and the MAC CMAC AES128, according to [27].

2.1.1.7 Key Derivation Functions (KDF)

Based on an initial secret value (key) and possibly additional input information, a key derivation function creates more keying material using e.g. hash functions or pseudo-random functions. The derived keying material can contain one or more secret keys.

The vSecPrim provides the following key derivation functions:

- PBKDF2 HMAC-SHA1 key derivation according to PKCS #5 [18].
- PBKDF2 HMAC-SHA256 key derivation according to PKCS #5 [18].
- KDF X9.63 SHA1 key derivation according to ANSI X9.63 [16].
- KDF X9.63 SHA256 key derivation according to ANSI X9.63 [16].
- KDF X9.63 SHA512 key derivation according to ANSI X9.63 [16].
- HKDF HMAC-SHA256 one step (Option 2) and two step variant based key derivation according to [25].
- HKDF SHA256 one step (Option 1) variant according to [25].
- HKDF HMAC-SHA-384 one step (Option 2) and two step variant based key derivation according to [25].

2.1.1.8 Authenticated Encryption with Associated Data (AEAD)

AEAD schemes combine the encryption of a message and the creation of a MAC to ensure the authenticity of the message’s content. Also, integrity and authenticity of possibly given and unencrypted associated authentication data is protected.

The vSecPrim provides the following AEAD algorithms:

- Galois Counter Mode authenticated (in-place) encryption according to NIST SP 800-38D [10] for AES key sizes 128, 192 and 256 bit.
- Counter with CBC-MAC (CCM) authenticated (in-place) encryption according to NIST SP 800-38C [26] for AES key size 128 bit.
- ChaCha20 and Poly1305 authenticated encryption according to RFC 7539 [4].

For these AEAD algorithms, in-place encryption and decryption is possible, see also Chapter 2.1.2.

2.1.1.9 Random Number Generation (PRNG)

The vSecPrim supports different pseudo random number generators. Given input entropy (and probably additional input data), these generators compute a sequence of random number in a deterministic way. This means that, given the same input data, they will generate the same random numbers again.

- FIPS 186-2 PRNG (SHA-1) according to FIPS 186-2 [19].
- CTR DRBG based on AES128, according to NIST SP 800-90Ar1 [22] with and without derivation function (DF).
- CTR DRBG based on AES256, according to NIST SP 800-90Ar1 [22] with and without derivation function (DF).
- Hash DRBG based on SHA2-512, according to NIST SP 800-90Ar1 [22].

2.1.2 In-place encryption and decryption

The following algorithms allow for in-place encryption and decryption, i.e., the addresses referenced by the input and output pointers can be identical:

- AES GCM,
- AES CCM,
- AES-128 in ECB, CBC and CTR mode with PKCS5 padding, ISO 9797-1 padding and without padding,
- AES-192 in ECB, CBC and CTR mode with PKCS5 padding, ISO 9797-1 padding and without padding,
- AES-256 in ECB, CBC and CTR mode with PKCS5 padding, ISO 9797-1 padding and without padding,
- ChaCha20,
- AEAD ChaCha20 and Poly1305 and
- RC2 in ECB and CBC mode and block encryption and decryption

2.1.3 Limitations

This chapter lists limitations of the vSecPrim Module.

2.1.3.1 Parameter Checking

The internal parameter checks of the API functions cannot be disabled.

2.1.3.2 Module Usage

The functionality according to the AUTOSAR standard is provided by the module Crypto_30_LibCv.



Caution
This module shall only be used by vector modules.

2.2 Additional Algorithm Specific Information

2.2.1 Random Number Generation with DRBG

The DRBG requires different input parameter for its seeding and generating operations. The required, optional and recommended parameters are shown in Table 2-1 Parameter classification for CTR DRBG and Hash DRBG operations.

DRBG Operation \ Parameter		Entropy	Nonce	Personalization String	Additional Input
CTR DRBG Without DF	Instantiation	X		R	
	Reseed	X			O
	Random Number Generation				O
CTR DRBG With DF	Instantiation	X	X	R	
	Reseed	X			O
	Random Number Generation				O
Hash DRBG	Instantiation	X	X	R	
	Reseed	X			O
	Random Number Generation				O
X: required R: recommended O: optional					

Table 2-1 Parameter classification for CTR DRBG and Hash DRBG operations

The parameters have to fulfill the length requirements shown in Table 2-2 DRBG algorithms, security strength and input sizes, to provide the security strength values given in [22].

Size parameter \ DRBG Algorithm	CTR DRBG AES128 with derivation function	CTR DRBG AES 128 without derivation function	CTR DRBG AES256 with derivation function	CTR DRBG AES256 without derivation function	Hash DRBG SHA2-512
Maximum security strength	128	128	256	256	256
Configurable security strength	112, 128	112, 128	-	-	-
Minimum entropy input length required to provide maximum security strength	128 bit	256 bit	256 bit	384 bit	256 bit

Maximum entropy input length	2^{32} – 10 bytes	256 bit	2^{32} – 10 bytes	384 bit	2^{32} bytes
Maximum personalization string input length	2^{32} – 10 bytes	256 bit	2^{32} – 10 bytes	384 bit	2^{32} bytes
Maximum additional input input length	2^{32} – 10 bytes	256 bit	2^{32} – 10 bytes	384 bit	2^{32} bytes
Maximum nonce input length	2^{32} – 10 bytes	2^{32} bytes	2^{32} – 10 bytes	2^{32} bytes	2^{32} bytes
Output length	1024 bytes	1024 bytes	1024 bytes	1024 bytes	65535 bytes

Table 2-2 DRBG algorithms, security strength and input sizes

Due to the implementation for embedded software all input parameters are limited to a length of the maximal value of uint32.

The security strength for CTR DRBG AES256 and Hash DRBG SHA2-512 is not configurable. Choose the DRBG algorithm according to its entropy and security strength requirements and to your entropy data.

The PRNG does not provide the features according to [22] for prediction resistance check and self / health test function.



Expert Knowledge
Reseed the DRBG instance as often as possible with new entropy to increase the numbers' randomness. The entropy source is an important part to increase the numbers randomness.

2.2.2 SPAKE2+

2.2.2.1 Cipher Suites

The SPAKE2+ implementation supports two versions, that differ in the way how the MAC keys are derived:

- a newer version according to [27] and
- a slightly modified version, see below.

Both versions derive the MAC keys as output keying material (OKM) of the key derivation function specified in the cipher suite. In [27], the key for party A is the first half of the OKM and the key for party B is the second half of the OKM. In the modified version, the key for party A is the second half of the (OKM) and the key for party B is the first half of the OKM. This also affects the key confirmation phase of the SPAKE2+ protocol.

The cipher suite, on which the SPAKE2+ algorithm is based, can be determined by the 'ciphersuite' parameter in esl_initSPAKE2P(). Table 2-3 shows the available values of this parameter.

Value of ‘mode’ in esl_initSPAKE2P()	Cipher suite
ESL_SPAKE2P_MODE_CIPHERSUITE_8_1	Curve: SECP256 Hash function: SHA2-256 Key derivation function: HKDF HMAC SHA2-256 MAC: CMAC AES128 Version: newer version according to [27]
ESL_SPAKE2P_MODE_CIPHERSUITE_8_2	Curve: SECP256 Hash function: SHA2-256 Key derivation function: HKDF HMAC SHA2-256 MAC: CMAC AES128 Version: modified version, see above

Table 2-3 Possible values of ‘ciphersuite’ in esl_initSPAKE2P()

2.2.2.2 Order of API Calls

The SPAKE2+ protocol consists of an (optional) preamble phase and a key exchange protocol. This is reflected in the vSecPrim in different APIs, which need to be called in the following order:

Preamble phase:

- esl_initSPAKE2PPreamble(..)
- esl_calcSPAKE2PPreamble(..)

Key exchange protocol:

- esl_initSPAKE2P(..)
- esl_setPreambleDataPartyASPAKE2P(..) or esl_setPreambleDataPartyBSPAKE2P(..)
- esl_calcPubValSPAKE2P(..)
- esl_setAdditionalInformationSPAKE2P(..)
  - Note: The call to this API is optional. It can be called any time after the call to esl_initSPAKE2P(..) and before the call to esl_calcSharedSecretSPAKE2P(..).
- esl_calcSharedSecretSPAKE2P(..)
- esl_confirmKeySPAKE2P(..)

Executing the preamble phase is optional. If the preamble data is already given, the key exchange protocol can be started without calling the preamble APIs before. In any case, the API esl_ setPreambleDataPartyASPAKE2P(..) or esl_ setPreambleDataPartyBSPAKE2P(..) needs to be called.

2.2.3 EC Based Algorithms over GF(p) Curves

For ECDSA, ECDH and ECBD over GF(p) curves, the vSecPrim expects the curve parameters as input parameters, see Chapters 4.2.2.4.2 and 4.2.2.6. The vSecPrim then parses the domain parameters and can therefore handle a large variety of curves with prime

bit lengths between 128 and 521 bits. For a list of supported standardized curves, such as NIST or SEC curves, please refer to [24].

2.3 Workspace and Call Concept

The vSecPrim uses a workspace concept for algorithmic calculations. Thereby the vSecPrim holds all data in the provided workspace. To support parallel processing the algorithm only needs to be processed on different workspace instances.

The processing of an algorithm is split in steps for preparation, data handling and finalization. The workspace needs to be kept for the complete processing and must not be modified between successive calls to functions from the list below. The naming of the functions is not consistent for all algorithms and some algorithms do not support all steps.

- esl_initWorkSpaceHeader:
  - Initialize workspace header.
  - It is mandatory to call this function before the initialization/preparation phase for any algorithm, see also the entries in field “Particularities and Limitations” in the respective API descriptions in Chapter 4.2.
- esl_init<algorithm>:
  - Initialize algorithm and set key or calculation parameters.
- esl_update<algorithm>:
  - Add new data to the algorithm and update internal calculation. For some algorithm there is also an intermediate result provided.
- esl_finalize<algorithm>:
  - Finalize calculation and provide result.

Alternatively, instead of esl_update<algorithm> and esl_finalize<algorithm>, some algorithms offer esl_<executeServiceAlgorithm>, e.g. for the ECDSA the respective APIs are called esl_initSignDSAEcP_prim, esl_signDSAEcP_prim, esl_initVerifyDSAEcP_prim and esl_verifyDSAEcP_prim. Other algorithms may follow similar naming schemes.



Caution
While calling esl_initWorkSpaceHeader the size of the workspace must be given as parameter. Therefor the defines from the vSecPrim shall be used: e.g. `ESL_MAXSIZEOF_WS_SHA256`. The given size refers to the size of the workspace without the header.

Expert Knowledge
The vSecPrim does not have any global variables. Hence, the vSecPrim does not occupy any RAM.

2.4 Clear Workspace

The vSecPrim provides a feature to clear the workspace for basic and symmetric algorithms. The workspace is cleared in the finalize/verification phase.

Caution
This feature can be disabled to improve runtime. If the workspace needs to be cleared, this is part of the security concept. In some case this is superfluous, e.g. if the keys are also held as plaintext in RAM.

2.5 Module Concept

The vSecPrim consists of two sub-modules, the EsLib and the actLib. The EsLib checks the parameters for consistency and handles sequences which use the basic cryptographic algorithms. The actLib provides the basic cryptographic algorithms.

For some services, there is a third, outermost, sub-module, called vSecPrim itself. This submodule provides easy access to the library vSecPrim by summarizing different algorithms of the same algorithm family into one API, such as e.g. a generic hashing API. All provided hash functions can be used via this API, where the respective hash algorithm is determined by a simple parameter. Hence, it is not necessary to call APIs dedicated to a single hashing algorithm.

A visualization of the different layers is shown in Figure 2-1.

© 2023 Vector Informatik GmbH

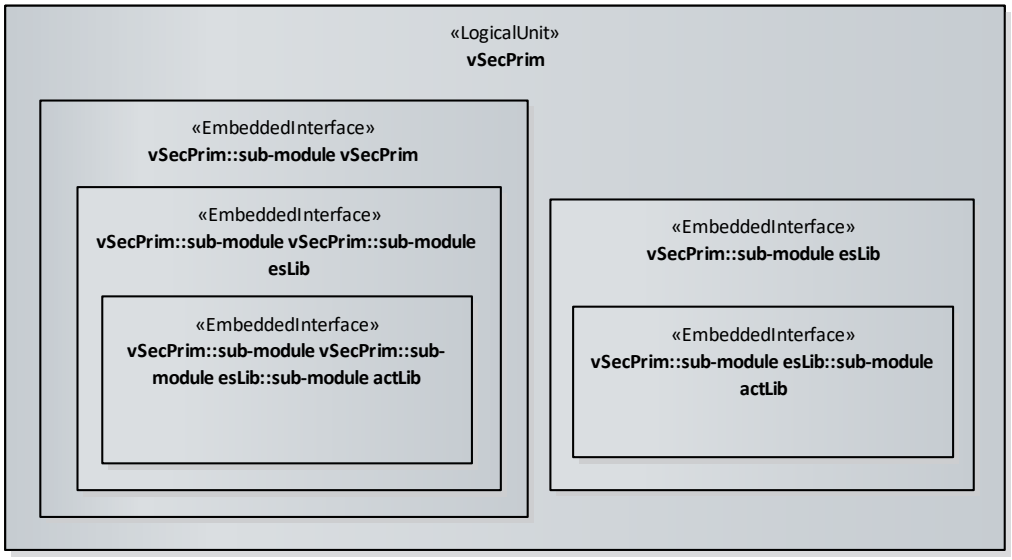


Figure 2-1 Module concept of vSecPrim

Table 2-4 lists all algorithms that can be accessed by the vSecPrim module layer:

API names	Accessible algorithms
vSecPrim_initHash() vSecPrim_updateHash() vSecPrim_finalizeHash() vsecPrim_digestHash()	SHA1 SHA2-224, SHA2-256 SHA2-384, SHA2-512, SHA2-512-224, SHA2-512-256 SHA3-224, SHA3-256, SHA3-384, SHA3-512 SHAKE128, SHAKE256 RIPEMD160 MD5

Table 2-4 Accessible algorithms in module layer vSecPrim

For more information about the respective APIs and how to determine which algorithm shall be used, please refer to Chapter 4.2.

3 Integration

This chapter gives necessary information for the integration of the MICROSAR Classic vSecPrim into an application environment of an ECU.

3.1 Scope of Delivery

The delivery of the vSecPrim contains the files which are described in Chapters 3.1.1 and 3.1.2.

3.1.1 Static Files

File Name	Description
vSecPrim.h	This is the module's header file
vSecPrim.c	This is the module's source file
vSecPrim*.h	These are the outermost module's header files
vSecPrim*.c	These are the outermost module's source files, providing interfaces to services on the outermost level
ESLib*.h	This is the internal module's header file
ESLib*.c	These are the internal module's source files
act*.h	These are the innermost module's header files
act*.c	These are the innermost module's source files

Table 3-1 Static files

For information about the different layers of the module, please refer to Chapter 2.5.

3.1.2 Dynamic Files

The dynamic files are generated by the configuration tool DaVinci Configurator 5 Pro.

File Name	Description
vSecPrim_Cfg.h	This is the configuration header file.
vSecPrim_MemMap.h	This file contains the module's memory section mappings. This file is generated for MICROSAR Classic R27 or newer projects; it is static in R26 and older projects.

Table 3-2 Generated files

3.2 Entropy Callout

The vSecPrim provides a callout for random number provision (esl_getBytesRNG: 4.4.1.1). There are some hints and requirements for the implementation. The callout declaration is available via vSecPrim.h. The callout is only required for the following algorithm types:

- Key Generation
- RSASSA and RSAES
- ECDSA and ECDHE

The callout needs to be implemented according the entropy requirement of the used algorithm.



Note
For more information about the entropy callout, please refer to [24]. The crypto driver can implement the callout itself.

3.3 Watchdog Callout

The vSecPrim provides a callout for a watchdog functionality to trigger a watchdog during algorithms with long computation times. The watchdog function must be provided to vSecPrim as a parameter when a workspace is initialized (see esl_initWorkSpaceHeader(..), Table 4-403).

It is possible to configure different watchdog levels:

- Watchdog Level 0: The watchdog functionality is disabled
- Watchdog Level 1: The watchdog is called from outer functions only
- Watchdog Level 2: The watchdog is also called in more internal functions
- Watchdog Level 3: The watchdog is called in the innermost hash loops

© 2023 Vector Informatik GmbH

4 API Description

For an interface overview please see Figure 1-1.

4.1 Type Definitions

The types defined by the vSecPrim are described in this chapter.

4.1.1 General Types

Type Name	C-Type	Description	Value Range
vSecPrimType_HashAlgorithm	uint8	Identifier for hash algorithms	VSECPRIM_DIGEST_SHA1: identifier for SHA1 VSECPRIM_DIGEST_SHA2_224: identifier for SHA2-224 VSECPRIM_DIGEST_SHA2_256: identifier for SHA2-256 VSECPRIM_DIGEST_SHA2_384: identifier for SHA2-384 VSECPRIM_DIGEST_SHA2_512: identifier for SHA2-512 VSECPRIM_DIGEST_SHA2_512_222: identifier for SHA2-512-224 VSECPRIM_DIGEST_SHA2_512_256: identifier for SHA2-512-256 VSECPRIM_DIGEST_SHA3_224: identifier for SHA3-224 VSECPRIM_DIGEST_SHA3_256: identifier for SHA3-256 VSECPRIM_DIGEST_SHA3_384: identifier for SHA3-384 VSECPRIM_DIGEST_SHA3_512: identifier for SHA3-512 VSECPRIM_DIGEST_SHAKE_128: identifier for SHAKE128 VSECPRIM_DIGEST_SHAKE_256: identifier for SHAKE256 VSECPRIM_DIGEST_RIPEMD160: identifier for RIPEMD160 VSECPRIM_DIGEST_MD5: identifier for MD5

Table 4-1 Type definitions for hash function identifiers

Type Name	C-Type	Description	Value Range
vSecPrimType_outputBufferFlag	uint16	Flag that indicates how	VSECPRIM_PROVIDED_BUFFER_LENGTH_MATCHES_DESIRED_OUTPUT_LENGTH: The length of the provided output buffer shall match

		vSecPrim shall handle the output buffer provided by the using module	the maximal possible output length provided by vSecPrim exactly; truncation of the output is not allowed VSECPRIM_PROVIDED_BUFFER_LENGTH_MATCHES_OR_EXCEEDS_DESIRED_OUTPUT_LENGTH: The length of the provided output buffer shall match or exceed the maximal possible output length provided by vSecPrim exactly; truncation of the output is not allowed VSECPRIM_OUTPUT_TRUNCATION_ALLOWED: The length of the provided output buffer shall match or exceed the maximal possible output length provided by vSecPrim exactly; truncation of the output is allowed

Table 4-2 Type definitions for output buffer flags

4.1.2 Return Types

Type Name	C-Type	Description	Value Range
eslt_ErrorCode	uint16	Error Codes	ESL_ERT_RETURNCODE: no error
			ESL_ERT_WARNING: a warning
			ESL_ERT_ERROR: a fatal error
			ESL_ERT_WS_ERROR: workspace and init-errorcodes; second nibble == 1
			ESL_ERC_NO_ERROR: OK, NO ERROR
			ESL_ERC_WARNING: WARNINGS
			ESL_ERC_ERROR: GENERAL ERRORS
			ESL_ERC_WS_STATE_INVALID: invalid workspace
			ESL_ERC_WS_TOO_SMALL: the given workSpace is too small
			ESL_ERC_PARAMETER_INVALID: some parameters contain invalid values
			ESL_ERC_MODE_INVALID: unsupported mode (eslt_Mode)
			ESL_ERC_MODE_INCOMPATIBLE: if two valid modes interfere
			ESL_ERC_MEMORY_CONFLICT: memory conflict
			ESL_ERC_OUTPUT_SIZE_TOO_SHORT: the given output length is not sufficient
			ESL_ERC_BUFFER_TOO_SMALL: the given output length is not sufficient
			ESL_ERC_INPUT_INVALID: invalid input

Type Name	C-Type	Description	Value Range
			ESL_ERC_RNG_INIT_FAILED: init (HW not available, no response)
			ESL_ERC_RNG_ENTROPY_TOO_SMALL: not enough entropy
			ESL_ERC_RNG_BAD_INTERNAL_STATE: bad internal state of RNG
			ESL_ERC_AES_PADDING_INVALID: the given padding is invalid
			ESL_ERC_SHA1_TOTAL_LENGTH_OVERFLOW: Hash and HashMAC/ ERRORS
			ESL_ERC_HMAC_KEY_LENGTH_OUT_OF_RANGE: key length is zero
			ESL_ERC_HMAC_INCORRECT_MAC: the given HashMAC is invalid
			ESL_ERC_INCORRECT_MAC: the given HashMAC is invalid
			ESL_ERC_KDF_ITERATION_COUNT_OUT_OF_RANGE: iteration count is zero
			ESL_ERC_ECC_DOMAIN_INVALID: the given domain parameter are invalid
			ESL_ERC_ECC_PUBKEY_INVALID: the public key is invalid
			ESL_ERC_ECC_MESSAGE_TOO_LONG: the input message is too long
			ESL_ERC_ECC_PRIVKEY_INVALID: the private key is invalid
			ESL_ERC_ECC_DOMAINEXT_INVALID: values do not match p and n
			ESL_ERC_ECC_SPEEDUPEXT_INVALID: value groups is not supported
			ESL_ERC_ECC_INTERNAL_ERROR: internal error of the
			ESL_ERC_ECC_SIGNATURE_INVALID: signature invalid
			ESL_ERC_RSA_MODULE_OUT_OF_RANGE: the given module is not supported
			ESL_ERC_RSA_CODE_OUT_OF_RANGE: input value (code) out of range
			ESL_ERC_RSA_MESSAGE_OUT_OF_RANGE: input value (message) out of range
			ESL_ERC_RSA_SIGNATURE_OUT_OF_RANGE: input value (signature) out of range
			ESL_ERC_RSA_SIGNATURE_INVALID: the given signature is invalid

Type Name	C-Type	Description	Value Range
			ESL_ERC_RSA_ENCODING_INVALID: encoding invalid
			ESL_ERC_RSA_PUBKEY_INVALID: RSA public key invalid
			ESL_ERC_RSA_PRIVKEY_INVALID: RSA private key invalid
			ESL_ERC_TOTAL_LENGTH_OVERFLOW: input length for hash algorithm too long
			ESL_ERC_SHA256_TOTAL_LENGTH_OVERFLOW: SHA-256 ERRORS
			ESL_ERC_SHA384_TOTAL_LENGTH_OVERFLOW: SHA-384 ERRORS
			ESL_ERC_RIPEMD160_TOTAL_LENGTH_OVERFLOW: RIPEMD160 ERRORS
			ESL_ERC_DES_PADDING_INVALID: the given padding is invalid
			ESL_ERC_TDES_KEY_LENGTH_INVALID: the given key length is invalid
			ESL_ERC_GCM_INVALID_KEY_LENGTH: key length must be one of the AES key lengths: 16, 24 or 32 bytes
			ESL_ERC_GCM_TOTAL_LENGTH_OVERFLOW: input too long
			ESL_ERC_GCM_TAG_VERIFICATION_FAILED: given tag does not match the computed tag
			ESL_ERC_GCM_STATE_INVALID: passing AAD after message data
			ESL_ERC_CCM_INVALID_KEY_LENGTH: key length must be 16 bytes
			ESL_ERC_CCM_TOTAL_LENGTH_OVERFLOW: aad input too long
			ESL_ERC_STATE_INVALID: SHA3 in invalid KECCAK state (squeeze instead of absorb) or CCM update function called more than once
			ESL_ERC_INVALID_IV_LENGTH: IV length must be at least 1 (bit) (NOT specific for GCM!!)
			ESL_ERC_CURVE_NOT_SUPPORTED: curve not supported (NOT specific for EdDSA!!)
			ESL_ERC_SIGNATURE_INVALID: signature invalid (NOT specific for EdDSA!!)
			ESL_ERC_RNG_FAILED: RNG failed - no private key generated (NOT specific for EdDSA!!)
			ESL_ERC_INSTANCE_NOT_SUPPORTED: EdDSA instance not supported

Type Name	C-Type	Description	Value Range
			ESL_ERC_INCORRECT_TAG: the given Poly1305 TAG is invalid
			ESL_ERC_ENTROPY_EXHAUSTION: entropy exhausted, reseeding (of DRBG) required
			ESL_ERC_INVALID_LENGTH: length of an input parameter invalid
			ESL_ERC_REQUESTED_LENGTH_INVALID: requested output length invalid
			ESL_ERC_UNRECOVERABLE_ERROR: unspecific Errors, unrecoverable error
			ESL_ERC_16BITS: 16 bit signed error codes

Table 4-3 Type definitions for error codes

4.1.3 Types for Elliptic Curve Domain Parameters

Type Name	C-Type	Description	Detailed Description
eslt_EccDomain	uint8	Elliptic curve domain parameters in ASN1 format	Variables of type eslt_EccDomain shall contain the elliptic curve domain parameters as defined in [14], namely • the field prime p, • the curve parameters a and b, • the base point G, • the order n, • and the cofactor h. This information shall be represented in the following format: TLV(SEQUENCE) { TLV(INTEGER(version)), TLV(SEQUENCE) { TLV(OBJECT_IDENTIFIER(prime -oID)), TLV(INTEGER(p) }, TLV(SEQUENCE) { TLV(OCTETSTRING(a)), TLV(OCTETSTRING(b)) }, TLV(OCTETSTRING(G)), TLV(INTEGER(n)), TLV(INTEGER(h)) },

			where TLV is short for (Tag - Length - Value) and shall be encoded using DER (Distinguished Encoding Rules). It is assumed that the base point representation is uncompressed, i.e., both coordinates are contained in TLV(OCTETSTING(G)). The length of the overall sequence is thus contained in the length-octets of the first TLV(SEQUENCE) object. The length is not passed separately, when a variable of type eslt_EccDomain is passed to a function and not stored in another global variable, either.

Table 4-4 Type definitions: Elliptic curve domain parameters

Type Name	C-Type	Description	Detailed Description

eslt_EccDomainExt	uint8	Elliptic curve domain extension parameters in ASN1 format	Variables of type eslt_EccDomainExt shall contain additional elliptic curve domain parameters as defined in [14], namely
			• the field prime Montgomery constant
			• the low digit of the negative field inverse
			• the Montgomery representations of the curve parameters a and b
			• the base point in Montgomery representation
			• the order Montgomery constant and
			• the negative order inverse
			This information shall be represented in the following format:
			TLV(SEQUENCE)
			{
			TLV(SEQUENCE)
			{
			TLV(OCTETSTRING(p_RR)),
			TLV(OCTETSTRING(p_bar)),
			TLV(SEQUENCE)
			{
			TLV(OCTETSTRING(a_R)),
			TLV(OCTETSTRING(b_R))
			},
			TLV(OCTETSTRING(G_R)),
			}
			TLV(SEQUENCE)
			{
			TLV(OCTETSTRING(n_RR)),
			TLV(OCTETSTRING(n_bar)),
			}

			} and shall be encoded using DER (Distinguished Encoding Rules). It is assumed that the base point representation is uncompressed, i.e., both coordinates are contained in TLV(OCTETSTING(G_R)). The length of the overall sequence is thus contained in the length-octets of the first TLV(SEQUENCE) object. The length is not passed separately, when a variable of type eslt_EccDomainExt is passed to a function and not stored in another global variable, either.

Table 4-5 Type definitions: Elliptic curve domain extension parameters

Type Name	C-Type	Description	Detailed Description

eslt_EccSpeedUpExt	uint8	Precomputed points on the elliptic curve described by eslt_EccDomain	Variables of type eslt_EccSpeedUpExt shall contain precomputed points on the elliptic curve specified by eslt_EccDomain as defined in [14] , namely
			• the group parameter, indicating how many precomputed points are provided and
			• the precomputed points.
			This information shall be represented in the following format:
			TLV(SEQUENCE)
			{
			TLV(OCTETSTRING(groups)),
			TLV(SEQUENCE)
			{
			TLV(OCTETSTRING(2*G_R)),
			...
			[(2^groups) - 1 precomputed points and correction point D_R]
			...
			TLV(OCTETSTRING(D_R))
			},
			}
			and shall be encoded using DER (Distinguished Encoding Rules).
			It is assumed that each point representation is uncompressed, i.e., both coordinates are contained in TLV(OCTETSTING(point)).
			The length of the overall sequence is thus contained in the length-octets of the first TLV(SEQUENCE) object. The length is not

			passed separately, when a variable of type eslt_SpeedUpExt is passed to a function and not stored in another global variable, either.

Table 4-6 Type definitions: Elliptic curve speed up parameters

4.1.4 Algorithm Specific Types

Type Name	C-Type	Description	Value Range
eslt_PaddingMode	Uint8	Padding Modes Defines how the plaintext, that shall be encrypted or decrypted with a block cipher algorithm, is padded	ESL_PM_PKCS5 apply PKCS#7 padding to the plaintext (equivalent to PKCS#5 padding)
			ESL_PM_OFF do not pad the plaintext
			ESL_PM_ONEWITHZEROES apply ISO 9797-1, padding method 2, to the plaintext sometimes also referred to as ISO 7816d4 padding, bit padding or padding ‘one with zeroes’

Table 4-7 Type definitions: Padding modes for block ciphers

Type Name	C-Type	Description	Value Range
eslt_BlockMode	uint8	operation mode for block ciphers	ESL_BM_ECB electronic codebook mode (ECB)
			ESL_BM_CBC cipher block chaining (CBC)
			ESL_BM_CTR Counter mode (also integer counter mode or segmented integer counter mode)

Table 4-8 Type definitions: operation modes for block ciphers

Type Name	C-Type	Description	Value Range
eslt_DRBGMode	Uint8	Identifier for variant of DRBG: specifies which algorithm the DRBG is based on (AES128) and whether a derivation	ESL_DRBGMODE_CTRDRBG_AES128_DF
			ESL_DRBGMODE_CTRDRBG_AES128_NODF
			ESL_DRBGMODE_CTRDRBG_AES256_DF
			ESL_DRBGMODE_CTRDRBG_AES256_NODF
			ESL_DRBGMODE_HASHDRBG_SHA512

		function shall be used	

Table 4-9 Type definitions: Mode identifiers for DRBG

Type Name	C-Type	Description
eslt_DRBGReseedCounterType	Uint32	Type for the DRBG reseed counter, used in the DRBG workSpace to store the reseed counter

Table 4-10 Type definitions: DRBG reseed counter type

Type Name	C-Type	Description	Value Range
eslt_DRBGSeedStatusType	Uint8	Type for the DRBG seed status	ESL_DRBG_SEED_STATUS_UNSEEDED
			ESL_DRBG_SEED_STATUS_SEEDED
			ESL_DRBG_SEED_STATUS_NEED_RESEED

Table 4-11 Type definitions: DRBG seed status type

Type Name	C-Type	Description	Value Range
eslt_HKDFMode	Uint8	Type for HDKF variant distinction	ESL_HKDF_ONE_STEP_VARIANT One-step variant of HKDF shall be used
			ESL_HKDF_TWO_STEP_VARIANT Two-step variant of HKDF shall be used, i.e., key derivation consists of an extraction and an expansion step

Table 4-12 Type definitions: HKDF variant type

Type Name	C-Type	Description	Value Range
eslt_SPAKE2PMode	Uint32	Type for SPAKE2+ mode, specifying the used cipher suite (curve, hash function, KDF, MAC) and the version	ESL_SPAKE2P_MODE_CIPHERSUITE_8_1 Cipher suite: curve SECP256, SHA2-256, HKDF HMAC SHA2-256, CMAC AES128 Version: version according to [27]; the output keying material of the HKDF contains the MAC key for party A as first half and the MAC key for party B as second half
			ESL_SPAKE2P_MODE_CIPHERSUITE_8_2 Cipher suite: curve SECP256, SHA2-256, HKDF HMAC SHA2-256, CMAC AES128 Version: modified version; the output keying material of the HKDF contains the MAC key for party B as first half and the MAC key for party A as second half

Table 4-13 Type definitions: SPAKE2+ mode type

4.2 Services provided by vSecPrim

4.2.1 Generalized Service Functions

4.2.1.1 Generalized Hash Service Functions

4.2.1.1.1 vSecPrim_initHash

```
eslt_ErrorCode vSecPrim_initHash (vSecPrimType_WorkSpaceHash) workSpace,
vSecPrimType_HashAlgorithm hashID, vSecPrimType_OutputBufferFlag
outputLengthFlag
```

Parameter	
workSpace [in,out]	(in) algorithm context buffer (out) initialized algorithm context structure
hashID	identifier of the hash function to be used
outputLengthFlag	flag indicating whether - the provided output buffer in finalize will be exactly as long as the desired digest length and truncation of the digest is not allowed - the provided output buffer in finalize will possibly be longer than the desired digest length and truncation of the digest is not allowed - the provided output buffer in finalize will either be exactly as long as or longer than the desired digest length and truncation of the digest is allowed
Return code	
eslt_ErrorCode	ESL_ERC_PARAMETER_INVALID input parameter is NULL
	ESL_ERC_INPUT_INVALID value of hashID or of outputLengthFlag invalid
	ESL_ERC_WS_TOO_SMALL work space too small
	ESL_ERC_NO_ERROR else
Functional Description	
Initializes the hash calculation. This function initializes the workspace header and the workspace itself by calling the hash function init routine corresponding to the value of hashID..	
Particularities and Limitations	
workSpace is initialized by esl_initWorkSpaceHeader(..), in particular the watchdog function pointer in the workspace header must be initialized. Please refer to the API descriptions in ESLib.h for further information about the specific hash initialization APIs. They can be found under esl_init[hash function name](..).	
Call context	
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces	

Table 4-14 vSecPrim_initHash

4.2.1.1.2 vSecPrim_updateHash

Prototype	
`eslt_ErrorCode` **`vSecPrim_updateHash`** `(VSECPRIM_P2VAR_PARA(vSecPrimType_WorkSpaceHash) workSpace, const eslt_Length inputLength, VSECPRIM_P2CONST_PARA(eslt_Byte) input)`	
Parameter	
workSpace [in,out]	(in) hash context buffer (out) updated hash context structure
inputLength [in]	length of input data in bytes
input [in]	pointer to data to be hashed
Return code	
eslt_ErrorCode	ESL_ERC_PARAMETER_INVALID input parameter is NULL
	ESL_ERC_WS_STATE_INVALID work space state invalid
	ESL_ERC_WS_TOO_SMALL work space too small
	ESL_ERC_TOTAL_LENGTH_OVERFLOW total input data too long
	ESL_ERC_STATE_INVALID update failed (e.g. SHA3 specific error occurred)
	ESL_ERC_NO_ERROR else
Functional Description	
Updates the hash calculation with the given input data. This function can be called several times until all data to be hashed has been passed. It calls the hash function update routine specified by workSpace->hashID.	
Particularities and Limitations	
workSpace is initialized by vSecPrim_initHash (..) Please refer to the API descriptions in ESLib.h for further information about the specific hash update APIs. They can be found under esl_update[hash function name](..).	
Call context	
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces	

Table 4-15 vSecPrim_updateHash

4.2.1.1.3 vSecPrim_finalizeHash

Prototype	
`eslt_ErrorCode` **`vSecPrim_finalizeHash`** `( VSECPRIM_P2VAR_PARA(vSecPrimType_WorkSpaceHash) workSpace, VSECPRIM_P2VAR_PARA(eslt_Length) messageDigestLength, VSECPRIM_P2VAR_PARA(eslt_Byte) messageDigest)`	
Parameter	
workSpace [in,out]	(in) hash context buffer (out) updated hash context structure

messageDigestLength [in,out]	(in) pointer to buffer, storing the length of messageDigest (out) pointer to buffer, storing the length of the digest written to the buffer referenced by 'messageDigest'
messageDigest [in,out]	(in) pointer to buffer to store the digest, possibly a null pointer (out) pointer to buffer storing the computed digest
Return code	
eslt_ErrorCode	ESL_ERC_PARAMETER_INVALID input parameter is NULL
	ESL_ERC_WS_STATE_INVALID work space state invalid
	ESL_ERC_WS_TOO_SMALL work space too small
	ESL_ERC_INPUT_INVALID *messageDigestLength invalid with respect to workSpace->outputLengthFlag
	ESL_ERC_OUTPUT_SIZE_TOO_SHORT *messageDigestLength too small
	ESL_ERC_NO_ERROR else
Functional Description	
Finalizes the hash calculation and returns the (possibly truncated) hash value. The hash function is specified by the parameter workSpace->hashID. If 'messageDigest' is not a null pointer, this function calls the hash function finalize routine specified by workSpace->hashID and returns the hash value. If 'messageDigest' is not a null pointer and if *messageDigestLength is shorter than the algorithm specific digest size and if truncation is allowed, the truncated digest is written to 'messageDigest'. If 'messageDigest' is a null pointer, this function writes the number of required bytes to store the (untruncated) digest to *messageDigestLength and does not compute the hash value. In this case, the function can be called again.	
Particularities and Limitations	
workSpace has been initialized by vSecPrim_initHash (...) If 'messageDigest' is not a null pointer, the buffer referenced by messageDigest must provide at least as many bytes as referenced by messageDigestLength. Please refer to the API descriptions in ESLib.h for further information about the specific hash finalization APIs. They can be found under esl_finalize[hash function name](..).	
Call context	
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces	

Table 4-16 vSecPrim_finalizeHash

4.2.1.1.4 vSecPrim_digestHash

Prototype

```
eslt_ErrorCode vSecPrim_digestHash (
VSECPRIM_P2VAR_PARA(vSecPrimType_WorkSpaceHash) workSpace, const eslt_Length
inputLength,VSECPRIM_P2CONST_PARA(eslt_Byte) input,
VSECPRIM_P2VAR_PARA(eslt_Length) messageDigestLength,
VSECPRIM_P2VAR_PARA(eslt_Byte) messageDigest)
```

Parameter	
workSpace [in,out]	(in) hash context buffer (out) updated hash context structure
inputLength [in]	length of input data in bytes

input [in]	pointer to data to be hashed
messageDigestLength [in,out]	(in) pointer to buffer, storing the length of messageDigest (out) pointer to buffer, storing the length of the digest written to the buffer referenced by 'messageDigest'
messageDigest [in,out]	(in) pointer to buffer to store the digest, possibly a null pointer (out) pointer to buffer storing the computed digest
Return code	
eslt_ErrorCode	ESL_ERC_PARAMETER_INVALID input parameter is NULL
	ESL_ERC_WS_STATE_INVALID work space state invalid
	ESL_ERC_WS_TOO_SMALL work space too small
	ESL_ERC_TOTAL_LENGTH_OVERFLOW total input data too long
	ESL_ERC_INPUT_INVALID *messageDigestLength invalid with respect to workSpace->outputLengthFlag
	ESL_ERC_OUTPUT_SIZE_TOO_SHORT *messageDigestLength too small
	ESL_ERC_NO_ERROR else
Functional Description	
Computes the hash value of input and writes it (possibly truncated) to messageDigest. This API simplifies the single call hashing use case, so vSecPrim_updateHash() must not be called before. The hash function is specified by the parameter workSpace->hashID. If 'messageDigest' is not a null pointer, this function calls the hash function update and finalize routines specified by workSpace->hashID and returns the hash value. If 'messageDigest' is not a null pointer and if *messageDigestLength is shorter than the algorithm specific digest size and if truncation is allowed, the truncated digest is written to 'messageDigest'. If 'messageDigest' is a null pointer, this function writes the number of required bytes to store the (untruncated) digest to *messageDigestLength and does not compute the hash value. In this case, the function can be called again.	
Particularities and Limitations	
workSpace has been initialized by vSecPrim_initHash (...) If 'messageDigest' is not a null pointer, the buffer referenced by messageDigest must provide at least as many bytes as referenced by messageDigestLength. Please refer to the API descriptions in ESLib.h for further information about the specific hash update and finalization APIs. They can be found under esl_update[hash function name](..) and esl_finalize[hash function name](..).	
Call context	
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces	

Table 4-17 vSecPrim_digestHash

4.2.2 Service Functions Sorted by Algorithms

4.2.2.1 Symmetric Encryption Algorithms

4.2.2.1.1 esl_initEncryptAES128

Prototype	
`eslt_ErrorCode` **`esl_initEncryptAES128`** `(VSECPRIM_P2VAR_PARA(eslt_WorkSpaceAES128) workSpace, VSECPRIM_P2CONST_PARA(eslt_Byte) key, const eslt_BlockMode blockMode, const eslt_PaddingMode paddingMode, VSECPRIM_P2CONST_PARA(eslt_Byte) initializationVector)`	
Parameter	
workSpace [in,out]	(in) algorithm context buffer, initialized by esl_initWorkSpaceHeader(..) function (out) initialized algorithm context structure
key [in]	pointer to user key of length ESL_SIZEOF_AES128_KEY
blockMode [in]	ECB, CBC, CTR, etc. block mode switch
paddingMode [in]	padding mode switch
initializationVector [in]	in CBC mode: pointer to initialization vector of length ESL_SIZEOF_AES128_BLOCK in CTR mode: pointer to counter of length ESL_SIZEOF_AES128_BLOCK if initializationVector==NULL, a trivial zero block is used as iv or counter redundant in ECB mode
Return code	
eslt_ErrorCode	ESL_ERC_PARAMETER_INVALID input parameter is NULL
	ESL_ERC_WS_TOO_SMALL work space too small
	ESL_ERC_MODE_INVALID block or padding mode is invalid
	ESL_ERC_NO_ERROR else
Functional Description	
Init AES128 encryption. This function initializes the AES128 stream encryption algorithm.	
Particularities and Limitations	
workSpace is initialized by esl_initWorkSpaceHeader(..)	
Call context	
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces	

Table 4-18 esl_initEncryptAES128

4.2.2.1.2 esl_encryptAES128

Prototype
`eslt_ErrorCode` **`esl_encryptAES128`** `(VSECPRIM_P2VAR_PARA(eslt_WorkSpaceAES128) workSpace, const eslt_Length inputSize, VSECPRIM_P2CONST_PARA(eslt_Byte)`

`input, VSECPRIM_P2VAR_PARA(eslt_Length) outputSize, VSECPRIM_P2VAR_PARA(eslt_Byte) output)`	
Parameter	
workSpace [in,out]	(in) algorithm context buffer, initialized by esl_initEncryptAES128(..) function (out) updated algorithm context structure
inputSize [in]	length of input data in bytes
input [in]	pointer to data to be encrypted
outputSize [in,out]	(in) output buffer size in bytes, a safe length is ((inputSize+15)/16)*16 (see output) (out) total size of encrypted blocks ((inputSize+buffered_bytes)/16)*16
output [in,out]	(in) pointer to encrypted data buffer (out) pointer to encrypted data blocks
Return code	
eslt_ErrorCode	ESL_ERC_PARAMETER_INVALID input parameter is NULL
	ESL_ERC_WS_STATE_INVALID work space state invalid
	ESL_ERC_WS_TOO_SMALL work space too small
	ESL_ERC_OUTPUT_SIZE_TOO_SHORT outputSize too small
	ESL_ERC_NO_ERROR else
Functional Description	
This function encrypts input data of any length and can be called arbitrarily often after the algorithm initialization. Exact (inputSize+buffered_bytes)/16 blocks will be encrypted and written to the output buffer, (inputSize+buffered_bytes)16 remaining bytes will be buffered, where 0 <= buffered_bytes < 16.	
Particularities and Limitations	
workSpace is initialized by esl_initEncryptAES128(..) input and output can point to the same address, i.e., inplace encryption is possible for ECB, CBC and CTR mode, independent from the chosen padding method	
Call context	
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces	

Table 4-19 esl_encryptAES128

4.2.2.1.3 esl_finalizeEncryptAES128

Prototype	
`eslt_ErrorCode` **`esl_finalizeEncryptAES128`** `(VSECPRIM_P2VAR_PARA(eslt_WorkSpaceAES128) workSpace, VSECPRIM_P2VAR_PARA(eslt_Length) outputSize, VSECPRIM_P2VAR_PARA(eslt_Byte) output)`	
Parameter	
workSpace [in,out]	(in) algorithm context structure (out) updated algorithm context structure
outputSize [in,out]	(in) output buffer size in bytes, a safe length is 2*AES_BLOCK_LENGTH (out) total size of encrypted input 16 or 32 bytes

output [in,out]	(in) pointer to encrypted data buffer (out) pointer to encrypted data blocks
Return code	
eslt_ErrorCode	ESL_ERC_PARAMETER_INVALID input parameter is NULL
	ESL_ERC_WS_STATE_INVALID work space state invalid
	ESL_ERC_WS_TOO_SMALL work space too small
	ESL_ERC_OUTPUT_SIZE_TOO_SHORT outputSize too small
	ESL_ERC_NO_ERROR else
Functional Description	
Finalize AES 128 encrypt. This function finalizes the AES encryption by processing the remaining input bytes in the internal buffer and padding the input according to the chosen padding in the corresponding init function.	
Particularities and Limitations	
> workSpace is initialized by esl_initEncryptAES128(..)esl_encryptAES128 called before	
Call context	
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces	

Table 4-20 esl_finalizeEncryptAES128

4.2.2.1.4 esl_initDecryptAES128

Prototype	
`eslt_ErrorCode esl_initDecryptAES128 (VSECPRIM_P2VAR_PARA(eslt_WorkSpaceAES128) workSpace, VSECPRIM_P2CONST_PARA(eslt_Byte) key, const eslt_BlockMode blockMode, const eslt_PaddingMode paddingMode, VSECPRIM_P2CONST_PARA(eslt_Byte) initializationVector)`	
Parameter	
workSpace [in,out]	(in) algorithm context buffer, initialized by esl_initWorkSpaceHeader(..) function (out) initialized algorithm context structure
key [in]	pointer to user key of length ESL_SIZEOF_AES128_KEY
blockMode [in]	ECB, CBC, CTR, etc. block mode switch
paddingMode [in]	padding mode switch
initializationVector [in]	in CBC mode: pointer to initialization vector of length ESL_SIZEOF_AES128_BLOCK in CTR mode: pointer to counter of length ESL_SIZEOF_AES128_BLOCK if initializationVector==NULL, a trivial zero block is used as iv or counter redundant in ECB mode
Return code	
eslt_ErrorCode	ESL_ERC_PARAMETER_INVALID input parameter is NULL
	ESL_ERC_WS_TOO_SMALL work space too small

	ESL_ERC_MODE_INVALID block or padding mode is invalid
	ESL_ERC_NO_ERROR else
Functional Description	
Init AES128 decryption. This function initializes the AES stream decryption algorithm.	
Particularities and Limitations	
workSpace is initialized by esl_initWorkSpaceHeader(..)	
Call context	
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces	

Table 4-21 esl_initDecryptAES128

4.2.2.1.5 esl_decryptAES128

Prototype	
`eslt_ErrorCode esl_decryptAES128 (VSECPRIM_P2VAR_PARA(eslt_WorkSpaceAES128) workSpace, const eslt_Length inputSize, VSECPRIM_P2CONST_PARA(eslt_Byte) input, VSECPRIM_P2VAR_PARA(eslt_Length) outputSize, VSECPRIM_P2VAR_PARA(eslt_Byte) output)`	
Parameter	
workSpace [in,out]	(in) algorithm context buffer, initialized by esl_initDecryptAES128(..) function (out) updated algorithm context structure
inputSize [in]	length of input data in bytes
input [in]	pointer to data to be encrypted
outputSize [in,out]	(in) output buffer size in bytes, a safe length is ((inputSize+15)/16)*16 (see output) (out) total size of encrypted blocks ((inputSize+buffered_bytes)/16)*16
output [in,out]	(in) pointer to encrypted data buffer (out) pointer to encrypted data blocks
Return code	
eslt_ErrorCode	ESL_ERC_PARAMETER_INVALID input parameter is NULL
	ESL_ERC_WS_STATE_INVALID work space state invalid
	ESL_ERC_WS_TOO_SMALL work space too small
	ESL_ERC_OUTPUT_SIZE_TOO_SHORT outputSize too small
	ESL_ERC_NO_ERROR else
Functional Description	
This function decrypts input data of any length and can be called arbitrarily often after the algorithm initialization. Exact (inputSize+buffered_bytes)/16 blocks will be decrypted and written to the output buffer, (inputSize+buffered_bytes)16 remaining bytes will be buffered, where 0 <= buffered_bytes < 16.	

Particularities and Limitations
workSpace is initialized by esl_initDecryptAES128(..) input and output can point to the same address, i.e., inplace decryption is possible for ECB, CBC and CTR mode, independent from the chosen padding method
Call context
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces

Table 4-22 esl_decryptAES128

4.2.2.1.6 esl_finalizeDecryptAES128

Prototype	
`eslt_ErrorCode esl_finalizeDecryptAES128 (VSECPRIM_P2VAR_PARA(eslt_WorkSpaceAES128) workSpace, VSECPRIM_P2VAR_PARA(eslt_Length) outputSize, VSECPRIM_P2VAR_PARA(eslt_Byte) output)`	
Parameter	
workSpace [in,out]	(in) algorithm context structure (out) updated algorithm context structure
outputSize [in,out]	(in) output buffer size in bytes, a safe length is AES_BLOCK_LENGTH (out) total size of decrypted input 16 or 32 bytes
output [in,out]	(in) pointer to decrypted data buffer (out) pointer to encrypted data blocks
Return code	
eslt_ErrorCode	ESL_ERC_PARAMETER_INVALID input parameter is NULL
	ESL_ERC_WS_STATE_INVALID work space state invalid
	ESL_ERC_WS_TOO_SMALL work space too small
	ESL_ERC_OUTPUT_SIZE_TOO_SHORT outputSize too small
	ESL_ERC_AES_PADDING_INVALID the padding is invalid
	ESL_ERC_INPUT_INVALID the total input length is not zero mod blocksize
	ESL_ERC_NO_ERROR else
Functional Description	
Finalize AES 128 decrypt. This function finalizes the AES decryption by processing the remaining input bytes in the internal buffer.	
Particularities and Limitations	
> workSpace is initialized by esl_initDecryptAES128(..)esl_decryptAES128 called before	
Call context	
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces	

Table 4-23 esl_finalizeDecryptAES128

4.2.2.1.7 esl_initEncryptAES128Block

Prototype	
`eslt_ErrorCode` **`esl_initEncryptAES128Block`** `(VSECPRIM_P2VAR_PARA(eslt_WorkSpaceAES128Block) workSpace, VSECPRIM_P2CONST_PARA(eslt_Byte) key)`	
Parameter	
workSpace [in,out]	(in) algorithm context buffer, initialized by esl_initWorkSpaceHeader(..) function (out) initialized algorithm context structure
key [in]	pointer to user key of length ESL_SIZEOF_AES128_KEY
Return code	
eslt_ErrorCode	ESL_ERC_PARAMETER_INVALID input parameter is NULL
	ESL_ERC_WS_TOO_SMALL work space too small
	ESL_ERC_MODE_INVALID block or padding mode is invalid
	ESL_ERC_NO_ERROR else
Functional Description	
Init AES128 block decryption. This function initializes the AES block encryption algorithm.	
Particularities and Limitations	
workSpace is initialized by esl_initWorkSpaceHeader(..)	
Call context	
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces	

Table 4-24 esl_initEncryptAES128Block

4.2.2.1.8 esl_encryptAES128Block

Prototype	
`eslt_ErrorCode` **`esl_encryptAES128Block`** `(VSECPRIM_P2VAR_PARA(eslt_WorkSpaceAES128Block) workSpace, VSECPRIM_P2CONST_PARA(eslt_Byte) inputBlock, VSECPRIM_P2VAR_PARA(eslt_Byte) outputBlock)`	
Parameter	
workSpace [in,out]	(in) algorithm context buffer, initialized by esl_initEncryptAES128Block(..) function (out) updated algorithm context structure
inputBlock [in]	pointer to the given input block (16 bytes)
outputBlock [in,out]	(in) pointer to output buffer (at least 16 bytes) (out) pointer to encrypted data block

Return code	
eslt_ErrorCode	ESL_ERC_PARAMETER_INVALID input parameter is NULL
	ESL_ERC_WS_STATE_INVALID work space state invalid
	ESL_ERC_WS_TOO_SMALL work space too small
	ESL_ERC_NO_ERROR else
Functional Description	
Encrypt AES Block. This function encrypts an AES Block (= 16 bytes)	
Particularities and Limitations	
workSpace is initialized by esl_initEncryptAES128Block(..) inputBlock and outputBlock can point to the same address, i.e., inplace encryption is possible	
Call context	
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces	

Table 4-25 esl_encryptAES128Block

4.2.2.1.9 esl_initDecryptAES128Block

Prototype	

`eslt_ErrorCode` **`esl_initDecryptAES128Block`**
`(VSECPRIM_P2VAR_PARA(eslt_WorkSpaceAES128Block) workSpace,`
`VSECPRIM_P2CONST_PARA(eslt_Byte) key)`	

Parameter	
workSpace [in,out]	(in) algorithm context buffer, initialized by esl_initWorkSpaceHeader(..) function (out) initialized algorithm context structure
key [in]	pointer to user key of length ESL_SIZEOF_AES128_KEY
Return code	
eslt_ErrorCode	ESL_ERC_PARAMETER_INVALID input parameter is NULL
	ESL_ERC_WS_TOO_SMALL work space too small
	ESL_ERC_MODE_INVALID block or padding mode is invalid
	ESL_ERC_NO_ERROR else
Functional Description	
Decrypt AES 128 block. This function initializes the AES block decryption algorithm.	
Particularities and Limitations	
workSpace is initialized by esl_initWorkSpaceHeader(..)	
Call context	
> TASK	

> This function is Synchronous > This function is Reentrant, for different workspaces

Table 4-26 esl_initDecryptAES128Block

4.2.2.1.10 esl_decryptAES128Block

Prototype	
`eslt_ErrorCode esl_decryptAES128Block (VSECPRIM_P2CONST_PARA(eslt_WorkSpaceAES128Block) workSpace, VSECPRIM_P2CONST_PARA(eslt_Byte) inputBlock, VSECPRIM_P2VAR_PARA(eslt_Byte) outputBlock)`	
Parameter	
workSpace [in]	(in) algorithm context buffer, initialized by esl_initDecryptAES128Block(..) function (out) updated algorithm context structure
inputBlock [in]	pointer to the given input block (16 bytes)
outputBlock [in,out]	(in) pointer to output buffer (at least 16 bytes) (out) pointer to encrypted data block
Return code	
eslt_ErrorCode	ESL_ERC_PARAMETER_INVALID input parameter is NULL
	ESL_ERC_WS_STATE_INVALID work space state invalid
	ESL_ERC_WS_TOO_SMALL work space too small
	ESL_ERC_NO_ERROR else
Functional Description	
Decrypt AES block. This function decrypts an AES Block (= 16 bytes)	
Particularities and Limitations	
workSpace is initialized by esl_initDecryptAES128Block(..) inputBlock and outputBlock can point to the same address, i.e., inplace decryption is possible	
Call context	
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces	

Table 4-27 esl_decryptAES128Block

4.2.2.1.11 esl_initEncryptAES192

Prototype
`eslt_ErrorCode esl_initEncryptAES192 (VSECPRIM_P2VAR_PARA(eslt_WorkSpaceAES192) workSpace, VSECPRIM_P2CONST_PARA(eslt_Byte) key, const eslt_BlockMode blockMode, const`

`eslt_PaddingMode paddingMode, VSECPRIM_P2CONST_PARA(eslt_Byte) initializationVector)`

Parameter	
workSpace [in,out]	(in) algorithm context buffer, initialized by esl_initWorkSpaceHeader(..) function (out) initialized algorithm context structure
key [in]	pointer to user key of length ESL_SIZEOF_AES192_KEY
blockMode [in]	ECB, CBC, CTR, etc. block mode switch
paddingMode [in]	padding mode switch
initializationVector [in]	in CBC mode: pointer to initialization vector of length ESL_SIZEOF_AES192_BLOCK in CTR mode: pointer to counter of length ESL_SIZEOF_AES192_BLOCK if initializationVector==NULL, a trivial zero block is used as iv or counter redundant in ECB mode
Return code	
eslt_ErrorCode	ESL_ERC_PARAMETER_INVALID input parameter is NULL
	ESL_ERC_WS_TOO_SMALL work space too small
	ESL_ERC_MODE_INVALID block or padding mode is invalid
	ESL_ERC_NO_ERROR else
Functional Description	
Init AES192 encryption. This function initializes the AES192 stream encryption algorithm.	
Particularities and Limitations	
workSpace is initialized by esl_initWorkSpaceHeader(..)	
Call context	
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces	

Table 4-28 esl_initEncryptAES192

4.2.2.1.12 esl_encryptAES192

Prototype
`eslt_ErrorCode esl_encryptAES192 (VSECPRIM_P2VAR_PARA(eslt_WorkSpaceAES192) workSpace, const eslt_Length inputSize, VSECPRIM_P2CONST_PARA(eslt_Byte) input, VSECPRIM_P2VAR_PARA(eslt_Length) outputSize, VSECPRIM_P2VAR_PARA(eslt_Byte) output)`

Parameter	
input [in]	pointer to data to be encrypted
outputSize [in,out]	(in) output buffer size in bytes, a safe length is ((inputSize+15)/16)*16 (see output) (out) total size of encrypted blocks ((inputSize+buffered_bytes)/16)*16
workSpace [in,out]	(in) algorithm context buffer, initialized by esl_initEncryptAES192(..) function (out) actualized algorithm context structure

output [in,out]	(in) pointer to encrypted data buffer (out) pointer to encrypted data blocks
inputSize [in]	length of input data in bytes
Return code	
eslt_ErrorCode	ESL_ERC_PARAMETER_INVALID input parameter is NULL
	ESL_ERC_WS_STATE_INVALID work space state invalid
	ESL_ERC_WS_TOO_SMALL work space too small
	ESL_ERC_OUTPUT_SIZE_TOO_SHORT outputSize too small
	ESL_ERC_NO_ERROR else
Functional Description	
This function encrypts input data of any length and can be called arbitrarily often after the algorithm initialization. Exact (inputSize+buffered_bytes)/16 blocks will be encrypted and written to the output buffer, (inputSize+buffered_bytes)16 remaining bytes will be buffered, where 0 <= buffered_bytes < 16.	
Particularities and Limitations	
workSpace is initialized by esl_initEncryptAES192(..) input and output can point to the same address, i.e., inplace encryption is possible for ECB, CBC and CTR mode, independent from the chosen padding method	
Call context	
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces	

Table 4-29 esl_encryptAES192

4.2.2.1.13 esl_finalizeEncryptAES192

Prototype	
`eslt_ErrorCode` **`esl_finalizeEncryptAES192`** `(VSECPRIM_P2VAR_PARA(eslt_WorkSpaceAES192) workSpace, VSECPRIM_P2VAR_PARA(eslt_Length) outputSize, VSECPRIM_P2VAR_PARA(eslt_Byte) output)`	
Parameter	
outputSize [in,out]	(in) output buffer size in bytes, a safe length is 2*AES_BLOCK_LENGTH (out) total size of encrypted input 16 or 32 bytes
workSpace [out]	actualized algorithm context structure
output [in,out]	(in) pointer to encrypted data buffer (out) pointer to encrypted data blocks
Return code	
eslt_ErrorCode	ESL_ERC_PARAMETER_INVALID input parameter is NULL
	ESL_ERC_WS_STATE_INVALID work space state invalid
	ESL_ERC_WS_TOO_SMALL work space too small
	ESL_ERC_OUTPUT_SIZE_TOO_SHORT outputSize too small
	ESL_ERC_NO_ERROR else

Functional Description
This function finalizes the AES encryption by processing the remaining input bytes in the internal buffer and padding the input according to the chosen padding in the corresponding init function.
Particularities and Limitations
workSpace is initialized by esl_initEncryptAES192(..) esl_encryptAES192 called before
Call context
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces

Table 4-30 esl_finalizeEncryptAES192

4.2.2.1.14 esl_initDecryptAES192

Prototype	
`eslt_ErrorCode` **`esl_initDecryptAES192`** `(VSECPRIM_P2VAR_PARA(eslt_WorkSpaceAES192) workSpace, VSECPRIM_P2CONST_PARA(eslt_Byte) key, const eslt_BlockMode blockMode, const eslt_PaddingMode paddingMode, VSECPRIM_P2CONST_PARA(eslt_Byte) initializationVector)`	
Parameter	
initializationVector [in]	in CBC mode: pointer to initialization vector of length ESL_SIZEOF_AES192_BLOCK in CTR mode: pointer to counter of length ESL_SIZEOF_AES192_BLOCK if initializationVector==NULL, a trivial zero block is used as iv or counter redundant in ECB mode
key [in]	pointer to user key of length ESL_SIZEOF_AES192_KEY
paddingMode [in]	padding mode switch
workSpace [in,out]	(in) algorithm context buffer, initialized by esl_initWorkSpaceHeader(..) function (out) initialized algorithm context structure
blockMode [in]	ECB, CBC, CTR, etc. block mode switch
Return code	
eslt_ErrorCode	ESL_ERC_PARAMETER_INVALID input parameter is NULL
	ESL_ERC_WS_TOO_SMALL work space too small
	ESL_ERC_MODE_INVALID block or padding mode is invalid
	ESL_ERC_NO_ERROR else
Functional Description	
This function initializes the AES stream decryption algorithm.	
Particularities and Limitations	
workSpace is initialized by esl_initWorkSpaceHeader(..)	
Call context	
> TASK	

> This function is Synchronous > This function is Reentrant, for different workspaces

Table 4-31 esl_initDecryptAES192

4.2.2.1.15 esl_decryptAES192

Prototype	
`eslt_ErrorCode esl_decryptAES192 (VSECPRIM_P2VAR_PARA(eslt_WorkSpaceAES192) workSpace, const eslt_Length inputSize, VSECPRIM_P2CONST_PARA(eslt_Byte) input, VSECPRIM_P2VAR_PARA(eslt_Length) outputSize, VSECPRIM_P2VAR_PARA(eslt_Byte) output)`	
Parameter	
input [in]	pointer to data to be encrypted
outputSize [in,out]	(in) output buffer size in bytes, a safe length is ((inputSize+15)/16)*16 (see output) (out) total size of encrypted blocks ((inputSize+buffered_bytes)/16)*16
workSpace [in,out]	(in) algorithm context buffer, initialized by esl_initDecryptAES192(..) function (out) actualized algorithm context structure
output [in,out]	(in) pointer to encrypted data buffer (out) pointer to encrypted data blocks
inputSize [in]	length of input data in bytes
Return code	
eslt_ErrorCode	ESL_ERC_PARAMETER_INVALID input parameter is NULL
	ESL_ERC_WS_STATE_INVALID work space state invalid
	ESL_ERC_WS_TOO_SMALL work space too small
	ESL_ERC_OUTPUT_SIZE_TOO_SHORT outputSize too small
	ESL_ERC_NO_ERROR else
Functional Description	
This function decrypts input data of any length and can be called arbitrarily often after the algorithm initialization. Exact (inputSize+buffered_bytes)/16 blocks will be decrypted and written to the output buffer, (inputSize+buffered_bytes)16 remaining bytes will be buffered, where 0 <= buffered_bytes < 16.	
Particularities and Limitations	
workSpace is initialized by esl_initDecryptAES192(..) input and output can point to the same address, i.e., inplace decryption is possible for ECB, CBC and CTR mode, independent from the chosen padding method	
Call context	
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces	

Table 4-32 esl_decryptAES192

4.2.2.1.16 esl_finalizeDecryptAES192

Prototype	
`eslt_ErrorCode` **`esl_finalizeDecryptAES192`** `(VSECPRIM_P2VAR_PARA(eslt_WorkSpaceAES192) workSpace, VSECPRIM_P2VAR_PARA(eslt_Length) outputSize, VSECPRIM_P2VAR_PARA(eslt_Byte) output)`	
Parameter	
outputSize [in,out]	(in) output buffer size in bytes, a safe length is AES_BLOCK_LENGTH (out) total size of decrypted input 16 or 32 bytes
workSpace [out]	actualized algorithm context structure
output [in,out]	(in) pointer to decrypted data buffer (out) pointer to encrypted data blocks
Return code	
eslt_ErrorCode	ESL_ERC_PARAMETER_INVALID input parameter is NULL
	ESL_ERC_WS_STATE_INVALID work space state invalid
	ESL_ERC_WS_TOO_SMALL work space too small
	ESL_ERC_OUTPUT_SIZE_TOO_SHORT outputSize too small
	ESL_ERC_AES_PADDING_INVALID the padding is invalid
	ESL_ERC_INPUT_INVALID the total input length is not zero mod blocksize
	ESL_ERC_NO_ERROR else
Functional Description	
This function finalizes the AES decryption by processing the remaining input bytes in the internal buffer.	
Particularities and Limitations	
workSpace is initialized by esl_initDecryptAES192(..) esl_decryptAES192 called before	
Call context	
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces	

Table 4-33 esl_finalizeDecryptAES192

4.2.2.1.17 esl_initEncryptAES192Block

Prototype	
`eslt_ErrorCode` **`esl_initEncryptAES192Block`** `(VSECPRIM_P2VAR_PARA(eslt_WorkSpaceAES192Block) workSpace, VSECPRIM_P2CONST_PARA(eslt_Byte) key)`	
Parameter	
key [in]	pointer to user key of length ESL_SIZEOF_AES192_KEY
workSpace [in,out]	(in) algorithm context buffer, initialized by esl_initWorkSpaceHeader(..) function (out) initialized algorithm context structure

Return code	
eslt_ErrorCode	ESL_ERC_PARAMETER_INVALID input parameter is NULL
	ESL_ERC_WS_TOO_SMALL work space too small
	ESL_ERC_MODE_INVALID block or padding mode is invalid
	ESL_ERC_NO_ERROR else
Functional Description	
This function initializes the AES block encryption algorithm.	
Particularities and Limitations	
workSpace is initialized by esl_initWorkSpaceHeader(..)	
Call context	
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces	

Table 4-34 esl_initEncryptAES192Block

4.2.2.1.18 esl_encryptAES192Block

Prototype	
`eslt_ErrorCode` **`esl_encryptAES192Block`** `(VSECPRIM_P2VAR_PARA(eslt_WorkSpaceAES192Block) workSpace, VSECPRIM_P2CONST_PARA(eslt_Byte) inputBlock, VSECPRIM_P2VAR_PARA(eslt_Byte) outputBlock)`	
Parameter	
outputBlock [in,out]	(in) pointer to output buffer (at least 16 bytes) (out) pointer to encrypted data block
inputBlock [in]	pointer to the given input block (16 bytes)
workSpace [in,out]	(in) algorithm context buffer, initialized by esl_initEncryptAES192Block(..) function (out) actualized algorithm context structure
Return code	
eslt_ErrorCode	ESL_ERC_PARAMETER_INVALID input parameter is NULL
	ESL_ERC_WS_STATE_INVALID work space state invalid
	ESL_ERC_WS_TOO_SMALL work space too small
	ESL_ERC_NO_ERROR else
Functional Description	
This function encrypts an AES Block (= 16 bytes).	
Particularities and Limitations	
workSpace is initialized by esl_initEncryptAES192Block(..) inputBlock and outputBlock can point to the same address, i.e., inplace encryption is possible	
Call context	

> TASK > This function is Synchronous > This function is Reentrant, for different workspaces

Table 4-35 esl_encryptAES192Block

4.2.2.1.19 esl_initDecryptAES192Block

Prototype	
`eslt_ErrorCode` **`esl_initDecryptAES192Block`** `(VSECPRIM_P2VAR_PARA(eslt_WorkSpaceAES192Block) workSpace, VSECPRIM_P2CONST_PARA(eslt_Byte) key)`	
Parameter	
key [in]	pointer to user key of length ESL_SIZEOF_AES192_KEY
workSpace [in,out]	(in) algorithm context buffer, initialized by esl_initWorkSpaceHeader(..) function (out) initialized algorithm context structure
Return code	
eslt_ErrorCode	ESL_ERC_PARAMETER_INVALID input parameter is NULL
	ESL_ERC_WS_TOO_SMALL work space too small
	ESL_ERC_MODE_INVALID block or padding mode is invalid
	ESL_ERC_NO_ERROR else
Functional Description	
This function initializes the AES block decryption algorithm.	
Particularities and Limitations	
workSpace is initialized by esl_initWorkSpaceHeader(..)	
Call context	
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces	

Table 4-36 esl_initDecryptAES192Block

4.2.2.1.20 esl_decryptAES192Block

Prototype	
`eslt_ErrorCode` **`esl_decryptAES192Block`** `(VSECPRIM_P2VAR_PARA(eslt_WorkSpaceAES192Block) workSpace, VSECPRIM_P2CONST_PARA(eslt_Byte) inputBlock, VSECPRIM_P2VAR_PARA(eslt_Byte) outputBlock)`	
Parameter	
outputBlock [in,out]	(in) pointer to output buffer (at least 16 bytes) (out) pointer to encrypted data block

inputBlock [in]	pointer to the given input block (16 bytes)
workSpace [in,out]	(in) algorithm context buffer, initialized by esl_initDecryptAES192Block(..) function (out) actualized algorithm context structure
Return code	
eslt_ErrorCode	ESL_ERC_PARAMETER_INVALID input parameter is NULL
	ESL_ERC_WS_STATE_INVALID work space state invalid
	ESL_ERC_WS_TOO_SMALL work space too small
	ESL_ERC_NO_ERROR else
Functional Description	
This function decrypts an AES Block (= 16 bytes).	
Particularities and Limitations	
workSpace is initialized by esl_initDecryptAES192Block(..) inputBlock and outputBlock can point to the same address, i.e., inplace decryption is possible	
Call context	
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces	

Table 4-37 esl_decryptAES192Block

4.2.2.1.21 esl_initEncryptAES256

Prototype	
`eslt_ErrorCode esl_initEncryptAES256 (VSECPRIM_P2VAR_PARA(eslt_WorkSpaceAES256) workSpace, VSECPRIM_P2CONST_PARA(eslt_Byte) key, const eslt_BlockMode blockMode, const eslt_PaddingMode paddingMode, VSECPRIM_P2CONST_PARA(eslt_Byte) initializationVector)`	
Parameter	
workSpace [in,out]	(in) algorithm context buffer, initialized by esl_initWorkSpaceHeader(..) function (out) initialized algorithm context structure
key [in]	pointer to user key of length ESL_SIZEOF_AES256_KEY
blockMode [in]	ECB, CBC, CTR, etc. block mode switch
paddingMode [in]	padding mode switch
initializationVector [in]	in CBC mode: pointer to initialization vector of length ESL_SIZEOF_AES256_BLOCK in CTR mode: pointer to counter of length ESL_SIZEOF_AES256_BLOCK if initializationVector==NULL, a trivial zero block is used as iv or counter redundant in ECB mode
Return code	
eslt_ErrorCode	ESL_ERC_PARAMETER_INVALID input parameter is NULL
	ESL_ERC_WS_TOO_SMALL work space too small

	ESL_ERC_MODE_INVALID block or padding mode is invalid
	ESL_ERC_NO_ERROR else
Functional Description	
Init AES256 encryption. This function initializes the AES256 stream encryption algorithm.	
Particularities and Limitations	
workSpace is initialized by esl_initWorkSpaceHeader(..)	
Call context	
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces	

Table 4-38 esl_initEncryptAES256

4.2.2.1.22 esl_encryptAES256

Prototype	
`eslt_ErrorCode esl_encryptAES256 (VSECPRIM_P2VAR_PARA(eslt_WorkSpaceAES256) workSpace, const eslt_Length inputSize, VSECPRIM_P2CONST_PARA(eslt_Byte) input, VSECPRIM_P2VAR_PARA(eslt_Length) outputSize, VSECPRIM_P2VAR_PARA(eslt_Byte) output)`	
Parameter	
input [in]	pointer to data to be encrypted
outputSize [in,out]	(in) output buffer size in bytes, a safe length is ((inputSize+15)/16)*16 (see output) (out) total size of encrypted blocks ((inputSize+buffered_bytes)/16)*16
workSpace [in,out]	(in) algorithm context buffer, initialized by esl_initEncryptAES256(..) function (out) actualized algorithm context structure
output [in,out]	(in) pointer to encrypted data buffer (out) pointer to encrypted data blocks
inputSize [in]	length of input data in bytes
Return code	
eslt_ErrorCode	ESL_ERC_PARAMETER_INVALID input parameter is NULL
	ESL_ERC_WS_STATE_INVALID work space state invalid
	ESL_ERC_WS_TOO_SMALL work space too small
	ESL_ERC_OUTPUT_SIZE_TOO_SHORT outputSize too small
	ESL_ERC_NO_ERROR else
Functional Description	
This function encrypts input data of any length and can be called arbitrarily often after the algorithm initialization. Exact (inputSize+buffered_bytes)/16 blocks will be encrypted and written to the output buffer, (inputSize+buffered_bytes)16 remaining bytes will be buffered, where 0 <= buffered_bytes < 16.	
Particularities and Limitations	
workSpace is initialized by esl_initEncryptAES256(..)	

input and output can point to the same address, i.e., inplace encryption is possible for ECB, CBC and CTR mode, independent from the chosen padding method
Call context
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces

Table 4-39 esl_encryptAES256

4.2.2.1.23 esl_finalizeEncryptAES256

Prototype	
`eslt_ErrorCode` **`esl_finalizeEncryptAES256`** `(VSECPRIM_P2VAR_PARA(eslt_WorkSpaceAES256) workSpace, VSECPRIM_P2VAR_PARA(eslt_Length) outputSize, VSECPRIM_P2VAR_PARA(eslt_Byte) output)`	
Parameter	
outputSize [in,out]	(in) output buffer size in bytes, a safe length is 2*AES_BLOCK_LENGTH (out) total size of encrypted input 16 or 32 bytes
workSpace [out]	actualized algorithm context structure
output [in,out]	(in) pointer to encrypted data buffer (out) pointer to encrypted data blocks
Return code	
eslt_ErrorCode	ESL_ERC_PARAMETER_INVALID input parameter is NULL
	ESL_ERC_WS_STATE_INVALID work space state invalid
	ESL_ERC_WS_TOO_SMALL work space too small
	ESL_ERC_OUTPUT_SIZE_TOO_SHORT outputSize too small
	ESL_ERC_NO_ERROR else
Functional Description	
This function finalizes the AES encryption by processing the remaining input bytes in the internal buffer and padding the input according to the chosen padding in the corresponding init function.	
Particularities and Limitations	
workSpace is initialized by esl_initEncryptAES256(..) esl_encryptAES256 called before	
Call context	
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces	

Table 4-40 esl_finalizeEncryptAES256

4.2.2.1.24 esl_initDecryptAES256

Prototype	
`eslt_ErrorCode` **`esl_initDecryptAES256`** `(VSECPRIM_P2VAR_PARA(eslt_WorkSpaceAES256) workSpace, VSECPRIM_P2CONST_PARA(eslt_Byte) key, const eslt_BlockMode blockMode, const eslt_PaddingMode paddingMode, VSECPRIM_P2CONST_PARA(eslt_Byte) initializationVector)`	
Parameter	
initializationVector [in]	in CBC mode: pointer to initialization vector of length ESL_SIZEOF_AES192_BLOCK in CTR mode: pointer to counter of length ESL_SIZEOF_AES192_BLOCK if initializationVector==NULL, a trivial zero block is used as iv or counter redundant in ECB mode
key [in]	pointer to user key of length ESL_SIZEOF_AES256_KEY
paddingMode [in]	padding mode switch
workSpace [in,out]	(in) algorithm context buffer, initialized by esl_initWorkSpaceHeader(..) function (out) initialized algorithm context structure
blockMode [in]	ECB, CBC, CTR, etc. block mode switch
Return code	
eslt_ErrorCode	ESL_ERC_PARAMETER_INVALID input parameter is NULL
	ESL_ERC_WS_TOO_SMALL work space too small
	ESL_ERC_MODE_INVALID block or padding mode is invalid
	ESL_ERC_NO_ERROR else
Functional Description	
This function initializes the AES stream decryption algorithm.	
Particularities and Limitations	
workSpace is initialized by esl_initWorkSpaceHeader(..)	
Call context	
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces	

Table 4-41 esl_initDecryptAES256

4.2.2.1.25 esl_decryptAES256

Prototype	
`eslt_ErrorCode` **`esl_decryptAES256`** `(VSECPRIM_P2VAR_PARA(eslt_WorkSpaceAES256) workSpace, const eslt_Length inputSize, VSECPRIM_P2CONST_PARA(eslt_Byte) input, VSECPRIM_P2VAR_PARA(eslt_Length) outputSize, VSECPRIM_P2VAR_PARA(eslt_Byte) output)`	
Parameter	
input [in]	pointer to data to be encrypted

outputSize [in,out]	(in) output buffer size in bytes, a safe length is ((inputSize+15)/16)*16 (see output) (out) total size of encrypted blocks ((inputSize+buffered_bytes)/16)*16
workSpace [in,out]	(in) algorithm context buffer, initialized by esl_initDecryptAES256(..) function (out) actualized algorithm context structure
output [in,out]	(in) pointer to encrypted data buffer (out) pointer to encrypted data blocks
inputSize [in]	length of input data in bytes
Return code	
eslt_ErrorCode	ESL_ERC_PARAMETER_INVALID input parameter is NULL
	ESL_ERC_WS_STATE_INVALID work space state invalid
	ESL_ERC_WS_TOO_SMALL work space too small
	ESL_ERC_OUTPUT_SIZE_TOO_SHORT outputSize too small
	ESL_ERC_NO_ERROR else
Functional Description	
This function decrypts input data of any length and can be called arbitrarily often after the algorithm initialization. Exact (inputSize+buffered_bytes)/16 blocks will be decrypted and written to the output buffer, (inputSize+buffered_bytes)16 remaining bytes will be buffered, where 0 <= buffered_bytes < 16.	
Particularities and Limitations	
workSpace is initialized by esl_initDecryptAES256(..) input and output can point to the same address, i.e., inplace decryption is possible for ECB, CBC and CTR mode, independent from the chosen padding method	
Call context	
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces	

Table 4-42 esl_decryptAES256

4.2.2.1.26 esl_finalizeDecryptAES256

Prototype	
`eslt_ErrorCode` **`esl_finalizeDecryptAES256`** `(VSECPRIM_P2VAR_PARA(eslt_WorkSpaceAES256) workSpace, VSECPRIM_P2VAR_PARA(eslt_Length) outputSize, VSECPRIM_P2VAR_PARA(eslt_Byte) output)`	
Parameter	
outputSize [in,out]	(in) output buffer size in bytes, a safe length is AES_BLOCK_LENGTH (out) total size of decrypted input 16 or 32 bytes
workSpace [out]	actualized algorithm context structure
output [in,out]	(in) pointer to decrypted data buffer (out) pointer to encrypted data blocks
Return code	
eslt_ErrorCode	ESL_ERC_PARAMETER_INVALID input parameter is NULL
	ESL_ERC_WS_STATE_INVALID work space state invalid

	ESL_ERC_WS_TOO_SMALL work space too small
	ESL_ERC_OUTPUT_SIZE_TOO_SHORT outputSize too small
	ESL_ERC_AES_PADDING_INVALID the padding is invalid
	ESL_ERC_INPUT_INVALID the total input length is not zero mod blocksize
	ESL_ERC_NO_ERROR else
Functional Description	
This function finalizes the AES decryption by processing the remaining input bytes in the internal buffer.	
Particularities and Limitations	
workSpace is initialized by esl_initDecryptAES256(..) esl_decryptAES256 called before	
Call context	
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces	

Table 4-43 esl_finalizeDecryptAES256

4.2.2.1.27 esl_initEncryptAES256Block

Prototype	

`eslt_ErrorCode` **`esl_initEncryptAES256Block`**
`(VSECPRIM_P2VAR_PARA(eslt_WorkSpaceAES256Block) workSpace,`
`VSECPRIM_P2CONST_PARA(eslt_Byte) key)`	

Parameter	
key [in]	pointer to user key of length ESL_SIZEOF_AES256_KEY
workSpace [in,out]	(in) algorithm context buffer, initialized by esl_initWorkSpaceHeader(..) function (out) initialized algorithm context structure
Return code	
eslt_ErrorCode	ESL_ERC_PARAMETER_INVALID input parameter is NULL
	ESL_ERC_WS_TOO_SMALL work space too small
	ESL_ERC_MODE_INVALID block or padding mode is invalid
	ESL_ERC_NO_ERROR else
Functional Description	
This function initializes the AES block encryption algorithm.	
Particularities and Limitations	
workSpace is initialized by esl_initWorkSpaceHeader(..)	
Call context	
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces	

Table 4-44 esl_initEncryptAES256Block

4.2.2.1.28 esl_encryptAES256Block

Prototype	
`eslt_ErrorCode` **`esl_encryptAES256Block`** `(VSECPRIM_P2VAR_PARA(eslt_WorkSpaceAES256Block) workSpace, VSECPRIM_P2CONST_PARA(eslt_Byte) inputBlock, VSECPRIM_P2VAR_PARA(eslt_Byte) outputBlock)`	
Parameter	
outputBlock [in,out]	(in) pointer to output buffer (at least 16 bytes) (out) pointer to encrypted data block
inputBlock [in]	pointer to the given input block (16 bytes)
workSpace [in,out]	(in) algorithm context buffer, initialized by esl_initEncryptAES256Block(..) function (out) actualized algorithm context structure
Return code	
eslt_ErrorCode	ESL_ERC_PARAMETER_INVALID input parameter is NULL
	ESL_ERC_WS_STATE_INVALID work space state invalid
	ESL_ERC_WS_TOO_SMALL work space too small
	ESL_ERC_NO_ERROR else
Functional Description	
This function encrypts an AES Block (= 16 bytes).	
Particularities and Limitations	
workSpace is initialized by esl_initEncryptAES256Block(..) inputBlock and outputBlock can point to the same address, i.e., inplace encryption is possible	
Call context	
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces	

Table 4-45 esl_encryptAES256Block

4.2.2.1.29 esl_initDecryptAES256Block

Prototype	
`eslt_ErrorCode` **`esl_initDecryptAES256Block`** `(VSECPRIM_P2VAR_PARA(eslt_WorkSpaceAES256Block) workSpace, VSECPRIM_P2CONST_PARA(eslt_Byte) key)`	
Parameter	
key [in]	pointer to user key of length ESL_SIZEOF_AES256_KEY
workSpace [in,out]	(in) algorithm context buffer, initialized by esl_initWorkSpaceHeader(..) function (out) initialized algorithm context structure

Return code	
eslt_ErrorCode	ESL_ERC_PARAMETER_INVALID input parameter is NULL
	ESL_ERC_WS_TOO_SMALL work space too small
	ESL_ERC_MODE_INVALID block or padding mode is invalid
	ESL_ERC_NO_ERROR else
Functional Description	
This function initializes the AES block decryption algorithm.	
Particularities and Limitations	
workSpace is initialized by esl_initWorkSpaceHeader(..)	
Call context	
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces	

Table 4-46 esl_initDecryptAES256Block

4.2.2.1.30 esl_decryptAES256Block

Prototype	
`eslt_ErrorCode` **`esl_decryptAES256Block`** `(VSECPRIM_P2VAR_PARA(eslt_WorkSpaceAES256Block) workSpace, VSECPRIM_P2CONST_PARA(eslt_Byte) inputBlock, VSECPRIM_P2VAR_PARA(eslt_Byte) outputBlock)`	
Parameter	
outputBlock [in,out]	(in) pointer to output buffer (at least 16 bytes) (out) pointer to encrypted data block
inputBlock [in]	pointer to the given input block (16 bytes)
workSpace [in,out]	(in) algorithm context buffer, initialized by esl_initDecryptAES256Block(..) function (out) actualized algorithm context structure
Return code	
eslt_ErrorCode	ESL_ERC_PARAMETER_INVALID input parameter is NULL
	ESL_ERC_WS_STATE_INVALID work space state invalid
	ESL_ERC_WS_TOO_SMALL work space too small
	ESL_ERC_NO_ERROR else
Functional Description	
This function decrypts an AES Block (= 16 bytes).	
Particularities and Limitations	
workSpace is initialized by esl_initDecryptAES256Block(..) inputBlock and outputBlock can point to the same address, i.e., inplace decryption is possible	
Call context	

> TASK > This function is Synchronous > This function is Reentrant, for different workspaces

Table 4-47 esl_decryptAES256Block

4.2.2.1.31 esl_generateKeyDES

Prototype	
`eslt_ErrorCode esl_generateKeyDES (VSECPRIM_P2VAR_PARA(eslt_Byte) key)`	
Parameter	
key [in,out]	(in) pointer to user key buffer of size ESL_SIZEOF_DES_KEY (out) pointer to the generated key
Return code	
eslt_ErrorCode	esl_getBytesRNG(..) return values are passed through
Functional Description	
This function generates a 64 bit DES key for usage in block or stream DES functions. It uses the RNG which is implemented by the call back function esl_getBytesRNG(..).	
Particularities and Limitations	
-	
Call context	
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces	

Table 4-48 esl_generateKeyDES

4.2.2.1.32 esl_initEncryptDES

Prototype	
`eslt_ErrorCode esl_initEncryptDES (VSECPRIM_P2VAR_PARA(eslt_WorkSpaceDES) workSpace, VSECPRIM_P2CONST_PARA(eslt_Byte) key, const eslt_BlockMode blockMode, const eslt_PaddingMode paddingMode, VSECPRIM_P2CONST_PARA(eslt_Byte) initializationVector)`	
Parameter	
initializationVector [in]	pointer to initialization vector of length ESL_SIZEOF_DES_BLOCK (redundant in ECB mode, if iV==NULL, a trivial zero block is used as iV)
key [in]	pointer to user key of length ESL_SIZEOF_DES_KEY
paddingMode [in]	padding mode switch
workSpace [in,out]	(in) algorithm context buffer, initialized by esl_initWorkSpaceHeader(..) function (out) initialized algorithm context structure
blockMode [in]	ECB, CBC, etc. block mode switch

Return code	
eslt_ErrorCode	ESL_ERC_PARAMETER_INVALID input parameter is NULL
	ESL_ERC_WS_TOO_SMALL work space too small
	ESL_ERC_MODE_INVALID block or padding mode is invalid
	ESL_ERC_NO_ERROR else
Functional Description	
This function initializes the DES stream encryption algorithm.	
Particularities and Limitations	
workSpace is initialized by esl_initWorkSpaceHeader(..)	
Call context	
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces	

Table 4-49 esl_initEncryptDES

4.2.2.1.33 esl_encryptDES

Prototype	
`eslt_ErrorCode esl_encryptDES (VSECPRIM_P2VAR_PARA(eslt_WorkSpaceDES) workSpace, const eslt_Length inputSize, VSECPRIM_P2CONST_PARA(eslt_Byte) input, VSECPRIM_P2VAR_PARA(eslt_Length) outputSize, VSECPRIM_P2VAR_PARA(eslt_Byte) output)`	
Parameter	
input [in]	pointer to data to be encrypted
outputSize [in,out]	(in) output buffer size in bytes, a safe length is ((inputSize+7)/8)*8 (see output) (out) total size of encrypted blocks ((inputSize+buffered_bytes)/8)*8
workSpace [in,out]	(in) algorithm context buffer, initialized by esl_initEncryptDES(..) function (out) actualized algorithm context structure
output [in,out]	(in) pointer to encrypted data buffer (out) pointer to encrypted data blocks
inputSize [in]	length of input data in bytes
Return code	
eslt_ErrorCode	ESL_ERC_PARAMETER_INVALID input parameter is NULL
	ESL_ERC_WS_STATE_INVALID work space state invalid
	ESL_ERC_WS_TOO_SMALL work space too small
	ESL_ERC_OUTPUT_SIZE_TOO_SHORT outputSize too small
	ESL_ERC_NO_ERROR else
Functional Description	
This function encrypts input data of any length and can be called arbitrarily often after the algorithm initialization. Exact (inputSize+buffered_bytes)/8 blocks will be encrypted and written to the output buffer, (inputSize+buffered_bytes)8 remaining bytes will be buffered, where 0 <= buffered_bytes < 8.	

Particularities and Limitations
workSpace is initialized by esl_initEncryptDES(..)
Call context
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces

Table 4-50 esl_encryptDES

4.2.2.1.34 esl_finalizeEncryptDES

Prototype	
`eslt_ErrorCode esl_finalizeEncryptDES (VSECPRIM_P2VAR_PARA(eslt_WorkSpaceDES) workSpace, VSECPRIM_P2VAR_PARA(eslt_Length) outputSize, VSECPRIM_P2VAR_PARA(eslt_Byte) output)`	
Parameter	
outputSize [in,out]	(in) output buffer size in bytes, a safe length is 2*DES_BLOCK_LENGTH (out) total size of encrypted input 8 or 16 bytes
workSpace [out]	actualized algorithm context structure
output [in,out]	(in) pointer to encrypted data buffer (out) pointer to encrypted data blocks
Return code	
eslt_ErrorCode	ESL_ERC_PARAMETER_INVALID input parameter is NULL
	ESL_ERC_WS_STATE_INVALID work space state invalid
	ESL_ERC_WS_TOO_SMALL work space too small
	ESL_ERC_OUTPUT_SIZE_TOO_SHORT outputSize too small
	ESL_ERC_NO_ERROR else
Functional Description	
This function finalizes the DES encryption by processing the remaining input bytes in the internal buffer and padding the input according to the chosen padding in the corresponding init function.	
Particularities and Limitations	
workSpace is initialized by esl_initEncryptDES(..) esl_encryptDES called before	
Call context	
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces	

Table 4-51 esl_finalizeEncryptDES

4.2.2.1.35 esl_initDecryptDES

Prototype	
`eslt_ErrorCode esl_initDecryptDES (VSECPRIM_P2VAR_PARA(eslt_WorkSpaceDES) workSpace, VSECPRIM_P2CONST_PARA(eslt_Byte) key, const eslt_BlockMode blockMode, const eslt_PaddingMode paddingMode, VSECPRIM_P2CONST_PARA(eslt_Byte) initializationVector)`	
Parameter	
initializationVector [in]	pointer to initialization vector of length ESL_SIZEOF_DES_BLOCK (redundant in ECB mode, if iV==NULL, a trivial zero block is used as iV)
key [in]	pointer to user key of length ESL_SIZEOF_DES_KEY
paddingMode [in]	padding mode switch
workSpace [in,out]	(in) algorithm context buffer, initialized by esl_initWorkSpaceHeader(..) function (out) initialized algorithm context structure
blockMode [in]	ECB, CBC, etc. block mode switch
Return code	
eslt_ErrorCode	ESL_ERC_PARAMETER_INVALID input parameter is NULL
	ESL_ERC_WS_TOO_SMALL work space too small
	ESL_ERC_MODE_INVALID block or padding mode is invalid
	ESL_ERC_NO_ERROR else
Functional Description	
This function initializes the DES stream decryption algorithm.	
Particularities and Limitations	
workSpace is initialized by esl_initWorkSpaceHeader(..)	
Call context	
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces	

Table 4-52 esl_initDecryptDES

4.2.2.1.36 esl_finalizeDecryptDES

Prototype	
`eslt_ErrorCode esl_finalizeDecryptDES (VSECPRIM_P2VAR_PARA(eslt_WorkSpaceDES) workSpace, VSECPRIM_P2VAR_PARA(eslt_Length) outputSize, VSECPRIM_P2VAR_PARA(eslt_Byte) output)`	
Parameter	
outputSize [in,out]	(in) output buffer size in bytes, a safe length is DES_BLOCK_LENGTH (out) total size of decrypted input 8 or 16 bytes
workSpace [out]	actualized algorithm context structure
output [in,out]	(in) pointer to decrypted data buffer (out) pointer to encrypted data blocks

Return code	
eslt_ErrorCode	ESL_ERC_PARAMETER_INVALID input parameter is NULL
	ESL_ERC_WS_STATE_INVALID work space state invalid
	ESL_ERC_WS_TOO_SMALL work space too small
	ESL_ERC_OUTPUT_SIZE_TOO_SHORT outputSize too small
	ESL_ERC_DES_PADDING_INVALID the padding is invalid
	ESL_ERC_INPUT_INVALID the total input length is not zero mod blocksize
	ESL_ERC_NO_ERROR else
Functional Description	
This function finalizes the DES decryption by processing the remaining input bytes in the internal buffer.	
Particularities and Limitations	
workSpace is initialized by esl_initDecryptDES(..) esl_decryptDES called before	
Call context	
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces	

Table 4-53 esl_finalizeDecryptDES

4.2.2.1.37 esl_decryptDES

Prototype
`eslt_ErrorCode esl_decryptDES (VSECPRIM_P2VAR_PARA(eslt_WorkSpaceDES) workSpace, const eslt_Length inputSize, VSECPRIM_P2CONST_PARA(eslt_Byte) input, VSECPRIM_P2VAR_PARA(eslt_Length) outputSize, VSECPRIM_P2VAR_PARA(eslt_Byte) output)`

Parameter	
input [in]	pointer to data to be encrypted
outputSize [in,out]	(in) output buffer size in bytes, a safe length is ((inputSize+7)/8)*8 (see output) (out) total size of encrypted blocks ((inputSize+buffered_bytes)/8)*8
workSpace [in,out]	(in) algorithm context buffer, initialized by esl_initDecryptDES(..) function (out) actualized algorithm context structure
output [in,out]	(in) pointer to encrypted data buffer (out) pointer to encrypted data blocks
inputSize [in]	length of input data in bytes
Return code	
eslt_ErrorCode	ESL_ERC_PARAMETER_INVALID input parameter is NULL
	ESL_ERC_WS_STATE_INVALID work space state invalid
	ESL_ERC_WS_TOO_SMALL work space too small
	ESL_ERC_OUTPUT_SIZE_TOO_SHORT outputSize too small
	ESL_ERC_NO_ERROR else

Functional Description
This function decrypts input data of any length and can be called arbitrarily often after the algorithm initialization. Exact (inputSize+buffered_bytes)/8 blocks will be decrypted and written to the output buffer, (inputSize+buffered_bytes)8 remaining bytes will be buffered, where 0 <= buffered_bytes < 8.
Particularities and Limitations
workSpace is initialized by esl_initDecryptDES(..)
Call context
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces

Table 4-54 esl_decryptDES

4.2.2.1.38 esl_initEncryptDESBlock

Prototype	

`eslt_ErrorCode` **`esl_initEncryptDESBlock`**
`(VSECPRIM_P2VAR_PARA(eslt_WorkSpaceDESBlock) workSpace,`
`VSECPRIM_P2CONST_PARA(eslt_Byte) key)`	

Parameter	
key [in]	pointer to user key of length ESL_SIZEOF_DES_KEY
workSpace [in,out]	(in) algorithm context buffer, initialized by esl_initWorkSpaceHeader(..) function (out) initialized algorithm context structure
Return code	
eslt_ErrorCode	ESL_ERC_PARAMETER_INVALID input parameter is NULL
	ESL_ERC_WS_TOO_SMALL work space too small
	ESL_ERC_MODE_INVALID block or padding mode is invalid
	ESL_ERC_NO_ERROR else
Functional Description	
This function initializes the DES block encryption algorithm.	
Particularities and Limitations	
workSpace is initialized by esl_initWorkSpaceHeader(..)	
Call context	
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces	

Table 4-55 esl_initEncryptDESBlock

4.2.2.1.39 esl_encryptDESBlock

Prototype	
`eslt_ErrorCode` **`esl_encryptDESBlock`** `(VSECPRIM_P2VAR_PARA(eslt_WorkSpaceDESBlock) workSpace, VSECPRIM_P2CONST_PARA(eslt_Byte) inputBlock, VSECPRIM_P2VAR_PARA(eslt_Byte) outputBlock)`	
Parameter	
outputBlock [in,out]	(in) pointer to output buffer (at least 8 bytes) (out) pointer to encrypted data block
inputBlock [in]	pointer to the given input block (8 bytes)
workSpace [in,out]	(in) algorithm context buffer, initialized by esl_initEncryptDESBlock(..) function (out) actualized algorithm context structure
Return code	
eslt_ErrorCode	ESL_ERC_PARAMETER_INVALID input parameter is NULL
	ESL_ERC_WS_STATE_INVALID work space state invalid
	ESL_ERC_WS_TOO_SMALL work space too small
	ESL_ERC_NO_ERROR else
Functional Description	
This function encrypts an DES Block (= 8 bytes).	
Particularities and Limitations	
workSpace is initialized by esl_initEncryptDESBlock(..)	
Call context	
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces	

Table 4-56 esl_encryptDESBlock

4.2.2.1.40 esl_initDecryptDESBlock

Prototype	
`eslt_ErrorCode` **`esl_initDecryptDESBlock`** `(VSECPRIM_P2VAR_PARA(eslt_WorkSpaceDESBlock) workSpace, VSECPRIM_P2CONST_PARA(eslt_Byte) key)`	
Parameter	
key [in]	pointer to user key of length ESL_SIZEOF_DES_KEY
workSpace [in,out]	(in) algorithm context buffer, initialized by esl_initWorkSpaceHeader(..) function (out) initialized algorithm context structure
Return code	
eslt_ErrorCode	ESL_ERC_PARAMETER_INVALID input parameter is NULL
	ESL_ERC_WS_TOO_SMALL work space too small

	ESL_ERC_MODE_INVALID block or padding mode is invalid
	ESL_ERC_NO_ERROR else
Functional Description	
This function initializes the DES block decryption algorithm.	
Particularities and Limitations	
workSpace is initialized by esl_initWorkSpaceHeader(..)	
Call context	
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces	

Table 4-57 esl_initDecryptDESBlock

4.2.2.1.41 esl_decryptDESBlock

Prototype	
`eslt_ErrorCode esl_decryptDESBlock (VSECPRIM_P2VAR_PARA(eslt_WorkSpaceDESBlock) workSpace, VSECPRIM_P2CONST_PARA(eslt_Byte) inputBlock, VSECPRIM_P2VAR_PARA(eslt_Byte) outputBlock)`	
Parameter	
outputBlock [in,out]	(in) pointer to output buffer (at least 8 bytes) (out) pointer to encrypted data block
inputBlock [in]	pointer to the given input block (8 bytes)
workSpace [in,out]	(in) algorithm context buffer, initialized by esl_initDecryptDESBlock(..) function (out) actualized algorithm context structure
Return code	
eslt_ErrorCode	ESL_ERC_PARAMETER_INVALID input parameter is NULL
	ESL_ERC_WS_STATE_INVALID work space state invalid
	ESL_ERC_WS_TOO_SMALL work space too small
	ESL_ERC_NO_ERROR else
Functional Description	
This function decrypts an DES Block (= 8 bytes).	
Particularities and Limitations	
workSpace is initialized by esl_initDecryptDESBlock(..)	
Call context	
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces	

Table 4-58 esl_decryptDESBlock

4.2.2.1.42 esl_generateKey3TDES

Prototype	
`eslt_ErrorCode esl_generateKey3TDES (VSECPRIM_P2VAR_PARA(eslt_Byte) key)`	
Parameter	
key [in,out]	(in) pointer to user key buffer of size ESL_SIZEOF_3TDES_KEY (out) pointer to the generated key
Return code	
eslt_ErrorCode	esl_getBytesRNG(..) return values are passed through
Functional Description	
This function generates a 192 bit TDES key for usage in block or stream TDES functions. It uses the RNG which is implemented by the call back function esl_getBytesRNG(..).	
Particularities and Limitations	
-	
Call context	
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces	

Table 4-59 esl_generateKey3TDES

4.2.2.1.43 esl_generateKey2TDES

Prototype	
`eslt_ErrorCode esl_generateKey2TDES (VSECPRIM_P2VAR_PARA(eslt_Byte) key)`	
Parameter	
key [in,out]	(in) pointer to user key buffer of size ESL_SIZEOF_2TDES_KEY (out) pointer to the generated key
Return code	
eslt_ErrorCode	esl_getBytesRNG(..) return values are passed through
Functional Description	
This function generates a 128 bit TDES key for usage in block or stream TDES functions. It uses the RNG which is implemented by the call back function esl_getBytesRNG(..).	
Particularities and Limitations	
-	
Call context	
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces	

Table 4-60 esl_generateKey2TDES

4.2.2.1.44 esl_initEncryptTDES

Prototype	
`eslt_ErrorCode esl_initEncryptTDES (VSECPRIM_P2VAR_PARA(eslt_WorkSpaceTDES) workSpace, VSECPRIM_P2CONST_PARA(eslt_Byte) key, const eslt_Byte key_len, const eslt_BlockMode blockMode, const eslt_PaddingMode paddingMode, VSECPRIM_P2CONST_PARA(eslt_Byte) initializationVector)`	
Parameter	
key [in]	pointer to user key
initializationVector [in]	pointer to initialization vector of length ESL_SIZEOF_DES_BLOCK (redundant in ECB mode, if iV==NULL, a trivial zero block is used as iV)
workSpace [in,out]	(in) algorithm context buffer, initialized by esl_initWorkSpaceHeader(..) function (out) initialized algorithm context structure
key_len [in]	length of user key
paddingMode [in]	padding mode switch
blockMode [in]	ECB, CBC, etc. block mode switch
Return code	
eslt_ErrorCode	ESL_ERC_PARAMETER_INVALID input parameter is NULL
	ESL_ERC_TDES_KEY_LENGTH_INVALID key length != 16 or 24
	ESL_ERC_WS_TOO_SMALL work space too small
	ESL_ERC_MODE_INVALID block or padding mode is invalid
	ESL_ERC_NO_ERROR else
Functional Description	
This function initializes the TDES stream encryption algorithm.	
Particularities and Limitations	
workSpace is initialized by esl_initWorkSpaceHeader(..)	
Call context	
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces	

Table 4-61 esl_initEncryptTDES

4.2.2.1.45 esl_encryptTDES

Prototype
`eslt_ErrorCode esl_encryptTDES (VSECPRIM_P2VAR_PARA(eslt_WorkSpaceTDES) workSpace, const eslt_Length inputSize, VSECPRIM_P2CONST_PARA(eslt_Byte) input, VSECPRIM_P2VAR_PARA(eslt_Length) outputSize, VSECPRIM_P2VAR_PARA(eslt_Byte) output)`

Parameter	
input [in]	pointer to data to be encrypted
outputSize [in,out]	(in) output buffer size in bytes, a safe length is ((inputSize+7)/8)*8 (see output) (out) total size of encrypted blocks ((inputSize+buffered_bytes)/8)*8
workSpace [in,out]	(in) algorithm context buffer, initialized by esl_initEncryptTDES(..) function (out) actualized algorithm context structure
output [in,out]	(in) pointer to encrypted data buffer (out) pointer to encrypted data blocks
inputSize [in]	length of input data in bytes
Return code	
eslt_ErrorCode	ESL_ERC_PARAMETER_INVALID input parameter is NULL
	ESL_ERC_WS_STATE_INVALID work space state invalid
	ESL_ERC_WS_TOO_SMALL work space too small
	ESL_ERC_OUTPUT_SIZE_TOO_SHORT outputSize too small
	ESL_ERC_NO_ERROR else
Functional Description	
This function encrypts input data of any length and can be called arbitrarily often after the algorithm initialization. Exact (inputSize+buffered_bytes)/8 blocks will be encrypted and written to the output buffer, (inputSize+buffered_bytes)8 remaining bytes will be buffered, where 0 <= buffered_bytes < 8.	
Particularities and Limitations	
workSpace is initialized by esl_initEncryptTDES(..)	
Call context	
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces	

Table 4-62 esl_encryptTDES

4.2.2.1.46 esl_finalizeEncryptTDES

Prototype	
`eslt_ErrorCode esl_finalizeEncryptTDES (VSECPRIM_P2VAR_PARA(eslt_WorkSpaceTDES) workSpace, VSECPRIM_P2VAR_PARA(eslt_Length) outputSize, VSECPRIM_P2VAR_PARA(eslt_Byte) output)`	
Parameter	
outputSize [in,out]	(in) output buffer size in bytes, a safe length is 2*DES_BLOCK_LENGTH (out) total size of encrypted input 8 or 16 bytes
workSpace [out]	actualized algorithm context structure
output [in,out]	(in) pointer to encrypted data buffer (out) pointer to encrypted data blocks
Return code	
eslt_ErrorCode	ESL_ERC_PARAMETER_INVALID input parameter is NULL

	ESL_ERC_WS_STATE_INVALID work space state invalid
	ESL_ERC_WS_TOO_SMALL work space too small
	ESL_ERC_OUTPUT_SIZE_TOO_SHORT outputSize too small
	ESL_ERC_NO_ERROR else
Functional Description	
This function finalizes the TDES encryption by processing the remaining input bytes in the internal buffer and padding the input according to the chosen padding in the corresponding init function.	
Particularities and Limitations	
workSpace is initialized by esl_initEncryptTDES(..) esl_encryptTDES called before	
Call context	
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces	

Table 4-63 esl_finalizeEncryptTDES

4.2.2.1.47 esl_initDecryptTDES

Prototype	
`eslt_ErrorCode esl_initDecryptTDES (VSECPRIM_P2VAR_PARA(eslt_WorkSpaceTDES) workSpace, VSECPRIM_P2CONST_PARA(eslt_Byte) key, const eslt_Byte key_len, const eslt_BlockMode blockMode, const eslt_PaddingMode paddingMode, VSECPRIM_P2CONST_PARA(eslt_Byte) initializationVector)`	
Parameter	
initializationVector [in]	pointer to initialization vector of length ESL_SIZEOF_DES_BLOCK (redundant in ECB mode, if iV==NULL, a trivial zero block is used as iV)
key [in]	pointer to user key of length ESL_SIZEOF_3TDES_KEY or ESL_SIZEOF_2TDES_KEY
key_len [in]	length of the key
paddingMode [in]	padding mode switch
workSpace [in,out]	(in) algorithm context buffer, initialized by esl_initWorkSpaceHeader(..) function (out) initialized algorithm context structure
blockMode [in]	ECB, CBC, etc. block mode switch
Return code	
eslt_ErrorCode	ESL_ERC_PARAMETER_INVALID input parameter is NULL
	ESL_ERC_WS_TOO_SMALL work space too small
	ESL_ERC_MODE_INVALID block or padding mode is invalid
	ESL_ERC_TDES_KEY_LENGTH_INVALID key length != 16 or 24
	ESL_ERC_NO_ERROR else
Functional Description	
This function initializes the TDES stream decryption algorithm.	

Particularities and Limitations
workSpace is initialized by esl_initWorkSpaceHeader(..)
Call context
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces

Table 4-64 esl_initDecryptTDES

4.2.2.1.48 esl_decryptTDES

Prototype	
`eslt_ErrorCode esl_decryptTDES (VSECPRIM_P2VAR_PARA(eslt_WorkSpaceTDES) workSpace, const eslt_Length inputSize, VSECPRIM_P2CONST_PARA(eslt_Byte) input, VSECPRIM_P2VAR_PARA(eslt_Length) outputSize, VSECPRIM_P2VAR_PARA(eslt_Byte) output)`	
Parameter	
input [in]	pointer to data to be encrypted
outputSize [in,out]	(in) output buffer size in bytes, a safe length is ((inputSize+7)/8)*8 (see output) (out) total size of encrypted blocks ((inputSize+buffered_bytes)/8)*8
workSpace [in,out]	(in) algorithm context buffer, initialized by esl_initDecryptTDES(..) function (out) actualized algorithm context structure
output [in,out]	(in) pointer to encrypted data buffer (out) pointer to encrypted data blocks
inputSize [in]	length of input data in bytes
Return code	
eslt_ErrorCode	ESL_ERC_PARAMETER_INVALID input parameter is NULL
	ESL_ERC_WS_STATE_INVALID work space state invalid
	ESL_ERC_WS_TOO_SMALL work space too small
	ESL_ERC_OUTPUT_SIZE_TOO_SHORT outputSize too small
	ESL_ERC_NO_ERROR else
Functional Description	
This function decrypts input data of any length and can be called arbitrarily often after the algorithm initialization. Exact (inputSize+buffered_bytes)/8 blocks will be decrypted and written to the output buffer, (inputSize+buffered_bytes)8 remaining bytes will be buffered, where 0 <= buffered_bytes < 8.	
Particularities and Limitations	
workSpace is initialized by esl_initDecryptTDES(..)	
Call context	
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces	

Table 4-65 esl_decryptTDES

4.2.2.1.49 esl_finalizeDecryptTDES

Prototype	
`eslt_ErrorCode` **`esl_finalizeDecryptTDES`** `(VSECPRIM_P2VAR_PARA(eslt_WorkSpaceTDES) workSpace, VSECPRIM_P2VAR_PARA(eslt_Length) outputSize, VSECPRIM_P2VAR_PARA(eslt_Byte) output)`	
Parameter	
outputSize [in,out]	(in) output buffer size in bytes, a safe length is DES_BLOCK_LENGTH (out) total size of decrypted input 8 or 16 bytes
workSpace [out]	actualized algorithm context structure
output [in,out]	(in) pointer to decrypted data buffer (out) pointer to encrypted data blocks
Return code	
eslt_ErrorCode	ESL_ERC_PARAMETER_INVALID input parameter is NULL
	ESL_ERC_WS_STATE_INVALID work space state invalid
	ESL_ERC_WS_TOO_SMALL work space too small
	ESL_ERC_OUTPUT_SIZE_TOO_SHORT outputSize too small
	ESL_ERC_DES_PADDING_INVALID the padding is invalid
	ESL_ERC_INPUT_INVALID the total input length is not zero mod blocksize
	ESL_ERC_NO_ERROR else
Functional Description	
This function finalizes the TDES decryption by processing the remaining input bytes in the internal buffer.	
Particularities and Limitations	
workSpace is initialized by esl_initDecryptTDES(..) esl_decryptTDES called before	
Call context	
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces	

Table 4-66 esl_finalizeDecryptTDES

4.2.2.1.50 esl_initEncryptTDESBlock

Prototype	
`eslt_ErrorCode` **`esl_initEncryptTDESBlock`** `(VSECPRIM_P2VAR_PARA(eslt_WorkSpaceTDESBlock) workSpace, VSECPRIM_P2CONST_PARA(eslt_Byte) key, const eslt_Byte key_len)`	
Parameter	
key [in]	pointer to user key
workSpace [in,out]	(in) algorithm context buffer, initialized by esl_initWorkSpaceHeader(..) function (out) initialized algorithm context structure

key_len [in]	length of user key
Return code	
eslt_ErrorCode	ESL_ERC_PARAMETER_INVALID input parameter is NULL
	ESL_ERC_TDES_KEY_LENGTH_INVALID key length != 16 or 24
	ESL_ERC_WS_TOO_SMALL work space too small
	ESL_ERC_MODE_INVALID block or padding mode is invalid
	ESL_ERC_NO_ERROR else
Functional Description	
This function initializes the TDES block encryption algorithm.	
Particularities and Limitations	
workSpace is initialized by esl_initWorkSpaceHeader(..)	
Call context	
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces	

Table 4-67 esl_initEncryptTDESBlock

4.2.2.1.51 esl_encryptTDESBlock

Prototype	
`eslt_ErrorCode esl_encryptTDESBlock (VSECPRIM_P2VAR_PARA(eslt_WorkSpaceTDESBlock) workSpace, VSECPRIM_P2CONST_PARA(eslt_Byte) inputBlock, VSECPRIM_P2VAR_PARA(eslt_Byte) outputBlock)`	
Parameter	
outputBlock [in,out]	(in) pointer to output buffer (at least 8 bytes) (out) pointer to encrypted data block
inputBlock [in]	pointer to the given input block (8 bytes)
workSpace [in,out]	(in) algorithm context buffer, initialized by esl_initEncryptTDESBlock(..) function (out) actualized algorithm context structure
Return code	
eslt_ErrorCode	ESL_ERC_PARAMETER_INVALID input parameter is NULL
	ESL_ERC_WS_STATE_INVALID work space state invalid
	ESL_ERC_WS_TOO_SMALL work space too small
	ESL_ERC_NO_ERROR else
Functional Description	
This function encrypts an TDES Block (= 8 bytes).	
Particularities and Limitations	
workSpace is initialized by esl_initEncryptTDESBlock(..)	

Call context
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces

Table 4-68 esl_encryptTDESBlock

4.2.2.1.52 esl_initDecryptTDESBlock

Prototype	
`eslt_ErrorCode esl_initDecryptTDESBlock (VSECPRIM_P2VAR_PARA(eslt_WorkSpaceTDESBlock) workSpace, VSECPRIM_P2CONST_PARA(eslt_Byte) key, const eslt_Byte key_len)`	
Parameter	
key [in]	pointer to user key of length ESL_SIZEOF_3TDES_KEY or ESL_SIZEOF_2TDES_KEY
key_len [in]	key length
workSpace [in,out]	(in) algorithm context buffer, initialized by esl_initWorkSpaceHeader(..) function (out) initialized algorithm context structure
Return code	
eslt_ErrorCode	ESL_ERC_PARAMETER_INVALID input parameter is NULL
	ESL_ERC_WS_TOO_SMALL work space too small
	ESL_ERC_TDES_KEY_LENGTH_INVALID key length != 16 or 24
	ESL_ERC_NO_ERROR else
Functional Description	
This function initializes the TDES block decryption algorithm.	
Particularities and Limitations	
workSpace is initialized by esl_initWorkSpaceHeader(..)	
Call context	
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces	

Table 4-69 esl_initDecryptTDESBlock

4.2.2.1.53 esl_decryptTDESBlock

Prototype
`eslt_ErrorCode esl_decryptTDESBlock (VSECPRIM_P2VAR_PARA(eslt_WorkSpaceTDESBlock) workSpace, VSECPRIM_P2CONST_PARA(eslt_Byte) inputBlock, VSECPRIM_P2VAR_PARA(eslt_Byte) outputBlock)`

Parameter	
outputBlock [in,out]	(in) pointer to output buffer (at least 8 bytes) (out) pointer to encrypted data block
inputBlock [in]	pointer to the given input block (8 bytes)
workSpace [in,out]	(in) algorithm context buffer, initialized by esl_initDecryptTDESBlock(..) function (out) actualized algorithm context structure
Return code	
eslt_ErrorCode	ESL_ERC_PARAMETER_INVALID input parameter is NULL
	ESL_ERC_WS_STATE_INVALID work space state invalid
	ESL_ERC_WS_TOO_SMALL work space too small
	ESL_ERC_NO_ERROR else
Functional Description	
This function decrypts an TDES Block (= 8 bytes).	
Particularities and Limitations	
workSpace is initialized by esl_initDecryptTDESBlock(..)	
Call context	
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces	

Table 4-70 esl_decryptTDESBlock

4.2.2.1.54 esl_generateKeyRC2

Prototype	
`eslt_ErrorCode esl_generateKeyRC2 (VSECPRIM_P2VAR_PARA(eslt_Byte) key)`	
Parameter	
key [in,out]	(in) pointer to user key buffer of size ESL_SIZEOF_RC2_KEY (out) pointer to the generated key
Return code	
eslt_ErrorCode	esl_getBytesRNG(..) return values are passed through
Functional Description	
This function generates a ESL_SIZEOF_RC2_KEY RC2 key for usage in block or stream RC2 functions. It uses the RNG which is implemented by the call back function esl_getBytesRNG(..).	
Particularities and Limitations	
-	
Call context	
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces	

Table 4-71 esl_generateKeyRC2

4.2.2.1.55 esl_initEncryptRC2

Prototype	
`eslt_ErrorCode esl_initEncryptRC2 (VSECPRIM_P2VAR_PARA(eslt_WorkSpaceRC2) workSpace, VSECPRIM_P2CONST_PARA(eslt_Byte) key, const eslt_Length key_len, const eslt_Length eff_key_len, const eslt_BlockMode blockMode, const eslt_PaddingMode paddingMode, VSECPRIM_P2CONST_PARA(eslt_Byte) initiallizationVector)`	
Parameter	
initializationVector [in]	pointer to initialization vector of length ESL_SIZEOF_RC2_BLOCK (redundant in ECB mode, if iV==NULL, a trivial zero block is used as iV)
key [in]	pointer to user key of length ESL_SIZEOF_RC2_KEY
key_len [in]	length of key in byte
eff_key_len [in]	eff_key_len length of effective key in Byte
paddingMode [in]	padding mode switch
workSpace [in,out]	(in) algorithm context buffer, initialized by esl_initWorkSpaceHeader(..) function (out) initialized algorithm context structure
blockMode [in]	ECB, CBC, etc. block mode switch
Return code	
eslt_ErrorCode	ESL_ERC_PARAMETER_INVALID input parameter is NULL
	ESL_ERC_WS_TOO_SMALL work space too small
	ESL_ERC_MODE_INVALID block or padding mode is invalid
	ESL_ERC_NO_ERROR else
Functional Description	
This function initializes the RC2 stream encryption algorithm.	
Particularities and Limitations	
workSpace is initialized by esl_initWorkSpaceHeader(..)	
Call context	
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces	

Table 4-72 esl_initEncryptRC2

4.2.2.1.56 esl_encryptRC2

Prototype
`eslt_ErrorCode esl_encryptRC2 (VSECPRIM_P2VAR_PARA(eslt_WorkSpaceRC2) workSpace, const eslt_Length inputSize, VSECPRIM_P2CONST_PARA(eslt_Byte) input, VSECPRIM_P2VAR_PARA(eslt_Length) outputSize, VSECPRIM_P2VAR_PARA(eslt_Byte) output)`

Parameter	
input [in]	pointer to data to be encrypted
outputSize [in,out]	(in) output buffer size in bytes, a safe length is ((inputSize+7)/8)*8 (see output) (out) total size of encrypted blocks ((inputSize+buffered_bytes)/8)*8
workSpace [in,out]	(in) algorithm context buffer, initialized by esl_initRC2(..) function (out) actualized algorithm context structure
output [in,out]	(in) pointer to encrypted data buffer (out) pointer to encrypted data blocks
inputSize [in]	length of input data in bytes
Return code	
eslt_ErrorCode	ESL_ERC_PARAMETER_INVALID input parameter is NULL
	ESL_ERC_WS_STATE_INVALID work space state invalid
	ESL_ERC_WS_TOO_SMALL work space too small
	ESL_ERC_OUTPUT_SIZE_TOO_SHORT outputSize too small
	ESL_ERC_NO_ERROR else
Functional Description	
This function encrypts input data of any length and can be called arbitrarily often after the algorithm initialization. Exact (inputSize+buffered_bytes)/8 blocks will be encrypted and written to the output buffer, (inputSize+buffered_bytes)8 remaining bytes will be buffered, where 0 <= buffered_bytes < 8.	
Particularities and Limitations	
workSpace is initialized by esl_initRC2(..) input and output can point to the same address, i.e., in-place encryption is possible	
Call context	
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces	

Table 4-73 esl_encryptRC2

4.2.2.1.57 esl_finalizeEncryptRC2

Prototype	
`eslt_ErrorCode esl_finalizeEncryptRC2 (VSECPRIM_P2VAR_PARA(eslt_WorkSpaceRC2) workSpace, VSECPRIM_P2VAR_PARA(eslt_Length) outputSize, VSECPRIM_P2VAR_PARA(eslt_Byte) output)`	
Parameter	
outputSize [in,out]	(in) output buffer size in bytes, a safe length is 2*RC2_BLOCK_LENGTH (out) total size of encrypted input
workSpace [out]	actualized algorithm context structure
output [in,out]	(in) pointer to encrypted data buffer (out) pointer to encrypted data blocks
Return code	
eslt_ErrorCode	ESL_ERC_PARAMETER_INVALID input parameter is NULL

	ESL_ERC_WS_STATE_INVALID work space state invalid
	ESL_ERC_WS_TOO_SMALL work space too small
	ESL_ERC_OUTPUT_SIZE_TOO_SHORT outputSize too small
	ESL_ERC_NO_ERROR else
Functional Description	
This function finalizes the RC2 encryption by processing the remaining input bytes in the internal buffer and padding the input according to the chosen padding in the corresponding init function.	
Particularities and Limitations	
workSpace is initialized by esl_initRC2(..) esl_encryptRC2 called before	
Call context	
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces	

Table 4-74 esl_finalizeEncryptRC2

4.2.2.1.58 esl_initDecryptRC2

Prototype	
`eslt_ErrorCode esl_initDecryptRC2 (VSECPRIM_P2VAR_PARA(eslt_WorkSpaceRC2) workSpace, VSECPRIM_P2CONST_PARA(eslt_Byte) key, const eslt_Length key_len, const eslt_Length eff_key_len, const eslt_BlockMode blockMode, const eslt_PaddingMode paddingMode, VSECPRIM_P2CONST_PARA(eslt_Byte) initiallizationVector)`	
Parameter	
initializationVector [in]	pointer to initialization vector of length ESL_SIZEOF_RC2_BLOCK (redundant in ECB mode, if iV==NULL, a trivial zero block is used as iV)
key [in]	pointer to user key of length ESL_SIZEOF_RC2_KEY
key_len [in]	length of key in byte
eff_key_len [in]	eff_key_len length of effective key in Byte
paddingMode [in]	padding mode switch
workSpace [in,out]	(in) algorithm context buffer, initialized by esl_initWorkSpaceHeader(..) function (out) initialized algorithm context structure
blockMode [in]	ECB, CBC, etc. block mode switch
Return code	
eslt_ErrorCode	ESL_ERC_PARAMETER_INVALID input parameter is NULL
	ESL_ERC_WS_TOO_SMALL work space too small
	ESL_ERC_MODE_INVALID block or padding mode is invalid
	ESL_ERC_NO_ERROR else
Functional Description	
This function initializes the RC2 stream decryption algorithm.	

Particularities and Limitations
workSpace is initialized by esl_initWorkSpaceHeader(..)
Call context
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces

Table 4-75 esl_initDecryptRC2

4.2.2.1.59 esl_decryptRC2

Prototype
`eslt_ErrorCode esl_decryptRC2 (VSECPRIM_P2VAR_PARA(eslt_WorkSpaceRC2) workSpace, const eslt_Length inputSize, VSECPRIM_P2CONST_PARA(eslt_Byte) input, VSECPRIM_P2VAR_PARA(eslt_Length) outputSize, VSECPRIM_P2VAR_PARA(eslt_Byte) output)`

Parameter	
input [in]	pointer to data to be encrypted
outputSize [in,out]	(in) output buffer size in bytes, a safe length is ((inputSize+7)/8)*8 (see output) (out) total size of encrypted blocks ((inputSize+buffered_bytes)/8)*8
workSpace [in,out]	(in) algorithm context buffer, initialized by esl_initDecryptRC2(..) function (out) actualized algorithm context structure
output [in,out]	(in) pointer to encrypted data buffer (out) pointer to encrypted data blocks
inputSize [in]	length of input data in bytes

Return code	
eslt_ErrorCode	ESL_ERC_PARAMETER_INVALID input parameter is NULL
	ESL_ERC_WS_STATE_INVALID work space state invalid
	ESL_ERC_WS_TOO_SMALL work space too small
	ESL_ERC_OUTPUT_SIZE_TOO_SHORT outputSize too small
	ESL_ERC_NO_ERROR else

Functional Description
This function decrypts input data of any length and can be called arbitrarily often after the algorithm initialization. Exact (inputSize+buffered_bytes)/8 blocks will be decrypted and written to the output buffer, (inputSize+buffered_bytes)8 remaining bytes will be buffered, where 0 <= buffered_bytes < 8.
Particularities and Limitations
workSpace is initialized by esl_initDecryptRC2(..) input and output can point to the same address, i.e., in-place decryption is possible
Call context
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces

Table 4-76 esl_decryptRC2

4.2.2.1.60 esl_finalizeDecryptRC2

Prototype	
`eslt_ErrorCode esl_finalizeDecryptRC2 (VSECPRIM_P2VAR_PARA(eslt_WorkSpaceRC2) workSpace, VSECPRIM_P2VAR_PARA(eslt_Length) outputSize, VSECPRIM_P2VAR_PARA(eslt_Byte) output)`	
Parameter	
outputSize [in,out]	(in) output buffer size in bytes, a safe length is RC2_BLOCK_LENGTH (out) total size of decrypted input
workSpace [out]	actualized algorithm context structure
output [in,out]	(in) pointer to decrypted data buffer (out) pointer to encrypted data blocks
Return code	
eslt_ErrorCode	ESL_ERC_PARAMETER_INVALID input parameter is NULL
	ESL_ERC_WS_STATE_INVALID work space state invalid
	ESL_ERC_WS_TOO_SMALL work space too small
	ESL_ERC_OUTPUT_SIZE_TOO_SHORT outputSize too small
	ESL_ERC_RC2_PADDING_INVALID the padding is invalid
	ESL_ERC_INPUT_INVALID the total input length is not zero mod blocksize
	ESL_ERC_NO_ERROR else
Functional Description	
This function finalizes the RC2 decryption by processing the remaining input bytes in the internal buffer.	
Particularities and Limitations	
workSpace is initialized by esl_initDecryptRC2(..) esl_decryptRC2 called before	
Call context	
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces	

Table 4-77 esl_finalizeDecryptRC2

4.2.2.1.61 esl_initEncryptRC2Block

Prototype	
`eslt_ErrorCode esl_initEncryptRC2Block (VSECPRIM_P2VAR_PARA(eslt_WorkSpaceRC2Block) workSpace, VSECPRIM_P2CONST_PARA(eslt_Byte) key, const eslt_Length key_len, const eslt_Length eff_key_len)`	
Parameter	
key [in]	pointer to user key of length ESL_SIZEOF_RC2_KEY

key_len [in]	length of key in byte
eff_key_len [in]	eff_key_len length of effective key in Byte
workSpace [in,out]	(in) algorithm context buffer, initialized by esl_initWorkSpaceHeader(..) function (out) initialized algorithm context structure
Return code	
eslt_ErrorCode	ESL_ERC_PARAMETER_INVALID input parameter is NULL
	ESL_ERC_WS_TOO_SMALL work space too small
	ESL_ERC_NO_ERROR else
Functional Description	
This function initializes the RC2 block encryption algorithm.	
Particularities and Limitations	
workSpace is initialized by esl_initWorkSpaceHeader(..)	
Call context	
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces	

Table 4-78 esl_initEncryptRC2Block

4.2.2.1.62 esl_encryptRC2Block

Prototype
`eslt_ErrorCode esl_encryptRC2Block (VSECPRIM_P2VAR_PARA(eslt_WorkSpaceRC2Block) workSpace, VSECPRIM_P2CONST_PARA(eslt_Byte) inputBlock, VSECPRIM_P2VAR_PARA(eslt_Byte) outputBlock)`

Parameter	
outputBlock [in,out]	(in) pointer to output buffer (at least 8 bytes) (out) pointer to encrypted data block
inputBlock [in]	pointer to the given input block (8 bytes)
workSpace [in,out]	(in) algorithm context buffer, initialized by esl_initEncryptRC2Block(..) function (out) actualized algorithm context structure
Return code	
eslt_ErrorCode	ESL_ERC_PARAMETER_INVALID input parameter is NULL
	ESL_ERC_WS_STATE_INVALID work space state invalid
	ESL_ERC_WS_TOO_SMALL work space too small
	ESL_ERC_NO_ERROR else
Functional Description	
This function encrypts an RC2 Block (= 8 bytes).	

Particularities and Limitations
workSpace is initialized by esl_initEncryptRC2Block(..) inputBlock and outputBlock can point to the same address, i.e., in-place encryption is possible
Call context
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces

Table 4-79 esl_encryptRC2Block

4.2.2.1.63 esl_initDecryptRC2Block

Prototype	
`eslt_ErrorCode esl_initDecryptRC2Block (VSECPRIM_P2VAR_PARA(eslt_WorkSpaceRC2Block) workSpace, VSECPRIM_P2CONST_PARA(eslt_Byte) key, const eslt_Length key_len, const eslt_Length eff_key_len)`	
Parameter	
key [in]	pointer to user key of length ESL_SIZEOF_RC2_KEY
key_len [in]	length of key in byte
eff_key_len [in]	eff_key_len length of effective key in Byte
workSpace [in,out]	(in) algorithm context buffer, initialized by esl_initWorkSpaceHeader(..) function (out) initialized algorithm context structure
Return code	
eslt_ErrorCode	ESL_ERC_PARAMETER_INVALID input parameter is NULL
	ESL_ERC_WS_TOO_SMALL work space too small
	ESL_ERC_MODE_INVALID block or padding mode is invalid
	ESL_ERC_NO_ERROR else
Functional Description	
This function initializes the RC2 block decryption algorithm.	
Particularities and Limitations	
workSpace is initialized by esl_initWorkSpaceHeader(..)	
Call context	
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces	

Table 4-80 esl_initDecryptRC2Block

4.2.2.1.64 esl_decryptRC2Block

Prototype	
`eslt_ErrorCode` **`esl_decryptRC2Block`** `(VSECPRIM_P2VAR_PARA(eslt_WorkSpaceRC2Block) workSpace, VSECPRIM_P2CONST_PARA(eslt_Byte) inputBlock, VSECPRIM_P2VAR_PARA(eslt_Byte) outputBlock)`	
Parameter	
outputBlock [in,out]	(in) pointer to output buffer (at least 8 bytes) (out) pointer to encrypted data block
inputBlock [in]	pointer to the given input block (8 bytes)
workSpace [in,out]	(in) algorithm context buffer, initialized by esl_initDecryptRC2Block(..) function (out) actualized algorithm context structure
Return code	
eslt_ErrorCode	ESL_ERC_PARAMETER_INVALID input parameter is NULL
	ESL_ERC_WS_STATE_INVALID work space state invalid
	ESL_ERC_WS_TOO_SMALL work space too small
	ESL_ERC_NO_ERROR else
Functional Description	
This function decrypts an RC2 Block (= 8 bytes).	
Particularities and Limitations	
workSpace is initialized by esl_initDecryptRC2Block(..) inputBlock and outputBlock can point to the same address, i.e., in-place decryption is possible	
Call context	
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces	

Table 4-81 esl_decryptRC2Block

4.2.2.1.65 esl_initEncryptChaCha20

Prototype	
`eslt_ErrorCode` **`esl_initEncryptChaCha20`** `(VSECPRIM_P2VAR_PARA(eslt_WorkSpaceChaCha20) workSpace, VSECPRIM_P2CONST_PARA(eslt_Byte) key, VSECPRIM_P2CONST_PARA(eslt_Byte) nonce, const eslt_Size32 blockCount)`	
Parameter	
key [in]	pointer to key of length ESL_SIZEOF_ChaCha20_KEY
nonce [in]	pointer to nonce of length ESL_SIZEOF_ChaCha20_NONCE
workSpace [in,out]	(in) algorithm context buffer, initialized by esl_initWorkSpaceHeader () function (out) initialized algorithm context structure
blockCount [in]	initial block count

Return code	
eslt_ErrorCode	ESL_ERC_PARAMETER_INVALID input parameter is NULL
	ESL_ERC_WS_TOO_SMALL work space too small
	ESL_ERC_NO_ERROR else
Functional Description	
This function initializes the ChaCha20 stream encryption.	
Particularities and Limitations	
workSpace is initialized by esl_initWorkSpaceHeader ()	
Call context	
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces	

Table 4-82 esl_initEncryptChaCha20

4.2.2.1.66 esl_encryptChaCha20

Prototype
`eslt_ErrorCode esl_encryptChaCha20 (VSECPRIM_P2VAR_PARA(eslt_WorkSpaceChaCha20) workSpace, VSECPRIM_P2CONST_PARA(eslt_Byte) in, const eslt_Length inSize, VSECPRIM_P2VAR_PARA(eslt_Byte) out, VSECPRIM_P2VAR_PARA(eslt_Length) outSize)`

Parameter	
in [in]	pointer to input data
outSize [out]	length of encrypted data
out [in,out]	pointer to output buffer
inSize [in]	length of input data (bytes)
workSpace [in,out]	(in) algorithm context buffer, initialized by esl_initChaCha20 () function (out) updated algorithm context buffer
outSize [in,out]	pointer to size of output buffer (bytes)
Return code	
eslt_ErrorCode	ESL_ERC_PARAMETER_INVALID input parameter is NULL
	ESL_ERC_BUFFER_TOO_SMALL output buffer too small
	ESL_ERC_WS_STATE_INVALID work space state invalid
	ESL_ERC_NO_ERROR else
Functional Description	
This function encrypts input data of any length and can be called arbitrarily often after the algorithm initialization.	
Particularities and Limitations	
workSpace is initialized by initChaCha20 ()	

in and out can point to the same address, i.e., inplace encryption is possible
Call context
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces

Table 4-83 esl_encryptChaCha20

4.2.2.1.67 esl_finalizeEncryptChaCha20

Prototype	

`eslt_ErrorCode` **`esl_finalizeEncryptChaCha20`**
`(VSECPRIM_P2VAR_PARA(eslt_WorkSpaceChaCha20) workSpace,`
`VSECPRIM_P2CONST_PARA(eslt_Byte) in, const eslt_Length inSize,`
`VSECPRIM_P2VAR_PARA(eslt_Byte) out, VSECPRIM_P2VAR_PARA(eslt_Length) outSize)`	

Parameter	
in [in]	pointer to input data
outSize [out]	length of encrypted data
out [in,out]	pointer to output buffer
inSize [in]	length of input data (bytes)
workSpace [in,out]	(in) algorithm context buffer, initialized by esl_initChaCha20 () function (out) updated algorithm context buffer
outSize [in,out]	pointer to size of output buffer (bytes)
Return code	
eslt_ErrorCode	ESL_ERC_PARAMETER_INVALID input parameter is NULL
	ESL_ERC_BUFFER_TOO_SMALL output buffer too small
	ESL_ERC_WS_STATE_INVALID work space state invalid
	ESL_ERC_NO_ERROR else
Functional Description	
This function encrypts the input data of any length and resets and clears the workspace.	
Particularities and Limitations	
workSpace is initialized by initChaCha20 () in and out can point to the same address, i.e., inplace encryption is possible	
Call context	
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces	

Table 4-84 esl_finalizeEncryptChaCha20

4.2.2.1.68 esl_initDecryptChaCha20

Prototype	
`eslt_ErrorCode esl_initDecryptChaCha20 (VSECPRIM_P2VAR_PARA(eslt_WorkSpaceChaCha20) workSpace, VSECPRIM_P2CONST_PARA(eslt_Byte) key, VSECPRIM_P2CONST_PARA(eslt_Byte) nonce, const eslt_Size32 blockCount)`	
Parameter	
key [in]	pointer to key of length ESL_SIZEOF_ChaCha20_KEY
nonce [in]	pointer to nonce of length ESL_SIZEOF_ChaCha20_NONCE
workSpace [in,out]	(in) algorithm context buffer, initialized by esl_initWorkSpaceHeader () function (out) initialized algorithm context structure
blockCount [in]	initial block count
Return code	
eslt_ErrorCode	ESL_ERC_PARAMETER_INVALID input parameter is NULL
	ESL_ERC_WS_TOO_SMALL work space too small
	ESL_ERC_NO_ERROR else
Functional Description	
This function initializes the ChaCha20 stream Decryption.	
Particularities and Limitations	
workSpace is initialized by esl_initWorkSpaceHeader ()	
Call context	
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces	

Table 4-85 esl_initDecryptChaCha20

4.2.2.1.69 esl_decryptChaCha20

Prototype	
`eslt_ErrorCode esl_decryptChaCha20 (VSECPRIM_P2VAR_PARA(eslt_WorkSpaceChaCha20) workSpace, VSECPRIM_P2CONST_PARA(eslt_Byte) in, const eslt_Length inSize, VSECPRIM_P2VAR_PARA(eslt_Byte) out, VSECPRIM_P2VAR_PARA(eslt_Length) outSize)`	
Parameter	
in [in]	pointer to input data
outSize [out]	length of encrypted data
out [in,out]	pointer to output buffer
inSize [in]	length of input data (bytes)
workSpace [in,out]	(in) algorithm context buffer, initialized by esl_initChaCha20 () function (out) updated algorithm context buffer

outSize [in,out]	size of output buffer (bytes)
Return code	
eslt_ErrorCode	ESL_ERC_PARAMETER_INVALID input parameter is NULL
	ESL_ERC_BUFFER_TOO_SMALL output buffer too small
	ESL_ERC_WS_STATE_INVALID work space state invalid
	ESL_ERC_NO_ERROR else
Functional Description	
This function decrypts input data of any length and can be called arbitrarily often after the algorithm initialization.	
Particularities and Limitations	
workSpace is initialized by initChaCha20 () in and out can point to the same address, i.e., inplace decryption is possible	
Call context	
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces	

Table 4-86 esl_decryptChaCha20

4.2.2.1.70 esl_finalizeDecryptChaCha20

Prototype	

`eslt_ErrorCode` **`esl_finalizeDecryptChaCha20`**
`(VSECPRIM_P2VAR_PARA(eslt_WorkSpaceChaCha20) workSpace,`
`VSECPRIM_P2CONST_PARA(eslt_Byte) in, const eslt_Length inSize,`
`VSECPRIM_P2VAR_PARA(eslt_Byte) out, VSECPRIM_P2VAR_PARA(eslt_Length) outSize)`	

Parameter	
in [in]	pointer to input data
outSize [out]	length of encrypted data
out [in,out]	pointer to output buffer
inSize [in]	length of input data (bytes)
workSpace [in,out]	(in) algorithm context buffer, initialized by esl_initChaCha20 () function (out) updated algorithm context buffer
outSize [in,out]	size of output buffer (bytes)
Return code	
eslt_ErrorCode	ESL_ERC_PARAMETER_INVALID input parameter is NULL
	ESL_ERC_BUFFER_TOO_SMALL output buffer too small
	ESL_ERC_WS_STATE_INVALID work space state invalid
	ESL_ERC_NO_ERROR else

Functional Description
This function decrypts the input data of any length and resets and clears the workspace.
Particularities and Limitations
workSpace is initialized by initChaCha20 () in and out can point to the same address, i.e., inplace decryption is possible
Call context
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces

Table 4-87 esl_finalizeDecryptChaCha20

4.2.2.2 Hash Algorithms

4.2.2.2.1 esl_initSHA1

Prototype	
`eslt_ErrorCode` **`esl_initSHA1`** `(VSECPRIM_P2VAR_PARA(eslt_WorkSpaceSHA1) workSpace)`	
Parameter	
workSpace [in,out]	(in) algorithm context buffer, initialized by esl_initWorkSpaceHeader(..) function (out) initialized algorithm context structure
Return code	
eslt_ErrorCode	ESL_ERC_PARAMETER_INVALID input parameter is NULL
	ESL_ERC_WS_TOO_SMALL work space too small
	ESL_ERC_NO_ERROR else
Functional Description	
Initializes the SHA-1 calculation. This function initializes the SHA-1 hash calculation.	
Particularities and Limitations	
workSpace is initialized by esl_initWorkSpaceHeader(..)	
Call context	
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces	

Table 4-88 esl_initSHA1

4.2.2.2.2 esl_updateSHA1

Prototype	
`eslt_ErrorCode esl_updateSHA1 (VSECPRIM_P2VAR_PARA(eslt_WorkSpaceSHA1) workSpace, const eslt_Length inputSize, VSECPRIM_P2CONST_PARA(eslt_Byte) input)`	
Parameter	
workSpace [in,out]	(in) algorithm context buffer, initialized by esl_initSHA1(..) function (out) updated algorithm context structure
inputSize [in]	length of input data in bytes
input [in]	pointer to data to be handled
Return code	
eslt_ErrorCode	ESL_ERC_PARAMETER_INVALID input parameter is NULL
	ESL_ERC_WS_STATE_INVALID work space state invalid
	ESL_ERC_WS_TOO_SMALL work space too small
	ESL_ERC_SHA1_TOTAL_LENGTH_OVERFLOW total input length overflow
	ESL_ERC_NO_ERROR else
Functional Description	
This function updates the SHA-1 calculation for the given input message (or parts of it). This function can be called several times until all input to be hashed is handled.	
Particularities and Limitations	
workSpace is initialized by esl_initSHA1(..)	
Call context	
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces	

Table 4-89 esl_updateSHA1

4.2.2.2.3 esl_finalizeSHA1

Prototype	
`eslt_ErrorCode esl_finalizeSHA1 (VSECPRIM_P2VAR_PARA(eslt_WorkSpaceSHA1) workSpace, VSECPRIM_P2VAR_PARA(eslt_Byte) messageDigest)`	
Parameter	
workSpace [in,out]	(in) algorithm context buffer, initialized by esl_initSHA1(..) function (out) updated algorithm context structure
messageDigest [in,out]	(in) pointer to buffer for the hash value (20 bytes) (out) pointer to computed hash value
Return code	
eslt_ErrorCode	ESL_ERC_PARAMETER_INVALID input parameter is NULL
	ESL_ERC_WS_STATE_INVALID work space state invalid

	ESL_ERC_WS_TOO_SMALL work space too small
	ESL_ERC_NO_ERROR else
Functional Description	
Finalizes the SHA-1 calculation. This function finalizes the SHA-1 calculation by doing the appropriate padding of the input value and returning the hash value.	
Particularities and Limitations	
workSpace is initialized by esl_initSHA1(..), esl_updateSHA1 called before	
Call context	
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces	

Table 4-90 esl_finalizeSHA1

4.2.2.2.4 esl_initSHA224

Prototype	
`eslt_ErrorCode` **`esl_initSHA224`** `(VSECPRIM_P2VAR_PARA(eslt_WorkSpaceSHA224) workSpace)`	
Parameter	
workSpace [in,out]	(in) algorithm context buffer, initialized by esl_initWorkSpaceHeader(..) function (out) initialized algorithm context structure
Return code	
eslt_ErrorCode	ESL_ERC_PARAMETER_INVALID input parameter is NULL
	ESL_ERC_WS_TOO_SMALL work space too small
	ESL_ERC_NO_ERROR else
Functional Description	
Initializes the SHA-224 calculation. This function initializes the SHA-224 hash calculation.	
Particularities and Limitations	
workSpace is initialized by esl_initWorkSpaceHeader(..)	
Call context	
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces	

Table 4-91 esl_initSHA224

4.2.2.2.5 esl_updateSHA224

Prototype	
`eslt_ErrorCode` **`esl_updateSHA224`** `(VSECPRIM_P2VAR_PARA(eslt_WorkSpaceSHA224) workSpace, const eslt_Length inputSize, VSECPRIM_P2CONST_PARA(eslt_Byte) input)`	
Parameter	
workSpace [in,out]	(in) algorithm context buffer, initialized by esl_initSHA224(..) function (out) updated algorithm context structure
inputSize [in]	length of input data in bytes
input [in]	pointer to data to be handled
Return code	
eslt_ErrorCode	ESL_ERC_PARAMETER_INVALID input parameter is NULL
	ESL_ERC_WS_STATE_INVALID work space state invalid
	ESL_ERC_WS_TOO_SMALL work space too small
	ESL_ERC_SHA1_TOTAL_LENGTH_OVERFLOW total input length overflow
	ESL_ERC_NO_ERROR else
Functional Description	
This function updates the SHA-224 calculation for the given input message (or parts of it). This function can be called several times until all input to be hashed is handled.	
Particularities and Limitations	
workSpace is initialized by esl_initSHA224(..)	
Call context	
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces	

Table 4-92 esl_updateSHA224

4.2.2.2.6 esl_finalizeSHA224

Prototype	
`eslt_ErrorCode` **`esl_finalizeSHA224`** `(VSECPRIM_P2VAR_PARA(eslt_WorkSpaceSHA224) workSpace, VSECPRIM_P2VAR_PARA(eslt_Byte) messageDigest)`	
Parameter	
workSpace [in,out]	(in) algorithm context buffer, initialized by esl_initSHA224(..) function (out) updated algorithm context structure
messageDigest [in,out]	(in) pointer to buffer for the hash value (28 bytes) (out) pointer to computed hash value
Return code	
eslt_ErrorCode	ESL_ERC_PARAMETER_INVALID input parameter is NULL
	ESL_ERC_WS_STATE_INVALID work space state invalid

	ESL_ERC_WS_TOO_SMALL work space too small
	ESL_ERC_NO_ERROR else
Functional Description	
Finalizes the SHA-224 calculation. This function finalizes the SHA-224 calculation by doing the appropriate padding of the input value and returning the hash value.	
Particularities and Limitations	
workSpace is initialized by esl_initSHA224(..), esl_updateSHA224 called before	
Call context	
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces	

Table 4-93 esl_finalizeSHA224

4.2.2.2.7 esl_initSHA256

Prototype	
`eslt_ErrorCode` **`esl_initSHA256`** `(VSECPRIM_P2VAR_PARA(eslt_WorkSpaceSHA256) workSpace)`	
Parameter	
workSpace [in,out]	(in) algorithm context buffer, initialized by esl_initWorkSpaceHeader(..) function (out) initialized algorithm context structure
Return code	
eslt_ErrorCode	ESL_ERC_PARAMETER_INVALID input parameter is NULL
	ESL_ERC_WS_TOO_SMALL work space too small
	ESL_ERC_NO_ERROR else
Functional Description	
Initializes the SHA-256 calculation. This function initializes the SHA-256 hash calculation.	
Particularities and Limitations	
workSpace is initialized by esl_initWorkSpaceHeader(..)	
Call context	
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces	

Table 4-94 esl_initSHA256

4.2.2.2.8 esl_updateSHA256

Prototype	
`eslt_ErrorCode esl_updateSHA256 (VSECPRIM_P2VAR_PARA(eslt_WorkSpaceSHA256) workSpace, const eslt_Length inputSize, VSECPRIM_P2CONST_PARA(eslt_Byte) input)`	
Parameter	
workSpace [in,out]	(in) algorithm context buffer, initialized by esl_initSHA256(..) function (out) updated algorithm context structure
inputSize [in]	length of input data in bytes
input [in]	pointer to data to be handled
Return code	
eslt_ErrorCode	ESL_ERC_PARAMETER_INVALID input parameter is NULL
	ESL_ERC_WS_STATE_INVALID work space state invalid
	ESL_ERC_WS_TOO_SMALL work space too small
	ESL_ERC_SHA1_TOTAL_LENGTH_OVERFLOW total input length overflow
	ESL_ERC_NO_ERROR else
Functional Description	
This function updates the SHA-256 calculation for the given input message (or parts of it). This function can be called several times until all input to be hashed is handled.	
Particularities and Limitations	
workSpace is initialized by esl_initSHA256(..)	
Call context	
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces	

Table 4-95 esl_updateSHA256

4.2.2.2.9 esl_finalizeSHA256

Prototype	
`eslt_ErrorCode esl_finalizeSHA256 (VSECPRIM_P2VAR_PARA(eslt_WorkSpaceSHA256) workSpace, VSECPRIM_P2VAR_PARA(eslt_Byte) messageDigest)`	
Parameter	
workSpace [in,out]	(in) algorithm context buffer, initialized by esl_initSHA256(..) function (out) updated algorithm context structure
messageDigest [in,out]	(in) pointer to buffer for the hash value (32 bytes) (out) pointer to computed hash value
Return code	
eslt_ErrorCode	ESL_ERC_PARAMETER_INVALID input parameter is NULL
	ESL_ERC_WS_STATE_INVALID work space state invalid

	ESL_ERC_WS_TOO_SMALL work space too small
	ESL_ERC_NO_ERROR else
Functional Description	
Finalizes the SHA-256 calculation. This function finalizes the SHA-256 calculation by doing the appropriate padding of the input value and returning the hash value.	
Particularities and Limitations	
workSpace is initialized by esl_initSHA256(..), esl_updateSHA256 called before	
Call context	
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces	

Table 4-96 esl_finalizeSHA256

4.2.2.2.10 esl_initSHA384

Prototype	
`eslt_ErrorCode` **`esl_initSHA384`** `(VSECPRIM_P2VAR_PARA(eslt_WorkSpaceSHA384) workSpace)`	
Parameter	
workSpace [in,out]	(in) algorithm context buffer, initialized by esl_initWorkSpaceHeader(..) function (out) initialized algorithm context structure
Return code	
eslt_ErrorCode	ESL_ERC_PARAMETER_INVALID input parameter is NULL
	ESL_ERC_WS_TOO_SMALL work space too small
	ESL_ERC_NO_ERROR else
Functional Description	
Initializes the SHA-384 calculation. This function initializes the SHA-384 hash calculation.	
Particularities and Limitations	
workSpace is initialized by esl_initWorkSpaceHeader(..)	
Call context	
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces	

Table 4-97 esl_initSHA384

4.2.2.2.11 esl_updateSHA384

Prototype	
`eslt_ErrorCode esl_updateSHA384 (VSECPRIM_P2VAR_PARA(eslt_WorkSpaceSHA384) workSpace, const eslt_Length inputSize, VSECPRIM_P2CONST_PARA(eslt_Byte) input)`	
Parameter	
workSpace [in,out]	(in) algorithm context buffer, initialized by esl_initSHA384(..) function (out) updated algorithm context structure
inputSize [in]	length of input data in bytes
input [in]	pointer to data to be handled
Return code	
eslt_ErrorCode	ESL_ERC_PARAMETER_INVALID input parameter is NULL
	ESL_ERC_WS_STATE_INVALID work space state invalid
	ESL_ERC_WS_TOO_SMALL work space too small
	ESL_ERC_SHA1_TOTAL_LENGTH_OVERFLOW total input length overflow
	ESL_ERC_NO_ERROR else
Functional Description	
This function updates the SHA-384 calculation for the given input message (or parts of it). This function can be called several times until all input to be hashed is handled.	
Particularities and Limitations	
workSpace is initialized by esl_initSHA384(..)	
Call context	
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces	

Table 4-98 esl_updateSHA384

4.2.2.2.12 esl_finalizeSHA384

Prototype	
`eslt_ErrorCode esl_finalizeSHA384 (VSECPRIM_P2VAR_PARA(eslt_WorkSpaceSHA384) workSpace, VSECPRIM_P2VAR_PARA(eslt_Byte) messageDigest)`	
Parameter	
workSpace [in,out]	(in) algorithm context buffer, initialized by esl_initSHA384(..) function (out) updated algorithm context structure
messageDigest [in,out]	(in) pointer to buffer for the hash value (48 bytes) (out) pointer to computed hash value
Return code	
eslt_ErrorCode	ESL_ERC_PARAMETER_INVALID input parameter is NULL
	ESL_ERC_WS_STATE_INVALID work space state invalid

	ESL_ERC_WS_TOO_SMALL work space too small
	ESL_ERC_NO_ERROR else
Functional Description	
Finalizes the SHA-384 calculation. This function finalizes the SHA-384 calculation by doing the appropriate padding of the input value and returning the hash value.	
Particularities and Limitations	
workSpace is initialized by esl_initSHA384(..), esl_updateSHA384 called before	
Call context	
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces	

Table 4-99 esl_finalizeSHA384

4.2.2.2.13 esl_initSHA512

Prototype	
`eslt_ErrorCode` **`esl_initSHA512`** `(VSECPRIM_P2VAR_PARA(eslt_WorkSpaceSHA512) workSpace)`	
Parameter	
workSpace [in,out]	(in) algorithm context buffer, initialized by esl_initWorkSpaceHeader(..) function (out) initialized algorithm context structure
Return code	
eslt_ErrorCode	ESL_ERC_PARAMETER_INVALID input parameter is NULL
	ESL_ERC_WS_TOO_SMALL work space too small
	ESL_ERC_NO_ERROR else
Functional Description	
Initializes the SHA-512 calculation. This function initializes the SHA-512 hash calculation.	
Particularities and Limitations	
workSpace is initialized by esl_initWorkSpaceHeader(..)	
Call context	
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces	

Table 4-100 esl_initSHA512

4.2.2.2.14 esl_updateSHA512

Prototype	
`eslt_ErrorCode esl_updateSHA512 (VSECPRIM_P2VAR_PARA(eslt_WorkSpaceSHA512) workSpace, const eslt_Length inputSize, VSECPRIM_P2CONST_PARA(eslt_Byte) input)`	
Parameter	
workSpace [in,out]	(in) algorithm context buffer, initialized by esl_initSHA512(..) function (out) updated algorithm context structure
inputSize [in]	length of input data in bytes
input [in]	pointer to data to be handled
Return code	
eslt_ErrorCode	ESL_ERC_PARAMETER_INVALID input parameter is NULL
	ESL_ERC_WS_STATE_INVALID work space state invalid
	ESL_ERC_WS_TOO_SMALL work space too small
	ESL_ERC_SHA1_TOTAL_LENGTH_OVERFLOW total input length overflow
	ESL_ERC_NO_ERROR else
Functional Description	
This function updates the SHA-512 calculation for the given input message (or parts of it). This function can be called several times until all input to be hashed is handled.	
Particularities and Limitations	
workSpace is initialized by esl_initSHA512(..)	
Call context	
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces	

Table 4-101 esl_updateSHA512

4.2.2.2.15 esl_finalizeSHA512

Prototype	
`eslt_ErrorCode esl_finalizeSHA512 (VSECPRIM_P2VAR_PARA(eslt_WorkSpaceSHA512) workSpace, VSECPRIM_P2VAR_PARA(eslt_Byte) messageDigest)`	
Parameter	
workSpace [in,out]	(in) algorithm context buffer, initialized by esl_initSHA512(..) function (out) updated algorithm context structure
messageDigest [in,out]	(in) pointer to buffer for the hash value (64 bytes) (out) pointer to computed hash value
Return code	
eslt_ErrorCode	ESL_ERC_PARAMETER_INVALID input parameter is NULL
	ESL_ERC_WS_STATE_INVALID work space state invalid

	ESL_ERC_WS_TOO_SMALL work space too small
	ESL_ERC_NO_ERROR else
Functional Description	
Finalizes the SHA-512 calculation. This function finalizes the SHA-512 calculation by doing the appropriate padding of the input value and returning the hash value.	
Particularities and Limitations	
workSpace is initialized by esl_initSHA512(..), esl_updateSHA512 called before	
Call context	
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces	

Table 4-102 esl_finalizeSHA512

4.2.2.2.16 esl_initSHA512_224

Prototype	
`eslt_ErrorCode` **`esl_initSHA512_224`** `(VSECPRIM_P2VAR_PARA(eslt_WorkSpaceSHA512_224) workSpace)`	
Parameter	
workSpace [in,out]	(in) algorithm context buffer, initialized by esl_initWorkSpaceHeader(..) function (out) initialized algorithm context structure
Return code	
eslt_ErrorCode	ESL_ERC_PARAMETER_INVALID input parameter is NULL
	ESL_ERC_WS_TOO_SMALL work space too small
	ESL_ERC_NO_ERROR else
Functional Description	
Initializes the SHA-512-224 calculation. This function initializes the SHA-512-224 hash calculation.	
Particularities and Limitations	
workSpace is initialized by esl_initWorkSpaceHeader(..)	
Call context	
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces	

Table 4-103 esl_initSHA512_224

4.2.2.2.17 esl_updateSHA512_224

Prototype	
`eslt_ErrorCode` **`esl_updateSHA512_224`** `(VSECPRIM_P2VAR_PARA(eslt_WorkSpaceSHA512_224) workSpace, const eslt_Length inputSize, VSECPRIM_P2CONST_PARA(eslt_Byte) input)`	
Parameter	
workSpace [in,out]	(in) algorithm context buffer, initialized by esl_initSHA512_224(..) function (out) updated algorithm context structure
inputSize [in]	length of input data in bytes
input [in]	pointer to data to be handled
Return code	
eslt_ErrorCode	ESL_ERC_PARAMETER_INVALID input parameter is NULL
	ESL_ERC_WS_STATE_INVALID work space state invalid
	ESL_ERC_WS_TOO_SMALL work space too small
	ESL_ERC_SHA1_TOTAL_LENGTH_OVERFLOW total input length overflow
	ESL_ERC_NO_ERROR else
Functional Description	
This function updates the SHA-512-224 calculation for the given input message (or parts of it). This function can be called several times until all input to be hashed is handled.	
Particularities and Limitations	
workSpace is initialized by esl_initSHA512_224(..)	
Call context	
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces	

Table 4-104 esl_updateSHA512_224

4.2.2.2.18 esl_finalizeSHA512_224

Prototype	
`eslt_ErrorCode` **`esl_finalizeSHA512_224`** `(VSECPRIM_P2VAR_PARA(eslt_WorkSpaceSHA512_224) workSpace, VSECPRIM_P2VAR_PARA(eslt_Byte) messageDigest)`	
Parameter	
workSpace [in,out]	(in) algorithm context buffer, initialized by esl_initSHA512_224(..) function (out) updated algorithm context structure
messageDigest [in,out]	(in) pointer to buffer for the hash value (28 bytes) (out) pointer to computed hash value
Return code	
eslt_ErrorCode	ESL_ERC_PARAMETER_INVALID input parameter is NULL

	ESL_ERC_WS_STATE_INVALID work space state invalid
	ESL_ERC_WS_TOO_SMALL work space too small
	ESL_ERC_NO_ERROR else
Functional Description	
Finalizes the SHA-512-224 calculation. This function finalizes the SHA-512-224 calculation by doing the appropriate padding of the input value and returning the hash value.	
Particularities and Limitations	
workSpace is initialized by esl_initSHA512_224(..), esl_updateSHA512_224 called before	
Call context	
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces	

Table 4-105 esl_finalizeSHA512_224

4.2.2.2.19 esl_initSHA512_256

Prototype	
`eslt_ErrorCode` **`esl_initSHA512_256`** `(VSECPRIM_P2VAR_PARA(eslt_WorkSpaceSHA512_256) workSpace)`	
Parameter	
workSpace [in,out]	(in) algorithm context buffer, initialized by esl_initWorkSpaceHeader(..) function (out) initialized algorithm context structure
Return code	
eslt_ErrorCode	ESL_ERC_PARAMETER_INVALID input parameter is NULL
	ESL_ERC_WS_TOO_SMALL work space too small
	ESL_ERC_NO_ERROR else
Functional Description	
Initializes the SHA-512-256 calculation. This function initializes the SHA-512-256 hash calculation.	
Particularities and Limitations	
workSpace is initialized by esl_initWorkSpaceHeader(..)	
Call context	
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces	

Table 4-106 esl_initSHA512_256

4.2.2.2.20 esl_updateSHA512_256

Prototype	
`eslt_ErrorCode` **`esl_updateSHA512_256`** `(VSECPRIM_P2VAR_PARA(eslt_WorkSpaceSHA512_256) workSpace, const eslt_Length inputSize, VSECPRIM_P2CONST_PARA(eslt_Byte) input)`	
Parameter	
workSpace [in,out]	(in) algorithm context buffer, initialized by esl_initSHA512_256(..) function (out) updated algorithm context structure
inputSize [in]	length of input data in bytes
input [in]	pointer to data to be handled
Return code	
eslt_ErrorCode	ESL_ERC_PARAMETER_INVALID input parameter is NULL
	ESL_ERC_WS_STATE_INVALID work space state invalid
	ESL_ERC_WS_TOO_SMALL work space too small
	ESL_ERC_SHA1_TOTAL_LENGTH_OVERFLOW total input length overflow
	ESL_ERC_NO_ERROR else
Functional Description	
This function updates the SHA-512-256 calculation for the given input message (or parts of it). This function can be called several times until all input to be hashed is handled.	
Particularities and Limitations	
workSpace is initialized by esl_initSHA512_256(..)	
Call context	
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces	

Table 4-107 esl_updateSHA512_256

4.2.2.2.21 esl_finalizeSHA512_256

Prototype	
`eslt_ErrorCode` **`esl_finalizeSHA512_256`** `(VSECPRIM_P2VAR_PARA(eslt_WorkSpaceSHA512_256) workSpace, VSECPRIM_P2VAR_PARA(eslt_Byte) messageDigest)`	
Parameter	
workSpace [in,out]	(in) algorithm context buffer, initialized by esl_initSHA512_256(..) function (out) updated algorithm context structure
messageDigest [in,out]	(in) pointer to buffer for the hash value (32 bytes) (out) pointer to computed hash value
Return code	
eslt_ErrorCode	ESL_ERC_PARAMETER_INVALID input parameter is NULL

	ESL_ERC_WS_STATE_INVALID work space state invalid
	ESL_ERC_WS_TOO_SMALL work space too small
	ESL_ERC_NO_ERROR else
Functional Description	
Finalizes the SHA-512-256 calculation. This function finalizes the SHA-512-256 calculation by doing the appropriate padding of the input value and returning the hash value.	
Particularities and Limitations	
workSpace is initialized by esl_initSHA512_256(..), esl_updateSHA512_256 called before	
Call context	
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces	

Table 4-108 esl_finalizeSHA512_256

4.2.2.2.22 esl_initSHA3

```
eslt_ErrorCode esl_initSHA3 (VSECPRIM_P2VAR_PARA(eslt_WorkSpaceSHA3)
workSpace, const eslt_Length bitLength)
```

Parameter	
workSpace [in,out]	(in) algorithm context buffer, initialized by esl_initWorkSpaceHeader(..) function (out) initialized algorithm context structure
bitlength	desired length of the digest, determines whether SHA3-224, SHA3-256, SHA3-384 or SHA3-512 is used
Return code	
eslt_ErrorCode	ESL_ERC_PARAMETER_INVALID input parameter is NULL, or bit length parameter is invalid, i.e., not 224, nor 256, nor 384 nor 512
	ESL_ERC_WS_TOO_SMALL work space too small
	ESL_ERC_NO_ERROR else
Functional Description	
Initializes the SHA-3 hash calculation.	
Particularities and Limitations	
workSpace is initialized by esl_initWorkSpaceHeader(..)	
Call context	
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces	

Table 4-109 esl_initSHA3

4.2.2.2.23 esl_updateSHA3

Prototype	
`eslt_ErrorCode esl_updateSHA3 (VSECPRIM_P2VAR_PARA(eslt_WorkSpaceSHA3) workSpace, const eslt_Length inputSize, VSECPRIM_P2CONST_PARA(eslt_Byte) input)`	
Parameter	
workSpace [in,out]	(in) SHA3 context buffer (out) updated SHA3 context structure
inputSize [in]	length of input data in bytes
input [in]	pointer to data to be hashed
Return code	
eslt_ErrorCode	ESL_ERC_PARAMETER_INVALID input parameter is NULL
	ESL_ERC_WS_STATE_INVALID work space state invalid
	ESL_ERC_WS_TOO_SMALL work space too small
	ESL_ERC_STATE_INVALID workSpace in invalid KECCAKc state, e.g. when called after esl_finalizeSHA3()
	ESL_ERC_NO_ERROR else
Functional Description	
Updates the SHA3 hash calculation for the given input data. This function can be called several times until all data to be hashed has been passed.	
Particularities and Limitations	
workSpace is initialized by esl_initSHA3(..)	
Call context	
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces	

Table 4-110 esl_updateSHA3

4.2.2.2.24 esl_finalizeSHA3

Prototype	
`eslt_ErrorCode esl_finalizeSHA3 (VSECPRIM_P2VAR_PARA(eslt_WorkSpaceSHA3) workSpace, VSECPRIM_P2VAR_PARA(eslt_Byte) messageDigest, VSECPRIM_P2VAR_PARA(eslt_Length) messageDigestLength)`	
Parameter	
workSpace [in,out]	(in) SHA3 context buffer (out) updated algorithm context structure
messageDigest [in,out]	(in) pointer to buffer for the hash value (out) pointer to computed hash value
messageDigestLength [in,out]	(in) pointer to buffer, storing the length of messageDigest (out) pointer to buffer, storing the length of the digest written to the buffer referenced by 'messageDigest'
Return code	
eslt_ErrorCode	ESL_ERC_PARAMETER_INVALID input parameter is NULL

	ESL_ERC_WS_STATE_INVALID work space state invalid
	ESL_ERC_WS_TOO_SMALL work space too small
	ESL_ERC_NO_ERROR else
Functional Description	
Initializes the SHA3 hash calculation and return the hash value. The input data is padded and the bit counter is added to the last hash input block. If SHA3 was initialized with bitLength 224 (256/384/512), the digest length will by default be 28 (32/48/64) bytes. However, the value referenced by messageDigestLength can be less than 28 (32/48/64), if only a truncated digest is required.	
Particularities and Limitations	
workSpace has been initialized by esl_initSHA3 (...) The buffer referenced by messageDigest must provide at least as many bytes as referenced by messageDigestLength.	
Call context	
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces	

Table 4-111 esl_finalizeSHA3

4.2.2.2.25 esl_initSHAKE

```
eslt_ErrorCode esl_initSHAKE (VSECPRIM_P2VAR_PARA(eslt_WorkSpaceSHAKE)
workSpace, const eslt_Length strength)
```

Parameter	
workSpace [in,out]	(in) algorithm context buffer, initialized by esl_initWorkSpaceHeader(..) function (out) initialized algorithm context structure
strength	Algorithm strength – 128 or 256 bits
Return code	
eslt_ErrorCode	ESL_ERC_PARAMETER_INVALID input parameter is NULL, or strength parameter is neither 128 nor 256 bits
	ESL_ERC_WS_TOO_SMALL work space too small
	ESL_ERC_NO_ERROR else
Functional Description	
Initializes the SHAKE hash calculation.	
Particularities and Limitations	
workSpace is initialized by esl_initWorkSpaceHeader(..)	
Call context	
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces	

Table 4-112 esl_initSHAKE

4.2.2.2.26 esl_updateSHAKE

```
eslt_ErrorCode esl_updateSHAKE (VSECPRIM_P2VAR_PARA(eslt_WorkSpaceSHAKE)
workSpace, const eslt_Length inputSize, VSECPRIM_P2CONST_PARA(eslt_Byte)
input)
```

Parameter	
workSpace [in,out]	(in) SHAKE context buffer (out) updated SHAKE context structure
inputSize [in]	length of input data in bytes
input [in]	pointer to data to be hashed
Return code	
eslt_ErrorCode	ESL_ERC_PARAMETER_INVALID input parameter is NULL
	ESL_ERC_WS_STATE_INVALID work space state invalid
	ESL_ERC_WS_TOO_SMALL work space too small
	ESL_ERC_STATE_INVALID workSpace in invalid KECCAKc state, e.g. when called after esl_getXOFDigestSHAKE() or after esl_finalizeSHAKE()
	ESL_ERC_NO_ERROR else

Functional Description

Updates the SHAKE hash calculation for the given input data.

This function can be called several times until all data to be hashed has been passed.

Particularities and Limitations

workSpace is initialized by esl_initSHAKE(..)

Call context

- TASK
- This function is Synchronous
- This function is Reentrant, for different workspaces

Table 4-113 esl_updateSHAKE

4.2.2.2.27 esl_getXOFDigestSHAKE

Prototype

```
eslt_ErrorCode esl_getXOFDigestSHAKE (VSECPRIM_P2VAR_PARA(eslt_WorkSpaceSHAKE)
workSpace, VSECPRIM_P2VAR_PARA(eslt_Byte) messageDigest, const eslt_Length
digestLength)
```

Parameter	
workSpace [in,out]	(in) SHA3 context buffer (out) updated algorithm context structure
messageDigest [in,out]	(in) pointer to buffer for the hash value (out) pointer to computed hash value
digestLength [in]	Length of requested digest in bits
Return code	
eslt_ErrorCode	ESL_ERC_PARAMETER_INVALID input parameter is NULL
	ESL_ERC_WS_STATE_INVALID work space state invalid
	ESL_ERC_WS_TOO_SMALL work space too small

	ESL_ERC_INVALID_LENGTH digest length invalid (not multiple of 8)
	ESL_ERC_NO_ERROR else

Functional Description

Get the digest from the SHAKE hash calculation.

This function can be used repeatedly to return more digest bits (XOF).

Particularities and Limitations

workSpace has been initialized by esl_initSHAKE(...)

The buffer referenced by messageDigest must provide at least digestLength many bytes.

Call context

- TASK
- This function is Synchronous
- This function is Reentrant, for different workspaces

Table 4-114 esl_getXOFDigestSHAKE

4.2.2.2.28 esl_finalizeSHAKE

Prototype

```
eslt_ErrorCode esl_finalizeSHAKE (VSECPRIM_P2VAR_PARA(eslt_WorkSpaceSHAKE)
workSpace, VSECPRIM_P2VAR_PARA(eslt_Byte) messageDigest,
VSECPRIM_P2VAR_PARA(eslt_Length) messageDigestLength)
```

Parameter

workSpace [in,out]	(in) SHA3 context buffer (out) updated algorithm context structure
messageDigest [in,out]	(in) pointer to buffer for the hash value, possibly a null pointer (out) pointer to computed hash value, if messageDigest is not a null pointer when NULL, no digest is returned, but just the context reset
messageDigestLength [in,out]	(in) pointer to buffer, storing the length of messageDigest, possibly a null pointer (out) pointer to buffer, storing the length of the digest written to the buffer referenced by 'messageDigest', if messageDigestLength is not a null pointer

Return code

eslt_ErrorCode	ESL_ERC_PARAMETER_INVALID work space is NULL or messageDigestLength is NULL, if messageDigest is not a null pointer
	ESL_ERC_WS_STATE_INVALID work space state invalid
	ESL_ERC_WS_TOO_SMALL work space too small
	ESL_ERC_NO_ERROR else

Functional Description

Finalizes the SHAKE hash calculation and returns (additional) digest bits (XOF). The input data is padded and the bit counter added to the last hash input block.
This function cannot be used repeatedly to return more digest bits. If messageDigest is a null pointer, the function only resets and clears the workSpace.
If SHAKE was initialized with strength 128 (256), the digest length will by default be 32 (64) bytes. However, the value referenced by messageDigestLength can be less than 32 (64), if only a truncated digest is required.

Particularities and Limitations

workSpace has been initialized by esl_initSHAKE(...)

If messageDigest is not a null pointer, the buffer provided by messageDigest must provide at least as many bytes as referenced by messageDigestLength.
Call context
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces

Table 4-115 esl_finalizeSHAKE

4.2.2.2.29 esl_initMD5

Prototype	
`eslt_ErrorCode esl_initMD5 (VSECPRIM_P2VAR_PARA(eslt_WorkSpaceMD5) workSpace)`	
Parameter	
workSpace [in,out]	(in) algorithm context buffer, initialized by esl_initWorkSpaceHeader(..) function (out) initialized algorithm context structure
Return code	
eslt_ErrorCode	ESL_ERC_PARAMETER_INVALID input parameter is NULL
	ESL_ERC_WS_TOO_SMALL work space too small
	ESL_ERC_NO_ERROR else
Functional Description	
This function initializes the MD5 hash calculation.	
Particularities and Limitations	
workSpace is initialized by esl_initWorkSpaceHeader(..)	
Call context	
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces	

Table 4-116 esl_initMD5

4.2.2.2.30 esl_updateMD5

Prototype	
`eslt_ErrorCode esl_updateMD5 (VSECPRIM_P2VAR_PARA(eslt_WorkSpaceMD5) workSpace, const eslt_Length inputSize, VSECPRIM_P2CONST_PARA(eslt_Byte) input)`	
Parameter	
input [in]	pointer to data to be handled
inputSize [in]	length of the input in bytes
workSpace [in,out]	(in) length of input data in bytes (out) actualized algorithm context structure

Return code	
eslt_ErrorCode	ESL_ERC_PARAMETER_INVALID input parameter is NULL
	ESL_ERC_WS_STATE_INVALID work space state invalid
	ESL_ERC_WS_TOO_SMALL work space too small
	ESL_ERC_NO_ERROR else
Functional Description	
This function updates the MD5 calculation for the given input message (or parts of it). This function can be called several times until all input to be hashed is handled.	
Particularities and Limitations	
workSpace is initialized by esl_initMD5(..)	
Call context	
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces	

Table 4-117 esl_updateMD5

4.2.2.2.31 esl_finalizeMD5

Prototype	
`eslt_ErrorCode esl_finalizeMD5 (VSECPRIM_P2VAR_PARA(eslt_WorkSpaceMD5) workSpace, VSECPRIM_P2VAR_PARA(eslt_Byte) messageDigest)`	
Parameter	
messageDigest [in,out]	(in) pointer to buffer for the hash value (20 bytes) (out) pointer to computed hash value
workSpace [in,out]	(in) algorithm context buffer, initialized by esl_initMD5(..) function (out) actualized algorithm context structure
Return code	
eslt_ErrorCode	ESL_ERC_PARAMETER_INVALID input parameter is NULL
	ESL_ERC_WS_STATE_INVALID work space state invalid
	ESL_ERC_WS_TOO_SMALL work space too small
	ESL_ERC_NO_ERROR else
Functional Description	
This function finalizes the MD5 calculation by doing the appropriate padding of the input value and returning the hash value.	
Particularities and Limitations	
workSpace is initialized by esl_initMD5(..) esl_updateMD5 called before	
Call context	
> TASK > This function is Synchronous	

> This function is Reentrant, for different workspaces

Table 4-118 esl_finalizeMD5

4.2.2.2.32 esl_initRIPEMD160

Prototype	
`eslt_ErrorCode` **`esl_initRIPEMD160`** `(VSECPRIM_P2VAR_PARA(eslt_WorkSpaceRIPEMD160) workSpace)`	
Parameter	
workSpace [in,out]	(in) algorithm context buffer, initialized by esl_initWorkSpaceHeader(..) function (out) initialized algorithm context structure
Return code	
eslt_ErrorCode	ESL_ERC_PARAMETER_INVALID input parameter is NULL
	ESL_ERC_WS_TOO_SMALL work space too small
	ESL_ERC_NO_ERROR else
Functional Description	
This function initializes the RIPEMD-160 hash calculation.	
Particularities and Limitations	
workSpace is initialized by esl_initWorkSpaceHeader(..)	
Call context	
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces	

Table 4-119 esl_initRIPEMD160

4.2.2.2.33 esl_updateRIPEMD160

Prototype	
`eslt_ErrorCode` **`esl_updateRIPEMD160`** `(VSECPRIM_P2VAR_PARA(eslt_WorkSpaceRIPEMD160) workSpace, const eslt_Length inputSize, VSECPRIM_P2CONST_PARA(eslt_Byte) input)`	
Parameter	
input [in]	pointer to data to be handled
inputSize [in]	length of the input in bytes
workSpace [in,out]	(in) length of input data in bytes (out) actualized algorithm context structure
Return code	
eslt_ErrorCode	ESL_ERC_PARAMETER_INVALID input parameter is NULL
	ESL_ERC_WS_STATE_INVALID work space state invalid
	ESL_ERC_WS_TOO_SMALL work space too small

	ESL_ERC_RIPEMD160_TOTAL_LENGTH_OVERFLOW total input length overflow
	ESL_ERC_NO_ERROR else
Functional Description	
This function updates the RIPEMD-160 calculation for the given input message (or parts of it). This function can be called several times until all input to be hashed is handled.	
Particularities and Limitations	
workSpace is initialized by esl_initRIPEMD160(..)	
Call context	
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces	

Table 4-120 esl_updateRIPEMD160

4.2.2.2.34 esl_finalizeRIPEMD160

Prototype	

`eslt_ErrorCode` **`esl_finalizeRIPEMD160`**
`(VSECPRIM_P2VAR_PARA(eslt_WorkSpaceRIPEMD160) workSpace,`
`VSECPRIM_P2VAR_PARA(eslt_Byte) messageDigest)`	

Parameter	
messageDigest [in,out]	(in) pointer to buffer for the hash value (20 bytes) (out) pointer to computed hash value
workSpace [in,out]	(in) algorithm context buffer, initialized by esl_initRIPEMD160(..) function (out) actualized algorithm context structure
Return code	
eslt_ErrorCode	ESL_ERC_PARAMETER_INVALID input parameter is NULL
	ESL_ERC_WS_STATE_INVALID work space state invalid
	ESL_ERC_WS_TOO_SMALL work space too small
	ESL_ERC_NO_ERROR else
Functional Description	
This function finalizes the RIPEMD-160 calculation by doing the appropriate padding of the input value and returning the hash value.	
Particularities and Limitations	
workSpace is initialized by esl_initRIPEMD160(..) esl_updateRIPEMD160 called before	
Call context	
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces	

Table 4-121 esl_finalizeRIPEMD160

4.2.2.3 Message Authentication Codes (MAC)

4.2.2.3.1 esl_generateKeyHashMACSHA1

Prototype	
`eslt_ErrorCode` **`esl_generateKeyHashMACSHA1`** `(const eslt_Length keyLength, VSECPRIM_P2VAR_PARA(eslt_Byte) key)`	
Parameter	
key [in,out]	(in) pointer to user key buffer of size keyLength (out) pointer to the generated key
keyLength [in]	the byte length of the key to be generated
Return code	
eslt_ErrorCode	esl_getBytesRNG(..) return values are passed through
Functional Description	
This function generates a HMAC key for an HashMac calculation. It uses the RNG which is implemented by the call back function esl_getBytesRNG(..).	
Particularities and Limitations	
-	
Call context	
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces	

Table 4-122 esl_generateKeyHashMACSHA1

4.2.2.3.2 esl_initHashMACSHA1

Prototype	
`eslt_ErrorCode` **`esl_initHashMACSHA1`** `(VSECPRIM_P2VAR_PARA(eslt_WorkSpaceHMACSHA1) workSpace, const eslt_Length keyLength, VSECPRIM_P2CONST_PARA(eslt_Byte) key)`	
Parameter	
key [in]	pointer to user key
workSpace [in,out]	(in) algorithm context buffer, initialized by esl_initWorkSpaceHeader(..) function (out) initialized algorithm context structure
keyLength [in]	the byte length of the key to be used
Return code	
eslt_ErrorCode	ESL_ERC_PARAMETER_INVALID input parameter is NULL
	ESL_ERC_WS_TOO_SMALL work space too small
	ESL_ERC_HMAC_KEY_LENGTH_OUT_OF_RANGE the given key length is zero

	ESL_ERC_NO_ERROR else
Functional Description	
This function initializes the HashMAC calculation according to RFC2104 based on the hash function SHA-1.	
Particularities and Limitations	
workSpace is initialized by esl_initWorkSpaceHeader(..)	
Call context	
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces	

Table 4-123 esl_initHashMACSHA1

4.2.2.3.3 esl_updateHashMACSHA1

Prototype	
`eslt_ErrorCode` **`esl_updateHashMACSHA1`** `(VSECPRIM_P2VAR_PARA(eslt_WorkSpaceHMACSHA1) workSpace, const eslt_Length inputLength, VSECPRIM_P2CONST_PARA(eslt_Byte) input)`	
Parameter	
input [in]	pointer to data to be handled
workSpace [in,out]	(in) algorithm context buffer, initialized by esl_initHashMACSHA1(..) function (out) updated algorithm context structure
inputLength [in]	length of input data in bytes
Return code	
eslt_ErrorCode	ESL_ERC_PARAMETER_INVALID input parameter is NULL
	ESL_ERC_WS_STATE_INVALID work space state invalid
	ESL_ERC_WS_TOO_SMALL work space too small
	ESL_ERC_SHA1_TOTAL_LENGTH_OVERFLOW total input length overflow
	ESL_ERC_NO_ERROR else
Functional Description	
This function updates the HashMAC calculation according to RFC2104 based on the hash function SHA-1.	
Particularities and Limitations	
workSpace is initialized by esl_initHashMACSHA1(..)	
Call context	
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces	

Table 4-124 esl_updateHashMACSHA1

4.2.2.3.4 esl_finalizeHashMACSHA1

Prototype	
`eslt_ErrorCode` **`esl_finalizeHashMACSHA1`** `(VSECPRIM_P2VAR_PARA(eslt_WorkSpaceHMACSHA1) workSpace, VSECPRIM_P2VAR_PARA(eslt_Byte) messageHashMAC)`	
Parameter	
workSpace [in,out]	(in) algorithm context buffer, initialized by esl_initHashMACSHA1(..) function (out) updated algorithm context structure
messageHashMAC [in,out]	(in) pointer to buffer for HashMAC (at least 20 bytes) (out) pointer to computed HashMAC
Return code	
eslt_ErrorCode	ESL_ERC_PARAMETER_INVALID input parameter is NULL
	ESL_ERC_WS_STATE_INVALID work space state invalid
	ESL_ERC_WS_TOO_SMALL work space too small
	ESL_ERC_NO_ERROR else
Functional Description	
This function finalizes the HashMAC calculation by returning the calculated HashMAC according to RFC2104 based on SHA-1.	
Particularities and Limitations	
workSpace is initialized by esl_initHashMACSHA1(..) esl_updateHashMACSHA1 called before	
Call context	
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces	

Table 4-125 esl_finalizeHashMACSHA1

4.2.2.3.5 esl_verifyHashMACSHA1

Prototype	
`eslt_ErrorCode` **`esl_verifyHashMACSHA1`** `(VSECPRIM_P2VAR_PARA(eslt_WorkSpaceHMACSHA1) workSpace, VSECPRIM_P2CONST_PARA(eslt_Byte) messageHashMAC)`	
Parameter	
workSpace [in,out]	(in) algorithm context buffer, initialized by esl_initHashMACSHA1(..) function (out) updated algorithm context structure
messageHashMAC [in]	pointer to the given HashMAC (20 bytes)
Return code	
eslt_ErrorCode	ESL_ERC_PARAMETER_INVALID input parameter is NULL
	ESL_ERC_WS_STATE_INVALID work space state invalid

	ESL_ERC_WS_TOO_SMALL work space too small
	ESL_ERC_HMAC_INCORRECT_MAC the given HashMAC is incorrect
	ESL_ERC_NO_ERROR HashMAC is correct
Functional Description	
This function finalizes the verification of a given HashMAC according to RFC2104 based on SHA-1.	
Particularities and Limitations	
workSpace is initialized by esl_initHashMACSHA1(..) esl_updateHashMACSHA1 called before	
Call context	
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces	

Table 4-126 esl_verifyHashMACSHA1

4.2.2.3.6 esl_generateKeyHashMACSHA256

Prototype	
`eslt_ErrorCode` **`esl_generateKeyHashMACSHA256`** `(const eslt_Length keyLength, VSECPRIM_P2VAR_PARA(eslt_Byte) key)`	
Parameter	
key [in,out]	(in) pointer to user key buffer of size keyLength (out) pointer to the generated key
keyLength [in]	the byte length of the key to be generated
Return code	
eslt_ErrorCode	esl_getBytesRNG(..) return values are passed through
Functional Description	
This function generates a HMAC key for an HashMac calculation. It uses the RNG which is implemented by the call back function esl_getBytesRNG(..).	
Particularities and Limitations	
-	
Call context	
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces	

Table 4-127 esl_generateKeyHashMACSHA256

4.2.2.3.7 esl_initHashMACSHA256

Prototype	
`eslt_ErrorCode` **`esl_initHashMACSHA256`** `(VSECPRIM_P2VAR_PARA(eslt_WorkSpaceHMACSHA256) workSpace, const eslt_Length keyLength, VSECPRIM_P2CONST_PARA(eslt_Byte) key)`	
Parameter	
key [in]	pointer to user key
workSpace [in,out]	(in) algorithm context buffer, initialized by esl_initWorkSpaceHeader(..) function (out) initialized algorithm context structure
keyLength [in]	the byte length of the key to be used
Return code	
eslt_ErrorCode	ESL_ERC_PARAMETER_INVALID input parameter is NULL
	ESL_ERC_WS_TOO_SMALL work space too small
	ESL_ERC_HMAC_KEY_LENGTH_OUT_OF_RANGE the given key length is zero
	ESL_ERC_NO_ERROR else
Functional Description	
This function initializes the HashMAC calculation according to RFC2104 based on the hash function SHA-256.	
Particularities and Limitations	
workSpace is initialized by esl_initWorkSpaceHeader(..)	
Call context	
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces	

Table 4-128 esl_initHashMACSHA256

4.2.2.3.8 esl_updateHashMACSHA256

Prototype	
`eslt_ErrorCode` **`esl_updateHashMACSHA256`** `(VSECPRIM_P2VAR_PARA(eslt_WorkSpaceHMACSHA256) workSpace, const eslt_Length inputLength, VSECPRIM_P2CONST_PARA(eslt_Byte) input)`	
Parameter	
input [in]	pointer to data to be handled
workSpace [in,out]	(in) algorithm context buffer, initialized by esl_initHashMACSHA256(..) function (out) actualized algorithm context structure
inputLength [in]	length of input data in bytes
Return code	
eslt_ErrorCode	ESL_ERC_PARAMETER_INVALID input parameter is NULL

	ESL_ERC_WS_STATE_INVALID work space state invalid
	ESL_ERC_WS_TOO_SMALL work space too small
	ESL_ERC_SHA256_TOTAL_LENGTH_OVERFLOW total input length overflow
	ESL_ERC_NO_ERROR else
Functional Description	
This function updates the HashMAC calculation according to RFC2104 based on the hash function SHA-256.	
Particularities and Limitations	
workSpace is initialized by esl_initHashMACSHA256(..)	
Call context	
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces	

Table 4-129 esl_updateHashMACSHA256

4.2.2.3.9 esl_finalizeHashMACSHA256

Prototype	

`eslt_ErrorCode` **`esl_finalizeHashMACSHA256`**
`(VSECPRIM_P2VAR_PARA(eslt_WorkSpaceHMACSHA256) workSpace,`
`VSECPRIM_P2VAR_PARA(eslt_Byte) messageHashMAC)`	

Parameter	
workSpace [in,out]	(in) algorithm context buffer, initialized by esl_initHashMACSHA256(..) function (out) actualized algorithm context structure
messageHashMAC [in,out]	(in) pointer to buffer for HashMAC (at least 32 bytes) (out) pointer to computed HashMAC
Return code	
eslt_ErrorCode	ESL_ERC_PARAMETER_INVALID input parameter is NULL
	ESL_ERC_WS_STATE_INVALID work space state invalid
	ESL_ERC_WS_TOO_SMALL work space too small
	ESL_ERC_NO_ERROR else
Functional Description	
This function finalizes the HashMAC calculation by returning the calculated HashMAC according to RFC2104 based on SHA-256.	
Particularities and Limitations	
workSpace is initialized by esl_initHashMACSHA256(..) esl_updateHashMACSHA256 called before	
Call context	
> TASK > This function is Synchronous	

> This function is Reentrant, for different workspaces

Table 4-130 esl_finalizeHashMACSHA256

4.2.2.3.10 esl_verifyHashMACSHA256

Prototype	
`eslt_ErrorCode` **`esl_verifyHashMACSHA256`** `(VSECPRIM_P2VAR_PARA(eslt_WorkSpaceHMACSHA256) workSpace, VSECPRIM_P2CONST_PARA(eslt_Byte) messageHashMAC)`	
Parameter	
workSpace [in,out]	(in) algorithm context buffer, initialized by esl_initHashMACSHA256(..) function (out) actualized algorithm context structure
messageHashMAC [in]	pointer to the given HashMAC (32 bytes)
Return code	
eslt_ErrorCode	ESL_ERC_PARAMETER_INVALID input parameter is NULL
	ESL_ERC_WS_STATE_INVALID work space state invalid
	ESL_ERC_WS_TOO_SMALL work space too small
	ESL_ERC_HMAC_INCORRECT_MAC the given HashMAC is incorrect
	ESL_ERC_NO_ERROR HashMAC is correct
Functional Description	
This function finalizes the verification of a given HashMAC according to RFC2104 based on SHA-256.	
Particularities and Limitations	
workSpace is initialized by esl_initHashMACSHA256(..) esl_updateHashMACSHA256 called before	
Call context	
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces	

Table 4-131 esl_verifyHashMACSHA256

4.2.2.3.11 esl_generateKeyHashMACSHA384

Prototype	
`eslt_ErrorCode` **`esl_generateKeyHashMACSHA384`** `(const eslt_Length keyLength, VSECPRIM_P2VAR_PARA(eslt_Byte) key)`	
Parameter	
key [in,out]	(in) pointer to user key buffer of size keyLength (out) pointer to the generated key
keyLength [in]	the byte length of the key to be generated

Return code	
eslt_ErrorCode	esl_getBytesRNG(..) return values are passed through
Functional Description	
This function generates a HMAC key for an HashMac calculation. It uses the RNG which is implemented by the call back function esl_getBytesRNG(..).	
Particularities and Limitations	
-	
Call context	
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces	

Table 4-132 esl_generateKeyHashMACSHA384

4.2.2.3.12 esl_initHashMACSHA384

Prototype	
`eslt_ErrorCode` **`esl_initHashMACSHA384`** `(VSECPRIM_P2VAR_PARA(eslt_WorkSpaceHMACSHA384) workSpace, const eslt_Length keyLength, VSECPRIM_P2CONST_PARA(eslt_Byte) key)`	
Parameter	
key [in]	pointer to user key
workSpace [in,out]	(in) algorithm context buffer, initialized by esl_initWorkSpaceHeader(..) function (out) initialized algorithm context structure
keyLength [in]	the byte length of the key to be used
Return code	
eslt_ErrorCode	ESL_ERC_PARAMETER_INVALID input parameter is NULL
	ESL_ERC_WS_TOO_SMALL work space too small
	ESL_ERC_HMAC_KEY_LENGTH_OUT_OF_RANGE the given key length is zero
	ESL_ERC_NO_ERROR else
Functional Description	
This function initializes the HashMAC calculation according to RFC2104 based on the hash function SHA-384.	
Particularities and Limitations	
workSpace is initialized by esl_initWorkSpaceHeader(..)	
Call context	
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces	

Table 4-133 esl_initHashMACSHA384

4.2.2.3.13 esl_updateHashMACSHA384

Prototype	
`eslt_ErrorCode` **`esl_updateHashMACSHA384`** `(VSECPRIM_P2VAR_PARA(eslt_WorkSpaceHMACSHA384) workSpace, const eslt_Length inputLength, VSECPRIM_P2CONST_PARA(eslt_Byte) input)`	
Parameter	
input [in]	pointer to data to be handled
workSpace [in,out]	(in) algorithm context buffer, initialized by esl_initHashMACSHA384(..) function (out) actualized algorithm context structure
inputLength [in]	length of input data in bytes
Return code	
eslt_ErrorCode	ESL_ERC_PARAMETER_INVALID input parameter is NULL
	ESL_ERC_WS_STATE_INVALID work space state invalid
	ESL_ERC_WS_TOO_SMALL work space too small
	ESL_ERC_SHA384_TOTAL_LENGTH_OVERFLOW total input length overflow
	ESL_ERC_NO_ERROR else
Functional Description	
This function updates the HashMAC calculation according to RFC2104 based on the hash function SHA-384.	
Particularities and Limitations	
workSpace is initialized by esl_initHashMACSHA384(..)	
Call context	
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces	

Table 4-134 esl_updateHashMACSHA384

4.2.2.3.14 esl_finalizeHashMACSHA384

Prototype	
`eslt_ErrorCode` **`esl_finalizeHashMACSHA384`** `(VSECPRIM_P2VAR_PARA(eslt_WorkSpaceHMACSHA384) workSpace, VSECPRIM_P2VAR_PARA(eslt_Byte) messageHashMAC)`	
Parameter	
workSpace [in,out]	(in) algorithm context buffer, initialized by esl_initHashMACSHA384(..) function (out) actualized algorithm context structure
messageHashMAC [in,out]	(in) pointer to buffer for HashMAC (at least 32 bytes) (out) pointer to computed HashMAC

Return code	
eslt_ErrorCode	ESL_ERC_PARAMETER_INVALID input parameter is NULL
	ESL_ERC_WS_STATE_INVALID work space state invalid
	ESL_ERC_WS_TOO_SMALL work space too small
	ESL_ERC_NO_ERROR else
Functional Description	
This function finalizes the HashMAC calculation by returning the calculated HashMAC according to RFC2104 based on SHA-384.	
Particularities and Limitations	
workSpace is initialized by esl_initHashMACSHA384(..) esl_updateHashMACSHA384 called before	
Call context	
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces	

Table 4-135 esl_finalizeHashMACSHA384

4.2.2.3.15 esl_verifyHashMACSHA384

Prototype	
`eslt_ErrorCode **esl_verifyHashMACSHA384** (VSECPRIM_P2VAR_PARA(eslt_WorkSpaceHMACSHA384) workSpace, VSECPRIM_P2CONST_PARA(eslt_Byte) messageHashMAC)`	
Parameter	
workSpace [in,out]	(in) algorithm context buffer, initialized by esl_initHashMACSHA384(..) function (out) actualized algorithm context structure
messageHashMAC [in]	pointer to the given HashMAC (32 bytes)
Return code	
eslt_ErrorCode	ESL_ERC_PARAMETER_INVALID input parameter is NULL
	ESL_ERC_WS_STATE_INVALID work space state invalid
	ESL_ERC_WS_TOO_SMALL work space too small
	ESL_ERC_HMAC_INCORRECT_MAC the given HashMAC is incorrect
	ESL_ERC_NO_ERROR HashMAC is correct
Functional Description	
This function finalizes the verification of a given HashMAC according to RFC2104 based on SHA-384.	
Particularities and Limitations	
workSpace is initialized by esl_initHashMACSHA384(..) esl_updateHashMACSHA384 called before	
Call context	
> TASK > This function is Synchronous	

> This function is Reentrant, for different workspaces

Table 4-136 esl_verifyHashMACSHA384

4.2.2.3.16 esl_generateKeyHashMACRIPEMD160

Prototype	
`eslt_ErrorCode` **`esl_generateKeyHashMACRIPEMD160`** `(const eslt_Length keyLength, VSECPRIM_P2VAR_PARA(eslt_Byte) key)`	
Parameter	
key [in,out]	(in) pointer to user key buffer of size keyLength (out) pointer to the generated key
keyLength [in]	the byte length of the key to be generated
Return code	
eslt_ErrorCode	esl_getBytesRNG(..) return values are passed through
Functional Description	
This function generates a HMAC key for an HashMac calculation. It uses the RNG which is implemented by the call back function esl_getBytesRNG(..).	
Particularities and Limitations	
-	
Call context	
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces	

Table 4-137 esl_generateKeyHashMACRIPEMD160

4.2.2.3.17 esl_initHashMACRIPEMD160

Prototype	
`eslt_ErrorCode` **`esl_initHashMACRIPEMD160`** `(VSECPRIM_P2VAR_PARA(eslt_WorkSpaceHMACRIPEMD160) workSpace, const eslt_Length keyLength, VSECPRIM_P2CONST_PARA(eslt_Byte) key)`	
Parameter	
key [in]	pointer to user key
workSpace [in,out]	(in) algorithm context buffer, initialized by esl_initWorkSpaceHeader(..) function (out) initialized algorithm context structure
keyLength [in]	the byte length of the key to be used
Return code	
eslt_ErrorCode	ESL_ERC_PARAMETER_INVALID input parameter is NULL
	ESL_ERC_WS_TOO_SMALL work space too small

	ESL_ERC_HMAC_KEY_LENGTH_OUT_OF_RANGE the given key length is zero
	ESL_ERC_NO_ERROR else
Functional Description	
This function initializes the HashMAC calculation according to RFC2104 based on the hash function RIPEMD-160.	
Particularities and Limitations	
workSpace is initialized by esl_initWorkSpaceHeader(..)	
Call context	
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces	

Table 4-138 esl_initHashMACRIPEMD160

4.2.2.3.18 esl_updateHashMACRIPEMD160

Prototype	
`eslt_ErrorCode esl_updateHashMACRIPEMD160 (VSECPRIM_P2VAR_PARA(eslt_WorkSpaceHMACRIPEMD160) workSpace, const eslt_Length inputLength, VSECPRIM_P2CONST_PARA(eslt_Byte) input)`	
Parameter	
input [in]	pointer to data to be handled
workSpace [in,out]	(in) algorithm context buffer, initialized by esl_initHashMACRIPEMD160(..) function (out) actualized algorithm context structure
inputLength [in]	length of input data in bytes
Return code	
eslt_ErrorCode	ESL_ERC_PARAMETER_INVALID input parameter is NULL
	ESL_ERC_WS_STATE_INVALID work space state invalid
	ESL_ERC_WS_TOO_SMALL work space too small
	ESL_ERC_RIPEMD160_TOTAL_LENGTH_OVERFLOW total input length overflow
	ESL_ERC_NO_ERROR else
Functional Description	
This function updates the HashMAC calculation according to RFC2104 based on the hash function RIPEMD-160.	
Particularities and Limitations	
workSpace is initialized by esl_initHashMACRIPEMD160(..)	
Call context	
> TASK > This function is Synchronous	

> This function is Reentrant, for different workspaces

Table 4-139 esl_updateHashMACRIPEMD160

4.2.2.3.19 esl_finalizeHashMACRIPEMD160

Prototype	
`eslt_ErrorCode` **`esl_finalizeHashMACRIPEMD160`** `(VSECPRIM_P2VAR_PARA(eslt_WorkSpaceHMACRIPEMD160) workSpace, VSECPRIM_P2VAR_PARA(eslt_Byte) messageHashMAC)`	
Parameter	
workSpace [in,out]	(in) algorithm context buffer, initialized by esl_initHashMACRIPEMD160(..) function (out) actualized algorithm context structure
messageHashMAC [in,out]	(in) pointer to buffer for HashMAC (at least 20 bytes) (out) pointer to computed HashMAC
Return code	
eslt_ErrorCode	ESL_ERC_PARAMETER_INVALID input parameter is NULL
	ESL_ERC_WS_STATE_INVALID work space state invalid
	ESL_ERC_WS_TOO_SMALL work space too small
	ESL_ERC_NO_ERROR else
Functional Description	
This function finalizes the HashMAC calculation by returning the calculated HashMAC according to RFC2104 based on RIPEMD-160.	
Particularities and Limitations	
workSpace is initialized by esl_initHashMACRIPEMD160(..) esl_updateHashMACRIPEMD160 called before	
Call context	
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces	

Table 4-140 esl_finalizeHashMACRIPEMD160

4.2.2.3.20 esl_verifyHashMACRIPEMD160

Prototype	
`eslt_ErrorCode` **`esl_verifyHashMACRIPEMD160`** `(VSECPRIM_P2VAR_PARA(eslt_WorkSpaceHMACRIPEMD160) workSpace, VSECPRIM_P2CONST_PARA(eslt_Byte) messageHashMAC)`	
Parameter	
workSpace [in,out]	(in) algorithm context buffer, initialized by esl_initHashMACRIPEMD160(..) function (out) actualized algorithm context structure
messageHashMAC [in]	pointer to the given HashMAC (20 bytes)

Return code	
eslt_ErrorCode	ESL_ERC_PARAMETER_INVALID input parameter is NULL
	ESL_ERC_WS_STATE_INVALID work space state invalid
	ESL_ERC_WS_TOO_SMALL work space too small
	ESL_ERC_HMAC_INCORRECT_MAC the given HashMAC is incorrect
	ESL_ERC_NO_ERROR HashMAC is correct
Functional Description	
This function finalizes the verification of a given HashMAC according to RFC2104 based on RIPEMD-160.	
Particularities and Limitations	
workSpace is initialized by esl_initHashMACRIPEMD160(..) esl_updateHashMACRIPEMD160 called before	
Call context	
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces	

Table 4-141 esl_verifyHashMACRIPEMD160

4.2.2.3.21 esl_getExpandedKeyCMACAES128

Prototype	
`eslt_ErrorCode` **`esl_getExpandedKeyCMACAES128`** `(VSECPRIM_P2CONST_PARA(eslt_WorkSpaceCMACAES) workSpace, VSECPRIM_P2VAR_PARA(eslt_Byte) keyPtr, VSECPRIM_P2VAR_PARA(eslt_Length) keyLengthPtr)`	
Parameter	
workSpace [in]	CMAC work space
keyPtr [out]	Pointer to buffer where the expanded key shall be stored.
keyLengthPtr [in,out]	In Size of provided buffer, Out Size of written data
Return code	
eslt_ErrorCode	ESL_ERC_NO_ERROR Operation successful
	ESL_ERC_PARAMETER_INVALID Input parameter is NULL
	ESL_ERC_WS_TOO_SMALL Work space too small
	ESL_ERC_WS_STATE_INVALID Work space state invalid
	ESL_ERC_INPUT_INVALID Invalid input length
Functional Description	
Extracts the expanded CMAC key. This function extracts the expanded CMAC key from the given workspace. Re-using the expanded key on a system with constant key usage will speed up the CMAC calculation.	

Particularities and Limitations
Workspace needs to be initialized.
Call context
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces

Table 4-142 esl_getExpandedKeyCMACAES128

4.2.2.3.22 esl_initExpandedCMACAES128

Prototype	
`eslt_ErrorCode esl_initExpandedCMACAES128 (VSECPRIM_P2VAR_PARA(eslt_WorkSpaceCMACAES) workSpace, VSECPRIM_P2CONST_PARA(eslt_Byte) expandedKey, const eslt_Length expandedKeyLength)`	
Parameter	
workSpace [in]	CMAC work space
expandedKey [in]	CMAC expanded key which shall be used
expandedKeyLength [in]	Size of provided key
Return code	
eslt_ErrorCode	ESL_ERC_NO_ERROR Operation successful
	ESL_ERC_PARAMETER_INVALID Input parameter is NULL
	ESL_ERC_WS_TOO_SMALL Work space too small
	ESL_ERC_HMAC_KEY_LENGTH_OUT_OF_RANGE AES key length is invalid
Functional Description	
Initialize CMAC calculation with a pre-calculated CMAC expanded key. This function initializes the CMAC calculation workspace using a pre-calculated CMAC expanded key. This will speed up the CMAC verification process.	
Particularities and Limitations	
Pointers must be valid	
Call context	
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces	

Table 4-143 esl_initExpandedCMACAES128

4.2.2.3.23 esl_initCMACAES128

Prototype	
`eslt_ErrorCode` **`esl_initCMACAES128`** `(VSECPRIM_P2VAR_PARA(eslt_WorkSpaceCMACAES) workSpace, const eslt_Length keyLength, VSECPRIM_P2CONST_PARA(eslt_Byte) key)`	
Parameter	
workSpace [in,out]	CMAC work space
keyLength [in]	Size of AES key to be used
key [in]	AES key to be used
Return code	
eslt_ErrorCode	ESL_ERC_NO_ERROR Operation successful
	ESL_ERC_PARAMETER_INVALID Input parameter is NULL
	ESL_ERC_WS_TOO_SMALL Work space too small
	ESL_ERC_HMAC_KEY_LENGTH_OUT_OF_RANGE AES key length is invalid
Functional Description	
Initialize CMAC calculation. This function initializes the CMAC calculation workspace.	
Particularities and Limitations	
Pointers must be valid	
Call context	
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces	

Table 4-144 esl_initCMACAES128

4.2.2.3.24 esl_updateCMACAES128

Prototype	
`eslt_ErrorCode` **`esl_updateCMACAES128`** `(VSECPRIM_P2VAR_PARA(eslt_WorkSpaceCMACAES) workSpace, const eslt_Length inputLength, VSECPRIM_P2CONST_PARA(eslt_Byte) input)`	
Parameter	
workSpace [in,out]	CMAC work space
inputLength [in]	Length of input data
input [in]	Pointer to input data
Return code	
eslt_ErrorCode	ESL_ERC_NO_ERROR Operation successful
	ESL_ERC_PARAMETER_INVALID Input parameter is NULL
	ESL_ERC_WS_STATE_INVALID Work space state invalid

Functional Description
Update CMAC calculation. This function is used to feed the CMAC calculation with input data.
Particularities and Limitations
Workspace needs to be initialized.
Call context
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces

Table 4-145 esl_updateCMACAES128

4.2.2.3.25 esl_finalizeCMACAES128

Prototype	

`eslt_ErrorCode` **`esl_finalizeCMACAES128`**
`(VSECPRIM_P2VAR_PARA(eslt_WorkSpaceCMACAES) workSpace,`
`VSECPRIM_P2VAR_PARA(eslt_Byte) messageCMAC)`	

Parameter	
workSpace [in,out]	CMAC work space
messageCMAC [out]	Pointer to computed CMAC. The size must be 16 bytes.
Return code	
eslt_ErrorCode	ESL_ERC_NO_ERROR Operation successful
	ESL_ERC_PARAMETER_INVALID Input parameter is NULL
	ESL_ERC_WS_TOO_SMALL Work space too small
	ESL_ERC_WS_STATE_INVALID Work space state invalid
	ESL_ERC_INPUT_INVALID Invalid input length
Functional Description	
Finalize CMAC calculation. This function finalizes the MAC calculation.	
Particularities and Limitations	
Workspace needs to be initialized. messageCMAC must point to valid memory of at least 16 bytes	
Call context	
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces	

Table 4-146 esl_finalizeCMACAES128

4.2.2.3.26 esl_verifyCMACAES128

Prototype	
`eslt_ErrorCode esl_verifyCMACAES128 (VSECPRIM_P2VAR_PARA(eslt_WorkSpaceCMACAES) workSpace, VSECPRIM_P2CONST_PARA(eslt_Byte) messageCMAC)`	
Parameter	
workSpace [in,out]	CMAC work space
messageCMAC [in]	Pointer to CMAC to be compared. The size must be 16 bytes.
Return code	
eslt_ErrorCode	ESL_ERC_NO_ERROR CMAC is correct
	ESL_ERC_PARAMETER_INVALID Input parameter is NULL
	ESL_ERC_WS_TOO_SMALL Work space too small
	ESL_ERC_WS_STATE_INVALID Work space state invalid
Functional Description	
Verify given CMAC. This function verifies the provided CMAC against the calculated one.	
Particularities and Limitations	
Workspace needs to be initialized.	
Call context	
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces	

Table 4-147 esl_verifyCMACAES128

4.2.2.3.27 esl_initExpandedCMACAES128

Prototype	
`eslt_ErrorCode esl_initExpandedCMACAES128 (VSECPRIM_P2VAR_PARA(eslt_WorkSpaceCMACAES) workSpace, VSECPRIM_P2CONST_PARA(eslt_Byte) expandedKey, const eslt_Length expandedKeyLength)`	
Parameter	
workSpace [in,out]	CMAC work space
expandedKey [in]	CMAC expanded key which shall be used
expandedKeyLength [in]	Size of provided key
Return code	
eslt_ErrorCode	ESL_ERC_NO_ERROR Operation successful
	ESL_ERC_PARAMETER_INVALID Input parameter is NULL
	ESL_ERC_WS_TOO_SMALL Work space too small
	ESL_ERC_HMAC_KEY_LENGTH_OUT_OF_RANGE AES key length is invalid

Functional Description
Initialize CMAC calculation with a pre-calculated CMAC expanded key. This function initializes the CMAC calculation workspace using a pre-calculated CMAC expanded key. This will speed up the CMAC verification process.
Particularities and Limitations
Pointers must be valid
Call context
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces

Table 4-148 esl_initExpandedCMACAES128

4.2.2.3.28 esl_getExpandedKeyCMACAES128

Prototype	
`eslt_ErrorCode` **`esl_getExpandedKeyCMACAES128`** `(VSECPRIM_P2CONST_PARA(eslt_WorkSpaceCMACAES) workSpace, VSECPRIM_P2VAR_PARA(eslt_Byte) keyPtr, VSECPRIM_P2VAR_PARA(eslt_Length) keyLengthPtr)`	
Parameter	
workSpace [in]	CMAC work space
keyPtr [in,out]	Pointer to buffer where the expanded key shall be stored.
keyLengthPtr [in,out]	(in) Size of provided buffer, (out) Size of written data
Return code	
eslt_ErrorCode	ESL_ERC_NO_ERROR Operation successful
	ESL_ERC_PARAMETER_INVALID Input parameter is NULL
	ESL_ERC_WS_TOO_SMALL Work space too small
	ESL_ERC_WS_STATE_INVALID Work space state invalid
	ESL_ERC_INPUT_INVALID Invalid input length
Functional Description	
Extracts the expanded CMAC key. This function extracts the expanded CMAC key from the given workspace. Re-using the expanded key on a system with constant key usage will speed up the CMAC calculation.	
Particularities and Limitations	
Workspace needs to be initialized.	
Call context	
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces	

Table 4-149 esl_getExpandedKeyCMACAES128

4.2.2.3.29 esl_initCMACAES256

Prototype	
`eslt_ErrorCode esl_initCMACAES256 (VSECPRIM_P2VAR_PARA(eslt_WorkSpaceCMACAES) workSpace, const eslt_Length keyLength, VSECPRIM_P2CONST_PARA(eslt_Byte) key)`	
Parameter	
workSpace [in,out]	CMAC work space
keyLength [in]	Size of AES key to be used
key [in]	AES key to be used
Return code	
eslt_ErrorCode	ESL_ERC_NO_ERROR Operation successful
	ESL_ERC_PARAMETER_INVALID Input parameter is NULL
	ESL_ERC_WS_TOO_SMALL Work space too small
	ESL_ERC_HMAC_KEY_LENGTH_OUT_OF_RANGE AES key length is invalid
Functional Description	
Initialize CMAC calculation. This function initializes the CMAC calculation workspace.	
Particularities and Limitations	
Pointers must be valid	
Call context	
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces	

Table 4-150 esl_initCMACAES256

4.2.2.3.30 esl_updateCMACAES256

Prototype	
`eslt_ErrorCode esl_updateCMACAES256 (VSECPRIM_P2VAR_PARA(eslt_WorkSpaceCMACAES) workSpace, const eslt_Length inputLength, VSECPRIM_P2CONST_PARA(eslt_Byte) input)`	
Parameter	
workSpace [in,out]	CMAC work space
inputLength [in]	Length of input data
input [in]	Pointer to input data
Return code	
eslt_ErrorCode	ESL_ERC_NO_ERROR Operation successful
	ESL_ERC_PARAMETER_INVALID Input parameter is NULL
	ESL_ERC_WS_STATE_INVALID Work space state invalid

Functional Description
Update CMAC calculation. This function is used to feed the CMAC AES 256 calculation with input data.
Particularities and Limitations
Workspace needs to be initialized.
Call context
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces

Table 4-151 esl_updateCMACAES256

4.2.2.3.31 esl_finalizeCMACAES256

Prototype	

`eslt_ErrorCode` **`esl_finalizeCMACAES256`**
`(VSECPRIM_P2VAR_PARA(eslt_WorkSpaceCMACAES) workSpace,`
`VSECPRIM_P2VAR_PARA(eslt_Byte) messageCMAC)`	

Parameter	
workSpace [in,out]	CMAC work space
messageCMAC [out]	Pointer to computed CMAC. The size must be 16 bytes.
Return code	
eslt_ErrorCode	ESL_ERC_NO_ERROR Operation successful
	ESL_ERC_PARAMETER_INVALID Input parameter is NULL
	ESL_ERC_WS_TOO_SMALL Work space too small
	ESL_ERC_WS_STATE_INVALID Work space state invalid
	ESL_ERC_INPUT_INVALID Invalid input length
	ESL_ERC_BUFFER_TOO_SMALL buffer referenced by messageCMAC not sufficient to store CMAC
Functional Description	
Finalize CMAC AES 256 calculation. This function finalizes the MAC AES 256 calculation.	
Particularities and Limitations	
Workspace needs to be initialized. The buffer referenced by messageCMAC must provide at least *messageCMACLength bytes.	
Call context	
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces	

Table 4-152 esl_finalizeCMACAES256

4.2.2.3.32 esl_verifyCMACAES256

Prototype	
`eslt_ErrorCode` **`esl_verifyCMACAES256`** `(VSECPRIM_P2VAR_PARA(eslt_WorkSpaceCMACAES) workSpace, VSECPRIM_P2CONST_PARA(eslt_Byte) messageCMAC)`	
Parameter	
workSpace [in,out]	CMAC work space
messageCMAC [in]	Pointer to CMAC to be compared. The size must be 16 bytes.
Return code	
eslt_ErrorCode	ESL_ERC_NO_ERROR CMAC is correct
	ESL_ERC_PARAMETER_INVALID Input parameter is NULL
	ESL_ERC_WS_TOO_SMALL Work space too small
	ESL_ERC_WS_STATE_INVALID Work space state invalid
Functional Description	
Verify given CMAC based on AES 256. This function verifies the provided CMAC against the calculated one.	
Particularities and Limitations	
Workspace needs to be initialized. The buffer referenced by messageCMAC must provide at least ESL_SIZOF_AES256BLOCK (= 16) bytes	
Call context	
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces	

Table 4-153 esl_verifyCMACAES256

4.2.2.3.33 esl_initExpandedCMACAES256

`eslt_ErrorCode` **`esl_initExpandedCMACAES256`** `(VSECPRIM_P2VAR_PARA(eslt_WorkSpaceCMACAES) workSpace, VSECPRIM_P2CONST_PARA(eslt_Byte) expandedKey, const eslt_Length expandedKeyLength)`	
Parameter	
workSpace [in,out]	CMAC work space
expandedKey [in]	CMAC expanded key which shall be used
expandedKeyLength [in]	Size of provided key
Return code	
eslt_ErrorCode	ESL_ERC_NO_ERROR Operation successful
	ESL_ERC_PARAMETER_INVALID Input parameter is NULL
	ESL_ERC_WS_TOO_SMALL Work space too small

	ESL_ERC_HMAC_KEY_LENGTH_OUT_OF_RANGE AES key length is invalid
Functional Description	
Initialize CMAC AES256 calculation with a pre-calculated CMAC expanded key. This function initializes the CMAC AES256 calculation workspace using a pre-calculated CMAC expanded key. This will speed up the CMAC verification process.	
Particularities and Limitations	
Pointers must be valid	
Call context	
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces	

Table 4-154 esl_initExpandedCMACAES256

4.2.2.3.34 esl_getExpandedKeyCMACAES256

Prototype	
`eslt_ErrorCode` **`esl_getExpandedKeyCMACAES256`** `(VSECPRIM_P2CONST_PARA(eslt_WorkSpaceCMACAES) workSpace, VSECPRIM_P2VAR_PARA(eslt_Byte) keyPtr, VSECPRIM_P2VAR_PARA(eslt_Length) keyLengthPtr)`	
Parameter	
workSpace [in]	CMAC work space
keyPtr [in,out]	Pointer to buffer where the expanded key shall be stored.
keyLengthPtr [in,out]	(in) Size of provided buffer, (out) Size of written data
Return code	
eslt_ErrorCode	ESL_ERC_NO_ERROR Operation successful
	ESL_ERC_PARAMETER_INVALID Input parameter is NULL
	ESL_ERC_WS_TOO_SMALL Work space too small
	ESL_ERC_WS_STATE_INVALID Work space state invalid
	ESL_ERC_INPUT_INVALID Invalid input length
Functional Description	
Extracts the expanded CMAC key. This function extracts the expanded CMAC key from the given workspace. Re-using the expanded key on a system with constant key usage will speed up the CMAC calculation.	
Particularities and Limitations	
Workspace needs to be initialized.	
Call context	
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces	

Table 4-155 esl_getExpandedKeyCMACAES256

4.2.2.3.35 esl_initGMAC

Prototype	
`eslt_ErrorCode esl_initGMAC (VSECPRIM_P2VAR_PARA(eslt_WorkSpaceGMAC) workSpace, const eslt_Length keyLength, VSECPRIM_P2CONST_PARA(eslt_Byte) key, const eslt_Length ivLength, VSECPRIM_P2CONST_PARA(eslt_Byte) iv)`	
Parameter	
key [in]	the symmetric AES master key
ivLength [in]	the length of the initialization vector in bytes
workSpace [in,out]	(in) algorithm context buffer, initialized by esl_initWorkSpaceHeader(..) function (out) initialized algorithm context structure
iv [in]	the initialization vector
keyLength [in]	the length of the AES key in bytes, must be 16, 24 or 32
Return code	
eslt_ErrorCode	ESL_ERC_PARAMETER_INVALID input parameter is NULL
	ESL_ERC_WS_TOO_SMALL work space too small
	ESL_ERC_GCM_INVALID_KEY_LENGTH no valid AES keyLength
	ESL_ERC_NO_ERROR else
Functional Description	
This function initializes the GMAC calculation.	
Particularities and Limitations	
workSpace is initialized by esl_initWorkSpaceHeader(..)	
Call context	
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces	

Table 4-156 esl_initGMAC

4.2.2.3.36 esl_updateGMAC

Prototype	
`eslt_ErrorCode esl_updateGMAC (VSECPRIM_P2VAR_PARA(eslt_WorkSpaceGMAC) workSpace, const eslt_Length inputLength, VSECPRIM_P2CONST_PARA(eslt_Byte) input)`	
Parameter	
input [in]	the input data
workSpace [in,out]	algorithm context buffer, initialized by esl_initWorkSpaceHeader(..) function
inputLength [in]	the length of the input

Return code	
eslt_ErrorCode	ESL_ERC_PARAMETER_INVALID input parameter is NULL
	ESL_ERC_WS_STATE_INVALID work space state invalid
	ESL_ERC_WS_TOO_SMALL work space too small
	ESL_ERC_GCM_TOTAL_LENGTH_OVERFLOW the input exceeded the length limit
	ESL_ERC_NO_ERROR else
Functional Description	
This function updates the GMAC operation.	
Particularities and Limitations	
workSpace is initialized by esl_initGMAC(..)	
Call context	
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces	

Table 4-157 esl_updateGMAC

4.2.2.3.37 esl_finalizeGMAC

Prototype	
`eslt_ErrorCode esl_finalizeGMAC (VSECPRIM_P2VAR_PARA(eslt_WorkSpaceGMAC) workSpace, VSECPRIM_P2VAR_PARA(eslt_Byte) tag)`	
Parameter	
workSpace [in,out]	algorithm context buffer, initialized by
tag [in,out]	(in) a pointer to a buffer for the authentication tag (out) this variable is incremented by the number of bytes that were handled in this iteration of updateEncryptGCM
Return code	
eslt_ErrorCode	ESL_ERC_PARAMETER_INVALID input parameter is NULL
	ESL_ERC_WS_STATE_INVALID work space state invalid
	ESL_ERC_WS_TOO_SMALL work space too small
	ESL_ERC_NO_ERROR else
Functional Description	
This function finalizes the GMAC operation.	
Particularities and Limitations	
workSpace is initialized by esl_initGMAC(..)	
Call context	
> TASK > This function is Synchronous	

> This function is Reentrant, for different workspaces

Table 4-158 esl_finalizeGMAC

4.2.2.3.38 esl_verifyGMAC

Prototype	
`eslt_ErrorCode` **`esl_verifyGMAC`** `(VSECPRIM_P2VAR_PARA(eslt_WorkSpaceGMAC) workSpace, VSECPRIM_P2CONST_PARA(eslt_Byte) tag)`	
Parameter	
workSpace [in,out]	algorithm context buffer, initialized by
tag [in]	a pointer to a buffer containing the authentication tag
Return code	
eslt_ErrorCode	ESL_ERC_PARAMETER_INVALID input parameter is NULL
	ESL_ERC_WS_STATE_INVALID work space state invalid
	ESL_ERC_WS_TOO_SMALL work space too small
	ESL_ERC_GCM_TAG_VERIFICATION_FAILED the provided tag does not
	match the computed one
	ESL_ERC_NO_ERROR else
Functional Description	
This function finalizes the GMAC operation.	
Particularities and Limitations	
workSpace is initialized by esl_initGMAC(..)	
Call context	
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces	

Table 4-159 esl_verifyGMAC

4.2.2.3.39 esl_initPoly1305

Prototype	
`eslt_ErrorCode` **`esl_initPoly1305`** `(VSECPRIM_P2VAR_PARA(eslt_WorkSpacePoly1305) workSpace, VSECPRIM_P2CONST_PARA(eslt_Byte) key)`	
Parameter	
key [in]	pointer to key of length ESL_SIZEOF_Poly1305_KEY
workSpace [in,out]	(in) algorithm context buffer, initialized by esl_initWorkSpaceHeader () function (out) initialized algorithm context structure
Return code	
eslt_ErrorCode	ESL_ERC_PARAMETER_INVALID input parameter is NULL

	ESL_ERC_WS_TOO_SMALL work space too small
	ESL_ERC_NO_ERROR else
Functional Description	
This function initializes the Poly1305 autenticator.	
Particularities and Limitations	
workSpace is initialized by esl_initWorkSpaceHeader ()	
Call context	
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces	

Table 4-160 esl_initPoly1305

4.2.2.3.40 esl_updatePoly1305

Prototype	
`eslt_ErrorCode` **`esl_updatePoly1305`** `(VSECPRIM_P2VAR_PARA(eslt_WorkSpacePoly1305) workSpace, VSECPRIM_P2CONST_PARA(eslt_Byte) message, const eslt_Length msgSize)`	
Parameter	
msgSize [in]	length of message data (bytes)
workSpace [in,out]	(in) algorithm context buffer, initialized by esl_initPoly1305 () function (out) updated algorithm context structure
message [in]	pointer to message data
Return code	
eslt_ErrorCode	ESL_ERC_PARAMETER_INVALID input parameter is NULL
	ESL_ERC_WS_STATE_INVALID work space state invalid
	ESL_ERC_NO_ERROR else
Functional Description	
This function updates the Poly1305 authenticator.	
Particularities and Limitations	
workSpace is initialized by initPoly1305 ()	
Call context	
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces	

Table 4-161 esl_updatePoly1305

4.2.2.3.41 esl_finalizePoly1305

Prototype	
`eslt_ErrorCode` **`esl_finalizePoly1305`** `(VSECPRIM_P2VAR_PARA(eslt_WorkSpacePoly1305) workSpace, VSECPRIM_P2VAR_PARA(eslt_Byte) tag)`	
Parameter	
workSpace [in,out]	(in) algorithm context buffer, initialized by esl_initPoly1305 () function (out) updated algorithm context buffer
tag [in,out]	(in) pointer to TAG (output) buffer; size must be at least ESL_SIZEOF_Poly1305_TAG (out) authentication TAG calculated
Return code	
eslt_ErrorCode	ESL_ERC_PARAMETER_INVALID input parameter is NULL
	ESL_ERC_WS_STATE_INVALID work space state invalid
	ESL_ERC_NO_ERROR else
Functional Description	
This function calculates and returns the Poly1305 TAG.	
Particularities and Limitations	
workSpace is initialized by initPoly1305 ()	
Call context	
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces	

Table 4-162 esl_finalizePoly1305

4.2.2.3.42 esl_verifyPoly1305

Prototype	
`eslt_ErrorCode` **`esl_verifyPoly1305`** `(VSECPRIM_P2VAR_PARA(eslt_WorkSpacePoly1305) workSpace, VSECPRIM_P2CONST_PARA(eslt_Byte) tag)`	
Parameter	
workSpace [in,out]	(in) algorithm context buffer, initialized by esl_initPoly1305 () function (out) updated algorithm context buffer
tag [in]	(in) pointer to TAG (input) (out) authentication TAG calculated
Return code	
eslt_ErrorCode	ESL_ERC_PARAMETER_INVALID input parameter is NULL
	ESL_ERC_WS_STATE_INVALID work space state invalid
	ESL_ERC_INCORRECT_TAG the given TAG is incorrect
	ESL_ERC_NO_ERROR Poly1305 TAG is correct

Functional Description
This function verifies a Poly1305 TAG.
Particularities and Limitations
workSpace is initialized by initPoly1305 ()
Call context
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces

Table 4-163 esl_verifyPoly1305

4.2.2.3.43 esl_initSipHash

Prototype	
`eslt_ErrorCode` **`esl_initSipHash`** `(VSECPRIM_P2VAR_PARA(eslt_WorkSpaceSipHash) workSpace, const eslt_Length keyLength, VSECPRIM_P2CONST_PARA(eslt_Byte) key)`	
Parameter	
workSpace [in,out]	SipHash work space
keyLength [in]	Size of SipHash key to be used
key [in]	SipHash key to be used
Return code	
eslt_ErrorCode	ESL_ERC_NO_ERROR Operation successful
	ESL_ERC_PARAMETER_INVALID Input parameter is NULL
	ESL_ERC_WS_TOO_SMALL Work space too small
	ESL_ERC_HMAC_KEY_LENGTH_OUT_OF_RANGE SipHash key length is invalid
Functional Description	
Initializes SipHash calculation. This function initializes SipHash calculation.	
Particularities and Limitations	
Pointers must be valid	
Call context	
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces	

Table 4-164 esl_initSipHash

4.2.2.3.44 esl_updateSipHash

Prototype	
`eslt_ErrorCode esl_updateSipHash (VSECPRIM_P2VAR_PARA(eslt_WorkSpaceSipHash) workSpace, const eslt_Length inputLength, VSECPRIM_P2CONST_PARA(eslt_Byte) input)`	
Parameter	
workSpace [in,out]	SipHash work space
inputLength [in]	Length of input data
input [in]	Pointer to input data
Return code	
eslt_ErrorCode	ESL_ERC_NO_ERROR Operation successful
	ESL_ERC_PARAMETER_INVALID Input parameter is NULL
	ESL_ERC_WS_TOO_SMALL Work space too small
	ESL_ERC_WS_STATE_INVALID Work space state invalid
	ESL_ERC_INPUT_INVALID Invalid input length
Functional Description	
Updates SipHash calculation. This function updates SipHash calculation.	
Particularities and Limitations	
Pointers must be valid	
Call context	
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces	

Table 4-165 esl_updateSipHash

4.2.2.3.45 esl_finalizeSipHash

Prototype	
`eslt_ErrorCode esl_finalizeSipHash (VSECPRIM_P2VAR_PARA(eslt_WorkSpaceSipHash) workSpace, VSECPRIM_P2VAR_PARA(eslt_Byte) messageMAC)`	
Parameter	
workSpace [in,out]	SipHash work space
messageMAC [out]	Pointer to computed MAC. Size must be 8 bytes.
Return code	
eslt_ErrorCode	ESL_ERC_NO_ERROR Operation successful
	ESL_ERC_PARAMETER_INVALID Input parameter is NULL
	ESL_ERC_WS_TOO_SMALL Work space too small

	ESL_ERC_WS_STATE_INVALID Work space state invalid
Functional Description	
Finalizes SipHash calculation. This function finalizes SipHash calculation.	
Particularities and Limitations	
Pointers must be valid messageMAC must point to valid memory of at least 8 bytes	
Call context	
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces	

Table 4-166 esl_finalizeSipHash

4.2.2.3.46 esl_verifySipHash

Prototype	
`eslt_ErrorCode esl_verifySipHash (VSECPRIM_P2VAR_PARA(eslt_WorkSpaceSipHash) workSpace, VSECPRIM_P2CONST_PARA(eslt_Byte) messageMAC)`	
Parameter	
workSpace [in,out]	SipHash work space
messageMAC [in]	Pointer to SipHash to be compared. Size must be 8 bytes.
Return code	
eslt_ErrorCode	ESL_ERC_NO_ERROR SipHash is correct
	ESL_ERC_PARAMETER_INVALID Input parameter is NULL
	ESL_ERC_WS_TOO_SMALL Work space too small
	ESL_ERC_WS_STATE_INVALID Work space state invalid
	ESL_ERC_HMAC_INCORRECT_MAC the given HashMAC is incorrect
Functional Description	
Verifies given SipHash against calculated one. This function verifies given SipHash against calculated one.	
Particularities and Limitations	
Pointers must be valid	
Call context	
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces	

Table 4-167 esl_verifySipHash

4.2.2.4 Asymmetric Signature Algorithms

4.2.2.4.1 RSA Based Signature Algorithms

4.2.2.4.1.1 esl_initSignRSASHA1_V15

Prototype	
`eslt_ErrorCode` **`esl_initSignRSASHA1_V15`** `(VSECPRIM_P2VAR_PARA(eslt_WorkSpaceRSAsig) workSpace, eslt_Length keyPairModuleSize, eslt_pRomByte keyPairModule, eslt_Length privateKeyExponentSize, eslt_pRomByte privateKeyExponent)`	
Parameter	
privateKeyExponentSize [in]	length of the privateKeyExponent (bytes)
keyPairModuleSize [in]	length of the keyPairModulus (bytes)
privateKeyExponent [in]	pointer to the private key exponent
workSpace [in,out]	algorithm context buffer, initialized by esl_initWorkSpaceHeader(..) function
keyPairModule [in]	pointer to the key modulus
Return code	
eslt_ErrorCode	ESL_ERC_NO_ERROR Operation successful
	ESL_ERC_WS_TOO_SMALL Work space too small
	ESL_ERC_PARAMETER_INVALID - format problems, nullpointer, ...
	ESL_ERC_RSA_PRIVKEY_INVALID The private exponent is invalid
	ESL_ERC_RSA_MODULE_OUT_OF_RANGE the given module is not supported the given module is not supported
	- if keyPairModuleSize < 46d = headLen+hLen+11, where
	+ hLen is the length of the Hashvalue (SHA-1 20d= 0x14)
	+ headLen is the AlgoIDentifier (RSA+SHA1 15d=0x0f)
Functional Description	
Initialize Signature for RSA PKCS#1 V1.5 with SHA-1.	
Particularities and Limitations	
workSpace is initialized by esl_initWorkSpaceHeader(..) privateKeyExponent < keyPairModule Module is a prime	
Call context	
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces	

Table 4-168 esl_initSignRSASHA1_V15

4.2.2.4.1.2 esl_updateSignRSASHA1_V15

Prototype	
`eslt_ErrorCode` **`esl_updateSignRSASHA1_V15`** `(VSECPRIM_P2VAR_PARA(eslt_WorkSpaceRSAsig) workSpace, eslt_Length inputSize, VSECPRIM_P2CONST_PARA(eslt_Byte) input)`	
Parameter	
input [in]	pointer to the input data
workSpace [in,out]	algorithm context buffer, initialized by esl_initWorkSpaceHeader(..) function
inputSize [in]	size of the input data (bytes)
Return code	
eslt_ErrorCode	ESL_ERC_NO_ERROR Operation successful
	ESL_ERC_WS_STATE_INVALID Invalid state
	ESL_ERC_PARAMETER_INVALID - format problems, nullpointer, ...
	ESL_ERC_SHA1_TOTAL_LENGTH_OVERFLOW total input length overflow
Functional Description	
Update Signature for RSA PKCS#1 V1.5 with SHA-1.	
Particularities and Limitations	
workSpace is initialized by esl_initSignRSASHA1_V15(..)	
Call context	
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces	

Table 4-169 esl_updateSignRSASHA1_V15

4.2.2.4.1.3 esl_finalizeSignRSASHA1_V15

Prototype	
`eslt_ErrorCode` **`esl_finalizeSignRSASHA1_V15`** `(VSECPRIM_P2VAR_PARA(eslt_WorkSpaceRSAsig) workSpace, VSECPRIM_P2VAR_PARA(eslt_Length) signatureSize, VSECPRIM_P2VAR_PARA(eslt_Byte) signature)`	
Parameter	
signature [out]	pointer to the signature
workSpace [in,out]	algorithm context buffer, initialized by esl_initWorkSpaceHeader(..) function
signatureSize [out]	length of the signature (bytes)
Return code	
eslt_ErrorCode	ESL_ERC_NO_ERROR Operation successful
	ESL_ERC_WS_STATE_INVALID Invalid state
	ESL_ERC_PARAMETER_INVALID - format problems, nullpointer, ...

	ESL_ERC_OUTPUT_SIZE_TOO_SHORT outputSize too small
Functional Description	
Finalize Signature for RSA PKCS#1 V1.5 with SHA-1.	
Particularities and Limitations	
workSpace is initialized by esl_initSignRSASHA1_V15(..)	
Call context	
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces	

Table 4-170 esl_finalizeSignRSASHA1_V15

4.2.2.4.1.4 esl_initSignRSACRTSHA1_V15

Prototype

```
eslt_ErrorCode esl_initSignRSACRTSHA1_V15
(VSECPRIM_P2VAR_PARA(eslt_WorkSpaceRSACRTsig) workSpace, eslt_Length
keyPairPrimePSize, eslt_pRomByte keyPairPrimeP, eslt_Length keyPairPrimeQSize,
eslt_pRomByte keyPairPrimeQ, eslt_Length privateKeyExponentDPSize,
eslt_pRomByte privateKeyExponentDP, esltt_Length privateKeyExponentDQSize,
eslt_pRomByte privateKeyExponentDQ, eslt_Length privateKeyInverseQISize,
eslt_pRomByte privateKeyInverseQI)
```

Parameter	
keyPairPrimeQSize [in]	length of the keyPairPrimeQ (bytes)
keyPairPrimeP [in]	pointer to the 1st key prime
privateKeyInverseQISize [in]	length of the privateKeyInverseQI (bytes)
privateKeyInverseQI [in]	pointer to the inverse
privateKeyExponentDPSize [in]	length of the privKeyExponenentDP (bytes)
keyPairPrimeQ [in]	pointer to the 2nd key prime
workSpace [in,out]	algorithm context buffer, initialized by esl_initWorkSpaceHeader(..) function
privateKeyExponentDQSize [in]	length of the privKeyExponenentDQ (bytes)
privateKeyExponentDP [in]	pointer to the partial privKey exponent
privateKeyExponentDQ [in]	pointer to the partial privKey exponent
keyPairPrimePSize [in]	length of the keyPairPrimeP (bytes)
Return code	
eslt_ErrorCode	ESL_ERC_NO_ERROR Operation successful
	ESL_ERC_WS_TOO_SMALL Work space too small
	ESL_ERC_PARAMETER_INVALID - format problems, nullpointer, ...
	ESL_ERC_RSA_PRIVKEY_INVALID The private exponent is invalid

	ESL_ERC_RSA_MODULE_OUT_OF_RANGE the given module is not supported
	- keyPairModuleSize < 46d = headLen+hLen+11, where
	+ hLen is the length of the Hashvalue (SHA-1 20d= 0x14)
	+ headLen is the AlgoIDentifier (RSA+SHA1 15d=0x0f)
Functional Description	
Initialize Signature for RSA PKCS#1 V1.5 CRT with SHA-1.	
Particularities and Limitations	
workSpace is initialized by esl_initWorkSpaceHeader(..) PrimeP and PrimeQ are a prime privateKeyExpDP<PrimeP privateKeyExpDQ<PrimeQ QI is inverse to Q, could be checked, then move this line to errorcodes the Hash-Function is SHA-1	
Call context	
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces	

Table 4-171 esl_initSignRSACRTSHA1_V15

4.2.2.4.1.5 esl_updateSignRSACRTSHA1_V15

Prototype	
`eslt_ErrorCode` **`esl_updateSignRSACRTSHA1_V15`** `(VSECPRIM_P2VAR_PARA(eslt_WorkSpaceRSACRTsig) workSpace, const eslt_Length inputSize, VSECPRIM_P2CONST_PARA(eslt_Byte) input)`	
Parameter	
input [in]	pointer to the input data
workSpace [in,out]	algorithm context buffer, initialized by esl_initWorkSpaceHeader(..) function
inputSize [in]	size of the input data (bytes)
Return code	
eslt_ErrorCode	ESL_ERC_NO_ERROR Operation successful
	ESL_ERC_WS_STATE_INVALID Invalid state
	ESL_ERC_PARAMETER_INVALID - format problems, nullpointer, ...
	ESL_ERC_SHA1_TOTAL_LENGTH_OVERFLOW total input length overflow
Functional Description	
Update Signature for RSA PKCS#1 V1.5 CRT with SHA-1.	
Particularities and Limitations	
workSpace is initialized by esl_initSignRSACRTSHA1_V15(..)	
Call context	
> TASK > This function is Synchronous	

> This function is Reentrant, for different workspaces

Table 4-172 esl_updateSignRSACRTSHA1_V15

4.2.2.4.1.6 esl_finalizeSignRSACRTSHA1_V15

Prototype	
`eslt_ErrorCode` **`esl_finalizeSignRSACRTSHA1_V15`** `(VSECPRIM_P2VAR_PARA(eslt_WorkSpaceRSACRTsig) workSpace, VSECPRIM_P2VAR_PARA(eslt_Length) signatureSize, VSECPRIM_P2VAR_PARA(eslt_Byte) signature)`	
Parameter	
signature [out]	pointer to the signature
workSpace [in,out]	algorithm context buffer, initialized by esl_initWorkSpaceHeader(..) function
signatureSize [out]	length of the signature (bytes)
Return code	
eslt_ErrorCode	ESL_ERC_NO_ERROR Operation successful
	ESL_ERC_WS_STATE_INVALID Invalid state
	ESL_ERC_PARAMETER_INVALID - format problems, nullpointer, ...
	ESL_ERC_OUTPUT_SIZE_TOO_SHORT outputSize too small
Functional Description	
Finalize Signature for RSA PKCS#1 V1.5 CRT with SHA-1.	
Particularities and Limitations	
workSpace is initialized by esl_initSignRSACRTSHA1_V15(..)	
Call context	
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces	

Table 4-173 esl_finalizeSignRSACRTSHA1_V15

4.2.2.4.1.7 esl_initVerifyRSASHA1_V15

Prototype	
`eslt_ErrorCode` **`esl_initVerifyRSASHA1_V15`** `(VSECPRIM_P2VAR_PARA(eslt_WorkSpaceRSAver) workSpace, eslt_Length keyPairModuleSize, eslt_pRomByte keyPairModule, eslt_Length publicKeyExponentSize, eslt_pRomByte publicKeyExponent)`	
Parameter	
publicKeyExponentSize [in]	length of the public publicKeyExponent (bytes)
keyPairModuleSize [in]	length of the keyPairModulus (bytes)

publicKeyExponent [in]	pointer to the public exponent
workSpace [in,out]	algorithm context buffer, initialized by esl_initWorkSpaceHeader(..) function
keyPairModule [in]	pointer to the key modulus
Return code	
eslt_ErrorCode	ESL_ERC_NO_ERROR Operation successful
	ESL_ERC_WS_TOO_SMALL Work space too small
	ESL_ERC_WS_STATE_INVALID Invalid state
	ESL_ERC_PARAMETER_INVALID - format problems, nullpointer, ...
Functional Description	
Initialize signature verification for RSA PKCS#1 V1.5 with SHA-1.	
Particularities and Limitations	
workSpace is initialized by esl_initWorkSpaceHeader(..) publicKeyExponent < keyPairModule	
Call context	
> TASK	
> This function is Synchronous	
> This function is Reentrant, for different workspaces	

Table 4-174 esl_initVerifyRSASHA1_V15

4.2.2.4.1.8 esl_updateVerifyRSASHA1_V15

Prototype	
`eslt_ErrorCode esl_updateVerifyRSASHA1_V15 (VSECPRIM_P2VAR_PARA(eslt_WorkSpaceRSAver) workSpace, eslt_Length inputSize, VSECPRIM_P2CONST_PARA(eslt_Byte) input)`	
Parameter	
input [in]	pointer to the input data
workSpace [in,out]	algorithm context buffer, initialized by esl_initWorkSpaceHeader(..) function
inputSize [in]	size of the input data (bytes)
Return code	
eslt_ErrorCode	ESL_ERC_NO_ERROR Operation successful
	ESL_ERC_WS_STATE_INVALID Invalid state
	ESL_ERC_PARAMETER_INVALID - format problems, nullpointer, ...
	ESL_ERC_SHA1_TOTAL_LENGTH_OVERFLOW total input length overflow
Functional Description	
Update signature verification for RSA PKCS#1 V1.5 with SHA-1.	
Particularities and Limitations	
workSpace is initialized by esl_initVerifyRSASHA1_V15(..)	
Call context	

> TASK > This function is Synchronous > This function is Reentrant, for different workspaces

Table 4-175 esl_updateVerifyRSASHA1_V15

4.2.2.4.1.9 esl_finalizeVerifyRSASHA1_V15

Prototype	
`eslt_ErrorCode` **`esl_finalizeVerifyRSASHA1_V15`** `(VSECPRIM_P2VAR_PARA(eslt_WorkSpaceRSAver) workSpace, eslt_Length signatureSize, VSECPRIM_P2CONST_PARA(eslt_Byte) signature)`	
Parameter	
signature [in]	pointer to the signature
workSpace [in,out]	algorithm context buffer, initialized by esl_initWorkSpaceHeader(..) function
signatureSize [in]	length of the signature (bytes)
Return code	
eslt_ErrorCode	ESL_ERC_NO_ERROR Operation successful
	ESL_ERC_WS_TOO_SMALL Work space too small
	ESL_ERC_WS_STATE_INVALID Invalid state
	ESL_ERC_PARAMETER_INVALID - format problems, nullpointer, ...
	ESL_ERC_RSA_MODULE_OUT_OF_RANGE modulus too short
	ESL_ERC_RSA_SIGNATURE_OUT_OF_RANGE - 0 <= (int) signature <= module-1, where module = PrimeP*PrimeQ
	ESL_ERC_RSA_ENCODING_INVALID Invalid padding
	ESL_ERC_RSA_PUBKEY_INVALID The public exponent is invalid
	ESL_ERC_RSA_SIGNATURE_INVALID Signature verification failed
Functional Description	
Finalize hash and signature for signature verification for RSA PKCS#1 V1.5 with SHA-1.	
Particularities and Limitations	
workSpace is initialized by esl_initVerifyRSASHA1_V15(..)	
Call context	
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces	

Table 4-176 esl_finalizeVerifyRSASHA1_V15

4.2.2.4.1.10 esl_finalizeHashVerifyRSASHA1_V15

Prototype	

`eslt_ErrorCode` **`esl_finalizeHashVerifyRSASHA1_V15`**
`(VSECPRIM_P2VAR_PARA(eslt_WorkSpaceRSAver) workSpace,`
`VSECPRIM_P2VAR_PARA(eslt_Byte) messageDigest)`	

Parameter	
messageDigest [in,out]	pointer to computed hash value
workSpace [in,out]	algorithm context buffer, initialized by esl_initWorkSpaceHeader(..) function
Return code	
eslt_ErrorCode	ESL_ERC_NO_ERROR Operation successful
	ESL_ERC_WS_TOO_SMALL Work space too small
	ESL_ERC_WS_STATE_INVALID Invalid state
	ESL_ERC_PARAMETER_INVALID - format problems, nullpointer, ...
Functional Description	
Finalize hash for signature verification for RSA PKCS#1 V1.5 with SHA-1.	
Particularities and Limitations	
workSpace is initialized by esl_initVerifyRSASHA1_V15(..)	
Call context	
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces	

Table 4-177 esl_finalizeHashVerifyRSASHA1_V15

4.2.2.4.1.11 esl_verifySigVerifyRSASHA1_V15

Prototype	

`eslt_ErrorCode` **`esl_verifySigVerifyRSASHA1_V15`**
`(VSECPRIM_P2VAR_PARA(eslt_WorkSpaceRSAver) workSpace,`
`VSECPRIM_P2CONST_PARA(eslt_Byte) messageDigest, eslt_Length signatureSize,`
`VSECPRIM_P2CONST_PARA(eslt_Byte) signature)`	

Parameter	
signature [in]	pointer to the signature
messageDigest [in]	pointer to computed hash value
workSpace [in,out]	algorithm context buffer, initialized by esl_initWorkSpaceHeader(..) function
signatureSize [in]	length of the signature (bytes)
Return code	
eslt_ErrorCode	ESL_ERC_NO_ERROR Operation successful
	ESL_ERC_WS_TOO_SMALL Work space too small
	ESL_ERC_WS_STATE_INVALID Invalid state

	ESL_ERC_PARAMETER_INVALID - format problems, nullpointer, ...
	ESL_ERC_RSA_MODULE_OUT_OF_RANGE modulus too short
	ESL_ERC_RSA_SIGNATURE_OUT_OF_RANGE - 0 <= (int) signature <= module-1, where module = PrimeP*PrimeQ
	ESL_ERC_RSA_ENCODING_INVALID Invalid padding
	ESL_ERC_RSA_PUBKEY_INVALID The public exponent is invalid
	ESL_ERC_RSA_SIGNATURE_INVALID Signature verification failed
Functional Description	
Finalize signature for signature verification for RSA PKCS#1 V1.5 with SHA-1.	
Particularities and Limitations	
workSpace is initialized by esl_initVerifyRSASHA1_V15(..)	
Call context	
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces	

Table 4-178 esl_verifySigVerifyRSASHA1_V15

4.2.2.4.1.12 esl_initSignRSASHA256_V15

Prototype	
`eslt_ErrorCode` **`esl_initSignRSASHA256_V15`** `(VSECPRIM_P2VAR_PARA(eslt_WorkSpaceRSAsig) workSpace, eslt_Length keyPairModuleSize, eslt_pRomByte keyPairModule, eslt_Length privateKeyExponentSize, eslt_pRomByte privateKeyExponent)`	
Parameter	
privateKeyExponentSize [in]	length of the privateKeyExponent (bytes)
keyPairModuleSize [in]	length of the keyPairModulus (bytes)
privateKeyExponent [in]	pointer to the private key exponent
workSpace [in,out]	algorithm context buffer, initialized by esl_initWorkSpaceHeader(..) function
keyPairModule [in]	pointer to the key modulus
Return code	
eslt_ErrorCode	ESL_ERC_NO_ERROR Operation successful
	ESL_ERC_WS_TOO_SMALL Work space too small
	ESL_ERC_PARAMETER_INVALID - format problems, nullpointer, ...
	ESL_ERC_RSA_PRIVKEY_INVALID The private exponent is invalid
	ESL_ERC_RSA_MODULE_OUT_OF_RANGE the given module is not supported
	- if keyPairModuleSize < 46d = headLen+hLen+11, where

	+ hLen is the length of the Hashvalue (SHA-256 32d= 0x20)
	+ headLen is the AlgoIDentifier (RSA+SHA256 19d=0x13)
Functional Description	
Initialize Signature for RSA PKCS#1 V1.5 with SHA-256.	
Particularities and Limitations	
workSpace is initialized by esl_initWorkSpaceHeader(..) privateKeyExponent < keyPairModule Module is a prime	
Call context	
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces	

Table 4-179 esl_initSignRSASHA256_V15

4.2.2.4.1.13 esl_updateSignRSASHA256_V15

Prototype	
`eslt_ErrorCode` **`esl_updateSignRSASHA256_V15`** `(VSECPRIM_P2VAR_PARA(eslt_WorkSpaceRSAsig) workSpace, eslt_Length inputSize, VSECPRIM_P2CONST_PARA(eslt_Byte) input)`	
Parameter	
input [in]	pointer to the input data
workSpace [in,out]	algorithm context buffer, initialized by esl_initWorkSpaceHeader(..) function
inputSize [in]	size of the input data (bytes)
Return code	
eslt_ErrorCode	ESL_ERC_NO_ERROR Operation successful
	ESL_ERC_WS_STATE_INVALID Invalid state
	ESL_ERC_PARAMETER_INVALID - format problems, nullpointer, ...
	ESL_ERC_SHA256_TOTAL_LENGTH_OVERFLOW Maximum total input length exceeded
Functional Description	
Update Signature for RSA PKCS#1 V1.5 with SHA-256.	
Particularities and Limitations	
workSpace is initialized by esl_initSignRSASHA256_V15(..)	
Call context	
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces	

Table 4-180 esl_updateSignRSASHA256_V15

4.2.2.4.1.14 esl_finalizeSignRSASHA256_V15

Prototype	
`eslt_ErrorCode` **`esl_finalizeSignRSASHA256_V15`** `(VSECPRIM_P2VAR_PARA(eslt_WorkSpaceRSAsig) workSpace, VSECPRIM_P2VAR_PARA(eslt_Length) signatureSize, VSECPRIM_P2VAR_PARA(eslt_Byte) signature)`	
Parameter	
signature [out]	pointer to the signature
workSpace [in,out]	algorithm context buffer, initialized by esl_initWorkSpaceHeader(..) function
signatureSize [out]	length of the signature (bytes)
Return code	
eslt_ErrorCode	ESL_ERC_NO_ERROR Operation successful
	ESL_ERC_WS_STATE_INVALID Invalid state
	ESL_ERC_PARAMETER_INVALID - format problems, nullpointer, ...
	ESL_ERC_OUTPUT_SIZE_TOO_SHORT outputSize too small
Functional Description	
Finalize Signature for RSA PKCS#1 V1.5 with SHA-256.	
Particularities and Limitations	
workSpace is initialized by esl_initSignRSASHA256_V15(..)	
Call context	
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces	

Table 4-181 esl_finalizeSignRSASHA256_V15

4.2.2.4.1.15 esl_initSignRSACRTSHA256_V15

Prototype	
`eslt_ErrorCode` **`esl_initSignRSACRTSHA256_V15`** `(VSECPRIM_P2VAR_PARA(eslt_WorkSpaceRSACRTsig) workSpace, eslt_Length keyPairPrimePSize, eslt_pRomByte keyPairPrimeP, eslt_Length keyPairPrimeQSize, eslt_pRomByte keyPairPrimeQ, eslt_Length privateKeyExponentDPSize, eslt_pRomByte privateKeyExponentDP, esltt_Length privateKeyExponentDQSize, eslt_pRomByte privateKeyExponentDQ, eslt_Length privateKeyInverseQISize, eslt_pRomByte privateKeyInverseQI)`	
Parameter	
keyPairPrimeQSize [in]	length of the keyPairPrimeQ (bytes)
keyPairPrimeP [in]	pointer to the 1st key prime
privateKeyInverseQISize [in]	length of the privateKeyInverseQI (bytes)

privateKeyInverseQI [in]	pointer to the inverse
privateKeyExponentDPSize [in]	length of the privKeyExponenentDP (bytes)
keyPairPrimeQ [in]	pointer to the 2nd key prime
workSpace [in,out]	algorithm context buffer, initialized by esl_initWorkSpaceHeader(..) function
privateKeyExponentDQSize [in]	length of the privKeyExponenentDQ (bytes)
privateKeyExponentDP [in]	pointer to the partial privKey exponent
privateKeyExponentDQ [in]	pointer to the partial privKey exponent
keyPairPrimePSize [in]	length of the keyPairPrimeP (bytes)
Return code	
eslt_ErrorCode	ESL_ERC_NO_ERROR Operation successful
	ESL_ERC_WS_TOO_SMALL Work space too small
	ESL_ERC_PARAMETER_INVALID - format problems, nullpointer, ...
	ESL_ERC_RSA_PRIVKEY_INVALID The private exponent is invalid
	ESL_ERC_RSA_MODULE_OUT_OF_RANGE the given module is not supported
	- keyPairModuleSize < 46d = headLen+hLen+11, where
	+ hLen is the length of the Hashvalue (SHA-256 32d= 0x20)
	+ headLen is the AlgoIDentifier (RSA+SHA256 19d=0x13)
Functional Description	
Initialize Signature for RSA PKCS#1 V1.5 CRT with SHA-256.	
Particularities and Limitations	
workSpace is initialized by esl_initWorkSpaceHeader(..) PrimeP and PrimeQ are a prime privateKeyExpDP<PrimeP privateKeyExpDQ<PrimeQ QI is inverse to Q, could be checked, then move this line to errorcodes the Hash-Function is SHA-1	
Call context	
> TASK	
> This function is Synchronous	
> This function is Reentrant, for different workspaces	

Table 4-182 esl_initSignRSACRTSHA256_V15

4.2.2.4.1.16 esl_updateSignRSACRTSHA256_V15

Prototype
`eslt_ErrorCode` **`esl_updateSignRSACRTSHA256_V15`** `(VSECPRIM_P2VAR_PARA(eslt_WorkSpaceRSACRTsig) workSpace, const eslt_Length inputSize, VSECPRIM_P2CONST_PARA(eslt_Byte) input)`
Parameter
input [in] — pointer to the input data

workSpace [in,out]	algorithm context buffer, initialized by esl_initWorkSpaceHeader(..) function
inputSize [in]	size of the input data (bytes)
Return code	
eslt_ErrorCode	ESL_ERC_NO_ERROR Operation successful
	ESL_ERC_WS_STATE_INVALID Invalid state
	ESL_ERC_PARAMETER_INVALID - format problems, nullpointer, ...
	ESL_ERC_SHA256_TOTAL_LENGTH_OVERFLOW Maximum total input length exceeded
Functional Description	
Update Signature for RSA PKCS#1 V1.5 CRT with SHA-256.	
Particularities and Limitations	
workSpace is initialized by esl_initSignRSACRTSHA256_V15(..)	
Call context	
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces	

Table 4-183 esl_updateSignRSACRTSHA256_V15

4.2.2.4.1.17 esl_finalizeSignRSACRTSHA256_V15

Prototype	
`eslt_ErrorCode` **`esl_finalizeSignRSACRTSHA256_V15`** `(VSECPRIM_P2VAR_PARA(eslt_WorkSpaceRSACRTsig) workSpace, VSECPRIM_P2VAR_PARA(eslt_Length) signatureSize, VSECPRIM_P2VAR_PARA(eslt_Byte) signature)`	
Parameter	
signature [out]	pointer to the signature
workSpace [in,out]	algorithm context buffer, initialized by esl_initWorkSpaceHeader(..) function
signatureSize [out]	length of the signature (bytes)
Return code	
eslt_ErrorCode	ESL_ERC_NO_ERROR Operation successful
	ESL_ERC_WS_STATE_INVALID Invalid state
	ESL_ERC_PARAMETER_INVALID - format problems, nullpointer, ...
	ESL_ERC_OUTPUT_SIZE_TOO_SHORT outputSize too small
Functional Description	
Finalize Signature for RSA PKCS#1 V1.5 CRT with SHA-256.	
Particularities and Limitations	
workSpace is initialized by esl_initSignRSACRTSHA256_V15(..)	
Call context	

> TASK > This function is Synchronous > This function is Reentrant, for different workspaces

Table 4-184 esl_finalizeSignRSACRTSHA256_V15

4.2.2.4.1.18 esl_initVerifyRSASHA256_V15

Prototype	
`eslt_ErrorCode` **`esl_initVerifyRSASHA256_V15`** `(VSECPRIM_P2VAR_PARA(eslt_WorkSpaceRSAver) workSpace, eslt_Length keyPairModuleSize, eslt_pRomByte keyPairModule, eslt_Length publicKeyExponentSize, eslt_pRomByte publicKeyExponent)`	
Parameter	
publicKeyExponentSize [in]	length of the public publicKeyExponent (bytes)
keyPairModuleSize [in]	length of the keyPairModulus (bytes)
publicKeyExponent [in]	pointer to the public exponent
workSpace [in,out]	algorithm context buffer, initialized by esl_initWorkSpaceHeader(..) function
keyPairModule [in]	pointer to the key modulus
Return code	
eslt_ErrorCode	ESL_ERC_NO_ERROR Operation successful
	ESL_ERC_WS_TOO_SMALL Work space too small
	ESL_ERC_WS_STATE_INVALID Invalid state
	ESL_ERC_PARAMETER_INVALID - format problems, nullpointer, ...
	ESL_ERC_RSA_PUBKEY_INVALID The public exponent is invalid
Functional Description	
Initialize signature verification for RSA PKCS#1 V1.5 with SHA-256.	
Particularities and Limitations	
workSpace is initialized by esl_initWorkSpaceHeader(..) publicKeyExponent < keyPairModule	
Call context	
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces	

Table 4-185 esl_initVerifyRSASHA256_V15

4.2.2.4.1.19 esl_updateVerifyRSASHA256_V15

Prototype	
`eslt_ErrorCode` **`esl_updateVerifyRSASHA256_V15`** `(VSECPRIM_P2VAR_PARA(eslt_WorkSpaceRSAver) workSpace, eslt_Length inputSize, VSECPRIM_P2CONST_PARA(eslt_Byte) input)`	
Parameter	
input [in]	pointer to the input data
workSpace [in,out]	algorithm context buffer, initialized by esl_initWorkSpaceHeader(..) function
inputSize [in]	size of the input data (bytes)
Return code	
eslt_ErrorCode	ESL_ERC_NO_ERROR Operation successful
	ESL_ERC_WS_STATE_INVALID Invalid state
	ESL_ERC_PARAMETER_INVALID - format problems, nullpointer, ...
	ESL_ERC_SHA256_TOTAL_LENGTH_OVERFLOW Maximum total input length exceeded
Functional Description	
Update signature verification for RSA PKCS#1 V1.5 with SHA-256.	
Particularities and Limitations	
workSpace is initialized by esl_initVerifyRSASHA256_V15(..)	
Call context	
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces	

Table 4-186 esl_updateVerifyRSASHA256_V15

4.2.2.4.1.20 esl_finalizeHashVerifyRSASHA256_V15

Prototype	
`eslt_ErrorCode` **`esl_finalizeHashVerifyRSASHA256_V15`** `(VSECPRIM_P2VAR_PARA(eslt_WorkSpaceRSAver) workSpace, VSECPRIM_P2VAR_PARA(eslt_Byte) messageDigest)`	
Parameter	
messageDigest [in,out]	pointer to computed hash value
workSpace [in,out]	algorithm context buffer, initialized by esl_initWorkSpaceHeader(..) function
Return code	
eslt_ErrorCode	ESL_ERC_NO_ERROR Operation successful
	ESL_ERC_WS_TOO_SMALL Work space too small
	ESL_ERC_WS_STATE_INVALID Invalid state
	ESL_ERC_PARAMETER_INVALID - format problems, nullpointer, ...

Functional Description
Finalize hash for signature verification for RSA PKCS#1 V1.5 with SHA-256.
Particularities and Limitations
workSpace is initialized by esl_initVerifyRSASHA256_V15(..)
Call context
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces

Table 4-187 esl_finalizeHashVerifyRSASHA256_V15

4.2.2.4.1.21 esl_verifySigVerifyRSASHA256_V15

Prototype	

`eslt_ErrorCode` **`esl_verifySigVerifyRSASHA256_V15`**
`(VSECPRIM_P2VAR_PARA(eslt_WorkSpaceRSAver) workSpace,`
`VSECPRIM_P2CONST_PARA(eslt_Byte) messageDigest, eslt_Length signatureSize,`
`VSECPRIM_P2CONST_PARA(eslt_Byte) signature)`	

Parameter	
signature [in]	pointer to the signature
messageDigest [in]	pointer to computed hash value
workSpace [in,out]	algorithm context buffer, initialized by esl_initWorkSpaceHeader(..) function
signatureSize [in]	length of the signature (bytes)
Return code	
eslt_ErrorCode	ESL_ERC_NO_ERROR Operation successful
	ESL_ERC_WS_TOO_SMALL Work space too small
	ESL_ERC_WS_STATE_INVALID Invalid state
	ESL_ERC_PARAMETER_INVALID - format problems, nullpointer, ...
	ESL_ERC_RSA_MODULE_OUT_OF_RANGE modulus too short
	ESL_ERC_RSA_SIGNATURE_OUT_OF_RANGE - 0 <= (int) signature <= module-1, where module = PrimeP*PrimeQ
	ESL_ERC_RSA_ENCODING_INVALID Invalid padding
	ESL_ERC_RSA_PUBKEY_INVALID The public exponent is invalid
	ESL_ERC_RSA_SIGNATURE_INVALID Signature verification failed
Functional Description	
Finalize signature for signature verification for RSA PKCS#1 V1.5 with SHA-256.	
Particularities and Limitations	
workSpace is initialized by esl_initVerifyRSASHA256_V15(..)	
Call context	

> TASK > This function is Synchronous > This function is Reentrant, for different workspaces

Table 4-188 esl_verifySigVerifyRSASHA256_V15

4.2.2.4.1.22 esl_finalizeVerifyRSASHA256_V15

Prototype	
`eslt_ErrorCode` **`esl_finalizeVerifyRSASHA256_V15`** `(VSECPRIM_P2VAR_PARA(eslt_WorkSpaceRSAver) workSpace, eslt_Length signatureSize, VSECPRIM_P2CONST_PARA(eslt_Byte) signature)`	
Parameter	
signature [in]	pointer to the signature
workSpace [in,out]	algorithm context buffer, initialized by esl_initWorkSpaceHeader(..) function
signatureSize [in]	length of the signature (bytes)
Return code	
eslt_ErrorCode	ESL_ERC_NO_ERROR Operation successful
	ESL_ERC_WS_TOO_SMALL Work space too small
	ESL_ERC_WS_STATE_INVALID Invalid state
	ESL_ERC_PARAMETER_INVALID - format problems, nullpointer, ...
	ESL_ERC_RSA_MODULE_OUT_OF_RANGE modulus too short
	ESL_ERC_RSA_SIGNATURE_OUT_OF_RANGE - 0 <= (int) signature <= module-1, where module = PrimeP*PrimeQ
	ESL_ERC_RSA_ENCODING_INVALID Invalid padding
	ESL_ERC_RSA_PUBKEY_INVALID The public exponent is invalid
	ESL_ERC_RSA_SIGNATURE_INVALID Signature verification failed
Functional Description	
Finalize hash and signature for signature verification for RSA PKCS#1 V1.5 with SHA-256.	
Particularities and Limitations	
workSpace is initialized by esl_initVerifyRSASHA256_V15(..)	
Call context	
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces	

Table 4-189 esl_finalizeVerifyRSASHA256_V15

4.2.2.4.1.23 esl_initSignRSARIPEMD160_V15

Prototype
`eslt_ErrorCode` **`esl_initSignRSARIPEMD160_V15`** `(VSECPRIM_P2VAR_PARA(eslt_WorkSpaceRSAsig) workSpace, eslt_Length`

keyPairModuleSize, eslt_pRomByte keyPairModule, eslt_Length privateKeyExponentSize, eslt_pRomByte privateKeyExponent)	
Parameter	
privateKeyExponentSize [in]	length of the privateKeyExponent (bytes)
keyPairModuleSize [in]	length of the keyPairModulus (bytes)
privateKeyExponent [in]	pointer to the private key exponent
workSpace [in,out]	algorithm context buffer, initialized by esl_initWorkSpaceHeader(..) function
keyPairModule [in]	pointer to the key modulus
Return code	
eslt_ErrorCode	ESL_ERC_NO_ERROR Operation successful
	ESL_ERC_WS_TOO_SMALL Work space too small
	ESL_ERC_PARAMETER_INVALID - format problems, nullpointer, ...
	ESL_ERC_RSA_PRIVKEY_INVALID The private exponent is invalid
	ESL_ERC_RSA_MODULE_OUT_OF_RANGE the given module is not supported
	- if keyPairModuleSize < 46d = headLen+hLen+11, where
	+ hLen is the length of the Hashvalue (RIPEMD160 20d= 0x14)
	+ headLen is the AlgoIDentifier (RSA+RIPEMD160 15d=0x0f)
Functional Description	
Initialize Signature for RSA PKCS#1 V1.5 with RIPEMD160.	
Particularities and Limitations	
workSpace is initialized by esl_initWorkSpaceHeader(..) privateKeyExponent < keyPairModule Module is a prime	
Call context	
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces	

Table 4-190 esl_initSignRSARIPEMD160_V15

4.2.2.4.1.24 esl_updateSignRSARIPEMD160_V15

Prototype	
eslt_ErrorCode **esl_updateSignRSARIPEMD160_V15** (VSECPRIM_P2VAR_PARA(eslt_WorkSpaceRSAsig) workSpace, eslt_Length inputSize, VSECPRIM_P2CONST_PARA(eslt_Byte) input)	
Parameter	
input [in]	pointer to the input data
workSpace [in,out]	algorithm context buffer, initialized by esl_initWorkSpaceHeader(..) function
inputSize [in]	size of the input data (bytes)

Return code	
eslt_ErrorCode	ESL_ERC_NO_ERROR Operation successful
	ESL_ERC_WS_STATE_INVALID Invalid state
	ESL_ERC_PARAMETER_INVALID - format problems, nullpointer, ...
	ESL_ERC_RIPEMD160_TOTAL_LENGTH_OVERFLOW Maximum total input length exceeded
Functional Description	
Update Signature for RSA PKCS#1 V1.5 with RIPEMD160.	
Particularities and Limitations	
workSpace is initialized by esl_initSignRSARIPEMD160_V15(..)	
Call context	
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces	

Table 4-191 esl_updateSignRSARIPEMD160_V15

4.2.2.4.1.25 esl_finalizeSignRSARIPEMD160_V15

Prototype	
`eslt_ErrorCode` **`esl_finalizeSignRSARIPEMD160_V15`** `(VSECPRIM_P2VAR_PARA(eslt_WorkSpaceRSAsig) workSpace, VSECPRIM_P2VAR_PARA(eslt_Length) signatureSize, VSECPRIM_P2VAR_PARA(eslt_Byte) signature)`	
Parameter	
signature [out]	pointer to the signature
workSpace [in,out]	algorithm context buffer, initialized by esl_initWorkSpaceHeader(..) function
signatureSize [out]	length of the signature (bytes)
Return code	
eslt_ErrorCode	ESL_ERC_NO_ERROR Operation successful
	ESL_ERC_WS_STATE_INVALID Invalid state
	ESL_ERC_PARAMETER_INVALID - format problems, nullpointer, ...
	ESL_ERC_OUTPUT_SIZE_TOO_SHORT outputSize too small
Functional Description	
Finalize Signature for RSA PKCS#1 V1.5 with RIPEMD160.	
Particularities and Limitations	
workSpace is initialized by esl_initSignRSARIPEMD160_V15(..)	
Call context	
> TASK > This function is Synchronous	

> This function is Reentrant, for different workspaces

Table 4-192 esl_finalizeSignRSARIPEMD160_V15

4.2.2.4.1.26 esl_initSignRSACRTRIPEMD160_V15

Prototype
`eslt_ErrorCode` **`esl_initSignRSACRTRIPEMD160_V15`** `(VSECPRIM_P2VAR_PARA(eslt_WorkSpaceRSACRTsig) workSpace, eslt_Length keyPairPrimePSize, eslt_pRomByte keyPairPrimeP, eslt_Length keyPairPrimeQSize, eslt_pRomByte keyPairPrimeQ, eslt_Length privateKeyExponentDPSize, eslt_pRomByte privateKeyExponentDP, esltt_Length privateKeyExponentDQSize, eslt_pRomByte privateKeyExponentDQ, eslt_Length privateKeyInverseQISize, eslt_pRomByte privateKeyInverseQI)`

Parameter	
keyPairPrimeQSize [in]	length of the keyPairPrimeQ (bytes)
keyPairPrimeP [in]	pointer to the 1st key prime
privateKeyInverseQISize [in]	length of the privateKeyInverseQI (bytes)
privateKeyInverseQI [in]	pointer to the inverse
privateKeyExponentDPSize [in]	length of the privKeyExponenentDP (bytes)
keyPairPrimeQ [in]	pointer to the 2nd key prime
workSpace [in,out]	algorithm context buffer, initialized by esl_initWorkSpaceHeader(..) function
privateKeyExponentDQSize [in]	length of the privKeyExponenentDQ (bytes)
privateKeyExponentDP [in]	pointer to the partial privKey exponent
privateKeyExponentDQ [in]	pointer to the partial privKey exponent
keyPairPrimePSize [in]	length of the keyPairPrimeP (bytes)

Return code	
eslt_ErrorCode	ESL_ERC_NO_ERROR Operation successful
	ESL_ERC_WS_TOO_SMALL Work space too small
	ESL_ERC_PARAMETER_INVALID - format problems, nullpointer, ...
	ESL_ERC_RSA_PRIVKEY_INVALID The private exponent is invalid
	ESL_ERC_RSA_MODULE_OUT_OF_RANGE the given module is not supported
	- keyPairModuleSize < 46d = headLen+hLen+11, where
	+ hLen is the length of the Hashvalue (RIPEMD160 20d= 0x14)
	+ headLen is the AlgoIDentifier (RSA+RIPEMD160 15d=0x0f)

Functional Description
Initialize Signature for RSA PKCS#1 V1.5 CRT with RIPEMD160.

Particularities and Limitations
workSpace is initialized by esl_initWorkSpaceHeader(..) PrimeP and PrimeQ are a prime privateKeyExpDP<PrimeP privateKeyExpDQ<PrimeQ QI is inverse to Q, could be checked, then move this line to errorcodes the Hash-Function is SHA-1
Call context
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces

Table 4-193 esl_initSignRSACRTRIPEMD160_V15

4.2.2.4.1.27 esl_updateSignRSACRTRIPEMD160_V15

Prototype	
`eslt_ErrorCode` **`esl_updateSignRSACRTRIPEMD160_V15`** `(VSECPRIM_P2VAR_PARA(eslt_WorkSpaceRSACRTsig) workSpace, const eslt_Length inputSize, VSECPRIM_P2CONST_PARA(eslt_Byte) input)`	
Parameter	
input [in]	pointer to the input data
workSpace [in,out]	algorithm context buffer, initialized by esl_initWorkSpaceHeader(..) function
inputSize [in]	size of the input data (bytes)
Return code	
eslt_ErrorCode	ESL_ERC_NO_ERROR Operation successful
	ESL_ERC_WS_STATE_INVALID Invalid state
	ESL_ERC_PARAMETER_INVALID - format problems, nullpointer, ...
	ESL_ERC_RIPEMD160_TOTAL_LENGTH_OVERFLOW Maximum total input length exceeded
Functional Description	
Update Signature for RSA PKCS#1 V1.5 CRT with RIPEMD160.	
Particularities and Limitations	
workSpace is initialized by esl_initSignRSACRTRIPEMD160_V15(..)	
Call context	
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces	

Table 4-194 esl_updateSignRSACRTRIPEMD160_V15

4.2.2.4.1.28 esl_finalizeSignRSACRTRIPEMD160_V15

Prototype	
`eslt_ErrorCode` **`esl_finalizeSignRSACRTRIPEMD160_V15`** `(VSECPRIM_P2VAR_PARA(eslt_WorkSpaceRSACRTsig) workSpace, VSECPRIM_P2VAR_PARA(eslt_Length) signatureSize, VSECPRIM_P2VAR_PARA(eslt_Byte) signature)`	
Parameter	
signature [out]	pointer to the signature
workSpace [in,out]	algorithm context buffer, initialized by esl_initWorkSpaceHeader(..) function
signatureSize [out]	length of the signature (bytes)
Return code	
eslt_ErrorCode	ESL_ERC_NO_ERROR Operation successful
	ESL_ERC_WS_STATE_INVALID Invalid state
	ESL_ERC_PARAMETER_INVALID - format problems, nullpointer, ...
	ESL_ERC_OUTPUT_SIZE_TOO_SHORT outputSize too small
Functional Description	
Finalize Signature for RSA PKCS#1 V1.5 CRT with RIPEMD160.	
Particularities and Limitations	
workSpace is initialized by esl_initSignRSACRTRIPEMD160_V15(..)	
Call context	
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces	

Table 4-195 esl_finalizeSignRSACRTRIPEMD160_V15

4.2.2.4.1.29 esl_initVerifyRSARIPEMD160_V15

Prototype	
`eslt_ErrorCode` **`esl_initVerifyRSARIPEMD160_V15`** `(VSECPRIM_P2VAR_PARA(eslt_WorkSpaceRSAver) workSpace, eslt_Length keyPairModuleSize, eslt_pRomByte keyPairModule, eslt_Length publicKeyExponentSize, eslt_pRomByte publicKeyExponent)`	
Parameter	
publicKeyExponentSize [in]	length of the public publicKeyExponent (bytes)
keyPairModuleSize [in]	length of the keyPairModulus (bytes)
publicKeyExponent [in]	pointer to the public exponent
workSpace [in,out]	algorithm context buffer, initialized by esl_initWorkSpaceHeader(..) function
keyPairModule [in]	pointer to the key modulus

Return code	
eslt_ErrorCode	ESL_ERC_NO_ERROR Operation successful
	ESL_ERC_WS_TOO_SMALL Work space too small
	ESL_ERC_WS_STATE_INVALID Invalid state
	ESL_ERC_PARAMETER_INVALID - format problems, nullpointer, ...
	ESL_ERC_RSA_PUBKEY_INVALID The public exponent is invalid
Functional Description	
Initialize signature verification for RSA PKCS#1 V1.5 with RIPEMD160.	
Particularities and Limitations	
workSpace is initialized by esl_initWorkSpaceHeader(..) publicKeyExponent < keyPairModule	
Call context	
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces	

Table 4-196 esl_initVerifyRSARIPEMD160_V15

4.2.2.4.1.30 esl_updateVerifyRSARIPEMD160_V15

Prototype	
`eslt_ErrorCode esl_updateVerifyRSARIPEMD160_V15 (VSECPRIM_P2VAR_PARA(eslt_WorkSpaceRSAver) workSpace, eslt_Length inputSize, VSECPRIM_P2CONST_PARA(eslt_Byte) input)`	
Parameter	
input [in]	pointer to the input data
workSpace [in,out]	algorithm context buffer, initialized by esl_initWorkSpaceHeader(..) function
inputSize [in]	size of the input data (bytes)
Return code	
eslt_ErrorCode	ESL_ERC_NO_ERROR Operation successful
	ESL_ERC_WS_STATE_INVALID Invalid state
	ESL_ERC_PARAMETER_INVALID - format problems, nullpointer, ...
	ESL_ERC_RIPEMD160_TOTAL_LENGTH_OVERFLOW Maximum total input length exceeded
Functional Description	
Update signature verification for RSA PKCS#1 V1.5 with RIPEMD160.	
Particularities and Limitations	
workSpace is initialized by esl_initVerifyRSARIPEMD160_V15(..)	
Call context	
> TASK	

> This function is Synchronous > This function is Reentrant, for different workspaces

Table 4-197 esl_updateVerifyRSARIPEMD160_V15

4.2.2.4.1.31 esl_finalizeHashVerifyRSARIPEMD160_V15

Prototype	

`eslt_ErrorCode` **`esl_finalizeHashVerifyRSARIPEMD160_V15`**
`(VSECPRIM_P2VAR_PARA(eslt_WorkSpaceRSAver) workSpace,`
`VSECPRIM_P2VAR_PARA(eslt_Byte) messageDigest)`	

Parameter	
messageDigest [in,out]	pointer to computed hash value
workSpace [in,out]	algorithm context buffer, initialized by esl_initWorkSpaceHeader(..) function
Return code	
eslt_ErrorCode	ESL_ERC_NO_ERROR Operation successful
	ESL_ERC_WS_TOO_SMALL Work space too small
	ESL_ERC_WS_STATE_INVALID Invalid state
	ESL_ERC_PARAMETER_INVALID - format problems, nullpointer, ...
Functional Description	
Finalize hash for signature verification for RSA PKCS#1 V1.5 with RIPEMD160.	
Particularities and Limitations	
workSpace is initialized by esl_initVerifyRSARIPEMD160_V15(..)	
Call context	
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces	

Table 4-198 esl_finalizeHashVerifyRSARIPEMD160_V15

4.2.2.4.1.32 esl_verifySigVerifyRSARIPEMD160_V15

Prototype	

`eslt_ErrorCode` **`esl_verifySigVerifyRSARIPEMD160_V15`**
`(VSECPRIM_P2VAR_PARA(eslt_WorkSpaceRSAver) workSpace,`
`VSECPRIM_P2CONST_PARA(eslt_Byte) messageDigest, eslt_Length signatureSize,`
`VSECPRIM_P2CONST_PARA(eslt_Byte) signature)`	

Parameter	
signature [in]	pointer to the signature
messageDigest [in]	pointer to computed hash value
workSpace [in,out]	algorithm context buffer, initialized by esl_initWorkSpaceHeader(..) function

signatureSize [in]	length of the signature (bytes)
Return code	
eslt_ErrorCode	ESL_ERC_NO_ERROR Operation successful
	ESL_ERC_WS_TOO_SMALL Work space too small
	ESL_ERC_WS_STATE_INVALID Invalid state
	ESL_ERC_PARAMETER_INVALID - format problems, nullpointer, ...
	ESL_ERC_RSA_MODULE_OUT_OF_RANGE modulus too short
	ESL_ERC_RSA_SIGNATURE_OUT_OF_RANGE - 0 <= (int) signature <= module-1, where module = PrimeP*PrimeQ
	ESL_ERC_RSA_ENCODING_INVALID Invalid padding
	ESL_ERC_RSA_PUBKEY_INVALID The public exponent is invalid
	ESL_ERC_RSA_SIGNATURE_INVALID Signature verification failed
Functional Description	
Finalize signature for signature verification for RSA PKCS#1 V1.5 with RIPEMD160.	
Particularities and Limitations	
workSpace is initialized by esl_initVerifyRSARIPEMD160_V15(..)	
Call context	
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces	

Table 4-199 esl_verifySigVerifyRSARIPEMD160_V15

4.2.2.4.1.33 esl_finalizeVerifyRSARIPEMD160_V15

Prototype
`eslt_ErrorCode` **`esl_finalizeVerifyRSARIPEMD160_V15`** `(VSECPRIM_P2VAR_PARA(eslt_WorkSpaceRSAver) workSpace, eslt_Length signatureSize, VSECPRIM_P2CONST_PARA(eslt_Byte) signature)`

Parameter	
signature [in]	pointer to the signature
workSpace [in,out]	algorithm context buffer, initialized by esl_initWorkSpaceHeader(..) function
signatureSize [in]	length of the signature (bytes)
Return code	
eslt_ErrorCode	ESL_ERC_NO_ERROR Operation successful
	ESL_ERC_WS_TOO_SMALL Work space too small
	ESL_ERC_WS_STATE_INVALID Invalid state
	ESL_ERC_PARAMETER_INVALID - format problems, nullpointer, ...
	ESL_ERC_RSA_MODULE_OUT_OF_RANGE modulus too short

	ESL_ERC_RSA_SIGNATURE_OUT_OF_RANGE - 0 <= (int) signature <= module-1, where module = PrimeP*PrimeQ
	ESL_ERC_RSA_ENCODING_INVALID Invalid padding
	ESL_ERC_RSA_PUBKEY_INVALID The public exponent is invalid
	ESL_ERC_RSA_SIGNATURE_INVALID Signature verification failed
Functional Description	
Finalize hash and signature for signature verification for RSA PKCS#1 V1.5 with RIPEMD160.	
Particularities and Limitations	
workSpace is initialized by esl_initVerifyRSARIPEMD160_V15(..)	
Call context	
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces	

Table 4-200 esl_finalizeVerifyRSARIPEMD160_V15

4.2.2.4.1.34 esl_verifyPaddingRSAEM_V15

Prototype	
`eslt_ErrorCode esl_verifyPaddingRSAEM_V15 (VSECPRIM_P2CONST_PARA(eslt_Byte) message, VSECPRIM_P2VAR_PARA(eslt_Length) paddingLength, eslt_Byte blockType)`	
Parameter	
paddingLength [in,out]	(in) Length of message (out) Length of padding
message [in]	Decrypted message
blockType [in]	Block type according to PKCS #1 1.5 ASN1_PADDING_BLOCK_TYPE_PRIVATE_ZERO ASN1_PADDING_BLOCK_TYPE_PRIVATE ASN1_PADDING_BLOCK_TYPE_PUBLIC
Return code	
eslt_ErrorCode	ESL_ERC_PARAMETER_INVALID Message length smaller than minimum padding
	ESL_ERC_RSA_ENCODING_INVALID Invalid padding
	ESL_ERC_NO_ERROR else
Functional Description	
Verify padding for RSA PKCS#1 V1.5.	
Particularities and Limitations	
Work space initialized	
Call context	
> TASK	

> This function is Synchronous > This function is Reentrant, for different workspaces

Table 4-201 esl_verifyPaddingRSAEM_V15

4.2.2.4.1.35 esl_initSignRSASHA1_PSS

Prototype	
`eslt_ErrorCode` **`esl_initSignRSASHA1_PSS`** `(VSECPRIM_P2VAR_PARA(eslt_WorkSpaceRSAPSSsig) workSpace, eslt_Length keyPairModuleSize, eslt_pRomByte keyPairModule, eslt_Length privateKeyExponentSize, eslt_pRomByte privateKeyExponent)`	
Parameter	
workSpace [in,out]	Signature generation work space
keyPairModuleSize [in]	Size of RSA key pair module
keyPairModule [in]	RSA key pair module
privateKeyExponentSize [in]	Size of private RSA key pair exponent
privateKeyExponent [in]	Private RSA key pair exponent
Return code	
eslt_ErrorCode	ESL_ERC_NO_ERROR Operation successful
	ESL_ERC_PARAMETER_INVALID Input parameter is NULL
	ESL_ERC_WS_TOO_SMALL Work space too small
	ESL_ERC_RSA_PRIVKEY_INVALID The private exponent is invalid
Functional Description	
Initialize signature generation with EMSA PSS encoding, based on SHA1	
Particularities and Limitations	
-	
Call context	
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces	

Table 4-202 esl_initSignRSASHA1_PSS

4.2.2.4.1.36 esl_updateSignRSASHA1_PSS

Prototype
`eslt_ErrorCode` **`esl_updateSignRSASHA1_PSS`** `(VSECPRIM_P2VAR_PARA(eslt_WorkSpaceRSAPSSsig) workSpace, eslt_Length inputSize, VSECPRIM_P2CONST_PARA(eslt_Byte) input)`

Parameter	
workSpace [in,out]	Signature generation work space
inputSize [in]	Size of data to be hashed
input [in]	Data to be hashed
Return code	
eslt_ErrorCode	ESL_ERC_NO_ERROR Operation successful
	ESL_ERC_PARAMETER_INVALID Input parameter is NULL
	ESL_ERC_WS_STATE_INVALID Invalid state
	ESL_ERC_WS_TOO_SMALL Work space too small
	ESL_ERC_SHA1_TOTAL_LENGTH_OVERFLOW Maximum total input length exceeded
Functional Description	
Update hash workspace with message to be signed During the RSA PSS signature generation scheme, the original message needs to be hashed. For RSA PSS SHA1, the hash function is SHA1. This function stores the message into the hash workspace,contained in the parameter workSpace.	
Particularities and Limitations	
workSpace is initialized by esl_initSignRSASHA1_PSS(..)	
Call context	
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces	

Table 4-203 esl_updateSignRSASHA1_PSS

4.2.2.4.1.37 esl_finalizeHashSignRSASHA1_PSS

Prototype

`eslt_ErrorCode` **`esl_finalizeHashSignRSASHA1_PSS`**
`(VSECPRIM_P2VAR_PARA(eslt_WorkSpaceRSAPSSsig) workSpace,`
`VSECPRIM_P2VAR_PARA(eslt_Byte) messageDigest)`

Parameter	
workSpace [in,out]	Signature generation work space
messageDigest [in,out]	Message digest (hash) of previously hashed data
Return code	
eslt_ErrorCode	ESL_ERC_NO_ERROR Operation successful
	ESL_ERC_PARAMETER_INVALID Input parameter is NULL
	ESL_ERC_WS_STATE_INVALID Invalid state
	ESL_ERC_WS_TOO_SMALL Work space too small
Functional Description	
Calculate the hash value of the data in hash workspace	

During the RSA PSS signature generation scheme, the original message needs to be hashed. For RSA PSS SHA1, the hash function is SHA1. The message was passed to the hash workSpace before, which is contained in the parameter 'workSpace'. This function computes its hash value.
Particularities and Limitations
workSpace is initialized by esl_initSignRSASHA1_PSS(..), esl_updateSignRSASHA1(..) called before The buffer referenced by messageDigest needs to be at least ESL_SIZEOF_SHA1_DIGEST (= 20) byte long, since this buffer is passed to actSHAFinalize. The buffer referenced by messageDigest needs to be at least ESL_SIZEOF_SHA1_DIGEST (= 20) byte long.
Call context
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces

Table 4-204 esl_finalizeHashSignRSASHA1_PSS

4.2.2.4.1.38 esl_finalizeSigSignRSASHA1_PSS

Prototype

```
eslt_ErrorCode esl_finalizeSigSignRSASHA1_PSS
(VSECPRIM_P2VAR_PARA(eslt_WorkSpaceRSAPSSsig) workSpace,
VSECPRIM_P2CONST_PARA(eslt_Byte) messageDigest, eslt_Length saltSize,
VSECPRIM_P2CONST_PARA(eslt_Byte) salt, VSECPRIM_P2VAR_PARA(eslt_Length)
signatureSize, VSECPRIM_P2VAR_PARA(eslt_Byte) signature)
```

Parameter	
workSpace [in,out]	Signature generation work space
messageDigest [in]	Message digest (hash) of previously hashed original message
saltSize [in]	Size of salt to be used
salt [in]	Salt to be used 0 (NULL) if salt should be generated randomly
signatureSize [in,out]	Input Maximum size of signature Output Actual size of signature
signature [out]	Generated signature value
Return code	
	ESL_ERC_NO_ERROR Operation successful
	ESL_ERC_PARAMETER_INVALID Input parameter is null or saltSize too high for salt to be stored in data block
	ESL_ERC_WS_STATE_INVALID Invalid state
	ESL_ERC_WS_TOO_SMALL Work space too small
	all possible error codes which can occur in esl_getBytesRNG
	ESL_ERC_SHA1_TOTAL_LENGTH_OVERFLOW total length of data to be hashed (while calculating the saltedHashValue) is too long
	ESL_ERC_INPUT_INVALID hashID that is passed to esl_calcSaltedHashRSA_PSS does not correspond to a supported hash function
	ESL_ERC_ERROR an error occured during the call to esl_generateMaskMGF1

	ESL_ERC_RSA_MESSAGE_OUT_OF_RANGE message to be signed is greater than the module
	ESL_ERC_OUTPUT_SIZE_TOO_SHORT *signatureSize is too small
	ESL_ERC_RSA_ENCODING_INVALID RSA encoded message length too short
Functional Description	
Generates the RSA PSS SHA1 signature by encoding and signing the message.	
Particularities and Limitations	
workSpace is initialized by esl_initSignRSASHA1_PSS(..), esl_updateSignRSASHA1(..) and esl_finalizeHashSignRSASHA1_PSS(..) called before The buffer referenced by signature needs to be at least signatureSize bytes long. The buffer referenced by messageDigest needs to be at least ESL_SIZEOF_SHA1_DIGEST (= 20) byte long.	
Call context	
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces	

Table 4-205 esl_finalizeSigSignRSASHA1_PSS

4.2.2.4.1.39 esl_finalizeSigSignRSASHA1_PSS_NoSalt

Prototype	
`eslt_ErrorCode` **`esl_finalizeSigSignRSASHA1_PSS_NoSalt`** `(VSECPRIM_P2VAR_PARA(eslt_WorkSpaceRSAPSSsig) workSpace, VSECPRIM_P2CONST_PARA(eslt_Byte) messageDigest, VSECPRIM_P2VAR_PARA(eslt_Length) signatureSize, VSECPRIM_P2VAR_PARA(eslt_Byte) signature)`	
Parameter	
workSpace [in,out]	Signature generation work space
messageDigest [in]	Message digest (hash) of previously hashed original message
signatureSize [in,out]	Input Maximum size of signature Output Actual size of signature
signature [out]	Generated signature value
Return code	
eslt_ErrorCode	ESL_ERC_NO_ERROR Operation successful
	all possible error codes which can occur in esl_finalizeSigSignRSASHA_PSS
Functional Description	
Generates the RSA PSS SHA1 signature by encoding and signing the message without using salt. This function calls esl_finalizeSigSignRSASHA1_PSS() with the given input parameters, saltSize = 0 and a null pointer for salt.	
Particularities and Limitations	
workSpace is initialized by esl_initSignRSASHA1_PSS(..), esl_updateSignRSASHA1(..) and	

esl_finalizeHashSignRSASHA1_PSS(..) called before The buffer referenced by signature needs to be at least signatureSize bytes long. The buffer referenced by messageDigest needs to be at least ESL_SIZEOF_SHA1_DIGEST (= 20) byte long.
Call context
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces

Table 4-206 esl_finalizeSigSignRSASHA1_PSS_NoSalt

4.2.2.4.1.40 esl_finalizeSigSignRSASHA1_PSS_DigestLengthSalt

Prototype	
`eslt_ErrorCode` **`esl_finalizeSigSignRSASHA1_PSS_DigestLengthSalt`** `(VSECPRIM_P2VAR_PARA(eslt_WorkSpaceRSAPSSsig) workSpace, VSECPRIM_P2CONST_PARA(eslt_Byte) messageDigest, VSECPRIM_P2VAR_PARA(eslt_Length) signatureSize, VSECPRIM_P2VAR_PARA(eslt_Byte) signature)`	
Parameter	
workSpace [in,out]	Signature generation work space
messageDigest [in]	Message digest (hash) of previously hashed original message
signatureSize [in,out]	Input Maximum size of signature Output Actual size of signature
signature [out]	Generated signature value
Return code	
eslt_ErrorCode	ESL_ERC_NO_ERROR Operation successful
	all possible error codes which can occur in esl_finalizeSigSignRSASHA_PSS
Functional Description	
Generates the RSA PSS SHA1 signature by encoding and signing the message with random salt. This function calls esl_finalizeSigSignRSASHA1_PSS() with the given input parameters, saltSize equal to message digest length and a null pointer for salt, which means that during encoding the message, a salt buffer of length ESL_SIZEOF_SHA1_DIGEST with random data will be generated.	
Particularities and Limitations	
workSpace is initialized by esl_initSignRSASHA1_PSS(..), esl_updateSignRSASHA1(..) and esl_finalizeHashSignRSASHA1_PSS(..) called before The buffer referenced by signature needs to be at least signatureSize bytes long. The buffer referenced by messageDigest needs to be at least ESL_SIZEOF_SHA1_DIGEST (= 20) byte long.	
Call context	
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces	

Table 4-207 esl_finalizeSigSignRSASHA1_PSS_DigestLengthSalt

4.2.2.4.1.41 esl_finalizeSignRSASHA1_PSS

Prototype	
`eslt_ErrorCode` **`esl_finalizeSignRSASHA1_PSS`** `(VSECPRIM_P2VAR_PARA(eslt_WorkSpaceRSAPSSsig) workSpace, eslt_Length saltSize, VSECPRIM_P2CONST_PARA(eslt_Byte) salt, VSECPRIM_P2VAR_PARA(eslt_Length) signatureSize, VSECPRIM_P2VAR_PARA(eslt_Byte) signature)`	
Parameter	
workSpace [in,out]	Signature generation work space
saltSize [in]	Size of salt to be used
salt [in]	Salt to be used 0 (NULL) if salt should be generated randomly
signatureSize [in,out]	Input Maximum size of signature Output Actual size of signature
signature [out]	Generated signature value
Return code	
eslt_ErrorCode	ESL_ERC_NO_ERROR Operation successful
	all possible error codes which can occur in esl_finalizeSigSignRSASHA_PSS
Functional Description	
Finalize message digest (hash) calculation and generate signature for RSA PSS SHA1 This function calls esl_finalizeHashSignRSASHA1_PSS() and esl_finalizeSigSignRSASHA1_PSS() with the given input parameters.	
Particularities and Limitations	
workSpace is initialized by esl_initSignRSASHA1_PSS(..), esl_updateSignRSASHA1(..) called before. The buffer referenced by signature needs to be at least signatureSize bytes long.	
Call context	
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces	

Table 4-208 esl_finalizeSignRSASHA1_PSS

4.2.2.4.1.42 esl_finalizeSignRSASHA1_PSS_NoSalt

Prototype	
`eslt_ErrorCode` **`esl_finalizeSignRSASHA1_PSS_NoSalt`** `(VSECPRIM_P2VAR_PARA(eslt_WorkSpaceRSAPSSsig) workSpace, VSECPRIM_P2VAR_PARA(eslt_Length) signatureSize, VSECPRIM_P2VAR_PARA(eslt_Byte) signature)`	
Parameter	
workSpace [in,out]	Signature generation work space
signatureSize [in,out]	Input Maximum size of signature Output Actual size of signature
signature [out]	Generated signature value

Return code	
eslt_ErrorCode	ESL_ERC_NO_ERROR Operation successful
	All possible error codes which can occur in esl_finalizeSignRSASHA256()
Functional Description	
Finalize message digest (hash) calculation and generate signature for RSA PSS SHA1 This function calls esl_finalizeSignRSASHA1_PSS() with the given input parameters, saltSize = 0 and a null pointer for salt.	
Particularities and Limitations	
workSpace is initialized by esl_initSignRSASHA1_PSS(..), esl_updateSignRSASHA1(..) called before. The buffer referenced by signature needs to be at least signatureSize bytes long.	
Call context	
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces	

Table 4-209 esl_finalizeSignRSASHA1_PSS_NoSalt

4.2.2.4.1.43 esl_finalizeSignRSASHA1_PSS_DigestLengthSalt

Prototype	
`eslt_ErrorCode esl_finalizeSignRSASHA1_PSS_DigestLengthSalt (VSECPRIM_P2VAR_PARA(eslt_WorkSpaceRSAPSSsig) workSpace, VSECPRIM_P2VAR_PARA(eslt_Length) signatureSize, VSECPRIM_P2VAR_PARA(eslt_Byte) signature)`	
Parameter	
workSpace [in,out]	Signature generation work space
signatureSize [in,out]	Input Maximum size of signature Output Actual size of signature
signature [out]	Generated signature value
Return code	
eslt_ErrorCode	ESL_ERC_NO_ERROR Operation successful
	all possible error codes which can occur in esl_finalizeSignRSASHA1_PSS
Functional Description	
Finalize message digest (hash) calculation and generate signature for RSA PSS SHA1 This function calls esl_finalizeSignRSASHA1_PSS()with a null pointer for salt and saltSize equal to message digest length for signature generation	
Particularities and Limitations	
workSpace is initialized by esl_initSignRSASHA1_PSS(..), esl_updateSignRSASHA1(..) called before. The buffer referenced by signature needs to be at least signatureSize bytes long.	
Call context	
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces	

Table 4-210 esl_finalizeSignRSASHA1_PSS_DigestLengthSalt

4.2.2.4.1.44 esl_initVerifyRSASHA1_PSS

Prototype	
`eslt_ErrorCode` **`esl_initVerifyRSASHA1_PSS`** `(VSECPRIM_P2VAR_PARA(eslt_WorkSpaceRSAPSSver) workSpace, eslt_Length keyPairModuleSize, eslt_pRomByte keyPairModule, eslt_Length publicKeyExponentSize, eslt_pRomByte publicKeyExponent)`	
Parameter	
workSpace [in,out]	Signature verification work space
keyPairModuleSize [in]	Size of RSA key pair module
keyPairModule [in]	RSA key pair module
publicKeyExponentSize [in]	Size of public RSA key pair exponent
publicKeyExponent [in]	Public RSA key pair exponent
Return code	
eslt_ErrorCode	ESL_ERC_NO_ERROR Operation successful
	ESL_ERC_PARAMETER_INVALID Input parameter is NULL
	ESL_ERC_WS_TOO_SMALL Work space too small
	ESL_ERC_RSA_PUBKEY_INVALID The public exponent is invalid
Functional Description	
Initialize signature verification with EMSA PSS encoding, based on SHA1	
Particularities and Limitations	
-	
Call context	
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces	

Table 4-211 esl_initVerifyRSASHA1_PSS

4.2.2.4.1.45 esl_updateVerifyRSASHA1_PSS

Prototype	
`eslt_ErrorCode` **`esl_updateVerifyRSASHA1_PSS`** `(VSECPRIM_P2VAR_PARA(eslt_WorkSpaceRSAPSSver) workSpace, eslt_Length inputSize, VSECPRIM_P2CONST_PARA(eslt_Byte) input)`	
Parameter	
workSpace [in,out]	Signature verification work space
inputSize [in]	Size of data to be hashed

input [in]	Data to be hashed, usually the original message
Return code	
eslt_ErrorCode	ESL_ERC_NO_ERROR Operation successful
	ESL_ERC_PARAMETER_INVALID Input parameter is NULL
	ESL_ERC_WS_STATE_INVALID Invalid state
	ESL_ERC_WS_TOO_SMALL Work space too small
	ESL_ERC_SHA1_TOTAL_LENGTH_OVERFLOW Maximum total input length exceeded
Functional Description	
Update hash workspace with message to be signed During the RSA PSS signature verification scheme, the original message needs to be hashed. For RSA PSS SHA1, the hash function is SHA1. This function stores the message into the hash workspace, contained in the parameter workSpace.	
Particularities and Limitations	
workSpace is initialized by esl_initVerifyRSASHA1_PSS(..)	
Call context	
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces	

Table 4-212 esl_updateVerifyRSASHA1_PSS

4.2.2.4.1.46 esl_finalizeHashVerifyRSASHA1_PSS

Prototype	

`eslt_ErrorCode` **`esl_finalizeHashVerifyRSASHA1_PSS`**
`(VSECPRIM_P2VAR_PARA(eslt_WorkSpaceRSAPSSver) workSpace,`
`VSECPRIM_P2VAR_PARA(eslt_Byte) messageDigest)`	

Parameter	
workSpace [in,out]	Signature verification work space
messageDigest [out]	Message digest (hash) of original message
Return code	
eslt_ErrorCode	ESL_ERC_NO_ERROR Operation successful
	ESL_ERC_PARAMETER_INVALID Input parameter is NULL
	ESL_ERC_WS_STATE_INVALID Invalid state
	ESL_ERC_WS_TOO_SMALL Work space too small
Functional Description	
Calculate the hash value of the data in the hash workspace During the RSA PSS signature generation scheme, the original message needs to be hashed. For RSA PSS SHA1, the hash function is SHA1. The message was passed to the hash workSpace before, which is contained in the parameter 'workSpace'. This function computes its hash value.	

Particularities and Limitations
workSpace is initialized by esl_initVerifyRSASHA1_PSS(..), esl_updateVerifyRSASHA1(..) called before. The buffer referenced by messageDigest needs to be at least ESL_SIZEOF_SHA1_DIGEST (= 20) byte long, since this buffer is passed to actSHAFinalize.
Call context
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces

Table 4-213 esl_finalizeHashVerifyRSASHA1_PSS

4.2.2.4.1.47 esl_verifySigVerifyRSASHA1_PSS

Prototype	
`eslt_ErrorCode` **`esl_verifySigVerifyRSASHA1_PSS`** `(VSECPRIM_P2VAR_PARA(eslt_WorkSpaceRSAPSSver) workSpace, VSECPRIM_P2CONST_PARA(eslt_Byte) messageDigest, eslt_Length saltSize, eslt_Length signatureSize, VSECPRIM_P2CONST_PARA(eslt_Byte) signature)`	
Parameter	
workSpace [in,out]	Signature verification work space
messageDigest [in]	Message digest (hash) of previously hashed original message
saltSize [in]	Expected size of salt, -1 if size of salt used during signature generation should be evaluated automatically
signatureSize [in]	Size of signature
signature [in]	Signature value
Return code	
eslt_ErrorCode	ESL_ERC_NO_ERROR Operation successful
	ESL_ERC_PARAMETER_INVALID input parameter null or saltSize too high for salt to be stored in data block
	ESL_ERC_WS_TOO_SMALL workSpace too small
	ESL_ERC_WS_STATE_INVALID workSpace state invalid
	ESL_ERC_SHA1_TOTAL_LENGTH_OVERFLOW total length of data to be hashed (while calculation the saltedHashValue) is too long
	ESL_ERC_INPUT_INVALID hashID that is passed to esl_calcSaltedHashRSA_PSS does not correspond to a supported hash function
	ESL_ERC_ERROR an error occured during the call to esl_generateMaskMGF1
	ESL_ERC_RSA_SIGNATURE_OUT_OF_RANGE signature to be verified is greater than the module
	ESL_ERC_RSA_SIGNATURE_INVALID signature verification failed
	ESL_ERC_RSA_ENCODING_INVALID RSA encoded message length too short

Functional Description
Verifies the RSA PSS SHA1 signature by applying the RSA verification primitive to the signature and decoding the message.
Particularities and Limitations
workSpace is initialized by esl_initVerifyRSASHA1_PSS(..), esl_updateVerifyRSASHA1(..) and esl_finalizeHashVerifyRSASHA1_PSS(..) called before. The buffer referenced by messageDigest needs to be at least ESL_SIZEOF_SHA1_DIGEST (= 20) byte long.
Call context
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces

Table 4-214 esl_verifySigVerifyRSASHA1_PSS

4.2.2.4.1.48 esl_verifySigVerifyRSASHA1_PSS_AutoSalt

Prototype	

`eslt_ErrorCode` **`esl_verifySigVerifyRSASHA1_PSS_AutoSalt`**
`(VSECPRIM_P2VAR_PARA(eslt_WorkSpaceRSAPSSver) workSpace,`
`VSECPRIM_P2CONST_PARA(eslt_Byte) messageDigest, eslt_Length signatureSize,`
`VSECPRIM_P2CONST_PARA(eslt_Byte) signature)`	

Parameter	
workSpace [in,out]	Signature verification work space
messageDigest [in]	Message digest (hash) of previously hashed original message
signatureSize [in]	Size of signature
signature [in]	Signature value to be verified
Return code	
eslt_ErrorCode	all possible error codes which can occur in esl_verifySigVerifyRSASHA1_PSS
Functional Description	
Verifies the RSA PSS SHA1 signature by encoding and signing the message. This function calls esl_verifySigVerifyRSASHA1_PSS() with the given input parameters and saltSize = -1. This requires that the size of the salt used during signature generation was evaluated automatically.	
Particularities and Limitations	
workSpace is initialized by esl_initVerifyRSASHA1_PSS(..), esl_updateVerifyRSASHA1(..) and esl_finalizeHashVerifyRSASHA1_PSS(..) called before. The buffer referenced by messageDigest needs to be at least ESL_SIZEOF_SHA1_DIGEST (= 20) byte long.	
Call context	
> TASK > This function is Synchronous	

> This function is Reentrant, for different workspaces

Table 4-215 esl_verifySigVerifyRSASHA1_PSS_AutoSalt

4.2.2.4.1.49 esl_verifySigVerifyRSASHA1_PSS_NoSalt

Prototype	

`eslt_ErrorCode` **`esl_verifySigVerifyRSASHA1_PSS_NoSalt`**
`(VSECPRIM_P2VAR_PARA(eslt_WorkSpaceRSAPSSver) workSpace,`
`VSECPRIM_P2CONST_PARA(eslt_Byte) messageDigest, eslt_Length signatureSize,`
`VSECPRIM_P2CONST_PARA(eslt_Byte) signature)`	

Parameter	
workSpace [in,out]	Signature verification work space
messageDigest [in]	Message digest (hash) of previously hashed original message
signatureSize [in]	Size of signature
signature [in]	Signature value to be verified
Return code	
eslt_ErrorCode	all possible error codes which can occur in esl_verifySigVerifyRSASHA1_PSS
Functional Description	
Verifies the RSA PSS SHA1 signature by encoding and signing the message. This function calls esl_verifySigVerifyRSASHA1_PSS() with the given input parameters and saltSize = 0. This requires that no salt was used during signature generation.	
Particularities and Limitations	
workSpace is initialized by esl_initVerifyRSASHA1_PSS(..), esl_updateVerifyRSASHA1(..) and esl_finalizeHashVerifyRSASHA1_PSS(..) called before. The buffer referenced by messageDigest needs to be at least ESL_SIZEOF_SHA1_DIGEST (= 20) byte long.	
Call context	
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces	

Table 4-216 esl_verifySigVerifyRSASHA1_PSS_NoSalt

4.2.2.4.1.50 esl_verifySigVerifyRSASHA1_PSS_DigestLengthSalt

Prototype

`eslt_ErrorCode` **`esl_verifySigVerifyRSASHA1_PSS_DigestLengthSalt`**
`(VSECPRIM_P2VAR_PARA(eslt_WorkSpaceRSAPSSver) workSpace,`
`VSECPRIM_P2CONST_PARA(eslt_Byte) messageDigest, eslt_Length signatureSize,`
`VSECPRIM_P2CONST_PARA(eslt_Byte) signature)`

Parameter	
workSpace [in,out]	Signature verification work space
messageDigest [in]	Message digest (hash) of previously hashed original message
signatureSize [in]	Size of signature
signature [in]	Signature value to be verified
Return code	
eslt_ErrorCode	all possible error codes which can occur in esl_verifySigVerifyRSASHA1_PSS
Functional Description	
Verifies the RSA PSS SHA1 signature by encoding and signing the message. This function calls esl_verifySigVerifyRSASHA1_PSS() with the given input parameters and saltSize = ESL_SIZEOF_SHA1_DIGEST. This requires that salt with size equal to message digest length was used during signature generation.	
Particularities and Limitations	
workSpace is initialized by esl_initVerifyRSASHA1_PSS(..), esl_updateVerifyRSASHA1(..) and esl_finalizeHashVerifyRSASHA1_PSS(..) called before. The buffer referenced by messageDigest needs to be at least ESL_SIZEOF_SHA1_DIGEST (= 20) byte long.	
Call context	
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces	

Table 4-217 esl_verifySigVerifyRSASHA1_PSS_DigestLengthSalt

4.2.2.4.1.51 esl_finalizeVerifyRSASHA1_PSS

Prototype	
`eslt_ErrorCode` **`esl_finalizeVerifyRSASHA1_PSS`** `(VSECPRIM_P2VAR_PARA(eslt_WorkSpaceRSAPSSver) workSpace, eslt_Length saltSize, eslt_Length signatureSize, VSECPRIM_P2CONST_PARA(eslt_Byte) signature)`	
Parameter	
workSpace [in,out]	Signature verification work space
saltSize [in]	Expected size of salt, -1 if size of salt used during signature generation should be evaluated automatically
signatureSize [in]	Size of signature
signature [in]	Signature value to be verified
Return code	
eslt_ErrorCode	all possible error codes which can occur in esl_verifySigVerifyRSASHA1_PSS
Functional Description	
Finalize message digest (hash) calculation and verify signature for RSA PSS SHA1	

This function calls esl_finalizeHashVerifyRSASHA1_PSS() and esl_verifySigVerifyRSASHA1_PSS() with the given input parameters.
Particularities and Limitations
workSpace is initialized by esl_initVerifyRSASHA1_PSS(..), esl_updateVerifyRSASHA1(..) called before
Call context
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces

Table 4-218 esl_finalizeVerifyRSASHA1_PSS

4.2.2.4.1.52 esl_finalizeVerifyRSASHA1_PSS_AutoSalt

Prototype	
`eslt_ErrorCode` **`esl_finalizeVerifyRSASHA1_PSS_AutoSalt`** `(VSECPRIM_P2VAR_PARA(eslt_WorkSpaceRSAPSSver) workSpace, eslt_Length signatureSize, VSECPRIM_P2CONST_PARA(eslt_Byte) signature)`	
Parameter	
workSpace [in,out]	Signature verification work space
signatureSize [in]	Size of signature
signature [in]	Signature value value to be verified
Return code	
eslt_ErrorCode	all possible error codes which can occur in esl_verifySigVerifyRSASHA1_PSS
Functional Description	
Finalize message digest (hash) calculation and verify signature for RSA PSS SHA1 This function calls esl_finalizeVerifyRSASHA1_PSS() with the given input parameters and saltSize = -1, thus it assumes that the size of the salt used during signature generation is evaluated automatically.	
Particularities and Limitations	
workSpace is initialized by esl_initVerifyRSASHA1_PSS(..), esl_updateVerifyRSASHA1(..) called before	
Call context	
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces	

Table 4-219 esl_finalizeVerifyRSASHA1_PSS_AutoSalt

4.2.2.4.1.53 esl_finalizeVerifyRSASHA1_PSS_NoSalt

Prototype
`eslt_ErrorCode` **`esl_finalizeVerifyRSASHA1_PSS_NoSalt`** `(VSECPRIM_P2VAR_PARA(eslt_WorkSpaceRSAPSSver) workSpace, eslt_Length signatureSize, VSECPRIM_P2CONST_PARA(eslt_Byte) signature)`

Parameter	
workSpace [in,out]	Signature verification work space
signatureSize [in]	Size of signature
signature [in]	Signature value to be verified
Return code	
eslt_ErrorCode	all possible error codes which can occur in esl_verifySigVerifyRSASHA1_PSS
Functional Description	
Finalize message digest (hash) calculation and verify signature for RSA PSS SHA1 This function calls esl_finalizeVerifyRSASHA1_PSS() with the given input parameters and saltSize = 0, thus it requires that no salt was used during signature generation.	
Particularities and Limitations	
workSpace is initialized by esl_initVerifyRSASHA1_PSS(..), esl_updateVerifyRSASHA1(..) called before	
Call context	
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces	

Table 4-220 esl_finalizeVerifyRSASHA1_PSS_NoSalt

4.2.2.4.1.54 esl_finalizeVerifyRSASHA1_PSS_DigestLengthSalt

Prototype
`eslt_ErrorCode` **`esl_finalizeVerifyRSASHA1_PSS_DigestLengthSalt`** `(VSECPRIM_P2VAR_PARA(eslt_WorkSpaceRSAPSSver) workSpace, eslt_Length signatureSize, VSECPRIM_P2CONST_PARA(eslt_Byte) signature)`

Parameter	
workSpace [in,out]	Signature verification work space
signatureSize [in]	Size of signature
signature [in]	Signature value to be verified
Return code	
eslt_ErrorCode	all possible error codes which can occur in esl_verifySigVerifyRSASHA1_PSS
Functional Description	
Finalize message digest (hash) calculation and verify signature for RSA PSS SHA1 This function calls esl_finalizeVerifyRSASHA1_PSS() with the given input parameters and saltSize = ESL_SIZEOF_SHA1_DIGEST, thus it requires that a salt buffer of size equal to message digest was used during signature generation.	
Particularities and Limitations	
workSpace is initialized by esl_initVerifyRSASHA1_PSS(..), esl_updateVerifyRSASHA1(..) called before	
Call context	
> TASK	

> This function is Synchronous > This function is Reentrant, for different workspaces

Table 4-221 esl_finalizeVerifyRSASHA1_PSS_DigestLengthSalt

4.2.2.4.1.55 esl_calcSaltedHashRSASHA1_PSS

Prototype	
`eslt_ErrorCode esl_calcSaltedHashRSASHA1_PSS (VSECPRIM_P2VAR_PARA(eslt_WorkSpaceSHA1) wsHash, eslt_Length saltSize, VSECPRIM_P2CONST_PARA(eslt_Byte) salt, VSECPRIM_P2CONST_PARA(eslt_Byte) messageDigest, VSECPRIM_P2VAR_PARA(eslt_Byte) saltedHash)`	
Parameter	
wsHash [in,out]	Pointer to hash workspace
saltSize [in]	Length of salt
salt [in]	Input salt
messageDigest [in]	Input hash (SHA-1)
saltedHash [out]	Output buffer for calculated hash (SHA-1), has to be large enough to hold requested length
Return code	
eslt_ErrorCode	ESL_ERC_NO_ERROR Operation successful
	ESL_ERC_PARAMETER_INVALID wsHash or saltedHash null
	ESL_ERC_WS_TOO_SMALL wsHash too small
	ESL_ERC_SHA1_TOTAL_LENGTH_OVERFLOW total length of data to be hashed too long
Functional Description	
Calculate salted hash value (SHA-1). Based on a given hash and a random salt the hash of M' = (0x) 00 00 00 00 00 00 00 00 || hash || salt is calculated, as described in chapter 9.1.1, steps 5-6 and 9.1.2, steps 12-13 of PKCS#1 2.2	
Particularities and Limitations	
The buffer referenced by saltedHash needs to be at least ESL_SIZEOF_SHA1_DIGEST (=20) byte long The buffer referenced by messageDigest needs to be at least ESL_SIZEOF_SHA1_DIGEST (= 20) byte long.	
Call context	
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces	

Table 4-222 esl_calcSaltedHashRSASHA1_PSS

4.2.2.4.1.56 esl_generateMaskMGF1RSASHA1_PSS

Prototype	
`eslt_ErrorCode` **`esl_generateMaskMGF1RSASHA1_PSS`** `(VSECPRIM_P2VAR_PARA(eslt_WorkSpaceSHA1) wsHash, VSECPRIM_P2VAR_PARA(eslt_Byte) tempHash, eslt_Length seedLength, VSECPRIM_P2CONST_PARA(eslt_Byte) seed, eslt_Length maskLength, VSECPRIM_P2VAR_PARA(eslt_Byte) mask)`	
Parameter	
wsHash [in,out]	Pointer to hash workspace
tempHash [in,out]	Temporary buffer for intermediate hash, has to be large enough to hold complete hash value
seedLength [in]	Length input seed
seed [in]	Input seed from which mask is generated
maskLength [in]	Length of mask to be generated
mask [out]	Output buffer for generated mask, has to be large enough to hold requested length
Return code	
eslt_ErrorCode	ESL_ERC_NO_ERROR Operation successful
	ESL_ERC_PARAMETER_INVALID Input parameter is NULL
	ESL_ERC_WS_TOO_SMALL wsHash too small
	ESL_ERC_ERROR an error occurred
Functional Description	
Mask generation function (MGF1). MGF1 implementation as described in appendix B.2.1 of PKCS#1 2.2 Based on hash function (SHA-1) this function generates a pseudorandom output string of arbitrary length depending on an input string (seed) of variable length	
Particularities and Limitations	
buffer referenced by tempHash has to be at least of length ESL_SIZEOF_SHA1_DIGEST buffer referenced by mask has to be at least of length maskLength	
Call context	
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces	

Table 4-223 esl_generateMaskMGF1RSASHA1_PSS

4.2.2.4.1.57 esl_initSignRSASHA256_PSS

Prototype
`eslt_ErrorCode` **`esl_initSignRSASHA256_PSS`** `(VSECPRIM_P2VAR_PARA(eslt_WorkSpaceRSAPSSsig) workSpace, eslt_Length keyPairModuleSize, eslt_pRomByte keyPairModule, eslt_Length privateKeyExponentSize, eslt_pRomByte privateKeyExponent)`

Parameter	
workSpace [in,out]	Signature generation work space
keyPairModuleSize [in]	Size of RSA key pair module
keyPairModule [in]	RSA key pair module
privateKeyExponentSize [in]	Size of private RSA key pair exponent
privateKeyExponent [in]	Private RSA key pair exponent
Return code	
eslt_ErrorCode	ESL_ERC_NO_ERROR Operation successful
	ESL_ERC_PARAMETER_INVALID Input parameter workSpace is NULL
	ESL_ERC_RSA_PRIVKEY_INVALID The private exponent is invalid
	ESL_ERC_WS_TOO_SMALL workSpace too small
Functional Description	
Initialize signature generation with EMSA PSS encoding, based on SHA2-256	
Particularities and Limitations	
-	
Call context	
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces	

Table 4-224 esl_initSignRSASHA256_PSS

4.2.2.4.1.58 esl_updateSignRSASHA256_PSS

Prototype	
`eslt_ErrorCode esl_updateSignRSASHA256_PSS (VSECPRIM_P2VAR_PARA(eslt_WorkSpaceRSAPSSsig) workSpace, eslt_Length inputSize, VSECPRIM_P2CONST_PARA(eslt_Byte) input)`	
Parameter	
workSpace [in,out]	Signature generation work space
inputSize [in]	Size of data to be hashed
input [in]	Data to be hashed
Return code	
eslt_ErrorCode	ESL_ERC_NO_ERROR Operation successful
	ESL_ERC_PARAMETER_INVALID input parameter workSpace is null
	ESL_ERC_WS_TOO_SMALL workSpace too small
	ESL_ERC_WS_STATE_INVALID workSpace in invalid state
	ESL_ERC_SHA256_TOTAL_LENGTH_OVERFLOW Maximum total input length exceeded

Functional Description
Update hash workspace with message to be signed During the RSA PSS signature generation scheme, the original message needs to be hashed. For RSA PSS SHA256, the hash function is SHA2-256. This function stores the message into the hash workspace, contained in the parameter workSpace.
Particularities and Limitations
workSpace is initialized by esl_initSignRSASHA256_PSS(..)
Call context
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces

Table 4-225 esl_updateSignRSASHA256_PSS

4.2.2.4.1.59 esl_finalizeHashSignRSASHA256_PSS

Prototype	

`eslt_ErrorCode` **`esl_finalizeHashSignRSASHA256_PSS`**
`(VSECPRIM_P2VAR_PARA(eslt_WorkSpaceRSAPSSsig) workSpace,`
`VSECPRIM_P2VAR_PARA(eslt_Byte) messageDigest)`	

Parameter	
workSpace [in,out]	Signature generation work space
messageDigest [in,out]	buffer to store the message digest (hash)
Return code	
eslt_ErrorCode	ESL_ERC_NO_ERROR Operation successful
	ESL_ERC_PARAMETER_INVALID input parameter workSpace null
	ESL_ERC_WS_TOO_SMALL workSpace too small
	ESL_ERC_WS_STATE_INVALID workSpace in invalid state
Functional Description	
Calculate the hash value of the data previously passed to the hash workspace, contained in the parameter workSpace. During the RSA PSS signature generation scheme, the original message needs to be hashed. For RSA PSS SHA256, the hash function is SHA2-256. The message was passed to the hash workSpace before. This function computes its hash value.	
Particularities and Limitations	
workSpace is initialized by esl_initSignRSASHA256_PSS(..), esl_updateSignRSASHA256(..) called before. The buffer referenced by messageDigest needs to be at least ESL_SIZEOF_SHA256_DIGEST (= 32) byte long, since this buffer is passed to actSHA256Finalize.	
Call context	
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces	

Table 4-226 esl_finalizeHashSignRSASHA256_PSS

4.2.2.4.1.60 esl_finalizeSigSignRSASHA256_PSS

Prototype	
`eslt_ErrorCode esl_finalizeSigSignRSASHA256_PSS (VSECPRIM_P2VAR_PARA(eslt_WorkSpaceRSAPSSsig) workSpace, VSECPRIM_P2CONST_PARA(eslt_Byte) messageDigest, eslt_Length saltSize, VSECPRIM_P2CONST_PARA(eslt_Byte) salt, VSECPRIM_P2VAR_PARA(eslt_Length) signatureSize, VSECPRIM_P2VAR_PARA(eslt_Byte) signature)`	
Parameter	
workSpace [in,out]	Signature generation work space
messageDigest [in]	Message digest (hash) of previously hashed data
saltSize [in]	Size of salt to be used
salt [in]	Salt to be used, 0 (NULL) if salt should be generated randomly
signatureSize [in,out]	Input Maximum size of signature, Output Actual size of signature
signature [in,out]	Buffer to store the generated signature value
Return code	
eslt_ErrorCode	ESL_ERC_NO_ERROR Operation successful
	ESL_ERC_PARAMETER_INVALID provided signature size is too small or saltSize too high for salt to be stored in data block or input parameter is null
	ESL_ERC_WS_TOO_SMALL workSpace too small
	ESL_ERC_WS_STATE_INVALID workSpace state invalid
	all possible error codes which can occur in esl_getBytesRNG
	ESL_ERC_SHA256_TOTAL_LENGTH_OVERFLOW total length of data to be hashed (while calculation the saltedHashValue) is too long
	ESL_ERC_INPUT_INVALID hashID that is passed to esl_calcSaltedHashRSA_PSS does not correspond to a supported hash function
	ESL_ERC_ERROR an error occured during the call to esl_generateMaskMGF1
	ESL_ERC_RSA_MESSAGE_OUT_OF_RANGE message to be signed is greater than the module
	ESL_ERC_OUTPUT_SIZE_TOO_SHORT *signatureSize is too small
	ESL_ERC_RSA_ENCODING_INVALID RSA encoded message length too short
Functional Description	
Generates the RSA PSS SHA256 signature by encoding and signing the message.	
Particularities and Limitations	
workSpace is initialized by esl_initSignRSASHA256_PSS(..), esl_updateSignRSASHA256(..) and esl_finalizeHashSignRSASHA256_PSS(..) called before. The buffer referenced by signature needs to be at least signatureSize bytes long. The buffer referenced by messageDigest needs to be at least ESL_SIZEOF_SHA256160_DIGEST (= 32) byte long.	
Call context	

> TASK > This function is Synchronous > This function is Reentrant, for different workspaces

Table 4-227 esl_finalizeSigSignRSASHA256_PSS

4.2.2.4.1.61 esl_finalizeSigSignRSASHA256_PSS_NoSalt

Prototype	
`eslt_ErrorCode` **`esl_finalizeSigSignRSASHA256_PSS_NoSalt`** `(VSECPRIM_P2VAR_PARA(eslt_WorkSpaceRSAPSSsig) workSpace, VSECPRIM_P2CONST_PARA(eslt_Byte) messageDigest, VSECPRIM_P2VAR_PARA(eslt_Length) signatureSize, VSECPRIM_P2VAR_PARA(eslt_Byte) signature)`	
Parameter	
workSpace [in,out]	Signature generation work space
messageDigest [in]	Message digest (hash) of previously hashed original message
signatureSize [in,out]	Input Maximum size of signature, Output Actual size of signature
signature [in,out]	Buffer to store the generated signature value
Return code	
eslt_ErrorCode	ESL_ERC_NO_ERROR Operation successful
	all possible error codes which can occur in esl_finalizeSigSignRSASHA256_PSS
Functional Description	
Generates the RSA PSS SHA256 signature by encoding and signing the message without using salt. This function calls esl_finalizeSigSignRSASHA256_PSS() with the given input parameters, saltSize = 0 and a null pointer for salt.	
Particularities and Limitations	
workSpace is initialized by esl_initSignRSASHA256_PSS(..), esl_updateSignRSASHA256(..) and esl_finalizeHashSignRSASHA256_PSS(..) called before. The buffer referenced by signature needs to be at least signatureSize bytes long. The buffer referenced by messageDigest needs to be at least ESL_SIZEOF_SHA256160_DIGEST (= 32) byte long.	
Call context	
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces	

Table 4-228 esl_finalizeSigSignRSASHA256_PSS_NoSalt

4.2.2.4.1.62 esl_finalizeSigSignRSASHA256_PSS_DigestLengthSalt

Prototype	
`eslt_ErrorCode` **`esl_finalizeSigSignRSASHA256_PSS_DigestLengthSalt`** `(VSECPRIM_P2VAR_PARA(eslt_WorkSpaceRSAPSSsig) workSpace, VSECPRIM_P2CONST_PARA(eslt_Byte) messageDigest, VSECPRIM_P2VAR_PARA(eslt_Length) signatureSize, VSECPRIM_P2VAR_PARA(eslt_Byte) signature)`	
Parameter	
workSpace [in,out]	Signature generation work space
messageDigest [in]	Message digest (hash) of previously hashed original message
signatureSize [in,out]	Input Maximum size of signature, Output Actual size of signature
signature [in,out]	Buffer to store the generated signature value
Return code	
eslt_ErrorCode	ESL_ERC_NO_ERROR Operation successful
	all possible error codes which can occur in esl_finalizeSigSignRSASHA256_PSS
Functional Description	
Generates the RSA PSS SHA256 signature by encoding and signing the message without using salt. This function calls esl_finalizeSigSignRSASHA256_PSS() with the given input parameters, saltSize equal to message digest length and a null pointer for salt.	
Particularities and Limitations	
workSpace is initialized by esl_initSignRSASHA256_PSS(..), esl_updateSignRSASHA256(..) and esl_finalizeHashSignRSASHA256_PSS(..) called before. The buffer referenced by signature needs to be at least signatureSize bytes long.The buffer signature needs to be able to store at least signatureSize bytes. The buffer referenced by messageDigest needs to be at least ESL_SIZEOF_SHA256160_DIGEST (= 32) byte long.	
Call context	
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces	

Table 4-229 esl_finalizeSigSignRSASHA256_PSS_DigestLengthSalt

4.2.2.4.1.63 esl_finalizeSignRSASHA256_PSS

Prototype	
`eslt_ErrorCode` **`esl_finalizeSignRSASHA256_PSS`** `(VSECPRIM_P2VAR_PARA(eslt_WorkSpaceRSAPSSsig) workSpace, eslt_Length saltSize, VSECPRIM_P2CONST_PARA(eslt_Byte) salt, VSECPRIM_P2VAR_PARA(eslt_Length) signatureSize, VSECPRIM_P2VAR_PARA(eslt_Byte) signature)`	
Parameter	
workSpace [in,out]	Signature generation work space

saltSize [in]	Size of salt to be used
salt [in]	Salt to be used, 0 (NULL) if salt should be generated randomly
signatureSize [in,out]	Input Maximum size of signature, Output Actual size of signature
signature [in,out]	Buffer to store the generated signature value
Return code	
eslt_ErrorCode	ESL_ERC_NO_ERROR Operation successful
	all possible error codes which can occur in esl_finalizeSigSignRSASHA256_PSS
Functional Description	
Finalize message digest (hash) calculation and generate signature for the RSA PSS SHA256 signature generation scheme. This function calls esl_finalizeHashSignRSASHA256_PSS() and esl_finalizeSigSignRSASHA256_PSS() with the given input parameters.	
Particularities and Limitations	
workSpace is initialized by esl_initSignRSASHA256_PSS(..), esl_updateSignRSASHA256(..) called before. The buffer referenced by signature needs to be at least signatureSize bytes long.	
Call context	
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces	

Table 4-230 esl_finalizeSignRSASHA256_PSS

4.2.2.4.1.64 esl_finalizeSignRSASHA256_PSS_NoSalt

Prototype	
`eslt_ErrorCode esl_finalizeSignRSASHA256_PSS_NoSalt (VSECPRIM_P2VAR_PARA(eslt_WorkSpaceRSAPSSsig) workSpace, VSECPRIM_P2VAR_PARA(eslt_Length) signatureSize, VSECPRIM_P2VAR_PARA(eslt_Byte) signature)`	
Parameter	
workSpace [in,out]	Signature generation work space
signatureSize [in,out]	Input Maximum size of signature, Output Actual size of signature
signature [in,out]	Buffer to store the generated signature value
Return code	
eslt_ErrorCode	ESL_ERC_NO_ERROR Operation successful
	all possible error codes which can occur in esl_finalizeSignRSASHA256_PSS
Functional Description	
Finalize message digest (hash) calculation and generate signature for the RSA PSS SHA256 signature generation scheme.	

This function calls esl_finalizeSignRSASHA256_PSS() with the given input parameters, saltSize = 0 and a null pointer for salt.
Particularities and Limitations
workSpace is initialized by esl_initSignRSASHA256_PSS(..), esl_updateSignRSASHA256(..) called before. The buffer referenced by signature needs to be at least signatureSize bytes long.
Call context
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces

Table 4-231 esl_finalizeSignRSASHA256_PSS_NoSalt

4.2.2.4.1.65 esl_finalizeSignRSASHA256_PSS_DigestLengthSalt

Prototype	
`eslt_ErrorCode` **`esl_finalizeSignRSASHA256_PSS_DigestLengthSalt`** `(VSECPRIM_P2VAR_PARA(eslt_WorkSpaceRSAPSSsig) workSpace, VSECPRIM_P2VAR_PARA(eslt_Length) signatureSize, VSECPRIM_P2VAR_PARA(eslt_Byte) signature)`	
Parameter	
workSpace [in,out]	Signature generation work space
signatureSize [in,out]	Input Maximum size of signature, Output Actual size of signature
signature [in,out]	Buffer to store the generated signature value
Return code	
eslt_ErrorCode	ESL_ERC_NO_ERROR Operation successful
	all possible error codes which can occur in esl_finalizeSignRSASHA256_PSS
Functional Description	
Finalize message digest (hash) calculation and generate signature for the RSA PSS SHA256 signature generation scheme This function calls esl_finalizeSignRSASHA256_PSS()with a null pointer for salt and saltSize equal to message digest length for signature generation.	
Particularities and Limitations	
workSpace is initialized by esl_initSignRSASHA256_PSS(..), esl_updateSignRSASHA256(..) called before. The buffer referenced by signature needs to be at least signatureSize bytes long.	
Call context	
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces	

Table 4-232 esl_finalizeSignRSASHA256_PSS_DigestLengthSalt

4.2.2.4.1.66 esl_initVerifyRSASHA256_PSS

Prototype	
`eslt_ErrorCode` **`esl_initVerifyRSASHA256_PSS`** `(VSECPRIM_P2VAR_PARA(eslt_WorkSpaceRSAPSSver) workSpace, eslt_Length keyPairModuleSize, eslt_pRomByte keyPairModule, eslt_Length publicKeyExponentSize, eslt_pRomByte publicKeyExponent)`	
Parameter	
workSpace [in,out]	Signature verification work space
keyPairModuleSize [in]	Size of RSA key pair module
keyPairModule [in]	RSA key pair module
publicKeyExponentSize [in]	Size of public RSA key pair exponent
publicKeyExponent [in]	Public RSA key pair exponent
Return code	
eslt_ErrorCode	ESL_ERC_NO_ERROR Operation successful
	ESL_ERC_PARAMETER_INVALID Input parameter workSpace/keyPairModule/publicKeyExponent is NULL or exponent is larger than module
	ESL_ERC_WS_TOO_SMALL workSpace too small
	ESL_ERC_RSA_PUBKEY_INVALID publicExponentKeySize is invalid
Functional Description	
Initialize signature verification with EMSA PSS encoding, based on SHA2-256	
Particularities and Limitations	
-	
Call context	
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces	

Table 4-233 esl_initVerifyRSASHA256_PSS

4.2.2.4.1.67 esl_updateVerifyRSASHA256_PSS

Prototype	
`eslt_ErrorCode` **`esl_updateVerifyRSASHA256_PSS`** `(VSECPRIM_P2VAR_PARA(eslt_WorkSpaceRSAPSSver) workSpace, eslt_Length inputSize, VSECPRIM_P2CONST_PARA(eslt_Byte) input)`	
Parameter	
workSpace [in,out]	Signature verification work space
inputSize [in]	Size of data to be hashed

input [in]	Data to be hashed
Return code	
eslt_ErrorCode	ESL_ERC_NO_ERROR Operation successful
	ESL_ERC_PARAMETER_INVALID input parameter workSpace is null
	ESL_ERC_WS_TOO_SMALL workSpace too small
	ESL_ERC_WS_STATE_INVALID workSpace in invalid state
	ESL_ERC_SHA256_TOTAL_LENGTH_OVERFLOW Maximum total input length exceeded
Functional Description	
Update hash workspace with message to be signed During the RSA PSS signature verification scheme, the original message needs to be hashed. For RSA PSS SHA256, the hash function is SHA2-256. This function stores the message into the hash workspace, contained in the parameter workSpace.	
Particularities and Limitations	
workSpace is initialized by esl_initVerifyRSASHA256_PSS(..)	
Call context	
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces	

Table 4-234 esl_updateVerifyRSASHA256_PSS

4.2.2.4.1.68 esl_finalizeHashVerifyRSASHA256_PSS

Prototype

`eslt_ErrorCode` **`esl_finalizeHashVerifyRSASHA256_PSS`**
`(VSECPRIM_P2VAR_PARA(eslt_WorkSpaceRSAPSSver) workSpace,`
`VSECPRIM_P2VAR_PARA(eslt_Byte) messageDigest)`

Parameter	
workSpace [in,out]	Signature verification work space
messageDigest [in,out]	Buffer to store the message digest (hash)
Return code	
eslt_ErrorCode	ESL_ERC_NO_ERROR Operation successful
	ESL_ERC_PARAMETER_INVALID parameter workSpace or messageDigest is null
	ESL_ERC_WS_TOO_SMALL workSpace too small
	ESL_ERC_WS_STATE_INVALID workSpace in invalid state
Functional Description	
Calculate the hash value of the data previously passed to the hash workspace, contained in the parameter workSpace.	

During the RSA PSS signature generation scheme, the original message needs to be hashed. For RSA PSS SHA256, the hash function is SHA2-256. The message was passed to the hash workSpace before. This function computes its hash value.
Particularities and Limitations
workSpace is initialized by esl_initVerifyRSASHA256_PSS(..), esl_updateVerifyRSASHA256(..) called before. The buffer referenced by messageDigest needs to be at least ESL_SIZEOF_SHA256_DIGEST (= 32) byte long, since this buffer is passed to actSHA256Finalize.
Call context
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces

Table 4-235 esl_finalizeHashVerifyRSASHA256_PSS

4.2.2.4.1.69 esl_verifySigVerifyRSASHA256_PSS

Prototype

```
eslt_ErrorCode esl_verifySigVerifyRSASHA256_PSS
(VSECPRIM_P2VAR_PARA(eslt_WorkSpaceRSAPSSver) workSpace,
VSECPRIM_P2CONST_PARA(eslt_Byte) messageDigest, eslt_Length saltSize,
eslt_Length signatureSize, VSECPRIM_P2CONST_PARA(eslt_Byte) signature)
```

Parameter

workSpace [in,out]	Signature verification work space
messageDigest [in]	Message digest (hash) of previously hashed data
saltSize [in]	Expected size of salt, -1 if size of salt used during signature generation should be evaluated automatically
signatureSize [in]	Size of signature
signature [in]	Signature value to be verified

Return code

eslt_ErrorCode	ESL_ERC_NO_ERROR Operation successful
	ESL_ERC_PARAMETER_INVALID provided signature size is too small or saltSize too high for salt to be stored in data block or input parameter is null
	ESL_ERC_WS_TOO_SMALL workSpace too small
	ESL_ERC_WS_STATE_INVALID workSpace state invalid
	ESL_ERC_SHA256_TOTAL_LENGTH_OVERFLOW total length of data to be hashed (while calculation the saltedHashValue) is too long
	ESL_ERC_INPUT_INVALID hashID that is passed to esl_calcSaltedHashRSA_PSS does not correspond to a supported hash function
	ESL_ERC_ERROR an error occured during the call to esl_generateMaskMGF1
	ESL_ERC_RSA_SIGNATURE_OUT_OF_RANGE signature to be verified is greater than the module
	ESL_ERC_RSA_SIGNATURE_INVALID signature verification failed
	ESL_ERC_RSA_ENCODING_INVALID RSA encoded message length too short

Functional Description
Verifies the RSA PSS SHA256 signature by applying the RSA verification primitive to the signature and decoding the message.
Particularities and Limitations
workSpace is initialized by esl_initVerifyRSASHA256_PSS(..), esl_updateVerifyRSASHA256(..) and esl_finalizeHashVerifyRSASHA256(..) called before. The buffer referenced by messageDigest needs to be at least ESL_SIZEOF_SHA256160_DIGEST (= 32) byte long.
Call context
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces

Table 4-236 esl_verifySigVerifyRSASHA256_PSS

4.2.2.4.1.70 esl_verifySigVerifyRSASHA256_PSS_AutoSalt

Prototype	
`eslt_ErrorCode esl_verifySigVerifyRSASHA256_PSS_AutoSalt (VSECPRIM_P2VAR_PARA(eslt_WorkSpaceRSAPSSver) workSpace, VSECPRIM_P2CONST_PARA(eslt_Byte) messageDigest, eslt_Length signatureSize, VSECPRIM_P2CONST_PARA(eslt_Byte) signature)`	
Parameter	
workSpace [in,out]	Signature verification work space
messageDigest [in]	Message digest (hash) of previously hashed original message
signatureSize [in]	Size of signature
signature [in]	Signature value to be verified
Return code	
eslt_ErrorCode	ESL_ERC_NO_ERROR Operation successful
	all possible error codes which can occur in esl_verifySigVerifyRSASHA256_PSS
Functional Description	
Verifies the RSA PSS SHA256 signature by encoding and signing the message. Requires that the size of the salt used during signature generation is evaluated automatically. This function calls esl_verifySigVerifyRSASHA256_PSS() with the given input parameters and saltSize = -1.	
Particularities and Limitations	
workSpace is initialized by esl_initVerifyRSASHA256_PSS(..), esl_updateVerifyRSASHA256(..) and esl_finalizeHashVerifyRSASHA256(..) called before. The buffer referenced by messageDigest needs to be at least ESL_SIZEOF_SHA256160_DIGEST (= 32) byte long.	
Call context	

> TASK > This function is Synchronous > This function is Reentrant, for different workspaces

Table 4-237 esl_verifySigVerifyRSASHA256_PSS_AutoSalt

4.2.2.4.1.71 esl_verifySigVerifyRSASHA256_PSS_NoSalt

Prototype	

`eslt_ErrorCode` **`esl_verifySigVerifyRSASHA256_PSS_NoSalt`**
`(VSECPRIM_P2VAR_PARA(eslt_WorkSpaceRSAPSSver) workSpace,`
`VSECPRIM_P2CONST_PARA(eslt_Byte) messageDigest, eslt_Length signatureSize,`
`VSECPRIM_P2CONST_PARA(eslt_Byte) signature)`	

Parameter	
workSpace [in,out]	Signature verification work space
messageDigest [in]	Message digest (hash) of previously hashed original message
signatureSize [in]	Size of signature
signature [in]	Signature value to be verified
Return code	
eslt_ErrorCode	ESL_ERC_NO_ERROR Operation successful
	all possible error codes which can occur in esl_verifySigVerifyRSASHA256_PSS
Functional Description	
Verifies the RSA PSS SHA256 signature by encoding and signing the message. This function requires that no salt was used during signature generation This function calls esl_verifySigVerifyRSASHA256_PSS() with the given input parameters and saltSize = 0.	
Particularities and Limitations	
workSpace is initialized by esl_initVerifyRSASHA256_PSS(..), esl_updateVerifyRSASHA256(..) and esl_finalizeHashVerifyRSASHA256(..) called before. The buffer referenced by messageDigest needs to be at least ESL_SIZEOF_SHA256160_DIGEST (= 32) byte long.	
Call context	
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces	

Table 4-238 esl_verifySigVerifyRSASHA256_PSS_NoSalt

4.2.2.4.1.72 esl_verifySigVerifyRSASHA256_PSS_DigestLengthSalt

Prototype	
`eslt_ErrorCode` **`esl_verifySigVerifyRSASHA256_PSS_DigestLengthSalt`** `(VSECPRIM_P2VAR_PARA(eslt_WorkSpaceRSAPSSver) workSpace, VSECPRIM_P2CONST_PARA(eslt_Byte) messageDigest, eslt_Length signatureSize, VSECPRIM_P2CONST_PARA(eslt_Byte) signature)`	
Parameter	
workSpace [in,out]	Signature verification work space
messageDigest [in]	Message digest (hash) of previously hashed original message
signatureSize [in]	Size of signature
signature [in]	Signature value to be verified
Return code	
eslt_ErrorCode	ESL_ERC_NO_ERROR Operation successful
	all possible error codes which can occur in esl_verifySigVerifyRSASHA256_PSS
Functional Description	
Verifies the RSA PSS SHA256 signature by encoding and signing the message. This function requires that salt with size equal to message digest length was used during signature generation This function calls esl_verifySigVerifyRSASHA256_PSS() with the given input parameters and saltSize = ESL_SIZEOF_SHA256_DIGEST.	
Particularities and Limitations	
workSpace is initialized by esl_initVerifyRSASHA256_PSS(..), esl_updateVerifyRSASHA256(..) and esl_finalizeHashVerifyRSASHA256(..) called before. The buffer referenced by messageDigest needs to be at least ESL_SIZEOF_SHA256160_DIGEST (= 32) byte long.	
Call context	
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces	

Table 4-239 esl_verifySigVerifyRSASHA256_PSS_DigestLengthSalt

4.2.2.4.1.73 esl_finalizeVerifyRSASHA256_PSS

Prototype	
`eslt_ErrorCode` **`esl_finalizeVerifyRSASHA256_PSS`** `(VSECPRIM_P2VAR_PARA(eslt_WorkSpaceRSAPSSver) workSpace, eslt_Length saltSize, eslt_Length signatureSize, VSECPRIM_P2CONST_PARA(eslt_Byte) signature)`	
Parameter	
workSpace [in,out]	Signature verification work space
saltSize [in]	Expected size of salt, -1 if size of salt used during signature generation should be evaluated automatically

signatureSize [in]	Size of signature
signature [in]	Signature value to be verified
Return code	
eslt_ErrorCode	ESL_ERC_NO_ERROR Operation successful
	all possible error codes which can occur in esl_verifySigVerifyRSASHA256_PSS
Functional Description	
Finalize message digest (hash) calculation and verify signature for the RSA PSS SHA256 signature verification scheme This function calls esl_finalizeHashVerifyRSASHA256_PSS() and esl_verifySigVerifyRSASHA256_PSS() with the given input parameters.	
Particularities and Limitations	
workSpace is initialized by esl_initVerifyRSASHA256_PSS(..), esl_updateVerifyRSASHA256(..) called before	
Call context	
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces	

Table 4-240 esl_finalizeVerifyRSASHA256_PSS

4.2.2.4.1.74 esl_finalizeVerifyRSASHA256_PSS_AutoSalt

Prototype	
`eslt_ErrorCode` **`esl_finalizeVerifyRSASHA256_PSS_AutoSalt`** `(VSECPRIM_P2VAR_PARA(eslt_WorkSpaceRSAPSSver) workSpace, eslt_Length signatureSize, VSECPRIM_P2CONST_PARA(eslt_Byte) signature)`	
Parameter	
workSpace [in,out]	Signature verification work space
signatureSize [in]	Size of signature
signature [in]	Signature value to be verified
Return code	
eslt_ErrorCode	ESL_ERC_NO_ERROR Operation successful
	all possible error codes which can occur in esl_verifySigVerifyRSASHA256_PSS
Functional Description	
Finalize message digest (hash) calculation and verify signature for the RSA PSS SHA256 signature verification scheme. Assumes that the size of the salt used during signature generation is evaluated automatically. This function calls esl_finalizeVerifyRSASHA256_PSS() with the given input parameters and saltSize = -1.	

Particularities and Limitations
workSpace is initialized by esl_initVerifyRSASHA256_PSS(..), esl_updateVerifyRSASHA256(..) called before
Call context
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces

Table 4-241 esl_finalizeVerifyRSASHA256_PSS_AutoSalt

4.2.2.4.1.75 esl_finalizeVerifyRSASHA256_PSS_NoSalt

Prototype	
`eslt_ErrorCode esl_finalizeVerifyRSASHA256_PSS_NoSalt (VSECPRIM_P2VAR_PARA(eslt_WorkSpaceRSAPSSver) workSpace, eslt_Length signatureSize, VSECPRIM_P2CONST_PARA(eslt_Byte) signature)`	
Parameter	
workSpace [in,out]	Signature verification work space
signatureSize [in]	Size of signature
signature [in]	Signature value to be verified
Return code	
eslt_ErrorCode	ESL_ERC_NO_ERROR Operation successful
	all possible error codes which can occur in esl_verifySigVerifyRSASHA256_PSS
all possible error codes which can occur in esl_verifySigVerifyRSASHA256_PSS	
Finalize message digest (hash) calculation and verify signature for the RSA PSS SHA256 signature verification scheme. Requires that no salt was used during signature generation. This function calls esl_finalizeVerifyRSASHA256_PSS() with the given input parameters and saltSize = 0.	
Particularities and Limitations	
workSpace is initialized by esl_initVerifyRSASHA256_PSS(..), esl_updateVerifyRSASHA256(..) called before	
Call context	
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces	

Table 4-242 esl_finalizeVerifyRSASHA256_PSS_NoSalt

4.2.2.4.1.76 esl_finalizeVerifyRSASHA256_PSS_DigestLengthSalt

Prototype	
`eslt_ErrorCode` **`esl_finalizeVerifyRSASHA256_PSS_DigestLengthSalt`** `(VSECPRIM_P2VAR_PARA(eslt_WorkSpaceRSAPSSver) workSpace, eslt_Length signatureSize, VSECPRIM_P2CONST_PARA(eslt_Byte) signature)`	
Parameter	
workSpace [in,out]	Signature verification work space
signatureSize [in]	Size of signature
signature [in]	Signature value to be verififed
Return code	
eslt_ErrorCode	ESL_ERC_NO_ERROR Operation successful
	all possible error codes which can occur in esl_verifySigVerifyRSASHA256_PSS
Functional Description	
Finalize message digest (hash) calculation and verify signature for the RSA PSS SHA256 signature verification scheme. Requires that salt with size equal to message digest was used during signature generation. This function calls esl_finalizeVerifyRSASHA256_PSS() with the given input parameters and saltSize = ESL_SIZEOF_SHA256_DIGEST.	
Particularities and Limitations	
workSpace is initialized by esl_initVerifyRSASHA256_PSS(..), esl_updateVerifyRSASHA256(..) called before	
Call context	
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces	

Table 4-243 esl_finalizeVerifyRSASHA256_PSS_DigestLengthSalt

4.2.2.4.1.77 esl_generateMaskMGF1RSASHA256_PSS

Prototype	
`eslt_ErrorCode` **`esl_generateMaskMGF1RSASHA256_PSS`** `(VSECPRIM_P2VAR_PARA(eslt_WorkSpaceSHA256) wsHash, VSECPRIM_P2VAR_PARA(eslt_Byte) tempHash, eslt_Length seedLength, VSECPRIM_P2CONST_PARA(eslt_Byte) seed, eslt_Length maskLength, VSECPRIM_P2VAR_PARA(eslt_Byte) mask)`	
Parameter	
wsHash [in,out]	Pointer to hash workspace
tempHash [in,out]	Temporary buffer for intermediate hash, has to be large enough to hold complete hash value
seedLength [in]	Length input seed
seed [in]	Input seed from which mask is generated

maskLength [in]	Length of mask to be generated
mask [in,out]	Output buffer for generated mask, has to be large enough to hold requested length
Return code	
eslt_ErrorCode	ESL_ERC_NO_ERROR Operation successful
	ESL_ERC_PARAMETER_INVALID input parameter null
	ESL_ERC_WS_TOO_SMALL wsHash too small
	ESL_ERC_ERROR an error occurred
Functional Description	
Mask generation function (MGF1). MGF1 implementation as described in appendix B.2.1 of PKCS#1 2.2 Based on hash function (SHA-256) this function generates a pseudorandom output string of arbitrary length depending on an input string (seed) of variable length	
Particularities and Limitations	
buffer referenced by tempHash has to be at least of length ESL_SIZEOF_SHA256_DIGEST buffer referenced by mask has to be at least of length maskLength	
Call context	
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces	

Table 4-244 esl_generateMaskMGF1RSASHA256_PSS

4.2.2.4.1.78 esl_calcSaltedHashRSASHA256_PSS

Prototype	
`eslt_ErrorCode` **`esl_calcSaltedHashRSASHA256_PSS`** `(VSECPRIM_P2VAR_PARA(eslt_WorkSpaceSHA256) wsHash, eslt_Length saltSize, VSECPRIM_P2CONST_PARA(eslt_Byte) salt, VSECPRIM_P2CONST_PARA(eslt_Byte) messageDigest, VSECPRIM_P2VAR_PARA(eslt_Byte) saltedHash)`	
Parameter	
wsHash [in,out]	Pointer to hash workspace
saltSize [in]	Length of salt
salt [in]	Input salt
messageDigest [in]	Input hash value of the original message, will be hashed again here (SHA-256)
saltedHash [in,out]	Output buffer for calculated hash (SHA-256), has to be large enough to hold requested length
Return code	
eslt_ErrorCode	ESL_ERC_NO_ERROR Operation successful
	ESL_ERC_PARAMETER_INVALID workSpace or saltedHash null
	ESL_ERC_WS_TOO_SMALL wsHash too small

	ESL_ERC_SHA256_TOTAL_LENGTH_OVERFLOW total length of data to be hashed too long
Functional Description	
Calculate salted hash value (SHA-256). Based on a given hash and a random salt, the hash of M' = [(0x) 00 00 00 00 00 00 00 00 || hash || salt] is calculated, as described in chapter 9.1.1, steps 5-6 and 9.1.2, steps 12-13 of PKCS#1 2.2	
Particularities and Limitations	
The buffer referenced by saltedHash needs to be at least ESL_SIZEOF_SHA256_DIGEST (=32) byte long. The buffer referenced by messageDigest needs to be at least ESL_SIZEOF_SHA256160_DIGEST (= 32) byte long.	
Call context	
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces	

Table 4-245 esl_calcSaltedHashRSASHA256_PSS

4.2.2.4.1.79 esl_initSignRSARIPEMD160_PSS

Prototype	
`eslt_ErrorCode` **`esl_initSignRSARIPEMD160_PSS`** `(VSECPRIM_P2VAR_PARA(eslt_WorkSpaceRSAPSSsig) workSpace, eslt_Length keyPairModuleSize, eslt_pRomByte keyPairModule, eslt_Length privateKeyExponentSize, eslt_pRomByte privateKeyExponent)`	
Parameter	
workSpace [in,out]	Signature generation work space
keyPairModuleSize [in]	Size of RSA key pair module
keyPairModule [in]	RSA key pair module
privateKeyExponentSize [in]	Size of private RSA key pair exponent
privateKeyExponent [in]	Private RSA key pair exponent
Return code	
eslt_ErrorCode	ESL_ERC_NO_ERROR Operation successful
	ESL_ERC_PARAMETER_INVALID Input parameter is NULL
	ESL_ERC_WS_TOO_SMALL Work space too small
	ESL_ERC_RSA_PRIVKEY_INVALID The private exponent is invalid
Functional Description	
Initialize signature generation with EMSA PSS encoding, based on RIPEMD160	
Particularities and Limitations	
-	

Call context
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces

Table 4-246 esl_initSignRSARIPEMD160_PSS

4.2.2.4.1.80 esl_updateSignRSARIPEMD160_PSS

Prototype	
`eslt_ErrorCode` **`esl_updateSignRSARIPEMD160_PSS`** `(VSECPRIM_P2VAR_PARA(eslt_WorkSpaceRSAPSSsig) workSpace, eslt_Length inputSize, VSECPRIM_P2CONST_PARA(eslt_Byte) input)`	
Parameter	
workSpace [in,out]	Signature generation work space. Out: hash workspace contained in signature generation workspace updated with data to be hashed
inputSize [in]	Size of data to be hashed
input [in]	Data to be hashed
Return code	
eslt_ErrorCode	ESL_ERC_NO_ERROR Operation successful
	ESL_ERC_PARAMETER_INVALID Input parameter is NULL
	ESL_ERC_WS_STATE_INVALID Invalid state
	ESL_ERC_WS_TOO_SMALL Work space too small
	ESL_ERC_RIPEMD160_TOTAL_LENGTH_OVERFLOW Maximum total input length exceeded
Functional Description	
Update hash workspace with message to be signed During the RSA PSS signature generation scheme, the original message needs to be hashed. For RSA PSS RIPEMD160, the hash function is RMD160. This function stores the message into the hash workspace, contained in the parameter workSpace.	
Particularities and Limitations	
workSpace is initialized by esl_initSignRSASHA256_PSS(..)	
Call context	
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces	

Table 4-247 esl_updateSignRSARIPEMD160_PSS

4.2.2.4.1.81 esl_finalizeHashSignRSARIPEMD160_PSS

Prototype	
`eslt_ErrorCode esl_finalizeHashSignRSARIPEMD160_PSS (VSECPRIM_P2VAR_PARA(eslt_WorkSpaceRSAPSSsig) workSpace, VSECPRIM_P2VAR_PARA(eslt_Byte) messageDigest)`	
Parameter	
workSpace [in,out]	Signature generation work space
messageDigest [in,out]	Message digest (hash) of previously hashed data
Return code	
eslt_ErrorCode	ESL_ERC_NO_ERROR Operation successful
	ESL_ERC_PARAMETER_INVALID Input parameter is NULL
	ESL_ERC_WS_STATE_INVALID Invalid state
	ESL_ERC_WS_TOO_SMALL Work space too small
Functional Description	
Calculate the hash value of the data in hash workspace During the RSA PSS signature generation scheme, the original message needs to be hashed. For RSA PSS RIPME160, the hash function is RMD160. The message was passed to the hash workSpace before, which is contained in the parameter 'workSpace'. This function computes its hash value.	
Particularities and Limitations	
workSpace is initialized by esl_initSignRSARIPEMD160_PSS(..), esl_updateSignRSARIPEM160(..) called before The buffer referenced by messageDigest needs to be at least ESL_SIZEOF_RIPEMD160_DIGEST (= 20) byte long, since this buffer is passed to actRMD160Finalize.	
Call context	
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces	

Table 4-248 esl_finalizeHashSignRSARIPEMD160_PSS

4.2.2.4.1.82 esl_finalizeSigSignRSARIPEMD160_PSS

Prototype	
`eslt_ErrorCode esl_finalizeSigSignRSARIPEMD160_PSS (VSECPRIM_P2VAR_PARA(eslt_WorkSpaceRSAPSSsig) workSpace, VSECPRIM_P2CONST_PARA(eslt_Byte) messageDigest, eslt_Length saltSize, VSECPRIM_P2CONST_PARA(eslt_Byte) salt, VSECPRIM_P2VAR_PARA(eslt_Length) signatureSize, VSECPRIM_P2VAR_PARA(eslt_Byte) signature)`	
Parameter	
workSpace [in,out]	Signature generation work space
messageDigest [in]	Message digest (hash) of previously hashed data
saltSize [in]	Size of salt to be used

salt [in]	Salt to be used 0 (NULL) if salt should be generated randomly
signatureSize [in,out]	Input Maximum size of signature Output Actual size of signature
signature [out]	Generated signature value
Return code	
eslt_ErrorCode	ESL_ERC_NO_ERROR Operation successful
	ESL_ERC_PARAMETER_INVALID Input parameter is NULL or saltSize too high for salt to be stored in data block
	ESL_ERC_WS_STATE_INVALID Invalid state
	ESL_ERC_WS_TOO_SMALL Work space too small
	all possible error codes which can occur in esl_getBytesRNG
	ESL_ERC_RIPEMD160_TOTAL_LENGTH_OVERFLOW total length of data to be hashed (while calculating the saltedHashValue) is too long
	ESL_ERC_INPUT_INVALID hashID that is passed to esl_calcSaltedHashRSA_PSS does not correspond to a supported hash function
	ESL_ERC_ERROR an error occured during the call to esl_generateMaskMGF1
	ESL_ERC_RSA_MESSAGE_OUT_OF_RANGE message to be signed is greater than the module
	ESL_ERC_OUTPUT_SIZE_TOO_SHORT *signatureSize is too small
	ESL_ERC_RSA_ENCODING_INVALID RSA encoded message length too short
Functional Description	
Generates the RSA PSS RIPEMD160 signature by encoding and signing the message.	
Particularities and Limitations	
workSpace is initialized by esl_initSignRSARIPEMD160_PSS(..), esl_updateSignRSARIPEMD160(..) and esl_finalizeHashSignRSARIPEMD160_PSS(..) called before The buffer referenced by signature needs to be at least signatureSize bytes long. The buffer referenced by messageDigest needs to be at least ESL_SIZEOF_RIPEMD160_DIGEST (= 20) byte long.	
Call context	
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces	

Table 4-249 esl_finalizeSigSignRSARIPEMD160_PSS

4.2.2.4.1.83 esl_finalizeSigSignRSARIPEMD160_PSS_NoSalt

Prototype

```
eslt_ErrorCode esl_finalizeSigSignRSARIPEMD160_PSS_NoSalt
(VSECPRIM_P2VAR_PARA(eslt_WorkSpaceRSAPSSsig) workSpace,
VSECPRIM_P2CONST_PARA(eslt_Byte) messageDigest,
```

```
VSECPRIM_P2VAR_PARA(eslt_Length) signatureSize, VSECPRIM_P2VAR_PARA(eslt_Byte)
signature)
```

Parameter	
workSpace [in,out]	Signature generation work space
messageDigest [in]	Message digest (hash) of previously hashed data
signatureSize [in,out]	Input Maximum size of signature Output Actual size of signature
signature [out]	Generated signature value
Return code	
eslt_ErrorCode	ESL_ERC_NO_ERROR Operation successful
	All possible error codes which can occur in esl_finalizeSigSignRSARIPEMD160_PSS()
Functional Description	
Generates the RSA PSS RIPEMD160 signature by encoding and signing the message without using salt. This function calls esl_finalizeSigSignRSARIPEMD160_PSS() with the given input parameters, saltSize = 0 and a null pointer for salt.	
Particularities and Limitations	
workSpace is initialized by esl_initSignRSARIPEMD160_PSS(..), esl_updateSignRSARIPEMD160(..) and esl_finalizeHashSignRSARIPEMD160_PSS(..) called before The buffer referenced by signature needs to be at least signatureSize bytes long. The buffer referenced by messageDigest needs to be at least ESL_SIZEOF_RIPEMD160_DIGEST (= 20) byte long.	
Call context	
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces	

Table 4-250 esl_finalizeSigSignRSARIPEMD160_PSS_NoSalt

4.2.2.4.1.84 esl_finalizeSigSignRSARIPEMD160_PSS_DigestLengthSalt

Prototype
`eslt_ErrorCode` **`esl_finalizeSigSignRSARIPEMD160_PSS_DigestLengthSalt`** `(VSECPRIM_P2VAR_PARA(eslt_WorkSpaceRSAPSSsig) workSpace, VSECPRIM_P2CONST_PARA(eslt_Byte) messageDigest, VSECPRIM_P2VAR_PARA(eslt_Length) signatureSize, VSECPRIM_P2VAR_PARA(eslt_Byte) signature)`

Parameter	
workSpace [in,out]	Signature generation work space
messageDigest [in]	Message digest (hash) of previously hashed data
signatureSize [in,out]	Input Maximum size of signature Output Actual size of signature
signature [out]	Generated signature value

Return code	
eslt_ErrorCode	ESL_ERC_NO_ERROR Operation successful
	All possible error codes which can occur in esl_finalizeSigSignRSARIPEMD160_PSS()
Functional Description	
Generates the RSA PSS RIPEMD160 signature by encoding and signing the message with random salt. This function calls esl_finalizeSigSignRSARIPEMD160_PSS() with the given input parameters, saltSize equal to message digest length and a null pointer for salt, which means that during encoding the message, a salt buffer of length ESL_SIZEOF_RIPEMD160_DIGEST with random data will be generated.	
Particularities and Limitations	
workSpace is initialized by esl_initSignRSARIPEMD160_PSS(..), esl_updateSignRSARIPEMD160(..) and esl_finalizeHashSignRSARIPEMD160_PSS(..) called before The buffer referenced by signature needs to be at least signatureSize bytes long. The buffer referenced by messageDigest needs to be at least ESL_SIZEOF_RIPEMD160_DIGEST (= 20) byte long.	
Call context	
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces	

Table 4-251 esl_finalizeSigSignRSARIPEMD160_PSS_DigestLengthSalt

4.2.2.4.1.85 esl_finalizeSignRSARIPEMD160_PSS

Prototype
`eslt_ErrorCode esl_finalizeSignRSARIPEMD160_PSS (VSECPRIM_P2VAR_PARA(eslt_WorkSpaceRSAPSSsig) workSpace, eslt_Length saltSize, VSECPRIM_P2CONST_PARA(eslt_Byte) salt, VSECPRIM_P2VAR_PARA(eslt_Length) signatureSize, VSECPRIM_P2VAR_PARA(eslt_Byte) signature)`

Parameter	
workSpace [in,out]	Signature generation work space
saltSize [in]	Size of salt to be used
salt [in]	Salt to be used 0 (NULL) if salt should be generated randomly
signatureSize [in,out]	Input Maximum size of signature Output Actual size of signature
signature [out]	Generated signature value
Return code	
eslt_ErrorCode	ESL_ERC_NO_ERROR Operation successful
	all possible error codes which can occur in esl_finalizeSigSignRSARIPEMD160_PSS
Functional Description	
Finalize message digest (hash) calculation and generate signature for RSA PSS RIPEMD160	

This function calls esl_finalizeHashSignRSARIPEMD160_PSS() and esl_finalizeSigSignRSARIPEMD160_PSS() with the given input parameters.
Particularities and Limitations
workSpace is initialized by esl_initSignRSARIPEMD160_PSS(..), esl_updateSignRSARIPEMD160(..) called before The buffer referenced by signature needs to be at least signatureSize bytes long.
Call context
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces

Table 4-252 esl_finalizeSignRSARIPEMD160_PSS

4.2.2.4.1.86 esl_finalizeSignRSARIPEMD160_PSS_NoSalt

Prototype	
`eslt_ErrorCode` **`esl_finalizeSignRSARIPEMD160_PSS_NoSalt`** `(VSECPRIM_P2VAR_PARA(eslt_WorkSpaceRSAPSSsig) workSpace, VSECPRIM_P2VAR_PARA(eslt_Length) signatureSize, VSECPRIM_P2VAR_PARA(eslt_Byte) signature)`	
Parameter	
workSpace [in,out]	Signature generation work space
signatureSize [in,out]	Input Maximum size of signature Output Actual size of signature
signature [out]	Generated signature value
Return code	
eslt_ErrorCode	ESL_ERC_NO_ERROR Operation successful
	all possible error codes which can occur in esl_finalizeSignRSARIPEMD160_PSS
Functional Description	
Finalize message digest (hash) calculation and generate signature for RSA PSS RIPEMD160 This function calls esl_finalizeSignRSARIPEMD160_PSS() with the given input parameters, saltSize = 0 and a null pointer for salt.	
Particularities and Limitations	
workSpace is initialized by esl_initSignRSARIPEMD160_PSS(..), esl_updateSignRSARIPEMD160(..) called before The buffer referenced by signature needs to be at least signatureSize bytes long.	
Call context	
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces	

Table 4-253 esl_finalizeSignRSARIPEMD160_PSS_NoSalt

4.2.2.4.1.87 esl_finalizeSignRSARIPEMD160_PSS_DigestLengthSalt

Prototype	
`eslt_ErrorCode` **`esl_finalizeSignRSARIPEMD160_PSS_DigestLengthSalt`** `(VSECPRIM_P2VAR_PARA(eslt_WorkSpaceRSAPSSsig) workSpace, VSECPRIM_P2VAR_PARA(eslt_Length) signatureSize, VSECPRIM_P2VAR_PARA(eslt_Byte) signature)`	
Parameter	
workSpace [in,out]	Signature generation work space
signatureSize [in,out]	Input Maximum size of signature Output Actual size of signature
signature [out]	Generated signature value
Return code	
eslt_ErrorCode	ESL_ERC_NO_ERROR Operation successful
	all possible error codes which can occur in esl_finalizeSignRSARIPEMD160_PSS
Functional Description	
Finalize message digest (hash) calculation and generate signature for RSA PSS RIPEMD160 This function calls esl_finalizeSignRSARIPEMD160_PSS()with a null pointer for salt and saltSize equal to message digest length for signature generation	
Particularities and Limitations	
workSpace is initialized by esl_initSignRSARIPEMD160_PSS(..), esl_updateSignRSARIPEMD160(..) called before The buffer referenced by signature needs to be at least signatureSize bytes long.	
Call context	
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces	

Table 4-254 esl_finalizeSignRSARIPEMD160_PSS_DigestLengthSalt

4.2.2.4.1.88 esl_initVerifyRSARIPEMD160_PSS

Prototype	
`eslt_ErrorCode` **`esl_initVerifyRSARIPEMD160_PSS`** `(VSECPRIM_P2VAR_PARA(eslt_WorkSpaceRSAPSSver) workSpace, eslt_Length keyPairModuleSize, eslt_pRomByte keyPairModule, eslt_Length publicKeyExponentSize, eslt_pRomByte publicKeyExponent)`	
Parameter	
workSpace [in,out]	Signature verification work space
keyPairModuleSize [in]	Size of RSA key pair module
keyPairModule [in]	RSA key pair module

publicKeyExponentSize [in]	Size of public RSA key pair exponent
publicKeyExponent [in]	Public RSA key pair exponent
Return code	
eslt_ErrorCode	ESL_ERC_NO_ERROR Operation successful
	ESL_ERC_PARAMETER_INVALID Input parameter is NULL
	ESL_ERC_WS_TOO_SMALL Work space too small
	ESL_ERC_RSA_PUBKEY_INVALID The public exponent is invalid
Functional Description	
Initialize signature verification with EMSA PSS encoding, based on RIPEMD160	
Particularities and Limitations	
-	
Call context	
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces	

Table 4-255 esl_initVerifyRSARIPEMD160_PSS

4.2.2.4.1.89 esl_updateVerifyRSARIPEMD160_PSS

Prototype	
`eslt_ErrorCode` **`esl_updateVerifyRSARIPEMD160_PSS`** `(VSECPRIM_P2VAR_PARA(eslt_WorkSpaceRSAPSSver) workSpace, eslt_Length inputSize, VSECPRIM_P2CONST_PARA(eslt_Byte) input)`	
Parameter	
workSpace [in,out]	Signature verification work space
inputSize [in]	Size of data to be hashed
input [in]	Data to be hashed
Return code	
eslt_ErrorCode	ESL_ERC_NO_ERROR Operation successful
	ESL_ERC_PARAMETER_INVALID Input parameter is NULL
	ESL_ERC_WS_STATE_INVALID Invalid state
	ESL_ERC_WS_TOO_SMALL Work space too small
	ESL_ERC_RIPEMD160_TOTAL_LENGTH_OVERFLOW Maximum total input length exceeded
Functional Description	
Update hash workspace with message to be signed During the RSA PSS signature verification scheme, the original message needs to be hashed. For RSA PSS RIPEMD160, the hash function is RMD160. This function stores the message into the hash	

workspace, contained in the parameter workSpace.
Particularities and Limitations
workSpace is initialized by esl_initVerifyRSARIPEMD160_PSS(..)
Call context
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces

Table 4-256 esl_updateVerifyRSARIPEMD160_PSS

4.2.2.4.1.90 esl_finalizeHashVerifyRSARIPEMD160_PSS

Prototype	
`eslt_ErrorCode` **`esl_finalizeHashVerifyRSARIPEMD160_PSS`** `(VSECPRIM_P2VAR_PARA(eslt_WorkSpaceRSAPSSver) workSpace, VSECPRIM_P2VAR_PARA(eslt_Byte) messageDigest)`	
Parameter	
workSpace [in,out]	Signature verification work space
messageDigest [out]	Message digest (hash) of previously hashed data
Return code	
eslt_ErrorCode	ESL_ERC_NO_ERROR Operation successful
	ESL_ERC_PARAMETER_INVALID Input parameter is NULL
	ESL_ERC_WS_STATE_INVALID Invalid state
	ESL_ERC_WS_TOO_SMALL Work space too small
Functional Description	
During the RSA PSS signature generation scheme, the original message needs to be hashed. For RSA PSS RIPEMD160, the hash function is RMD160. The message was passed to the hash workSpace before, which is contained in the parameter 'workSpace'. This function computes its hash value.	
Particularities and Limitations	
workSpace is initialized by esl_initVerifyRSARIPEMD160_PSS(..), esl_updateVerifyRSARIPEMD160(..) called before. The buffer referenced by messageDigest needs to be at least ESL_SIZEOF_RIPEMD160_DIGEST (= 20) byte long, since this buffer is passed to actRMD160Finalize.	
Call context	
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces	

Table 4-257 esl_finalizeHashVerifyRSARIPEMD160_PSS

4.2.2.4.1.91 esl_verifySigVerifyRSARIPEMD160_PSS

Prototype
`eslt_ErrorCode esl_verifySigVerifyRSARIPEMD160_PSS (VSECPRIM_P2VAR_PARA(eslt_WorkSpaceRSAPSSver) workSpace, VSECPRIM_P2CONST_PARA(eslt_Byte) messageDigest, eslt_Length saltSize, eslt_Length signatureSize, VSECPRIM_P2CONST_PARA(eslt_Byte) signature)`

Parameter	
workSpace [in,out]	Signature verification work space
messageDigest [in]	Message digest (hash) of previously hashed data
saltSize [in]	Expected size of salt, -1 if size of salt used during signature generation should be evaluated automatically
signatureSize [in]	Size of signature
signature [in]	Signature value

Return code	
eslt_ErrorCode	ESL_ERC_NO_ERROR Operation successful
	ESL_ERC_PARAMETER_INVALID Input parameter is NULL or saltSize too high for salt to be stored in data block
	ESL_ERC_WS_STATE_INVALID Invalid state
	ESL_ERC_WS_TOO_SMALL Work space too small
	ESL_ERC_RIPEMD160_TOTAL_LENGTH_OVERFLOW total length of data to be hashed (while calculation the saltedHashValue) is too long
	ESL_ERC_INPUT_INVALID hashID that is passed to esl_calcSaltedHashRSA_PSS does not correspond to a supported hash function
	ESL_ERC_ERROR an error occured during the call to esl_generateMaskMGF1
	ESL_ERC_RSA_SIGNATURE_OUT_OF_RANGE signature to be verified is greater than the module
	ESL_ERC_RSA_SIGNATURE_INVALID signature verification failed
	ESL_ERC_RSA_ENCODING_INVALID RSA encoded message length too short

Functional Description
Verifies the RSA PSS RIPEMD160 signature by applying the RSA verification primitive to the signature and decoding the message.

Particularities and Limitations
workSpace is initialized by esl_initVerifyRSARIPEMD160_PSS(..), esl_updateVerifyRSARIPEMD160(..) and esl_finalizeHashVerifyRSARIPEMD160_PSS(..) called before
The buffer referenced by messageDigest needs to be at least ESL_SIZEOF_RIPEMD160_DIGEST (= 20) byte long.

Call context
> TASK
> This function is Synchronous

> This function is Reentrant, for different workspaces

Table 4-258 esl_verifySigVerifyRSARIPEMD160_PSS

4.2.2.4.1.92 esl_verifySigVerifyRSARIPEMD160_PSS_AutoSalt

Prototype	
`eslt_ErrorCode esl_verifySigVerifyRSARIPEMD160_PSS_AutoSalt (VSECPRIM_P2VAR_PARA(eslt_WorkSpaceRSAPSSver) workSpace, VSECPRIM_P2CONST_PARA(eslt_Byte) messageDigest, eslt_Length signatureSize, VSECPRIM_P2CONST_PARA(eslt_Byte) signature)`	
Parameter	
workSpace [in,out]	Signature verification work space
messageDigest [in]	Message digest (hash) of previously hashed data
signatureSize [in]	Size of signature
signature [in]	Signature value
Return code	
eslt_ErrorCode	all possible error codes which can occur in esl_verifySigVerifyRSARIPEMD160_PSS
Functional Description	
Verifies the RSA PSS RIPEMD160 signature by encoding and signing the message. This function calls esl_verifySigVerifyRSARIPEMD160_PSS() with the given input parameters and saltSize = -1. This requires that the size of the salt used during signature generation was evaluated automatically.	
Particularities and Limitations	
workSpace is initialized by esl_initVerifyRSARIPEMD160_PSS(..), esl_updateVerifyRSARIPEMD160(..) and esl_finalizeHashVerifyRSARIPEMD160_PSS(..) called before The buffer referenced by messageDigest needs to be at least ESL_SIZEOF_RIPEMD160_DIGEST (= 20) byte long.	
Call context	
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces	

Table 4-259 esl_verifySigVerifyRSARIPEMD160_PSS_AutoSalt

4.2.2.4.1.93 esl_verifySigVerifyRSARIPEMD160_PSS_NoSalt

Prototype
`eslt_ErrorCode esl_verifySigVerifyRSARIPEMD160_PSS_NoSalt (VSECPRIM_P2VAR_PARA(eslt_WorkSpaceRSAPSSver) workSpace, VSECPRIM_P2CONST_PARA(eslt_Byte) messageDigest, eslt_Length signatureSize, VSECPRIM_P2CONST_PARA(eslt_Byte) signature)`

Parameter	
workSpace [in,out]	Signature verification work space
messageDigest [in]	Message digest (hash) of previously hashed data
signatureSize [in]	Size of signature
signature [in]	Signature value
Return code	
eslt_ErrorCode	all possible error codes which can occur in esl_verifySigVerifyRSARIPEMD160_PSS
Functional Description	
Verifies the RSA PSS RIPEMD160 signature by encoding and signing the message. This function calls esl_verifySigVerifyRSARIPEMD160_PSS() with the given input parameters and saltSize = 0. This requires that no salt was used during signature generation.	
Particularities and Limitations	
workSpace is initialized by esl_initVerifyRSARIPEMD160_PSS(..), esl_updateVerifyRSARIPEMD160(..) and esl_finalizeHashVerifyRSARIPEMD160_PSS(..) called before The buffer referenced by messageDigest needs to be at least ESL_SIZEOF_RIPEMD160_DIGEST (= 20) byte long.	
Call context	
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces	

Table 4-260 esl_verifySigVerifyRSARIPEMD160_PSS_NoSalt

4.2.2.4.1.94 esl_verifySigVerifyRSARIPEMD160_PSS_DigestLengthSalt

Prototype
`eslt_ErrorCode` **`esl_verifySigVerifyRSARIPEMD160_PSS_DigestLengthSalt`** `(VSECPRIM_P2VAR_PARA(eslt_WorkSpaceRSAPSSver) workSpace, VSECPRIM_P2CONST_PARA(eslt_Byte) messageDigest, eslt_Length signatureSize, VSECPRIM_P2CONST_PARA(eslt_Byte) signature)`

Parameter	
workSpace [in,out]	Signature verification work space
messageDigest [in]	Message digest (hash) of previously hashed data
signatureSize [in]	Size of signature
signature [in]	Signature value
Return code	
eslt_ErrorCode	all possible error codes which can occur in esl_verifySigVerifyRSARIPEMD160_PSS
Functional Description	
Verifies the RSA PSS RIPEMD160 signature by encoding and signing the message.	

This function calls esl_verifySigVerifyRSARIPEMD160_PSS() with the given input parameters and saltSize = ESL_SIZEOF_RIPEMD160_DIGEST. This requires that salt with size equal to message digest length was used during signature generation.
Particularities and Limitations
workSpace is initialized by esl_initVerifyRSARIPEMD160_PSS(..), esl_updateVerifyRSARIPEMD160(..) and esl_finalizeHashVerifyRSARIPEMD160_PSS(..) called before The buffer referenced by messageDigest needs to be at least ESL_SIZEOF_RIPEMD160_DIGEST (= 20) byte long.
Call context
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces

Table 4-261 esl_verifySigVerifyRSARIPEMD160_PSS_DigestLengthSalt

4.2.2.4.1.95 esl_finalizeVerifyRSARIPEMD160_PSS

Prototype	
`eslt_ErrorCode esl_finalizeVerifyRSARIPEMD160_PSS (VSECPRIM_P2VAR_PARA(eslt_WorkSpaceRSAPSSver) workSpace, eslt_Length saltSize, eslt_Length signatureSize, VSECPRIM_P2CONST_PARA(eslt_Byte) signature)`	
Parameter	
workSpace [in,out]	Signature verification work space
saltSize [in]	Expected size of salt, -1 if size of salt used during signature generation should be evaluated automatically
signatureSize [in]	Size of signature
signature [in]	Signature value
Return code	
eslt_ErrorCode	all possible error codes which can occur in esl_verifySigVerifyRSARIPEMD160_PSS
Functional Description	
Finalize message digest (hash) calculation and verify signature for RSA PSS RIPEMD160 This function calls esl_finalizeHashVerifyRSARIPEMD160_PSS() and esl_verifySigVerifyRSARIPEMD160_PSS() with the given input parameters.	
Particularities and Limitations	
workSpace is initialized by esl_initVerifyRSARIPEMD160_PSS(..), esl_updateVerifyRSARIPEMD160(..) called before	
Call context	
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces	

Table 4-262 esl_finalizeVerifyRSARIPEMD160_PSS

4.2.2.4.1.96 esl_finalizeVerifyRSARIPEMD160_PSS_AutoSalt

Prototype	
`eslt_ErrorCode` **`esl_finalizeVerifyRSARIPEMD160_PSS_AutoSalt`** `(VSECPRIM_P2VAR_PARA(eslt_WorkSpaceRSAPSSver) workSpace, eslt_Length signatureSize, VSECPRIM_P2CONST_PARA(eslt_Byte) signature)`	
Parameter	
workSpace [in,out]	Signature verification work space
signatureSize [in]	Size of signature
signature [in]	Signature value
Return code	
eslt_ErrorCode	all possible error codes which can occur in esl_verifySigVerifyRSARIPEMD160_PSS
Functional Description	
Finalize message digest (hash) calculation and verify signature for RSA PSS RIPEMD160 This function calls esl_finalizeVerifyRSARIPEMD160_PSS() with the given input parameters and saltSize = -1, thus it assumes that the size of the salt used during signature generation is evaluated automatically.	
Particularities and Limitations	
workSpace is initialized by esl_initVerifyRSARIPEMD160_PSS(..), esl_updateVerifyRSARIPEMD160(..) called before	
Call context	
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces	

Table 4-263 esl_finalizeVerifyRSARIPEMD160_PSS_AutoSalt

4.2.2.4.1.97 esl_finalizeVerifyRSARIPEMD160_PSS_NoSalt

Prototype	
`eslt_ErrorCode` **`esl_finalizeVerifyRSARIPEMD160_PSS_NoSalt`** `(VSECPRIM_P2VAR_PARA(eslt_WorkSpaceRSAPSSver) workSpace, eslt_Length signatureSize, VSECPRIM_P2CONST_PARA(eslt_Byte) signature)`	
Parameter	
workSpace [in,out]	Signature verification work space
signatureSize [in]	Size of signature
signature [in]	Signature value
Return code	
eslt_ErrorCode	all possible error codes which can occur in esl_verifySigVerifyRSARIPEMD160_PSS

Functional Description
Finalize message digest (hash) calculation and verify signature for RSA PSS RIPEMD160 This function calls esl_finalizeVerifyRSARIPEMD160_PSS() with the given input parameters and saltSize = 0, thus it requires that no salt was used during signature generation.
Particularities and Limitations
workSpace is initialized by esl_initVerifyRSARIPEMD160_PSS(..), esl_updateVerifyRSARIPEMD160(..) called before
Call context
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces

Table 4-264 esl_finalizeVerifyRSARIPEMD160_PSS_NoSalt

4.2.2.4.1.98 esl_finalizeVerifyRSARIPEMD160_PSS_DigestLengthSalt

Prototype	
`eslt_ErrorCode esl_finalizeVerifyRSARIPEMD160_PSS_DigestLengthSalt (VSECPRIM_P2VAR_PARA(eslt_WorkSpaceRSAPSSver) workSpace, eslt_Length signatureSize, VSECPRIM_P2CONST_PARA(eslt_Byte) signature)`	
Parameter	
workSpace [in,out]	Signature verification work space
signatureSize [in]	Size of signature
signature [in]	Signature value
Return code	
eslt_ErrorCode	all possible error codes which can occur in esl_verifySigVerifyRSARIPEMD160_PSS
Functional Description	
Finalize message digest (hash) calculation and verify signature for RSA PSS RIPEMD160 This function calls esl_finalizeVerifyRSARIPEMD160_PSS() with the given input parameters and saltSize = ESL_SIZEOF_RIPEMD160_DIGEST, thus it requires that a salt buffer of size equal to message digest was used during signature generation.	
Particularities and Limitations	
workSpace is initialized by esl_initVerifyRSARIPEMD160_PSS(..), esl_updateVerifyRSARIPEMD160(..) called before	
Call context	
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces	

Table 4-265 esl_finalizeVerifyRSARIPEMD160_PSS_DigestLengthSalt

4.2.2.4.1.99 esl_calcSaltedHashRSARIPEMD160_PSS

Prototype	
`eslt_ErrorCode esl_calcSaltedHashRSARIPEMD160_PSS (VSECPRIM_P2VAR_PARA(eslt_WorkSpaceRIPEMD160) wsHash, eslt_Length saltSize, VSECPRIM_P2CONST_PARA(eslt_Byte) salt, VSECPRIM_P2CONST_PARA(eslt_Byte) messageDigest, VSECPRIM_P2VAR_PARA(eslt_Byte) saltedHash)`	
Parameter	
wsHash [in,out]	Pointer to hash workspace
saltSize [in]	Length of salt
salt [in]	Input salt
messageDigest [in]	Input hash (RIPEMD-160)
saltedHash [out]	Output buffer for calculated hash (RIPEMD-160), has to be large enough to hold requested length
Return code	
eslt_ErrorCode	ESL_ERC_NO_ERROR Operation successful
	ESL_ERC_PARAMETER_INVALID workSpace or saltedHash null
	ESL_ERC_WS_TOO_SMALL wsHash too small
	ESL_ERC_RIPEMD160_TOTAL_LENGTH_OVERFLOW total length of data to be hashed too long
Functional Description	
Calculate salted hash value (RIPEMD-160). Based on a given hash and a random salt the hash of M' = (0x) 00 00 00 00 00 00 00 00 || hash || salt is calculated, as described in chapter 9.1.1, steps 5-6 and 9.1.2, steps 12-13 of PKCS#1 2.2	
Particularities and Limitations	
The buffer referenced by saltedHash needs to be at least ESL_SIZEOF_RIPEMD160_DIGEST (=20) byte long. The buffer referenced by messageDigest needs to be at least ESL_SIZEOF_RIPEMD160_DIGEST (= 20) byte long.	
Call context	
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces	

Table 4-266 esl_calcSaltedHashRSARIPEMD160_PSS

4.2.2.4.1.100 esl_generateMaskMGF1RSARIPEMD160_PSS

Prototype
`eslt_ErrorCode esl_generateMaskMGF1RSARIPEMD160_PSS (VSECPRIM_P2VAR_PARA(eslt_WorkSpaceRIPEMD160) wsHash, VSECPRIM_P2VAR_PARA(eslt_Byte) tempHash, eslt_Length seedLength, VSECPRIM_P2CONST_PARA(eslt_Byte) seed, eslt_Length maskLength, VSECPRIM_P2VAR_PARA(eslt_Byte) mask)`

Parameter	
wsHash [in,out]	Pointer to hash workspace
tempHash [in,out]	Temporary buffer for intermediate hash, has to be large enough to hold complete hash value
seedLength [in]	Length input seed
seed [in]	Input seed from which mask is generated
maskLength [in]	Length of mask to be generated
mask [out]	Output buffer for generated mask, has to be large enough to hold requested length
Return code	
eslt_ErrorCode	ESL_ERC_NO_ERROR Operation successful
	ESL_ERC_PARAMETER_INVALID Input parameter is NULL
	ESL_ERC_WS_TOO_SMALL Work space too small
	ESL_ERC_ERROR an error occurred
Functional Description	
Mask generation function (MGF1). MGF1 implementation as described in appendix B.2.1 of PKCS#1 2.2 Based on hash function (RIPEMD-160) this function generates a pseudorandom output string of arbitrary length depending on an input string (seed) of variable length	
Particularities and Limitations	
buffer referenced by tempHash has to be at least of length ESL_SIZEOF_RIPEMD160_DIGEST buffer referenced by mask has to be at least of length maskLength	
Call context	
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces	

Table 4-267 esl_generateMaskMGF1RSARIPEMD160_PSS

4.2.2.4.1.101 esl_calcSaltedHashRSA_PSS

Prototype
`eslt_ErrorCode` **`esl_calcSaltedHashRSASHA256_PSS`** `(VSECPRIM_P2VAR_PARA(eslt_WorkSpaceSHA256) wsHash, eslt_Length saltSize, VSECPRIM_P2CONST_PARA(eslt_Byte) salt, VSECPRIM_P2CONST_PARA(eslt_Byte) messageDigest, VSECPRIM_P2VAR_PARA(eslt_Byte) saltedHash)`

Parameter	
wsHash [in,out]	Pointer to hash workspace union, containing either a SHA1, a SHA2 or a RMD160 workspace
saltSize [in]	Length of salt
salt [in]	Input salt
messageDigest [in]	Input hash value of the original message, will be hashed again here
saltedHash [in,out]	Output buffer for calculated digest, has to be large enough to hold requested length

hashID [in]	Parameter to determine the hash function to be used
Return code	
eslt_ErrorCode	ESL_ERC_NO_ERROR Operation successful
	ESL_ERC_TOTAL_LENGTH_OVERFLOW total length of data to be hashed is too long
	ESL_ERC_INPUT_INVALID hashID does not correspond to a supported hash function
Functional Description	
Calculate salted hash value for RSA PSS signature generation and verification schemes, independent of underlying hash function. Based on a given hash and a random salt the hash of M' = (0x) 00 00 00 00 00 00 00 00 || hash(orig. msg) || salt is calculated, as described in chapter 9.1.1, steps 5-6 and 9.1.2, steps 12-13 of PKCS#1 2.2. The parameter hashID indicates which specific hash function is used.	
Particularities and Limitations	
wsHash, salt, messageDigest and saltedHash have to be valid pointers wsHash has to be initialized buffer referenced by saltedHash has to be at least of length ESL_SIZEOF_SHA256_DIGEST	
Call context	
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces	

Table 4-268 esl_calcSaltedHashRSASHA256_PSS

4.2.2.4.1.102 esl_initSignRSA_prim

Prototype	
`eslt_ErrorCode` **`esl_initSignRSA_prim`** `(VSECPRIM_P2VAR_PARA(eslt_WorkSpaceRSAsig_prim) workSpace, eslt_Length keyPairModuleSize, eslt_pRomByte keyPairModule, eslt_Length privateKeyExponentSize, eslt_pRomByte privateKeyExponent)`	
Parameter	
privateKeyExponentSize [in]	the byte length of the private exponent d
keyPairModuleSize [in]	the byte length of the modulus n
privateKeyExponent [in]	pointer to the private exponent d
workSpace [in,out]	(in) algorithm context buffer, initialized by esl_initWorkSpaceHeader(..) function (out) initialized algorithm context structure
keyPairModule [in]	pointer to the modulus n
Return code	
eslt_ErrorCode	ESL_ERC_PARAMETER_INVALID input parameter is NULL
	ESL_ERC_WS_TOO_SMALL work space too small
	ESL_ERC_RSA_PRIVKEY_INVALID the private exponent d is invalid
	ESL_ERC_NO_ERROR else

Functional Description
This function initializes the RSA signature primitive (plain version).
Particularities and Limitations
-
Call context
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces

Table 4-269 esl_initSignRSA_prim

4.2.2.4.1.103 esl_signRSA_prim

Prototype	
`eslt_ErrorCode esl_signRSA_prim (VSECPRIM_P2VAR_PARA(eslt_WorkSpaceRSAsig_prim) workSpace, eslt_Length messageSize, VSECPRIM_P2CONST_PARA(eslt_Byte) message, VSECPRIM_P2VAR_PARA(eslt_Length) signatureSize, VSECPRIM_P2VAR_PARA(eslt_Byte) signature)`	
Parameter	
message [in]	pointer to the input
signatureSize [in,out]	pointer to byte length of the output buffer
signature [in,out]	pointer to the output buffer
workSpace [in,out]	(in) algorithm context buffer, initialized by esl_initWorkSpaceHeader(..) function (out) initialized algorithm context structure
messageSize [in]	the byte length of the input
Return code	
eslt_ErrorCode	ESL_ERC_PARAMETER_INVALID input parameter is NULL
	ESL_ERC_RSA_MESSAGE_OUT_OF_RANGE messag > module-1
	ESL_ERC_OUTPUT_SIZE_TOO_SHORT *signatureSize is too small
	ESL_ERC_WS_TOO_SMALL workSpace too small
	ESL_ERC_WS_STATE_INVALID workSpace in invalid state
	ESL_ERC_NO_ERROR else
Functional Description	
This function performs the RSA signature generation (RSA_SP1).	
Particularities and Limitations	
workSpace state set by esl_initSignRSA_prim(..) the buffer referenced by signature must be at least of length signatureSize	
Call context	
> TASK > This function is Synchronous	

> This function is Reentrant, for different workspaces

Table 4-270 esl_signRSA_prim

4.2.2.4.1.104 esl_initVerifyRSA_prim

Prototype	
`eslt_ErrorCode esl_initVerifyRSA_prim (VSECPRIM_P2VAR_PARA(eslt_WorkSpaceRSAver_prim) workSpace, eslt_Length keyPairModuleSize, eslt_pRomByte keyPairModule, eslt_Length publicKeyExponentSize, eslt_pRomByte publicKeyExponent)`	
Parameter	
publicKeyExponentSize [in]	the byte length of the public exponent e
keyPairModuleSize [in]	the byte length of the modulus n
publicKeyExponent [in]	pointer to the public exponent e
workSpace [in,out]	(in) algorithm context buffer, initialized by esl_initWorkSpaceHeader(..) function (out) initialized algorithm context structure
keyPairModule [in]	pointer to the modulus n
Return code	
eslt_ErrorCode	ESL_ERC_PARAMETER_INVALID input parameter is NULL
	ESL_ERC_WS_TOO_SMALL work space too small
	ESL_ERC_RSA_PUBKEY_INVALID the public exponent is
	invalid
	ESL_ERC_NO_ERROR else
Functional Description	
This function initializes the RSA verification primitive.	
Particularities and Limitations	
-	
Call context	
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces	

Table 4-271 esl_initVerifyRSA_prim

4.2.2.4.1.105 esl_verifyRSA_prim

Prototype
`eslt_ErrorCode esl_verifyRSA_prim (VSECPRIM_P2VAR_PARA(eslt_WorkSpaceRSAver_prim) workSpace, eslt_Length signatureSize, VSECPRIM_P2CONST_PARA(eslt_Byte) signature,`

VSECPRIM_P2VAR_PARA(eslt_Length) messageSize, VSECPRIM_P2VAR_PARA(eslt_Byte) message)	
Parameter	
messageSize [in,out]	pointer to byte length of the output buffer
message [in,out]	pointer to the output buffer
signature [in]	pointer to the given signature
workSpace [in,out]	(in) algorithm context buffer, initialized by esl_initWorkSpaceHeader(..) function (out) initialized algorithm context structure
signatureSize [in]	the byte length of the given signature
Return code	
eslt_ErrorCode	ESL_ERC_PARAMETER_INVALID input parameter is NULL
	ESL_ERC_WS_TOO_SMALL workSpace too small
	ESL_ERC_RSA_SIGNATURE_OUT_OF_RANGE (int) signature > module -1
	ESL_ERC_OUTPUT_SIZE_TOO_SHORT *codeSize is too small
	ESL_ERC_WS_STATE_INVALID workSpace in invalid state
	ESL_ERC_NO_ERROR else
Functional Description	
This function performs the RSA verification primitive (RSA_VP1).	
Particularities and Limitations	
workSpace state set by esl_initVerifyRSA_prim(..) the buffer referenced by message must be at least of length messageSize	
Call context	
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces	

Table 4-272 esl_verifyRSA_prim

4.2.2.4.1.106 esl_initSignRSACRT_prim

Prototype	
eslt_ErrorCode **esl_initSignRSACRT_prim** (VSECPRIM_P2VAR_PARA(eslt_WorkSpaceRSACRTsig_prim) workSpace, eslt_Length keyPairPrimePSize, eslt_pRomByte keyPairPrimeP, eslt_Length keyPairPrimeQSize, eslt_pRomByte keyPairPrimeQ, eslt_Length privateKeyExponentDPSize, eslt_pRomByte privateKeyExponentDP,, eslt_Length privateKeyExponentDQSize, eslt_pRomByte privateKeyExponentDQ, eslt_Length privateKeyInverseQISize, eslt_pRomByte privateKeyInverseQI)	
Parameter	
keyPairPrimeQSize [in]	the byte length of the prime q
keyPairPrimeP [in]	pointer to prime p

privateKeyInverseQISize [in]	the byte length of CRT key component qInv
privateKeyInverseQI [in]	pointer to CRT key component qInv
privateKeyExponentDPSize [in]	the byte length of dp
keyPairPrimeQ [in]	pointer to prime q
workSpace [in,out]	(in) algorithm context buffer, initialized by esl_initWorkSpaceHeader(..) function (out) initialized algorithm context structure
privateKeyExponentDQSize [in]	the byte length of dq
privateKeyExponentDP [in]	pointer to CRT key component dp
privateKeyExponentDQ [in]	pointer to CRT key component dq
keyPairPrimePSize [in]	the byte length of the prime p
Return code	
eslt_ErrorCode	ESL_ERC_PARAMETER_INVALID input parameter is NULL
	ESL_ERC_WS_TOO_SMALL work space too small
	ESL_ERC_RSA_PRIVKEY_INVALID the private key
	is invalid
	ESL_ERC_NO_ERROR else
Functional Description	
This function initializes the RSA signature primitive (CRT version).	
Particularities and Limitations	
workSpace is initialized by esl_initWorkSpaceHeader(..)	
Call context	
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces	

Table 4-273 esl_initSignRSACRT_prim

4.2.2.4.1.107 esl_signRSACRT_prim

Prototype	
`eslt_ErrorCode esl_signRSACRT_prim (VSECPRIM_P2VAR_PARA(eslt_WorkSpaceRSACRTsig_prim) workSpace, eslt_Length messageSize, VSECPRIM_P2CONST_PARA(eslt_Byte) message, VSECPRIM_P2VAR_PARA(eslt_Length) signatureSize, VSECPRIM_P2VAR_PARA(eslt_Byte) signature)`	
Parameter	
message [in]	pointer to the input
signatureSize [in,out]	pointer to byte length of the output buffer

signature [in,out]	pointer to the output buffer
workSpace [in,out]	(in) algorithm context buffer, initialized by esl_initWorkSpaceHeader(..) function (out) initialized algorithm context structure
messageSize [in]	the byte length of the input
Return code	
eslt_ErrorCode	ESL_ERC_PARAMETER_INVALID input parameter is NULL
	ESL_ERC_RSA_MESSAGE_OUT_OF_RANGE (int) code > n-1
	ESL_ERC_OUTPUT_SIZE_TOO_SHORT *signatureSize is too small
	ESL_ERC_WS_STATE_INVALID invalid Algo
	ESL_ERC_NO_ERROR else
Functional Description	
This function performs the RSA signature operation (RSA_SP1).	
Particularities and Limitations	
workSpace is initialized by esl_initSignRSACRT_prim(..)	
Call context	
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces	

Table 4-274 esl_signRSACRT_prim

4.2.2.4.2 EC Based Signature Algorithms

4.2.2.4.2.1 esl_initSignDSAEcP_prim

Prototype
`eslt_ErrorCode esl_initSignDSAEcP_prim (VSECPRIM_P2VAR_PARA(eslt_WorkSpaceEcP) workSpace, VSECPRIM_P2ROMCONST_PARA(eslt_EccDomain) domain, VSECPRIM_P2ROMCONST_PARA(eslt_EccDomainExt) domainExt, VSECPRIM_P2ROMCONST_PARA(eslt_EccSpeedUpExt) speedUpExt)`

Parameter	
domain [in]	pointer to domain parameter structure
speedUpExt [in]	pointer to precomputation structure
workSpace [in,out]	(in) algorithm context buffer, initialized by esl_initWorkSpaceHeader(..) function (out) initialized algorithm context structure
domainExt [in]	pointer to domain parameter extension structure
Return code	
eslt_ErrorCode	ESL_ERC_PARAMETER_INVALID input parameter is NULL
	ESL_ERC_WS_TOO_SMALL work space too small
	ESL_ERC_ECC_DOMAIN_INVALID domain structure is invalid
	ESL_ERC_ECC_DOMAINEXT_INVALID domainExt structure is invalid

	ESL_ERC_ECC_SPEEDUPEXT_INVALID speedUpExt structure is invalid
	ESL_ERC_NO_ERROR else
Functional Description	
This function initializes the DSAEcP sign algorithm.	
Particularities and Limitations	
workSpace is initialized by esl_initWorkSpaceHeader(..) domain and domainExt have to be valid ASN1 structures for EC domain parameters speedUpExt has to be a valid ASN1 struct for precomputed EC points The addresses of domain, domainExt and speedUpExt have to be valid until returning from esl_initSignDSAEcP_prim(), because the pointers to the data stored in domain, domainExt and speedUpExt are stored in the workspace. These pointers are used as long as the workspace is used. For information about the expected data format in domain, domainExt and speedUpExt see Chapter 4.1.	
Call context	
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces	

Table 4-275 esl_initSignDSAEcP_prim

4.2.2.4.2.2 esl_signDSAEcP_prim

Prototype	
`eslt_ErrorCode esl_signDSAEcP_prim (VSECPRIM_P2VAR_PARA(eslt_WorkSpaceEcP) workSpace, eslt_Length messageLength, VSECPRIM_P2CONST_PARA(eslt_Byte) message, VSECPRIM_P2CONST_PARA(eslt_Byte) privateKey, VSECPRIM_P2VAR_PARA(eslt_Length) signature_rLength, VSECPRIM_P2VAR_PARA(eslt_Byte) signatuure_r, VSECPRIM_P2VAR_PARA(eslt_Length) signature_sLength, VSECPRIM_P2VAR_PARA(eslt_Byte) signature_s)`	
Parameter	
signature_r [in,out]	(in) pointer to signature_r buffer (out) signature_r
signature_s [in,out]	(in) pointer to signature_s buffer (out) signature_s
privateKey [in]	pointer to private key (of length getLengthOfEcPprivateKey(domain))
message [in]	pointer to message
workSpace [in,out]	algorithm context buffer, initialized by esl_initWorkSpaceHeader(..) function
signature_sLength [in,out]	(in) signature_s buffer length in bytes (>= esl_getLengthOfEcPsignature_comp(domain)) (out) signature_s length in bytes
messageLength [in]	message length in bytes (<= esl_getMaxLengthOfEcPmessage(domain))
signature_rLength [in,out]	(in) signature_r buffer length in bytes (>= esl_getLengthOfEcPsignature_comp(domain)) (out) signature_r length in bytes
Return code	
eslt_ErrorCode	ESL_ERC_PARAMETER_INVALID input parameter is NULL

	ESL_ERC_WS_TOO_SMALL work space too small
	ESL_ERC_WS_STATE_INVALID wrong work space state
	ESL_ERC_OUTPUT_SIZE_TOO_SHORT signature buffer too small
	ESL_ERC_ECC_MESSAGE_TOO_LONG wrong messageLength
	ESL_ERC_ECC_PRIVKEY_INVALID privateKey is invalid
	ESL_ERC_INTERNAL_ERROR calculation of affine point representation from projective Montgomery representation failed
	All possible error codes which can occur in esl_getBytesRNG
	ESL_ERC_NO_ERROR else
Functional Description	
This function signs the given message with the passed private key.	
Particularities and Limitations	
The buffer referenced by signature_r must provide at least signature_rLength bytes. The buffer referenced by signature_s must provide at least signature_sLength bytes.	
Call context	
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces	

Table 4-276 esl_signDSAEcP_prim

Note
vSecPrim expects the message which is to be signed to be hashed already, when esl_signDSAEcP_prim is called, in order to comply with [14]. If vSecPrim is used in combination with LibCv, this is fulfilled automatically.

4.2.2.4.2.3 esl_initVerifyDSAEcP_prim

Prototype	
`eslt_ErrorCode esl_initVerifyDSAEcP_prim (VSECPRIM_P2VAR_PARA(eslt_WorkSpaceEcP) workSpace, VSECPRIM_P2ROMCONST_PARA(eslt_EccDomain) domain, VSECPRIM_P2ROMCONST_PARA(eslt_EccDomainExt) domainExt)`	
Parameter	
domain [in]	pointer to domain parameter structure
workSpace [in,out]	(in) algorithm context buffer, initialized by esl_initWorkSpaceHeader(..) function (out) initialized algorithm context structure
domainExt [in]	pointer to domain parameter extension structure
Return code	
eslt_ErrorCode	ESL_ERC_PARAMETER_INVALID input parameter is NULL

	ESL_ERC_WS_TOO_SMALL work space too small
	ESL_ERC_ECC_DOMAIN_INVALID domain structure is invalid
	ESL_ERC_ECC_DOMAINEXT_INVALID domainExt structure is invalid
	ESL_ERC_NO_ERROR else
Functional Description	
This function initializes the DSAEcP verification algorithm.	
Particularities and Limitations	
workSpace is initialized by esl_initWorkSpaceHeader(..) domain and domainExt have to be valid ASN1 structures for EC domain parameters The addresses of domain and domainExt have to be valid until returning from esl_verifyDSAEcP_prim(), because the pointers to the data stored in domain and domainExt are stored in the workspace. These pointers are used as long as the workspace is used. For information about the expected data format in domain and domainExt see Chapter 4.1.	
Call context	
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces	

Table 4-277 esl_initVerifyDSAEcP_prim

4.2.2.4.2.4 esl_verifyDSAEcP_prim

Prototype	
`eslt_ErrorCode esl_verifyDSAEcP_prim (VSECPRIM_P2VAR_PARA(eslt_WorkSpaceEcP) workSpace, eslt_Length messageLength, VSECPRIM_P2CONST_PARA(eslt_Byte) message, VSECPRIM_P2CONST_PARA(eslt_Byte) publicKey_x, VSECPRIM_P2CONST_PARA(eslt_Byte) publicKey_y, eslt_Length signature_rLength, VSECPRIM_P2CCONST_PARA(eslt_Byte) signature_r, eslt_Length signature_sLength, VSECPRIM_P2CONST_PARA(eslt_Byte) signature_s)`	
Parameter	
workSpace [in,out]	algorithm context buffer, initialized by esl_initWorkSpaceHeader(..) function
messageLength [in]	message length in bytes (<= esl_getMaxLengthOfEcPmessage(domain))
message [in]	pointer to message
publicKey_x [in]	pointer to x-coordinate of public key (of length getLengthOfEcPpublicKey_comp(domain))
publicKey_y [in]	pointer to y-coordinate of public key (of length getLengthOfEcPpublicKey_comp(domain))
signature_rLength [in]	signature_r length in bytes
signature_r [in]	pointer to signature_r
signature_sLength [in]	signature_slength in bytes
signature_s [in]	pointer to signature_s
Return code	
eslt_ErrorCode	ESL_ERC_PARAMETER_INVALID input parameter is NULL

	ESL_ERC_WS_TOO_SMALL work space too small
	ESL_ERC_WS_STATE_INVALID wrong work space state
	ESL_ERC_ECC_MESSAGE_TOO_LONG wrong messageLength
	ESL_ERC_ECC_PUBKEY_INVALID publicKey is invalid
	ESL_ERC_ECC_SIGNATURE_INVALID invalid signature
	ESL_ERC_INTERNAL_ERROR calculation of affine point representation from projective Montgomery representation failed
	ESL_ERC_NO_ERROR else

Functional Description

This function verifies the given signature of the input message with the passed public key.

Particularities and Limitations

workSpace is initialized by esl_initVerifyDSAEcP_prim(..)
The buffer referenced by signature_r must provide at least signature_rLength bytes.
The buffer referenced by signature_s must provide at least signature_sLength bytes.

If one of the coordinates publicKey_x or publicKey_y is shorter than getLengthOfEcPpublicKey_comp(domain) bytes, the coordinate must be padded with zeros to reach a total length of getLengthOfEcPpublicKey_comp(domain) bytes. 'domain' corresponds to the EC domain parameter used in esl_initVerifyDSAEcP_prim().

The only return code that indicates that the signature is valid is ESL_ERC_NO_ERROR. Any other return code implies that the signature shall not be accepted as valid.

Call context

- TASK
- This function is Synchronous
- This function is Reentrant, for different workspaces

Table 4-278 esl_verifyDSAEcP_prim



Note
vSecPrim expects the message which is to be verified to be hashed already, when esl_verifyDSAEcP_prim is called, in order to comply with [14]. If vSecPrim is used in combination with LibCv, this is fulfilled automatically.

4.2.2.4.2.5 esl_initSignEdDSA

Prototype

```
eslt_ErrorCode esl_initSignEdDSA (VSECPRIM_P2VAR_PARA(eslt_WorkSpaceEd25519)
workSpace, eslt_Size16 curve, const eslt_Byte instance,
VSECPRIM_P2CONST_PARA(eslt_Byte) context, const eslt_Length contextLength)
```

Parameter

workSpace [in,out]	algorithm context buffer, initialized by esl_initWorkSpaceHeader(..) function
curve [in]	underlying curve; supports

	- ESL_Curve25519
instance [in]	EdDSA instance to use; one of - ESL_INSTANCE_Ed25519 - ESL_INSTANCE_Ed25519ctx - ESL_INSTANCE_Ed25519ph
context [in]	Context data pointer
contextLength [in]	The length of the context data
Return code	
eslt_ErrorCode	ESL_ERC_NO_ERROR - success
	ESL_ERC_PARAMETER_INVALID - some parameter is NULL
	ESL_ERC_CURVE_NOT_SUPPORTED - unknown curve
	ESL_ERC_INSTANCE_NOT_SUPPORTED - unknown EdDSA instance
	ESL_ERC_WS_TOO_SMALL - work space too small
Functional Description	
Init EdDSA. This function initializes the EdDSA signature.	
Particularities and Limitations	
workSpace is initialized by esl_initWorkSpaceHeader(..)	
Call context	
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces	

Table 4-279 esl_initSignEdDSA

4.2.2.4.2.6 esl_signEdDSA

Prototype
`eslt_ErrorCode esl_signEdDSA (VSECPRIM_P2VAR_PARA(eslt_WorkSpaceEd25519) workSpace, VSECPRIM_P2CONST_PARA(eslt_Byte) input, eslt_Length inputLength, VSECPRIM_P2CONST_PARA(eslt_Byte) secretKey, VSECPRIM_P2CONST_PARA(eslt_Byte) publicKey, VSECPRIM_P2VAR_PARA(eslt_Byte) signature, VSECPPRIM_P2VAR_PARA(eslt_Length) signatureLength)`

Parameter	
workSpace [in,out]	algorithm context buffer, initialized by esl_initWorkSpaceHeader(..) function
input [in]	Pointer to the input data
inputLength [in]	The length of the provided input data
secretKey [in]	Pointer to the secret key of length 32 byte to use for signature
publicKey [in]	Pointer to the public key of length 32 byte. Pass null pointer to calculate from private key.
signature [out]	Pointer to where the signature shall be stored.

signatureLength [in,out]	Pointer to where the size of the buffer is provided
Return code	
eslt_ErrorCode	ESL_ERC_NO_ERROR - success
	ESL_ERC_PARAMETER_INVALID - some parameter is NULL
	ESL_ERC_WS_STATE_INVALID - work space state invalid
	ESL_ERC_WS_TOO_SMALL - work space too small
	ESL_ERC_BUFFER_TOO_SMALL - buffer for signature too small
	ESL_ERC_TOTAL_LENGTH_OVERFLOW - internal overflow (== message too long - SHA-512)
Functional Description	
Sign data EdDSA. This function signs the given input data.	
Particularities and Limitations	
workSpace is initialized by esl_initWorkSpaceHeader(..)	
Call context	
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces	

Table 4-280 esl_signEdDSA

4.2.2.4.2.7 esl_initVerifyEdDSA

Prototype	
`eslt_ErrorCode esl_initVerifyEdDSA (VSECPRIM_P2VAR_PARA(eslt_WorkSpaceEd25519) workSpace, eslt_Size16 curve, const eslt_Byte instance, VSECPRIM_P2CONST_PARA(eslt_Byte) context, const eslt_Length contextLength)`	
Parameter	
workSpace [in,out]	algorithm context buffer, initialized by esl_initWorkSpaceHeader(..) function
curve [in]	Underlying curve; supports - ESL_Curve25519
instance [in]	EdDSA instance to use; one of - ESL_INSTANCE_Ed25519 - ESL_INSTANCE_Ed25519ctx - ESL_INSTANCE_Ed25519ph
context [in]	Pointer to the context data
contextLength [in]	Size of provided context data
Return code	
eslt_ErrorCode	ESL_ERC_NO_ERROR - success
	ESL_ERC_PARAMETER_INVALID - some parameter is NULL

	ESL_ERC_CURVE_NOT_SUPPORTED - unknown curve
	ESL_ERC_INSTANCE_NOT_SUPPORTED - unknown EdDSA instance
	ESL_ERC_WS_TOO_SMALL - work space too small
Functional Description	
Init verification for EdDSA. This function initializes the EdDSA verification.	
Particularities and Limitations	
workSpace is initialized by esl_initWorkSpaceHeader(..)	
Call context	
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces	

Table 4-281 esl_initVerifyEdDSA

4.2.2.4.2.8 esl_verifyEdDSA

Prototype	
`eslt_ErrorCode esl_verifyEdDSA (VSECPRIM_P2VAR_PARA(eslt_WorkSpaceEd25519) workSpace, VSECPRIM_P2CONST_PARA(eslt_Byte) input, eslt_Length inputLength, VSECPRIM_P2CONST_PARA(eslt_Byte) publicKey, VSECPRIM_P2CONST_PARA(eslt_Byte) signature, eslt_Length signatureLength)`	
Parameter	
workSpace [in,out]	algorithm context buffer, initialized by esl_initWorkSpaceHeader(..) function
input [in]	Pointer to the input data.
inputLength [in]	The length of the input data
publicKey [in]	Pointer to the public key
signature [in]	Pointer to the signature to be validated against input data
signatureLength [in]	Length of the signature
Return code	
eslt_ErrorCode	ESL_ERC_NO_ERROR - success
	ESL_ERC_PARAMETER_INVALID - some parameter is NULL
	ESL_ERC_WS_STATE_INVALID - work space state invalid
	ESL_ERC_WS_TOO_SMALL - work space too small
	ESL_ERC_TOTAL_LENGTH_OVERFLOW - internal overflow (== message too long - SHA-512)
	ESL_ERC_SIGNATURE_INVALID - the signature does NOT verify
Functional Description	
EdDSA verification. This function verifies given signature against the given data.	

Particularities and Limitations
workSpace is initialized by esl_initWorkSpaceHeader(..)
Call context
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces

Table 4-282 esl_verifyEdDSA

4.2.2.4.2.9 esl_updateEdDSA

Prototype	
`eslt_ErrorCode esl_updateEdDSA (VSECPRIM_P2VAR_PARA(eslt_WorkSpaceEd25519) workSpace, VSECPRIM_P2CONST_PARA(eslt_Byte) input, eslt_Length inputLength)`	
Parameter	
workSpace [in,out]	algorithm context buffer, initialized by esl_initWorkSpaceHeader(..) function
input [in]	Pointer to the input data.
inputLength [in]	The length of the input data
Return code	
eslt_ErrorCode	ESL_ERC_NO_ERROR - success
	ESL_ERC_PARAMETER_INVALID - some parameter is NULL
	ESL_ERC_WS_STATE_INVALID - work space state invalid
	ESL_ERC_WS_TOO_SMALL - work space too small
	ESL_ERC_TOTAL_LENGTH_OVERFLOW - internal overflow (== message too long - SHA-512)
Functional Description	
Perform hashing. This function updates the internal hash value in a pre-hash EdDSA signature OR verification.	
Particularities and Limitations	
workSpace is initialized by esl_initWorkSpaceHeader(..)	
Call context	
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces	

Table 4-283 esl_updateEdDSA

4.2.2.4.3 Key Generation Algorithms

4.2.2.4.3.1 esl_initGenerateKeyPairEdDSA

Prototype	
`eslt_ErrorCode esl_initGenerateKeyPairEdDSA (VSECPRIM_P2VAR_PARA(eslt_WorkSpaceEd25519) workSpace, eslt_Size16 curve)`	
Parameter	
workSpace [in,out]	algorithm context buffer, initialized by esl_initWorkSpaceHeader(..) function
curve [in]	Underlying curve; supports - ESL_Curve25519
Return code	
eslt_ErrorCode	ESL_ERC_NO_ERROR - success
	ESL_ERC_PARAMETER_INVALID - some parameter is NULL
	ESL_ERC_CURVE_NOT_SUPPORTED - unknown curve
	ESL_ERC_WS_TOO_SMALL - work space too small
Functional Description	
Initialize generated key pair. This function initializes the EdDSA key-pair generation.	
Particularities and Limitations	
workSpace is initialized by esl_initWorkSpaceHeader(..)	
Call context	
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces	

Table 4-284 esl_initGenerateKeyPairEdDSA

4.2.2.4.3.2 esl_generateKeyPairEdDSA

Prototype	
`eslt_ErrorCode esl_generateKeyPairEdDSA (VSECPRIM_P2VAR_PARA(eslt_WorkSpaceEd25519) workSpace, VSECPRIM_P2VAR_PARA(eslt_Byte) secretKey, VSECPRIM_P2VAR_PARA(eslt_Length) secretKeyLength, VSECPRIM_P2VAR_PARA(eslt_Byte) publicKey, VSECPRIM_P2VAR_PARA(eslt_Length) publicKeyLength)`	
Parameter	
workSpace [in,out]	algorithm context buffer, initialized by esl_initWorkSpaceHeader(..) function
secretKey [out]	Pointer to where the secret key shall be stored
secretKeyLength [in,out]	Length of the provided buffer/Length of the written data
publicKey [out]	Pointer to where the public key shall be stored
publicKeyLength [in,out]	Length of the provided buffer/Length of the written data

Return code	
eslt_ErrorCode	ESL_ERC_NO_ERROR - success
	ESL_ERC_PARAMETER_INVALID - some parameter is NULL
	ESL_ERC_WS_STATE_INVALID - work space state invalid
	ESL_ERC_WS_TOO_SMALL - work space too small
	ESL_ERC_BUFFER_TOO_SMALL - buffer for private or public key too small
	ESL_ERC_RNG_FAILED - the random number generator failed
Functional Description	
Generate Key Pair. This function generates an EdDSA key-pair.	
Particularities and Limitations	
workSpace is initialized by esl_initWorkSpaceHeader(..)	
Call context	
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces	

Table 4-285 esl_generateKeyPairEdDSA

4.2.2.4.3.3 esl_initGenerateKeyEcP_prim

Prototype	
`eslt_ErrorCode esl_initGenerateKeyEcP_prim (VSECPRIM_P2VAR_PARA(eslt_WorkSpaceEcP) workSpace, VSECPRIM_P2ROMCONST_PARA(eslt_EccDomain) domain, VSECPRIM_P2ROMCONST_PARA(eslt_EccDomainExt) domainExt, VSECPRIM_P2ROMCONST_PARA(eslt_EccSpeedUpExt) speedUpExt)`	
Parameter	
workSpace [in,out]	(in) algorithm context buffer, initialized by esl_initWorkSpaceHeader(..) function (out) initialized algorithm context structure
domain [in]	pointer to domain parameter structure
domainExt [in]	pointer to domain parameter extension structure
speedUpExt [in]	pointer to precomputation structure
Return code	
eslt_ErrorCode	ESL_ERC_PARAMETER_INVALID input parameter is NULL
	ESL_ERC_WS_TOO_SMALL work space too small
	ESL_ERC_ECC_DOMAIN_INVALID domain structure is invalid
	ESL_ERC_ECC_DOMAINEXT_INVALID domainExt structure is invalid
	ESL_ERC_ECC_SPEEDUPEXT_INVALID speedUpExt structure is invalid
	ESL_ERC_NO_ERROR else

Functional Description
This function initializes the EcP key generation.
Particularities and Limitations
workSpace is initialized by esl_initWorkSpaceHeader(..) domain and domainExt have to be valid ASN1 structures for EC domain parameters speedUpExt has to be a valid ASN1 struct for precomputed EC points The addresses of domain, domainExt and speedUpExt have to be valid until returning from esl_generateKeyEcP_prim(), because the pointers to the data stored in domain, domainExt and speedUpExt are stored in the workspace. These pointers are used as long as the workspace is used. For information about the expected data format in domain, domainExt and speedUpExt see Chapter 4.1.
Call context
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces

Table 4-286 esl_initGenerateKeyEcP_prim

4.2.2.4.3.4 esl_generateKeyEcP_prim

Prototype

```
eslt_ErrorCode esl_generateKeyEcP_prim (VSECPRIM_P2VAR_PARA(eslt_WorkSpaceEcP) workSpace, VSECPRIM_P2VAR_PARA(eslt_Byte) privateKey, VSECPRIM_P2VAR_PARA(eslt_Byte) publicKey_x, VSECPRIM_P2VAR_PARA(eslt_Byte) publicKey_y)
```

Parameter	
workSpace [in,out]	algorithm context buffer, initialized by esl_initWorkSpaceHeader(..) function
privateKey [in,out]	(in) buffer to store the private key d (of length getLengthOfEcPprivateKey(domain)) (out) the private key d
publicKey_x [in,out]	(in) buffer to store x-coordinate of the public key (out) x-coordinate of the public key Q
publicKey_y [in,out]	(in) buffer to store y-coordinate of the public key (both of length getLengthOfEcPpublicKey_comp(domain)) (out) y-coordinate of the public key Q
Return code	
eslt_ErrorCode	ESL_ERC_PARAMETER_INVALID input parameter is NULL
	ESL_ERC_WS_TOO_SMALL work space too small
	ESL_ERC_WS_STATE_INVALID wrong work space state
	ESL_ERC_ECC_PRIVKEY_INVALID privateKey invalid
	ESL_ERC_ECC_INTERNAL_ERROR calculation of affine point representation from projective Montgomery representation failed or generation of random number by call-out failed
	ESL_ERC_NO_ERROR else

Functional Description
This function generates an ECC key pair (d=privateKey, Q=(publicKey_x, publicKey_y)).
Particularities and Limitations
workSpace is initialized by esl_initGenerateKeyEcP_prim(..) The pointers privateKey, publicKey_x and publicKey_y passed to the function esl_generateKeyEcP_prim must reference valid memory locations and the sizes of the arrays referenced by the parameters privateKey, publicKey_x and publicKey_y must be greater than or equal to the return values of esl_getLengthOfEcPprivateKey(domain) for privateKey and esl_getLengthOfEcPpublicKey_comp(domain) for publicKey_x and publicKey_y in bytes, where domain is the ASN1 structure describing the domain parameters of the underlying curve. The maximally required buffer size is 32 byte, if ECP_MAX_KEY_SIZE = ECP_MAX_KEY_SIZE_256, and 48 byte, if ECP_MAX_KEY_SIZE = ECP_MAX_KEY_SIZE_384. The interface of this function does not pass the lengths of the arrays referenced by the pointers privateKey, publicKey_x and publicKey_y.
Call context
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces

Table 4-287 esl_generateKeyEcP_prim

4.2.2.4.4 Service APIs to Retrieve EC Length Information

4.2.2.4.4.1 esl_getMaxLengthOfEcPmessage

Prototype	
`eslt_Length` **`esl_getMaxLengthOfEcPmessage`** `(VSECPRIM_P2ROMCONST_PARA(eslt_EccDomain) domain)`	
Parameter	
domain [in]	pointer to domain parameter structure
Return code	
eslt_Length	0 if domain==NULL or domain decoding error
eslt_Length	maximally possible message length in bytes, else
Functional Description	
Returns the maximally possible byte length of a message to be signed or verified by the ECDSA protocol.	
Particularities and Limitations	
domain has to be a valid ASN1 structure for EC domain parameters For information about the expected data format in domain see Chapter 4.1.	
Call context	
> TASK > This function is Synchronous > This function is Reentrant	

Table 4-288 esl_getMaxLengthOfEcPmessage

4.2.2.4.4.2 esl_getLengthOfEcPpublicKey_comp

Prototype	
`eslt_Length` **`esl_getLengthOfEcPpublicKey_comp`** `(VSECPRIM_P2ROMCONST_PARA(eslt_EccDomain) domain)`	
Parameter	
domain [in]	pointer to domain parameter structure
Return code	
eslt_Length	0 if domain==NULL or domain decoding error,
eslt_Length	length of public key in bytes, else
Functional Description	
Returns the byte length of a ECC public key. An ECC public key is a point on a curve. The returned length is the length of one coordinate of the point (and not the joint length of all coordinates).	
Particularities and Limitations	
domain has to be a valid ASN1 structure for EC domain parameters For information about the expected data format in domain see Chapter 4.1.	
Call context	
> TASK > This function is Synchronous > This function is Reentrant	

Table 4-289 esl_getLengthOfEcPpublicKey_comp

4.2.2.4.4.3 esl_getLengthOfEcPprivateKey

Prototype	
`eslt_Length` **`esl_getLengthOfEcPprivateKey`** `(VSECPRIM_P2ROMCONST_PARA(eslt_EccDomain) domain)`	
Parameter	
domain [in]	pointer to domain parameter structure
Return code	
eslt_Length	0 if domain==NULL or domain decoding error,
eslt_Length	maximally possible byte length of the ECC private key, else
Functional Description	
Returns the maximally possible byte length of the private key in an ECC protocol.	
Particularities and Limitations	
domain has to be a valid ASN1 structure for EC domain parameters For information about the expected data format in domain see Chapter 4.1.	
Call context	
> TASK	

> This function is Synchronous > This function is Reentrant

Table 4-290 esl_getLengthOfEcPprivateKey

4.2.2.4.4.4 esl_getLengthOfEcPsignature_comp

Prototype	
`eslt_Length esl_getLengthOfEcPsignature_comp (VSECPRIM_P2ROMCONST_PARA(eslt_EccDomain) domain)`	
Parameter	
domain [in]	pointer to domain parameter structure
Return code	
eslt_Length	0 if domain==NULL or domain decoding error,
eslt_Length	maximally possible byte length of ECDSA signature component
Functional Description	
Returns the maximally possible byte length of an ECDSA signature (for signature component r or s) This function only returns the length of one component, r or s, and not the joint length of both components together.	
Particularities and Limitations	
domain has to be a valid ASN1 structure for EC domain parameters For information about the expected data format in domain see Chapter 4.1.	
Call context	
> TASK > This function is Synchronous > This function is Reentrant	

Table 4-291 esl_getLengthOfEcPsignature_comp

4.2.2.4.4.5 esl_getLengthOfEcPsecret_comp

Prototype	
`eslt_Length esl_getLengthOfEcPsecret_comp (VSECPRIM_P2ROMCONST_PARA(eslt_EccDomain) domain)`	
Parameter	
domain [in]	pointer to domain parameter structure
Return code	
eslt_Length	0 if domain==NULL or domain decoding error,
eslt_Length	maximally possible byte length of the shared secret, else
Functional Description	
Returns the maximally possible length of a shared secret in an ECC protocol in bytes.	

A shared secret in an ECC protocol like Diffie Hellman is a point on a curve. This function only returns the length of one coordinate of this point and not the joint length of all coordinates together.
Particularities and Limitations
domain has to be a valid ASN1 structure for EC domain parameters For information about the expected data format in domain see Chapter 4.1.
Call context
> TASK > This function is Synchronous > This function is Reentrant

Table 4-292 esl_getLengthOfEcPsecret_comp

4.2.2.5 Asymmetric Encryption Algorithms

4.2.2.5.1 esl_initEncryptRSA_prim

Prototype	
`eslt_ErrorCode` **`esl_initEncryptRSA_prim`** `(VSECPRIM_P2VAR_PARA(eslt_WorkSpaceRSAenc_prim) workSpace, eslt_Length keyPairModuleSize, eslt_pRomByte keyPairModule, eslt_Length publicKeyExponentSize, eslt_pRomByte publicKeyExponent)`	
Parameter	
publicKeyExponentSize [in]	the byte length of the public exponent e
keyPairModuleSize [in]	the byte length of the modulus n
publicKeyExponent [in]	pointer to the public exponent e
workSpace [in,out]	(in) algorithm context buffer, initialized by esl_initWorkSpaceHeader(..) function (out) initialized algorithm context structure
keyPairModule [in]	pointer to the modulus n
Return code	
eslt_ErrorCode	ESL_ERC_PARAMETER_INVALID input parameter is NULL
	ESL_ERC_WS_TOO_SMALL work space too small
	ESL_ERC_RSA_PUBKEY_INVALID the public exponent is
	invalid
	ESL_ERC_NO_ERROR else
Functional Description	
This function initializes the RSA encryption primitive.	
Particularities and Limitations	
workSpace is initialized by esl_initWorkSpaceHeader(..)	
Call context	
> TASK	

> This function is Synchronous > This function is Reentrant, for different workspaces

Table 4-293 esl_initEncryptRSA_prim

4.2.2.5.2 esl_encryptRSA_prim

Prototype	
`eslt_ErrorCode esl_encryptRSA_prim (VSECPRIM_P2VAR_PARA(eslt_WorkSpaceRSAenc_prim) workSpace, eslt_Length messageSize, VSECPRIM_P2CONST_PARA(eslt_Byte) message, VSECPRIM_P2VAR_PARA(eslt_Length) cipherSize, VSECPRIM_P2VAR_PARA(eslt_Byte) cipher)`	
Parameter	
messageSize [in]	the byte length of the input message
cipher [in,out]	pointer to the output buffer
cipherSize [in,out]	pointer to byte length of the output buffer
workSpace [in,out]	(in) algorithm context buffer, initialized by esl_initWorkSpaceHeader(..) function (out) initialized algorithm context structure
message [in]	pointer to the input message
Return code	
eslt_ErrorCode	ESL_ERC_PARAMETER_INVALID input parameter is NULL
	ESL_ERC_RSA_MESSAGE_OUT_OF_RANGE (int) message > n-1
	ESL_ERC_OUTPUT_SIZE_TOO_SHORT *cipherSize is too small
	ESL_ERC_WS_STATE_INVALID invalid Algo
	ESL_ERC_NO_ERROR else
Functional Description	
This function performs the RSA encryption operation (RSA_EP1).	
Particularities and Limitations	
workSpace is initialized by esl_initEncryptRSA_prim(..)	
Call context	
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces	

Table 4-294 esl_encryptRSA_prim

4.2.2.5.3 esl_initDecryptRSA_prim

Prototype	
`eslt_ErrorCode` **`esl_initDecryptRSA_prim`** `(VSECPRIM_P2VAR_PARA(eslt_WorkSpaceRSAdec_prim) workSpace, eslt_Length keyPairModuleSize, eslt_pRomByte keyPairModule, eslt_Length privateKeyExponentSize, eslt_pRomByte privateKeyExponent)`	
Parameter	
privateKeyExponentSize [in]	the byte length of the private exponent d
keyPairModuleSize [in]	the byte length of the modulus n
privateKeyExponent [in]	pointer to the private exponent d
workSpace [in,out]	(in) algorithm context buffer, initialized by esl_initWorkSpaceHeader(..) function (out) initialized algorithm context structure
keyPairModule [in]	pointer to the modulus n
Return code	
eslt_ErrorCode	ESL_ERC_PARAMETER_INVALID input parameter is NULL
	ESL_ERC_WS_TOO_SMALL work space too small
	ESL_ERC_RSA_PRIVKEY_INVALID the private exponent d
	is invalid
	ESL_ERC_NO_ERROR else
Functional Description	
This function initializes the RSA decryption primitive (plain version).	
Particularities and Limitations	
workSpace is initialized by esl_initWorkSpaceHeader(..)	
Call context	
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces	

Table 4-295 esl_initDecryptRSA_prim

4.2.2.5.4 esl_decryptRSA_prim

Prototype	
`eslt_ErrorCode` **`esl_decryptRSA_prim`** `(VSECPRIM_P2VAR_PARA(eslt_WorkSpaceRSAdec_prim) workSpace, eslt_Length cipherSize, VSECPRIM_P2CONST_PARA(eslt_Byte) cipher, VSECPRIM_P2VAR_PARA(eslt_Length) messageSize, VSECPRIM_P2VAR_PARA(eslt_Byte) message)`	
Parameter	
messageSize [in,out]	pointer to byte length of the output buffer
message [in,out]	pointer to the output buffer

cipher [in]	pointer to the input
workSpace [in,out]	(in) algorithm context buffer, initialized by esl_initWorkSpaceHeader(..) function (out) initialized algorithm context structure
cipherSize [in]	the byte length of the input
Return code	
eslt_ErrorCode	ESL_ERC_PARAMETER_INVALID input parameter is NULL
	ESL_ERC_RSA_CODE_OUT_OF_RANGE (int) cipher > n-1
	ESL_ERC_OUTPUT_SIZE_TOO_SHORT *messageSize is too small
	ESL_ERC_WS_STATE_INVALID invalid Algo
	ESL_ERC_NO_ERROR else
Functional Description	
This function performs the RSA decryption operation (RSA_DP1).	
Particularities and Limitations	
workSpace is initialized by esl_initDecryptRSA_prim(..)	
Call context	
> TASK	
> This function is Synchronous	
> This function is Reentrant, for different workspaces	

Table 4-296 esl_decryptRSA_prim

4.2.2.5.5 esl_initDecryptRSACRT_prim

Prototype

```
eslt_ErrorCode esl_initDecryptRSACRT_prim
(VSECPRIM_P2VAR_PARA(eslt_WorkSpaceRSACRTdec_prim) workSpace, eslt_Length
keyPairPrimePSize, eslt_pRomByte keyPairPrimeP, eslt_Length keyPairPrimeQSize,
eslt_pRomByte keyPairPrimeQ, eslt_Length privateKeyExponentDPSize,
eslt_pRomByte privateKeyExponentDP,, eslt_Length privateKeyExponentDQSize,
eslt_pRomByte privateKeyExponentDQ, eslt_Length privateKeyInverseQISize,
eslt_pRomByte privateKeyInverseQI)
```

Parameter	
keyPairPrimeQSize [in]	the byte length of the prime q
keyPairPrimeQ [in]	pointer to prime q
keyPairPrimePSize [in]	the byte length of the prime p
keyPairPrimeP [in]	pointer to prime p
privateKeyInverseQISize [in]	the byte length of CRT key component qInv
privateKeyInverseQI [in]	pointer to CRT key component qInv
privateKeyExponentDPSize [in]	the byte length of dp

workSpace [in,out]	(in) algorithm context buffer, initialized by esl_initWorkSpaceHeader(..) function (out) initialized algorithm context structure
privateKeyExponentDQSize [in]	the byte length of dq
privateKeyExponentDP [in]	pointer to CRT key component dp
privateKeyExponentDQ [in]	pointer to CRT key component dq
Return code	
eslt_ErrorCode	ESL_ERC_PARAMETER_INVALID input parameter is NULL
	ESL_ERC_WS_TOO_SMALL work space too small
	ESL_ERC_RSA_PRIVKEY_INVALID the private key
	is invalid
	ESL_ERC_NO_ERROR else
Functional Description	
This function initializes the RSA decryption primitive (CRT version).	
Particularities and Limitations	
workSpace is initialized by esl_initWorkSpaceHeader(..)	
Call context	
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces	

Table 4-297 esl_initDecryptRSACRT_prim

4.2.2.5.6 esl_decryptRSACRT_prim

Prototype	
`eslt_ErrorCode esl_decryptRSACRT_prim (VSECPRIM_P2VAR_PARA(eslt_WorkSpaceRSACRTdec_prim) workSpace, eslt_Length cipherSize, VSECPRIM_P2CONST_PARA(eslt_Byte) cipher, VSECPRIM_P2VAR_PARA(eslt_Length) messageSize, VSECPRIM_P2VAR_PARA(eslt_Byte) message)`	
Parameter	
messageSize [in,out]	pointer to byte length of the output buffer
message [in,out]	pointer to the output buffer
cipher [in]	pointer to the input
workSpace [in,out]	(in) algorithm context buffer, initialized by esl_initWorkSpaceHeader(..) function (out) initialized algorithm context structure
cipherSize [in]	the byte length of the input
Return code	
eslt_ErrorCode	ESL_ERC_PARAMETER_INVALID input parameter is NULL
	ESL_ERC_RSA_CODE_OUT_OF_RANGE (int) cipher > n-1

	ESL_ERC_OUTPUT_SIZE_TOO_SHORT *messageSize is too small
	ESL_ERC_WS_STATE_INVALID invalid Algo
	ESL_ERC_NO_ERROR else
Functional Description	
This function performs the RSA decryption operation (RSA_DP1).	
Particularities and Limitations	
workSpace is initialized by esl_initDecryptRSACRT_prim(..)	
Call context	
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces	

Table 4-298 esl_decryptRSACRT_prim

4.2.2.5.7 esl_initEncryptRSA_V15

Prototype	
`eslt_ErrorCode` **`esl_initEncryptRSA_V15`** `(VSECPRIM_P2VAR_PARA(eslt_WorkSpaceRSAenc) workSpace, eslt_Length keyPairModuleSize, eslt_pRomByte keyPairModule, eslt_Length publicKeyExponentSize, eslt_pRomByte publicKeyExponent)`	
Parameter	
publicKeyExponentSize [in]	length of the public publicKeyExponent (bytes)
keyPairModuleSize [in]	length of the keyPairModulus (bytes)
publicKeyExponent [in]	pointer to the public exponent
workSpace [in,out]	algorithm context buffer, initialized by esl_initWorkSpaceHeader(..) function
keyPairModule [in]	pointer to the key modulus
Return code	
eslt_ErrorCode	ESL_ERC_NO_ERROR Operation successful
	ESL_ERC_WS_TOO_SMALL Work space too small
	ESL_ERC_WS_STATE_INVALID Invalid state
	ESL_ERC_PARAMETER_INVALID - format problems, nullpointer, ...
	ESL_ERC_RSA_PUBKEY_INVALID The public exponent is invalid
Functional Description	
Initialize encryption for RSA PKCS#1 V1.5.	
Particularities and Limitations	
workSpace is initialized by esl_initWorkSpaceHeader(..) privateKeyExponent < keyPairModule Module is a prime	

Call context
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces

Table 4-299 esl_initEncryptRSA_V15

4.2.2.5.8 esl_encryptRSA_V15

Prototype
`eslt_ErrorCode esl_encryptRSA_V15 (VSECPRIM_P2VAR_PARA(eslt_WorkSpaceRSAenc) workSpace, eslt_Length messageSize, VSECPRIM_P2CONST_PARA(eslt_Byte) message, VSECPRIM_P2VAR_PARA(eslt_Length) cipherSize, VSECPRIM_P2VAR_PARA(eslt_Byte) cipher)`

Parameter	
messageSize [in]	pointer to length of the message buffer (bytes)
cipher [out]	pointer to the cipher
cipherSize [out]	length of the cipher (bytes)
workSpace [in,out]	algorithm context buffer, initialized by esl_initWorkSpaceHeader(..) function
message [in]	pointer to the message buffer

Return code	
eslt_ErrorCode	ESL_ERC_NO_ERROR Operation successful
	ESL_ERC_WS_TOO_SMALL Work space too small
	ESL_ERC_WS_STATE_INVALID Invalid state
	ESL_ERC_PARAMETER_INVALID - format problems, nullpointer, ...
	ESL_ERC_RSA_MESSAGE_OUT_OF_RANGE
	- if messageSize > keyPairModuleSize - ASN1_SIZEOF_MINIMAL_PADDING
	ESL_ERC_OUTPUT_SIZE_TOO_SHORT outputSize too small
	and all possible error codes which can occur in esl_getBytesRNG

Functional Description
Perform encryption for RSA PKCS#1 V1.5 , thereby using the RNG which is implemented by the call back function esl_getBytesRNG(..).

Particularities and Limitations
workSpace is initialized by esl_initEncryptRSA_V15(..) *cipherSize >= keyPairModuleSize

Call context
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces

Table 4-300 esl_encryptRSA_V15

4.2.2.5.9 esl_initDecryptRSA_V15

Prototype	
`eslt_ErrorCode` **`esl_initDecryptRSA_V15`** `(VSECPRIM_P2VAR_PARA(eslt_WorkSpaceRSAdec) workSpace, eslt_Length keyPairModuleSize, eslt_pRomByte keyPairModule, eslt_Length privateKeyExponentSize, eslt_pRomByte privateKeyExponent)`	
Parameter	
keyPairModuleSize [in]	length of the keyPairModulus (bytes)
privateKeyExponent [in]	pointer to the public exponent
privateKeyExponentSize [in]	length of the public keyExponent (bytes)
workSpace [in,out]	algorithm context buffer, initialized by esl_initWorkSpaceHeader(..) function
keyPairModule [in]	pointer to the key modulus
Return code	
eslt_ErrorCode	ESL_ERC_NO_ERROR Operation successful
	ESL_ERC_WS_TOO_SMALL Work space too small
	ESL_ERC_PARAMETER_INVALID - format problems, nullpointer, ...
Functional Description	
Initialize decryption for RSA PKCS#1 V1.5.	
Particularities and Limitations	
workSpace is initialized by esl_initWorkSpaceHeader(..) keyExponent < keyPairModule Module is a prime	
Call context	
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces	

Table 4-301 esl_initDecryptRSA_V15

4.2.2.5.10 esl_decryptRSA_V15

Prototype	
`eslt_ErrorCode` **`esl_decryptRSA_V15`** `(VSECPRIM_P2VAR_PARA(eslt_WorkSpaceRSAdec) workSpace, eslt_Length cipherSize, VSECPRIM_P2CONST_PARA(eslt_Byte) cipher, VSECPRIM_P2VAR_PARA(eslt_Length) messageSize, VSECPRIM_P2VAR_PARA(eslt_Byte) message)`	
Parameter	
messageSize [out]	pointer to length of the message buffer (bytes)
message [out]	pointer to the message buffer
cipher [in]	pointer to the cipher
workSpace [in,out]	algorithm context buffer, initialized by esl_initWorkSpaceHeader(..) function
cipherSize [in]	length of the cipher (bytes)

Return code	
eslt_ErrorCode	ESL_ERC_NO_ERROR Operation successful
	ESL_ERC_WS_STATE_INVALID Invalid state
	ESL_ERC_PARAMETER_INVALID - format problems, nullpointer, ...
	ESL_ERC_RSA_CODE_OUT_OF_RANGE (int) cipher > n-1
	ESL_ERC_RSA_ENCODING_INVALID Invalid padding
	ESL_ERC_OUTPUT_SIZE_TOO_SHORT outputSize too small
Functional Description	
Perform decryption for RSA PKCS#1 V1.5 (default with private key).	
Particularities and Limitations	
workSpace is initialized by esl_initDecryptRSA_V15(..) private keyExponent < keyPairModule Module is a prime	
Call context	
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces	

Table 4-302 esl_decryptRSA_V15

4.2.2.5.11 esl_decryptPubRSA_V15

Prototype	
`eslt_ErrorCode` **`esl_decryptPubRSA_V15`** `(VSECPRIM_P2VAR_PARA(eslt_WorkSpaceRSAdec) workSpace, eslt_Length cipherSize, VSECPRIM_P2CONST_PARA(eslt_Byte) cipher, VSECPRIM_P2VAR_PARA(eslt_Length) messageSize, VSECPRIM_P2VAR_PARA(eslt_Byte) message)`	
Parameter	
messageSize [out]	pointer to length of the message buffer (bytes)
message [out]	pointer to the message buffer
cipher [in]	pointer to the cipher
workSpace [in,out]	algorithm context buffer, initialized by esl_initWorkSpaceHeader(..) function
cipherSize [in]	length of the cipher (bytes)
Return code	
eslt_ErrorCode	ESL_ERC_NO_ERROR Operation successful
	ESL_ERC_WS_STATE_INVALID Invalid state
	ESL_ERC_PARAMETER_INVALID - format problems, nullpointer, ...
	ESL_ERC_RSA_CODE_OUT_OF_RANGE (int) cipher > n-1
	ESL_ERC_RSA_ENCODING_INVALID Invalid padding
	ESL_ERC_OUTPUT_SIZE_TOO_SHORT outputSize too small

Functional Description
Perform decryption for RSA PKCS#1 V1.5 with public key.
Particularities and Limitations
workSpace is initialized by esl_initDecryptRSA_V15(..) public keyExponent < keyPairModule Module is a prime
Call context
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces

Table 4-303 esl_decryptPubRSA_V15

4.2.2.5.12 esl_decryptPrivRSA_V15

Prototype	
`eslt_ErrorCode` **`esl_decryptPrivRSA_V15`** `(VSECPRIM_P2VAR_PARA(eslt_WorkSpaceRSAdec) workSpace, eslt_Length cipherSize, VSECPRIM_P2CONST_PARA(eslt_Byte) cipher, VSECPRIM_P2VAR_PARA(eslt_Length) messageSize, VSECPRIM_P2VAR_PARA(eslt_Byte) message)`	
Parameter	
messageSize [out]	pointer to length of the message buffer (bytes)
message [out]	pointer to the message buffer
cipher [in]	pointer to the cipher
workSpace [in,out]	algorithm context buffer, initialized by esl_initWorkSpaceHeader(..) function
cipherSize [in]	length of the cipher (bytes)
Return code	
eslt_ErrorCode	ESL_ERC_NO_ERROR Operation successful
	ESL_ERC_WS_STATE_INVALID Invalid state
	ESL_ERC_PARAMETER_INVALID - format problems, nullpointer, ...
	ESL_ERC_RSA_CODE_OUT_OF_RANGE (int) cipher > n-1
	ESL_ERC_RSA_ENCODING_INVALID Invalid padding
	ESL_ERC_OUTPUT_SIZE_TOO_SHORT outputSize too small
Functional Description	
Perform decryption for RSA PKCS#1 V1.5 with private key.	
Particularities and Limitations	
workSpace is initialized by esl_initDecryptRSA_V15(..) private keyExponent < keyPairModule Module is a prime	
Call context	
> TASK > This function is Synchronous	

> This function is Reentrant, for different workspaces

Table 4-304 esl_decryptPrivRSA_V15

4.2.2.5.13 esl_initDecryptRSACRT_V15

Prototype	
`eslt_ErrorCode esl_initDecryptRSACRT_V15 (VSECPRIM_P2VAR_PARA(eslt_WorkSpaceRSACRTdec) workSpace, eslt_Length keyPairPrimePSize, eslt_pRomByte keyPairPrimeP, eslt_Length keyPairPrimeQSize, eslt_pRomByte keyPairPrimeQ, eslt_Length privateKeyExponentDPSize, eslt_pRomByte privateKeyExponentDP, esltt_Length privateKeyExponentDQSize, eslt_pRomByte privateKeyExponentDQ, eslt_Length privateKeyInverseQISize, eslt_pRomByte privateKeyInverseQI)`	
Parameter	
keyPairPrimeQSize [in]	length of the keyPairPrimeQ (bytes)
keyPairPrimeP [in]	pointer to the 1st key prime
privateKeyInverseQISize [in]	length of the privateKeyInverseQI (bytes)
privateKeyInverseQI [in]	pointer to the inverse
privateKeyExponentDPSize [in]	length of the privKeyExponenentDP (bytes)
keyPairPrimeQ [in]	pointer to the 2nd key prime
workSpace [in,out]	algorithm context buffer, initialized by esl_initWorkSpaceHeader(..) function
privateKeyExponentDQSize [in]	length of the privKeyExponenentDQ (bytes)
privateKeyExponentDP [in]	pointer to the partial privKey exponent
privateKeyExponentDQ [in]	pointer to the partial privKey exponent
keyPairPrimePSize [in]	length of the keyPairPrimeP (bytes)
Return code	
eslt_ErrorCode	ESL_ERC_NO_ERROR Operation successful
	ESL_ERC_WS_TOO_SMALL Work space too small
	ESL_ERC_PARAMETER_INVALID format problems, nullpointer, ..
	ESL_ERC_RSA_PRIVKEY_INVALID The private exponent is invalid
Functional Description	
Initialize decryption for RSA PKCS#1 V1.5 CRT.	
Particularities and Limitations	
workSpace is initialized by esl_initWorkSpaceHeader(..) PrimeP and Q are a prime privateKeyExpDP<PrimeP privateKeyExpDQ<PrimeQ QI is inverse to Q, could be checked, then move this line to errorcodes	
Call context	
> TASK	

> This function is Synchronous > This function is Reentrant, for different workspaces

Table 4-305 esl_initDecryptRSACRT_V15

4.2.2.5.14 esl_decryptRSACRT_V15

Prototype	
`eslt_ErrorCode esl_decryptRSACRT_V15 (VSECPRIM_P2VAR_PARA(eslt_WorkSpaceRSACRTdec) workSpace, eslt_Length cipherSize, VSECPRIM_P2CONST_PARA(eslt_Byte) cipher, VSECPRIM_P2VAR_PARA(eslt_Length) messageSize, VSECPRIM_P2VAR_PARA(eslt_Byte) message)`	
Parameter	
messageSize [out]	pointer to length of the message buffer (bytes)
message [out]	pointer to the message buffer
cipher [in]	pointer to the cipher
workSpace [in,out]	algorithm context buffer, initialized by esl_initWorkSpaceHeader(..) function
cipherSize [in]	length of the cipher (bytes)
Return code	
eslt_ErrorCode	ESL_ERC_NO_ERROR Operation successful
	ESL_ERC_WS_STATE_INVALID Invalid state
	ESL_ERC_PARAMETER_INVALID - format problems, nullpointer, ...
	ESL_ERC_RSA_MODULE_OUT_OF_RANGE modulus too short
	ESL_ERC_RSA_CODE_OUT_OF_RANGE (int) cipher > n-1
	- 0 <= (int) cipher <= module-1, where module = PrimeP*PrimeQ
	ESL_ERC_RSA_ENCODING_INVALID Invalid padding
	ESL_ERC_OUTPUT_SIZE_TOO_SHORT outputSize too small
Functional Description	
Perform decryption for RSA PKCS#1 V1.5 CRT.	
Particularities and Limitations	
workSpace is initialized by esl_initDecryptRSACRT_V15(..)	
Call context	
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces	

Table 4-306 esl_decryptRSACRT_V15

4.2.2.5.15 esl_initEncryptRSASHA1_OAEP

Prototype	
`eslt_ErrorCode` **`esl_initEncryptRSASHA1_OAEP`** `(VSECPRIM_P2VAR_PARA(eslt_WorkSpaceRSAOAEPenc) workSpace, eslt_Length keyPairModuleSize, eslt_pRomByte keyPairModule, eslt_Length publicKeyExponentSize, eslt_pRomByte publicKeyExponent)`	
Parameter	
workSpace [in,out]	Encryption work space
keyPairModuleSize [in]	Size of RSA key pair module
keyPairModule [in]	RSA key pair module
publicKeyExponentSize [in]	Size of public RSA key pair exponent
publicKeyExponent [in]	Public RSA key pair exponent
Return code	
eslt_ErrorCode	ESL_ERC_NO_ERROR Operation successful
	ESL_ERC_PARAMETER_INVALID Input parameter is NULL
	ESL_ERC_WS_TOO_SMALL Work space too small
	ESL_ERC_RSA_PUBKEY_INVALID The public exponent is invalid
Functional Description	
Initialize encryption.	
Particularities and Limitations	
Work space initialized	
Call context	
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces	

Table 4-307 esl_initEncryptRSASHA1_OAEP

4.2.2.5.16 esl_encryptRSASHA1_OAEP_Label

Prototype	
`eslt_ErrorCode` **`esl_encryptRSASHA1_OAEP_Label`** `(VSECPRIM_P2VAR_PARA(eslt_WorkSpaceRSAOAEPenc) workSpace, eslt_Length messageSize, VSECPRIM_P2CONST_PARA(eslt_Byte) message, eslt_Length labelSize, VSECPRIM_P2CONST_PARA(eslt_Byte) label, VSECPRIM_P2VAR_PARA(eslt_Length) cipherSize, VSECPRIM_P2VAR_PARA(esslt_Byte) cipher)`	
Parameter	
workSpace [in,out]	Encryption work space
messageSize [in]	Size of data to be encrypted
message [in]	Data to be encrypted

labelSize [in]	Size of optional label
label [in]	Optional label to be associated with the message
cipherSize [in,out]	Input Maximum size of encrypted data Output Actual size of encrypted data
cipher [out]	Encrypted data
Return code	
eslt_ErrorCode	ESL_ERC_NO_ERROR Operation successful
	ESL_ERC_PARAMETER_INVALID Input parameter is NULL or out of range
	ESL_ERC_WS_STATE_INVALID Invalid state
	ESL_ERC_WS_TOO_SMALL Work space too small
	ESL_ERC_RSA_MESSAGE_OUT_OF_RANGE Input message out of range
Functional Description	
Encrypt message. Random seed generated internally	
Particularities and Limitations	
Work space initialized	
Call context	
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces	

Table 4-308 esl_encryptRSASHA1_OAEP_Label

4.2.2.5.17 esl_encryptRSASHA1_OAEP_Seed

Prototype
`eslt_ErrorCode` **`esl_encryptRSASHA1_OAEP_Seed`** `(VSECPRIM_P2VAR_PARA(eslt_WorkSpaceRSAOAEPenc) workSpace, eslt_Length messageSize, VSECPRIM_P2CONST_PARA(eslt_Byte) message, eslt_Length inSeedSize, VSECPRIM_P2CONST_PARA(eslt_Byte) inSeed, VSECPRIM_P2VAR_PARA(eslt_Length) cipherSize, VSECPRIM_P2VAR_PARA((eslt_Byte) cipher)`

Parameter	
workSpace [in,out]	Encryption work space
messageSize [in]	Size of data to be encrypted
message [in]	Data to be encrypted
inSeedSize [in]	Size of seed (must match size if hash digest)
inSeed [in]	Seed value (pass null pointer to trigger internal generation of random seed)
cipherSize [in,out]	Input Maximum size of encrypted data Output Actual size of encrypted data
cipher [out]	Encrypted data
Return code	
eslt_ErrorCode	ESL_ERC_NO_ERROR Operation successful

	ESL_ERC_PARAMETER_INVALID Input parameter is NULL or out of range
	ESL_ERC_WS_STATE_INVALID Invalid state
	ESL_ERC_WS_TOO_SMALL Work space too small
	ESL_ERC_RSA_MESSAGE_OUT_OF_RANGE Input message out of range
Functional Description	
Encrypt message. Empty string used as default label	
Particularities and Limitations	
Work space initialized	
Call context	
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces	

Table 4-309 esl_encryptRSASHA1_OAEP_Seed

4.2.2.5.18 esl_encryptRSASHA1_OAEP_Label_Seed

Prototype	
`eslt_ErrorCode esl_encryptRSASHA1_OAEP_Label_Seed (VSECPRIM_P2VAR_PARA(eslt_WorkSpaceRSAOAEPenc) workSpace, eslt_Length messageSize, VSECPRIM_P2CONST_PARA(eslt_Byte) message, eslt_Length labelSize, VSECPRIM_P2CONST_PARA(eslt_Byte) label, eslt_Length inSeedSize, VSECPRIM_P2CONST_PARA(eslt_Byte) inSeed, VSSECPRIM_P2VAR_PARA(eslt_Length) cipherSize, VSECPRIM_P2VAR_PARA(eslt_Byte) cipher)`	
Parameter	
workSpace [in,out]	Encryption work space
messageSize [in]	Size of data to be encrypted
message [in]	Data to be encrypted
labelSize [in]	Size of optional label
label [in]	Optional label to be associated with the message
inSeedSize [in]	Size of seed (must match size if hash digest)
inSeed [in]	Seed value (pass null pointer to trigger internal generation of random seed)
cipherSize [in,out]	Input Maximum size of encrypted data Output Actual size of encrypted data
cipher [out]	Encrypted data
Return code	
eslt_ErrorCode	ESL_ERC_NO_ERROR Operation successful
	ESL_ERC_PARAMETER_INVALID Input parameter is NULL or out of range
	ESL_ERC_WS_STATE_INVALID Invalid state
	ESL_ERC_WS_TOO_SMALL Work space too small

	ESL_ERC_RSA_MESSAGE_OUT_OF_RANGE Input message out of range
	all possible error codes which can occur in esl_getBytesRNG
Functional Description	
Encrypt message. If no inSeed is provided, this function uses the RNG which is implemented by the call back function esl_getBytesRNG(..) to generate random seed values.	
Particularities and Limitations	
Work space initialized	
Call context	
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces	

Table 4-310 esl_encryptRSASHA1_OAEP_Label_Seed

4.2.2.5.19 esl_encryptRSASHA1_OAEP

Prototype	
`eslt_ErrorCode` **`esl_encryptRSASHA1_OAEP`** `(VSECPRIM_P2VAR_PARA(eslt_WorkSpaceRSAOAEPenc) workSpace, eslt_Length messageSize, VSECPRIM_P2CONST_PARA(eslt_Byte) message, VSECPRIM_P2VAR_PARA(eslt_Length) cipherSize, VSECPRIM_P2VAR_PARA(eslt_Byte) cipher)`	
Parameter	
workSpace [in,out]	Encryption work space
messageSize [in]	Size of data to be encrypted
message [in]	Data to be encrypted
cipherSize [in,out]	Input Maximum size of encrypted data Output Actual size of encrypted data
cipher [out]	Encrypted data
Return code	
eslt_ErrorCode	ESL_ERC_NO_ERROR Operation successful
	ESL_ERC_PARAMETER_INVALID Input parameter is NULL or out of range
	ESL_ERC_WS_STATE_INVALID Invalid state
	ESL_ERC_WS_TOO_SMALL Work space too small
	ESL_ERC_RSA_MESSAGE_OUT_OF_RANGE Input message out of range
Functional Description	
Encrypt message. Random seed generated internally, empty string used as default label	
Particularities and Limitations	
Work space initialized	
Call context	

> TASK > This function is Synchronous > This function is Reentrant, for different workspaces

Table 4-311 esl_encryptRSASHA1_OAEP

4.2.2.5.20 esl_initDecryptRSASHA1_OAEP

Prototype	
`eslt_ErrorCode` **`esl_initDecryptRSASHA1_OAEP`** `(VSECPRIM_P2VAR_PARA(eslt_WorkSpaceRSAOAEPdec) workSpace, eslt_Length keyPairModuleSize, eslt_pRomByte keyPairModule, eslt_Length privateKeyExponentSize, eslt_pRomByte privateKeyExponent)`	
Parameter	
workSpace [in,out]	Decryption work space
keyPairModuleSize [in]	Size of RSA key pair module
keyPairModule [in]	RSA key pair module
privateKeyExponentSize [in]	Size of private RSA key pair exponent
privateKeyExponent [in]	Private RSA key pair exponent
Return code	
eslt_ErrorCode	ESL_ERC_NO_ERROR Operation successful
	ESL_ERC_PARAMETER_INVALID Input parameter is NULL
	ESL_ERC_WS_TOO_SMALL Work space too small
	ESL_ERC_RSA_PRIVKEY_INVALID The private exponent is invalid
Functional Description	
Initialize decryption.	
Particularities and Limitations	
Work space initialized	
Call context	
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces	

Table 4-312 esl_initDecryptRSASHA1_OAEP

4.2.2.5.21 esl_decryptRSASHA1_OAEP_Label

Prototype
`eslt_ErrorCode` **`esl_decryptRSASHA1_OAEP_Label`** `(VSECPRIM_P2VAR_PARA(eslt_WorkSpaceRSAOAEPdec) workSpace, eslt_Length cipherSize, VSECPRIM_P2CONST_PARA(eslt_Byte) cipher, eslt_Length labelSize,`

VSECPRIM_P2CONST_PARA(eslt_Byte) label, VSECPRIM_P2VAR_PARA(eslt_Length) messageSize, VSECPRIM_P2VAR_PARA(esllt_Byte) message)	
Parameter	
workSpace [in,out]	Decryption work space
cipherSize [in]	Size of encrypted data
cipher [in]	Encrypted data
labelSize [in]	Size of optional label
label [in]	Optional label associated with the message
messageSize [in,out]	Input Maximum size of decrypted data Output Actual size of decrypted data
message [out]	Decrypted data
Return code	
eslt_ErrorCode	ESL_ERC_NO_ERROR Operation successful
	ESL_ERC_PARAMETER_INVALID Input parameter is NULL or out of range
	ESL_ERC_WS_STATE_INVALID Invalid state
	ESL_ERC_WS_TOO_SMALL Work space too small
	ESL_ERC_RSA_MESSAGE_OUT_OF_RANGE Input message out of range
Functional Description	
Decrypt message.	
Particularities and Limitations	
Work space initialized	
Call context	
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces	

Table 4-313 esl_decryptRSASHA1_OAEP_Label

4.2.2.5.22 esl_decryptRSASHA1_OAEP

Prototype	
eslt_ErrorCode **esl_decryptRSASHA1_OAEP** (VSECPRIM_P2VAR_PARA(eslt_WorkSpaceRSAOAEPdec) workSpace, eslt_Length cipherSize, VSECPRIM_P2CONST_PARA(eslt_Byte) cipher, VSECPRIM_P2VAR_PARA(eslt_Length) messageSize, VSECPRIM_P2VAR_PARA(eslt_Byte) message)	
Parameter	
workSpace [in,out]	Decryption work space
cipherSize [in]	Size of encrypted data
cipher [in]	Encrypted data
messageSize [in,out]	Input Maximum size of decrypted data Output Actual size of decrypted data

message [out]	Decrypted data
Return code	
eslt_ErrorCode	ESL_ERC_NO_ERROR Operation successful
	ESL_ERC_PARAMETER_INVALID Input parameter is NULL or out of range
	ESL_ERC_WS_STATE_INVALID Invalid state
	ESL_ERC_WS_TOO_SMALL Work space too small
	ESL_ERC_RSA_MESSAGE_OUT_OF_RANGE Input message out of range
Functional Description	
Decrypt message. Empty string used as default label	
Particularities and Limitations	
Work space initialized	
Call context	
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces	

Table 4-314 esl_decryptRSASHA1_OAEP

4.2.2.5.23 esl_decodeRSASHA1_OAEP

Prototype	
`eslt_ErrorCode esl_decodeRSASHA1_OAEP (VSECPRIM_P2VAR_PARA(eslt_WorkSpace) workSpace, eslt_Length encodedMessageSize, eslt_Length labelSize, VSECPRIM_P2CONST_PARA(eslt_Byte) label, VSECPRIM_P2VAR_PARA(eslt_Length) messageSize, VSECPRIM_P2VAR_PARA(eslt_Byte) message)`	
Parameter	
workSpace [in,out]	Decryption work space
encodedMessageSize [in]	Size of encoded message located in work space
labelSize [in]	Size of optional label
label [in]	Optional label associated with the message
messageSize [in,out]	Input Maximum size of decrypted data Output Actual size of decrypted data
message [out]	Decrypted data
Return code	
eslt_ErrorCode	ESL_ERC_NO_ERROR Operation successful
	ESL_ERC_PARAMETER_INVALID Input parameter is NULL or out of range
	ESL_ERC_WS_STATE_INVALID Invalid state
	ESL_ERC_WS_TOO_SMALL Work space too small
	ESL_ERC_RSA_MESSAGE_OUT_OF_RANGE Input message out of range

Functional Description
Decode an OAEP encoded message.
Particularities and Limitations
Work space initialized
Call context
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces

Table 4-315 esl_decodeRSASHA1_OAEP

4.2.2.5.24 esl_initDecryptRSACRTSHA1_OAEP

Prototype

```
eslt_ErrorCode esl_initDecryptRSACRTSHA1_OAEP
(VSECPRIM_P2VAR_PARA(eslt_WorkSpaceRSACRTOAEPdec) workSpace, eslt_Length
keyPairPrimePSize, eslt_pRomByte keyPairPrimeP, eslt_Length keyPairPrimeQSize,
eslt_pRomByte keyPairPrimeQ, eslt_Length privateKeyExponentDPSize,
eslt_pRomByte privateKeyExponentDP,  eslt_Length privateKeyExponentDQSize,
eslt_pRomByte privateKeyExponentDQ, eslt_Length privateKeyInverseQISize,
eslt_pRomByte privateKeyInverseQI)
```

Parameter	
workSpace [in,out]	Decryption work space
keyPairPrimePSize [in]	Size of secret prime p
keyPairPrimeP [in]	Secret prime p
keyPairPrimeQSize [in]	Size of secret prime q
keyPairPrimeQ [in]	Secret prime q
privateKeyExponentDPSize [in]	Size of private key component dp
privateKeyExponentDP [in]	Private key component dp
privateKeyExponentDQSize [in]	Size of private key component dq
privateKeyExponentDQ [in]	Private key component dq
privateKeyInverseQISize [in]	Size of private key component q^-1 mod p
privateKeyInverseQI [in]	Private key component q^-1 mod p
Return code	
eslt_ErrorCode	ESL_ERC_NO_ERROR Operation successful
	ESL_ERC_PARAMETER_INVALID Input parameter is NULL
	ESL_ERC_WS_TOO_SMALL Work space too small
	ESL_ERC_RSA_PRIVKEY_INVALID The private exponent is invalid

Functional Description
Initialize decryption (CRT variant).
Particularities and Limitations
Work space initialized
Call context
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces

Table 4-316 esl_initDecryptRSACRTSHA1_OAEP

4.2.2.5.25 esl_decryptRSACRTSHA1_OAEP_Label

Prototype	
`eslt_ErrorCode` **`esl_decryptRSACRTSHA1_OAEP_Label`** `(VSECPRIM_P2VAR_PARA(eslt_WorkSpaceRSACRTOAEPdec) workSpace, eslt_Length cipherSize, VSECPRIM_P2CONST_PARA(eslt_Byte) cipher, eslt_Length labelSize, VSECPRIM_P2CONST_PARA(eslt_Byte) label, VSECPRIM_P2VAR_PARA(eslt_Length) messageSize, VSECPRIM_P2VAR_PARA((eslt_Byte) message)`	
Parameter	
workSpace [in,out]	Decryption work space
cipherSize [in]	Size of encrypted data
cipher [in]	Encrypted data
labelSize [in]	Size of optional label
label [in]	Optional label associated with the message
messageSize [in,out]	Input Maximum size of decrypted data Output Actual size of decrypted data
message [out]	Decrypted data
Return code	
eslt_ErrorCode	ESL_ERC_NO_ERROR Operation successful
	ESL_ERC_PARAMETER_INVALID Input parameter is NULL or out of range
	ESL_ERC_WS_STATE_INVALID Invalid state
	ESL_ERC_WS_TOO_SMALL Work space too small
	ESL_ERC_RSA_MESSAGE_OUT_OF_RANGE Input message out of range
Functional Description	
Decrypt message (CRT variant).	
Particularities and Limitations	
Work space initialized	
Call context	
> TASK > This function is Synchronous	

> This function is Reentrant, for different workspaces

Table 4-317 esl_decryptRSACRTSHA1_OAEP_Label

4.2.2.5.26 esl_decryptRSACRTSHA1_OAEP

Prototype	
`eslt_ErrorCode` **`esl_decryptRSACRTSHA1_OAEP`** `(VSECPRIM_P2VAR_PARA(eslt_WorkSpaceRSACRTOAEPdec) workSpace, eslt_Length cipherSize, VSECPRIM_P2CONST_PARA(eslt_Byte) cipher, VSECPRIM_P2VAR_PARA(eslt_Length) messageSize, VSECPRIM_P2VAR_PARA(eslt_Byte) message)`	
Parameter	
workSpace [in,out]	Decryption work space
cipherSize [in]	Size of encrypted data
cipher [in]	Encrypted data
messageSize [in,out]	Input Maximum size of decrypted data Output Actual size of decrypted data
message [out]	Decrypted data
Return code	
eslt_ErrorCode	ESL_ERC_NO_ERROR Operation successful
	ESL_ERC_PARAMETER_INVALID Input parameter is NULL or out of range
	ESL_ERC_WS_STATE_INVALID Invalid state
	ESL_ERC_WS_TOO_SMALL Work space too small
	ESL_ERC_RSA_MESSAGE_OUT_OF_RANGE Input message out of range
Functional Description	
Decrypt message (CRT variant). Empty string used as default label	
Particularities and Limitations	
Work space initialized	
Call context	
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces	

Table 4-318 esl_decryptRSACRTSHA1_OAEP

4.2.2.5.27 esl_initEncryptRSASHA256_OAEP

Prototype
`eslt_ErrorCode` **`esl_initEncryptRSASHA256_OAEP`** `(VSECPRIM_P2VAR_PARA(eslt_WorkSpaceRSAOAEPenc) workSpace, eslt_Length`

keyPairModuleSize, eslt_pRomByte keyPairModule, eslt_Length publicKeyExponentSize, eslt_pRomByte publicKeyExponent)	
Parameter	
workSpace [in,out]	Encryption work space
keyPairModuleSize [in]	Size of RSA key pair module
keyPairModule [in]	RSA key pair module
publicKeyExponentSize [in]	Size of public RSA key pair exponent
publicKeyExponent [in]	Public RSA key pair exponent
Return code	
eslt_ErrorCode	ESL_ERC_NO_ERROR Operation successful
	ESL_ERC_PARAMETER_INVALID Input parameter is NULL
	ESL_ERC_WS_TOO_SMALL Work space too small
	ESL_ERC_RSA_PUBKEY_INVALID The public exponent is invalid
Functional Description	
Initialize encryption.	
Particularities and Limitations	
Work space initialized	
Call context	
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces	

Table 4-319 esl_initEncryptRSASHA256_OAEP

4.2.2.5.28 esl_encryptRSASHA256_OAEP_Label

Prototype	
eslt_ErrorCode **esl_encryptRSASHA256_OAEP_Label** (VSECPRIM_P2VAR_PARA(eslt_WorkSpaceRSAOAEPenc) workSpace, eslt_Length messageSize, VSECPRIM_P2CONST_PARA(eslt_Byte) message, eslt_Length labelSize, VSECPRIM_P2CONST_PARA(eslt_Byte) label, VSECPRIM_P2VAR_PARA(eslt_Length) cipherSize, VSECPRIM_P2VAR_PARA(esslt_Byte) cipher)	
Parameter	
workSpace [in,out]	Encryption work space
messageSize [in]	Size of data to be encrypted
message [in]	Data to be encrypted
labelSize [in]	Size of optional label
label [in]	Optional label to be associated with the message
cipherSize [in,out]	Input Maximum size of encrypted data Output Actual size of encrypted data
cipher [out]	Encrypted data

Return code	
eslt_ErrorCode	ESL_ERC_NO_ERROR Operation successful
	ESL_ERC_PARAMETER_INVALID Input parameter is NULL or out of range
	ESL_ERC_WS_STATE_INVALID Invalid state
	ESL_ERC_WS_TOO_SMALL Work space too small
	ESL_ERC_RSA_MESSAGE_OUT_OF_RANGE Input message out of range
Functional Description	
Encrypt message. Random seed generated internally	
Particularities and Limitations	
Work space initialized	
Call context	
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces	

Table 4-320 esl_encryptRSASHA256_OAEP_Label

4.2.2.5.29 esl_encryptRSASHA256_OAEP_Seed

Prototype
`eslt_ErrorCode esl_encryptRSASHA256_OAEP_Seed (VSECPRIM_P2VAR_PARA(eslt_WorkSpaceRSAOAEPenc) workSpace, eslt_Length messageSize, VSECPRIM_P2CONST_PARA(eslt_Byte) message, eslt_Length inSeedSize, VSECPRIM_P2CONST_PARA(eslt_Byte) inSeed, VSECPRIM_P2VAR_PARA(eslt_Length) cipherSize, VSECPRIM_P2VAR_PARA((eslt_Byte) cipher)`

Parameter	
workSpace [in,out]	Encryption work space
messageSize [in]	Size of data to be encrypted
message [in]	Data to be encrypted
inSeedSize [in]	Size of seed (must match size if hash digest)
inSeed [in]	Seed value (pass null pointer to trigger internal generation of random seed)
cipherSize [in,out]	Input Maximum size of encrypted data Output Actual size of encrypted data
cipher [out]	Encrypted data
Return code	
eslt_ErrorCode	ESL_ERC_NO_ERROR Operation successful
	ESL_ERC_PARAMETER_INVALID Input parameter is NULL or out of range
	ESL_ERC_WS_STATE_INVALID Invalid state
	ESL_ERC_WS_TOO_SMALL Work space too small
	ESL_ERC_RSA_MESSAGE_OUT_OF_RANGE Input message out of range

Functional Description
Encrypt message. Empty string used as default label
Particularities and Limitations
Work space initialized
Call context
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces

Table 4-321 esl_encryptRSASHA256_OAEP_Seed

4.2.2.5.30 esl_encryptRSASHA256_OAEP_Label_Seed

Prototype	
`eslt_ErrorCode` **`esl_encryptRSASHA256_OAEP_Label_Seed`** `(VSECPRIM_P2VAR_PARA(eslt_WorkSpaceRSAOAEPenc) workSpace, eslt_Length messageSize, VSECPRIM_P2CONST_PARA(eslt_Byte) message, eslt_Length labelSize, VSECPRIM_P2CONST_PARA(eslt_Byte) label, eslt_Length inSeedSize, VSECPRIM_P2CONST_PARA(eslt_Byte) inSeed, VSSECPRIM_P2VAR_PARA(eslt_Length) cipherSize, VSECPRIM_P2VAR_PARA(eslt_Byte) cipher)`	
Parameter	
workSpace [in,out]	Encryption work space
messageSize [in]	Size of data to be encrypted
message [in]	Data to be encrypted
labelSize [in]	Size of optional label
label [in]	Optional label to be associated with the message
inSeedSize [in]	Size of seed (must match size if hash digest)
inSeed [in]	Seed value (pass null pointer to trigger internal generation of random seed)
cipherSize [in,out]	Input Maximum size of encrypted data Output Actual size of encrypted data
cipher [out]	Encrypted data
Return code	
eslt_ErrorCode	ESL_ERC_NO_ERROR Operation successful
	ESL_ERC_PARAMETER_INVALID Input parameter is NULL or out of range
	ESL_ERC_WS_STATE_INVALID Invalid state
	ESL_ERC_WS_TOO_SMALL Work space too small
	ESL_ERC_RSA_MESSAGE_OUT_OF_RANGE Input message out of range
	all possible error codes which can occur in esl_getBytesRNG
Functional Description	
Encrypt message.	

If no inSeed is provided, this function uses the RNG which is implemented by the call back function esl_getBytesRNG(..) to generate random seed values.
Particularities and Limitations
Work space initialized
Call context
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces

Table 4-322 esl_encryptRSASHA256_OAEP_Label_Seed

4.2.2.5.31 esl_encryptRSASHA256_OAEP

Prototype	
`eslt_ErrorCode` **`esl_encryptRSASHA256_OAEP`** `(VSECPRIM_P2VAR_PARA(eslt_WorkSpaceRSAOAEPenc) workSpace, eslt_Length messageSize, VSECPRIM_P2CONST_PARA(eslt_Byte) message, VSECPRIM_P2VAR_PARA(eslt_Length) cipherSize, VSECPRIM_P2VAR_PARA(eslt_Byte) cipher)`	
Parameter	
workSpace [in,out]	Encryption work space
messageSize [in]	Size of data to be encrypted
message [in]	Data to be encrypted
cipherSize [in,out]	Input Maximum size of encrypted data Output Actual size of encrypted data
cipher [out]	Encrypted data
Return code	
eslt_ErrorCode	ESL_ERC_NO_ERROR Operation successful
	ESL_ERC_PARAMETER_INVALID Input parameter is NULL or out of range
	ESL_ERC_WS_STATE_INVALID Invalid state
	ESL_ERC_WS_TOO_SMALL Work space too small
	ESL_ERC_RSA_MESSAGE_OUT_OF_RANGE Input message out of range
Functional Description	
Encrypt message. Random seed generated internally, empty string used as default label	
Particularities and Limitations	
Work space initialized	
Call context	
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces	

Table 4-323 esl_encryptRSASHA256_OAEP

4.2.2.5.32 esl_initDecryptRSASHA256_OAEP

Prototype	
`eslt_ErrorCode` **`esl_initDecryptRSASHA256_OAEP`** `(VSECPRIM_P2VAR_PARA(eslt_WorkSpaceRSAOAEPdec) workSpace, eslt_Length keyPairModuleSize, eslt_pRomByte keyPairModule, eslt_Length privateKeyExponentSize, eslt_pRomByte privateKeyExponent)`	
Parameter	
workSpace [in,out]	Decryption work space
keyPairModuleSize [in]	Size of RSA key pair module
keyPairModule [in]	RSA key pair module
privateKeyExponentSize [in]	Size of private RSA key pair exponent
privateKeyExponent [in]	Private RSA key pair exponent
Return code	
eslt_ErrorCode	ESL_ERC_NO_ERROR Operation successful
	ESL_ERC_PARAMETER_INVALID Input parameter is NULL
	ESL_ERC_WS_TOO_SMALL Work space too small
	ESL_ERC_RSA_PRIVKEY_INVALID The private exponent is invalid
Functional Description	
Initialize decryption.	
Particularities and Limitations	
Work space initialized	
Call context	
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces	

Table 4-324 esl_initDecryptRSASHA256_OAEP

4.2.2.5.33 esl_decryptRSASHA256_OAEP_Label

Prototype	
`eslt_ErrorCode` **`esl_decryptRSASHA256_OAEP_Label`** `(VSECPRIM_P2VAR_PARA(eslt_WorkSpaceRSAOAEPdec) workSpace, eslt_Length cipherSize, VSECPRIM_P2CONST_PARA(eslt_Byte) cipher, eslt_Length labelSize, VSECPRIM_P2CONST_PARA(eslt_Byte) label, VSECPRIM_P2VAR_PARA(eslt_Length) messageSize, VSECPRIM_P2VAR_PARA(esllt_Byte) message)`	
Parameter	
workSpace [in,out]	Decryption work space
cipherSize [in]	Size of encrypted data

cipher [in]	Encrypted data
labelSize [in]	Size of optional label
label [in]	Optional label associated with the message
messageSize [in,out]	Input Maximum size of decrypted data Output Actual size of decrypted data
message [out]	Decrypted data
Return code	
eslt_ErrorCode	ESL_ERC_NO_ERROR Operation successful
	ESL_ERC_PARAMETER_INVALID Input parameter is NULL or out of range
	ESL_ERC_WS_STATE_INVALID Invalid state
	ESL_ERC_WS_TOO_SMALL Work space too small
	ESL_ERC_RSA_MESSAGE_OUT_OF_RANGE Input message out of range
Functional Description	
Decrypt message.	
Particularities and Limitations	
Work space initialized	
Call context	
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces	

Table 4-325 esl_decryptRSASHA256_OAEP_Label

4.2.2.5.34 esl_decryptRSASHA256_OAEP

Prototype
`eslt_ErrorCode` **`esl_decryptRSASHA256_OAEP`** `(VSECPRIM_P2VAR_PARA(eslt_WorkSpaceRSAOAEPdec) workSpace, eslt_Length cipherSize, VSECPRIM_P2CONST_PARA(eslt_Byte) cipher, VSECPRIM_P2VAR_PARA(eslt_Length) messageSize, VSECPRIM_P2VAR_PARA(eslt_Byte) message)`

Parameter	
workSpace [in,out]	Decryption work space
cipherSize [in]	Size of encrypted data
cipher [in]	Encrypted data
messageSize [in,out]	Input Maximum size of decrypted data Output Actual size of decrypted data
message [out]	Decrypted data
Return code	
eslt_ErrorCode	ESL_ERC_NO_ERROR Operation successful
	ESL_ERC_PARAMETER_INVALID Input parameter is NULL or out of range
	ESL_ERC_WS_STATE_INVALID Invalid state

	ESL_ERC_WS_TOO_SMALL Work space too small
	ESL_ERC_RSA_MESSAGE_OUT_OF_RANGE Input message out of range
Functional Description	
Decrypt message. Empty string used as default label	
Particularities and Limitations	
Work space initialized	
Call context	
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces	

Table 4-326 esl_decryptRSASHA256_OAEP

4.2.2.5.35 esl_decodeRSASHA256_OAEP

Prototype	
`eslt_ErrorCode esl_decodeRSASHA256_OAEP (VSECPRIM_P2VAR_PARA(eslt_WorkSpace) workSpace, eslt_Length encodedMessageSize, eslt_Length labelSize, VSECPRIM_P2CONST_PARA(eslt_Byte) label, VSECPRIM_P2VAR_PARA(eslt_Length) messageSize, VSECPRIM_P2VAR_PARA(eslt_Byte) message)`	
Parameter	
workSpace [in,out]	Decryption work space
encodedMessageSize [in]	Size of encoded message located in work space
labelSize [in]	Size of optional label
label [in]	Optional label associated with the message
messageSize [in,out]	Input Maximum size of decrypted data Output Actual size of decrypted data
message [out]	Decrypted data
Return code	
eslt_ErrorCode	ESL_ERC_NO_ERROR Operation successful
	ESL_ERC_PARAMETER_INVALID Input parameter is NULL or out of range
	ESL_ERC_WS_STATE_INVALID Invalid state
	ESL_ERC_WS_TOO_SMALL Work space too small
	ESL_ERC_RSA_MESSAGE_OUT_OF_RANGE Input message out of range
Functional Description	
Decode an OAEP encoded message.	
Particularities and Limitations	
Work space initialized	
Call context	
> TASK	

> This function is Synchronous > This function is Reentrant, for different workspaces

Table 4-327 esl_decodeRSASHA256_OAEP

4.2.2.5.36 esl_initDecryptRSACRTSHA256_OAEP

Prototype	
`eslt_ErrorCode esl_initDecryptRSACRTSHA256_OAEP (VSECPRIM_P2VAR_PARA(eslt_WorkSpaceRSACRTOAEPdec) workSpace, eslt_Length keyPairPrimePSize, eslt_pRomByte keyPairPrimeP, eslt_Length keyPairPrimeQSize, eslt_pRomByte keyPairPrimeQ, eslt_Length privateKeyExponentDPSize, eslt_pRomByte privateKeyExponentDP, eslt_Length privateKeyExponentDQSize, eslt_pRomByte privateKeyExponentDQ, eslt_Length privateKeyInverseQISize, eslt_pRomByte privateKeyInverseQI)`	
Parameter	
workSpace [in,out]	Decryption work space
keyPairPrimePSize [in]	Size of secret prime p
keyPairPrimeP [in]	Secret prime p
keyPairPrimeQSize [in]	Size of secret prime q
keyPairPrimeQ [in]	Secret prime q
privateKeyExponentDPSize [in]	Size of private key component dp
privateKeyExponentDP [in]	Private key component dp
privateKeyExponentDQSize [in]	Size of private key component dq
privateKeyExponentDQ [in]	Private key component dq
privateKeyInverseQISize [in]	Size of private key component q^-1 mod p
privateKeyInverseQI [in]	Private key component q^-1 mod p
Return code	
eslt_ErrorCode	ESL_ERC_NO_ERROR Operation successful
	ESL_ERC_PARAMETER_INVALID Input parameter is NULL
	ESL_ERC_WS_TOO_SMALL Work space too small
	ESL_ERC_RSA_PRIVKEY_INVALID The private exponent is invalid
Functional Description	
Initialize decryption (CRT variant).	
Particularities and Limitations	
Work space initialized	
Call context	
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces	

Table 4-328 esl_initDecryptRSACRTSHA256_OAEP

4.2.2.5.37 esl_decryptRSACRTSHA256_OAEP_Label

Prototype	
`eslt_ErrorCode esl_decryptRSACRTSHA256_OAEP_Label (VSECPRIM_P2VAR_PARA(eslt_WorkSpaceRSACRTOAEPdec) workSpace, eslt_Length cipherSize, VSECPRIM_P2CONST_PARA(eslt_Byte) cipher, eslt_Length labelSize, VSECPRIM_P2CONST_PARA(eslt_Byte) label, VSECPRIM_P2VAR_PARA(eslt_Length) messageSize, VSECPRIM_P2VAR_PARA((eslt_Byte) message)`	
Parameter	
workSpace [in,out]	Decryption work space
cipherSize [in]	Size of encrypted data
cipher [in]	Encrypted data
labelSize [in]	Size of optional label
label [in]	Optional label associated with the message
messageSize [in,out]	Input Maximum size of decrypted data Output Actual size of decrypted data
message [out]	Decrypted data
Return code	
eslt_ErrorCode	ESL_ERC_NO_ERROR Operation successful
	ESL_ERC_PARAMETER_INVALID Input parameter is NULL or out of range
	ESL_ERC_WS_STATE_INVALID Invalid state
	ESL_ERC_WS_TOO_SMALL Work space too small
	ESL_ERC_RSA_MESSAGE_OUT_OF_RANGE Input message out of range
Functional Description	
Decrypt message (CRT variant).	
Particularities and Limitations	
Work space initialized	
Call context	
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces	

Table 4-329 esl_decryptRSACRTSHA256_OAEP_Label

4.2.2.5.38 esl_decryptRSACRTSHA256_OAEP

Prototype
`eslt_ErrorCode esl_decryptRSACRTSHA256_OAEP (VSECPRIM_P2VAR_PARA(eslt_WorkSpaceRSACRTOAEPdec) workSpace, eslt_Length cipherSize, VSECPRIM_P2CONST_PARA(eslt_Byte) cipher,`

`VSECPRIM_P2VAR_PARA(eslt_Length) messageSize, VSECPRIM_P2VAR_PARA(eslt_Byte) message)`	
Parameter	
workSpace [in,out]	Decryption work space
cipherSize [in]	Size of encrypted data
cipher [in]	Encrypted data
messageSize [in,out]	Input Maximum size of decrypted data Output Actual size of decrypted data
message [out]	Decrypted data
Return code	
eslt_ErrorCode	ESL_ERC_NO_ERROR Operation successful
	ESL_ERC_PARAMETER_INVALID Input parameter is NULL or out of range
	ESL_ERC_WS_STATE_INVALID Invalid state
	ESL_ERC_WS_TOO_SMALL Work space too small
	ESL_ERC_RSA_MESSAGE_OUT_OF_RANGE Input message out of range
Functional Description	
Decrypt message (CRT variant). Empty string used as default label	
Particularities and Limitations	
Work space initialized	
Call context	
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces	

Table 4-330 esl_decryptRSACRTSHA256_OAEP

4.2.2.6 Key Exchange Algorithms

4.2.2.6.1 esl_initECDH

Prototype	
`eslt_ErrorCode` **`esl_initECDH`** `(VSECPRIM_P2VAR_PARA(eslt_WorkSpaceX25519) workSpace, eslt_Size16 curve)`	
Parameter	
workSpace [in,out]	algorithm context buffer, initialized by esl_initWorkSpaceHeader(..) function
curve [in]	underlying curve; supports - ESL_Curve25519

Return code	
eslt_ErrorCode	ESL_ERC_NO_ERROR - success
	ESL_ERC_PARAMETER_INVALID - some parameter is NULL
	ESL_ERC_CURVE_NOT_SUPPORTED - unknown curve
	ESL_ERC_WS_TOO_SMALL - work space too small
Functional Description	
Init ECDH. This function initializes the EC-D/H key-exchange.	
Particularities and Limitations	
workSpace is initialized by esl_initWorkSpaceHeader(..)	
Call context	
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces	

Table 4-331 esl_initECDH

4.2.2.6.2 esl_initExchangeKeyDHEcP_key

Prototype

`eslt_ErrorCode` **`esl_initExchangeKeyDHEcP_key`**
`(VSECPRIM_P2VAR_PARA(eslt_WorkSpaceEcP) workSpace,`
`VSECPRIM_P2ROMCONST_PARA(eslt_EccDomain) domain,`
`VSECPRIM_P2ROMCONST_PARA(eslt_EccDomainExt) domainExt)`

Parameter	
domain [in]	pointer to domain parameter structure
workSpace [in,out]	(in) algorithm context buffer, initialized by esl_initWorkSpaceHeader(..) function (out) initialized algorithm context structure
domainExt [in]	pointer to domain parameter extension structure
Return code	
eslt_ErrorCode	ESL_ERC_PARAMETER_INVALID input parameter is NULL
	ESL_ERC_WS_TOO_SMALL work space too small
	ESL_ERC_ECC_DOMAIN_INVALID domain structure is invalid
	ESL_ERC_ECC_DOMAINEXT_INVALID domainExt structure is invalid
	ESL_ERC_NO_ERROR else
Functional Description	
This function initializes the DHEcP key exchange algorithm. For key derivation KDF2HMACSHA1 of the library is used.	
Particularities and Limitations	
workSpace is initialized by esl_initWorkSpaceHeader(..) domain and domainExt have to be valid ASN1 structures for EC domain parameters	

The addresses of domain and domainExt have to be valid until returning from esl_exchangeKeyDHecP_key(), because the pointers to the data stored in domain and domainExt are stored in the workspace. These pointers are used as long as the workspace is used. For information about the expected data format in domain and domainExt see Chapter 4.1.
Call context
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces

Table 4-332 esl_initExchangeKeyDHEcP_key

4.2.2.6.3 esl_exchangeKeyDHEcP_key

Prototype	
`eslt_ErrorCode` **`esl_exchangeKeyDHEcP_key`** `(VSECPRIM_P2VAR_PARA(eslt_WorkSpaceEcP) workSpace, VSECPRIM_P2CONST_PARA(eslt_Byte) privateKey, VSECPRIM_P2CONST_PARA(eslt_Byte) publicKey_x, VSECPRIM_P2CONST_PARA(eslt_Byte) publicKey_y, eslt_Length infoLength, VSECPRIM_P2CONST_PARA(eslt_Byte) info, esltt_Length iterationCount, eslt_Length keyLength, VSECPRIM_P2VAR_PARA(eslt_Byte) key)`	
Parameter	
publicKey_x [in]	x-coordinate of the public key Q2 of the key pair (d2, Q2)
publicKey_y [in]	y-coordinate of the public key Q2 of the key pair (d2, Q2) (both of length getLengthOfEcPpublicKey_comp(domain))
privateKey [in]	the private key d1 of the key pair (d1, Q1) (of length getLengthOfEcPprivateKey(domain))
key [in,out]	output key buffer
workSpace [in,out]	algorithm context buffer, initialized by esl_initWorkSpaceHeader(..) function
infoLength [in]	KDF2 salt length in bytes
info [in]	pointer to KDF2 salt (optional, maybe NULL)
iterationCount [in]	KDF2 key derivation iteration count
keyLength [in]	output key buffer length in bytes
Return code	
eslt_ErrorCode	ESL_ERC_PARAMETER_INVALID input parameter is NULL
	ESL_ERC_WS_STATE_INVALID wrong work space state
	ESL_ERC_ECC_PRIVKEY_INVALID privateKey is invalid
	ESL_ERC_ECC_PUBKEY_INVALID publicKey is invalid
	ESL_ERC_KDF_ITERATION_COUNT_OUT_OF_RANGE iterationCount==0
	ESL_ERC_NO_ERROR else
Functional Description	
This function calculates the common secret d1 * Q2 (= d1 * d2 * G) and derives the output key from the x-coordinate of the secret with the KDF2HMACSHA1 derivation function of the library. if (secret_x ==	

publicKey_x) secret_x will replace publicKey_x if (secret_y == publicKey_y) secret_y will replace publicKey_y.
Particularities and Limitations
workSpace is initialized by esl_initExchangeKeyDHEcP_key(..) The buffer referenced by key has to provide at least keyLength bytes
Call context
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces

Table 4-333 esl_exchangeKeyDHEcP_key

4.2.2.6.4 esl_generateEphemeralKeyPairECDH

Prototype	
`eslt_ErrorCode` **`esl_generateEphemeralKeyPairECDH`** `(VSECPRIM_P2VAR_PARA(eslt_WorkSpaceX25519) workSpace, VSECPRIM_P2VAR_PARA(eslt_Byte) publicKey, VSECPRIM_P2VAR_PARA(eslt_Length) publicKeyLength)`	
Parameter	
workSpace [in,out]	Algorithm context buffer, initialized by esl_initWorkSpaceHeader(..) function
publicKey [in,out]	Pointer to public key memory
publicKeyLength [in,out]	Pointer to public key memory size
Return code	
eslt_ErrorCode	ESL_ERC_NO_ERROR - success
	ESL_ERC_PARAMETER_INVALID - some parameter is NULL
	ESL_ERC_WS_STATE_INVALID - work space state invalid
	ESL_ERC_WS_TOO_SMALL - work space too small
	ESL_ERC_BUFFER_TOO_SMALL - buffer for private or public key too small
	ESL_ERC_RNG_FAILED - the random number generator failed
Functional Description	
Generate Ephemeral Key Pair. This function generates an ephemeral key-pair for EC-D/H.	
Particularities and Limitations	
workSpace is initialized by esl_initWorkSpaceHeader(..)	
Call context	
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces	

Table 4-334 esl_generateEphemeralKeyPairECDH

4.2.2.6.5 esl_importStaticPrivateKeyECDH

Prototype	
`eslt_ErrorCode` **`esl_importStaticPrivateKeyECDH`** `(VSECPRIM_P2VAR_PARA(eslt_WorkSpaceX25519) workSpace, VSECPRIM_P2CONST_PARA(eslt_Byte) privateKey)`	
Parameter	
workSpace [in,out]	Algorithm context buffer, initialized by esl_initWorkSpaceHeader(..) function
privateKey [in]	Pointer to private key
Return code	
eslt_ErrorCode	ESL_ERC_NO_ERROR - success
	ESL_ERC_PARAMETER_INVALID - some parameter is NULL
	ESL_ERC_WS_STATE_INVALID - work space state invalid
	ESL_ERC_WS_TOO_SMALL - work space too small
	ESL_ERC_BUFFER_TOO_SMALL - buffer for private or public key too small
	ESL_ERC_RNG_FAILED - the random number generator failed
Functional Description	
Import private key. This function imports a static private key for EC-D/H.	
Particularities and Limitations	
workSpace is initialized by esl_initWorkSpaceHeader(..)	
Call context	
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces	

Table 4-335 esl_importStaticPrivateKeyECDH

4.2.2.6.6 esl_generateSharedSecretECDH

Prototype	
`eslt_ErrorCode` **`esl_generateSharedSecretECDH`** `(VSECPRIM_P2VAR_PARA(eslt_WorkSpaceX25519) workSpace, VSECPRIM_P2VAR_PARA(eslt_Byte) publicKey, VSECPRIM_P2VAR_PARA(eslt_Byte) sharedSecret, VSECPRIM_P2VAR_PARA(eslt_Length) sharedSecretLength)`	
Parameter	
workSpace [in,out]	Algorithm context buffer, initialized by esl_initWorkSpaceHeader(..) function
publicKey [in,out]	Pointer to public key
sharedSecret [out]	Pointer to shared secret memory
sharedSecretLength [in,out]	Pointer to shared secret memory size

Return code	
eslt_ErrorCode	ESL_ERC_NO_ERROR - success
	ESL_ERC_PARAMETER_INVALID - some parameter is NULL
	ESL_ERC_WS_STATE_INVALID - work space state invalid
	ESL_ERC_WS_TOO_SMALL - work space too small
	ESL_ERC_BUFFER_TOO_SMALL - buffer for shared secret too small
Functional Description	
Generate shared secret. This function generates the shared secret EC-D/H.	
Particularities and Limitations	
workSpace is initialized by esl_initWorkSpaceHeader(..)	
Call context	
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces	

Table 4-336 esl_generateSharedSecretECDH

4.2.2.6.7 esl_initGenerateSharedSecretDHEcP_prim

Prototype

`eslt_ErrorCode` **`esl_initGenerateSharedSecretDHEcP_prim`**
`(VSECPRIM_P2VAR_PARA(eslt_WorkSpaceEcP) workSpace,`
`VSECPRIM_P2ROMCONST_PARA(eslt_EccDomain) domain,`
`VSECPRIM_P2ROMCONST_PARA(eslt_EccDomainExt) domainExt)`

Parameter
domain [in] — pointer to domain parameter structure
workSpace [in,out] — (in) algorithm context buffer, initialized by esl_initWorkSpaceHeader(..) function (out) initialized algorithm context structure
domainExt [in] — pointer to domain parameter extension structure
Return code
eslt_ErrorCode: ESL_ERC_PARAMETER_INVALID input parameter is NULL
ESL_ERC_WS_TOO_SMALL work space too small
ESL_ERC_ECC_DOMAIN_INVALID domain structure is invalid
ESL_ERC_ECC_DOMAINEXT_INVALID domainExt structure is invalid
ESL_ERC_NO_ERROR else
Functional Description
This function initializes the DHEcP generate secret algorithm.
Particularities and Limitations
workSpace is initialized by esl_initWorkSpaceHeader(..) domain and domainExt have to be valid ASN1 structures for EC domain parameters

The addresses of domain and domainExt have to be valid until returning from esl_generateSharedSecretDHEcP_prim(), because the pointers to the data stored in domain and domainExt are stored in the workspace. These pointers are used as long as the workspace is used. For information about the expected data format in domain and domainExt see Chapter 4.1.
Call context
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces

Table 4-337 esl_initGenerateSharedSecretDHEcP_prim

4.2.2.6.8 esl_generateSharedSecretDHEcP_prim

Prototype	
`eslt_ErrorCode` **`esl_generateSharedSecretDHEcP_prim`** `(VSECPRIM_P2VAR_PARA(eslt_WorkSpaceEcP) workSpace, VSECPRIM_P2CONST_PARA(eslt_Byte) privateKey, VSECPRIM_P2CONST_PARA(eslt_Byte) publicKey_x, VSECPRIM_P2CONST_PARA(eslt_Byte) publicKey_y, VSECPRIM_P2VAR_PARA(eslt_Byte) secret_x, VSECPRIM_P2VAR_PARA(eslt_BByte) secret_y)`	
Parameter	
workSpace [in,out]	algorithm context buffer, initialized by esl_initWorkSpaceHeader(..) function
privateKey [in]	the private key d1 of the key pair (d1, Q1) (of length getLengthOfEcPprivateKey(domain))
publicKey_x [in]	x-coordinate of the public key Q2 of the key pair (d2, Q2)
publicKey_y [in]	y-coordinate of the public key Q2 of the key pair (d2, Q2) (both of length getLengthOfEcPpublicKey_comp(domain))
secret_x [in,out]	(in) buffer to store x-coordinate of the secret d1 * Q2 (out) x-coordinate of the secret d1 * Q2
secret_y [in,out]	(in) buffer to store y-coordinate of the secret d1 * Q2 (both of length getLengthOfEcPsecret_comp(domain)) (out) y-coordinate of the secret d1 * Q2
Return code	
eslt_ErrorCode	ESL_ERC_PARAMETER_INVALID input parameter is NULL
	ESL_ERC_WS_STATE_INVALID wrong work space state
	ESL_ERC_ECC_PRIVKEY_INVALID privateKey is invalid
	ESL_ERC_ECC_PUBKEY_INVALID publicKey is invalid
	ESL_ERC_NO_ERROR else
Functional Description	
This function calculates the common secret d1 * Q2 (= d1 * d2 * G).	
Particularities and Limitations	
workSpace is initialized by esl_initGenerateSharedSecretDHEcP_prim(..)	
Call context	

> TASK > This function is Synchronous > This function is Reentrant, for different workspaces

Table 4-338 esl_generateSharedSecretDHEcP_prim

4.2.2.6.9 esl_initECBD

Prototype	
`eslt_ErrorCode esl_initECBD (VSECPRIM_P2VAR_PARA(eslt_WorkSpaceECBD) workSpace, const eslt_Size32 nDevices, const eslt_Size32 deviceID, VSECPRIM_P2CONST_PARA(eslt_EccDomain) domain, VSECPRIM_P2CONST_PARA(eslt_EccDomainExt) domainExt, VSECPRIM_P2CONST_PARA(eslt_EccSpeedUpExt) speedUppExt)`	
Parameter	
deviceID [in]	ID of this device (1..nDevices)
domainExt [in]	domain parameter extension structure
workSpace [in,out]	algorithm context buffer, initialized by esl_initWorkSpaceHeader(..) function
nDevices [in]	number of devices partaking in the key agreement
domain [in]	domain parameter structure
speedUpExt [in]	pre-computation structure
Return code	
eslt_ErrorCode	ESL_ERC_PARAMETER_INVALID some parameter is NULL
	ESL_ERC_WS_TOO_SMALL work space too small
	ESL_ERC_INPUT_INVALID deviceID is not matching nDevices
	ESL_ERC_NO_ERROR otherwise
Functional Description	
This function initializes the EC-B/D key agreement.	
Particularities and Limitations	
workSpace is initialized by esl_initWorkSpaceHeader(..) domain and domainExt have to be valid ASN1 structures for EC domain parameters speedUpExt has to be a valid ASN1 struct for precomputed EC points The addresses of domain, domainExt and speedUpExt have to be valid until returning from esl_generateEphemeralKeyPairECBD(), esl_calculateIntermediateECBD(), esl_initSharedSecretECBD, esl_updateSharedSecretECBD and esl_retrieveSharedSecretECBD, because the pointers to the data stored in domain, domainExt and speedUpExt are stored in the workspace. These pointers are used as long as the workspace is used. For information about the expected data format in domain, domainExt and speedUpExt see Chapter 4.1.	

Call context
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces

Table 4-339 esl_initECBD

4.2.2.6.10 esl_generateEphemeralKeyPairECBD

Prototype	
`eslt_ErrorCode` **`esl_generateEphemeralKeyPairECBD`** `(VSECPRIM_P2VAR_PARA(eslt_WorkSpaceECBD) workSpace, VSECPRIM_P2VAR_PARA(eslt_Byte) privateKey, VSECPRIM_P2VAR_PARA(eslt_Byte) publicKey_x, VSECPRIM_P2VAR_PARA(eslt_Byte) publicKey_y)`	
Parameter	
publicKey_x [in,out]	(in) pointer to memory for public key (X coordinate) (out) public key (X coordinate)
publicKey_y [in,out]	(in) pointer to memory for public key (Y coordinate) (out) public key (Y coordinate)
privateKey [in,out]	(in) pointer to memory for private key (if NULL, key is not returned) (out) private key (if pointer not NULL)
workSpace [in,out]	algorithm context buffer, initialized by esl_initECBD(..) function
Return code	
eslt_ErrorCode	ESL_ERC_PARAMETER_INVALID some parameter is NULL
	ESL_ERC_WS_STATE_INVALID work space state invalid
	ESL_ERC_WS_TOO_SMALL work space too small
	ESL_ERC_RNG_FAILED the random number generator failed
	ESL_ERC_ECC_INTERNAL_ERROR calculation failed
	ESL_ERC_NO_ERROR otherwise
Functional Description	
This function generates an ephemeral key-pair for EC-B/D.	
Particularities and Limitations	
workSpace is initialized by esl_initECBD(..) both publicKey_x and .._y are sufficiently large memory areas (>= prime length of the underlying curve)	
Call context	
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces	

Table 4-340 esl_generateEphemeralKeyPairECBD

4.2.2.6.11 esl_calculateIntermediateECBD

Prototype	
`eslt_ErrorCode esl_calculateIntermediateECBD (VSECPRIM_P2VAR_PARA(eslt_WorkSpaceECBD) workSpace, VSECPRIM_P2CONST_PARA(eslt_Byte) pubKeyLeft_x, VSECPRIM_P2CONST_PARA(eslt_Byte) pubKeyLeft_y, VSECPRIM_P2CONST_PARA(eslt_Byte) pubKeyRight_x, VSECPRIM_P2CONST_PARA(eslt_Byte) pubKeyRight_y, VSECPRIM_P2VAAR_PARA(eslt_Byte) intermediate_x, VSECPRIM_P2VAR_PARA(eslt_Byte) intermediate_y)`	
Parameter	
intermediate_y [in,out]	(in) pointer to memory for intermediate value (Xi - Y coordinate) (out) intermediate value (Xi - Y coordinate)
workSpace [in,out]	algorithm context buffer, initialized by esl_initECBD(..) function
intermediate_x [in,out]	(in) pointer to memory for intermediate value (Xi - X coordinate) (out) intermediate value (Xi - X coordinate)
pubKeyLeft_y [in]	pointer to left neighbor's public key (Zi-1 - Y coordinate)
pubKeyLeft_x [in]	pointer to left neighbor's public key (Zi-1 - X coordinate)
pubKeyRight_y [in]	pointer to right neighbor's public key (Zi+1 - Y coordinate)
pubKeyRight_x [in]	pointer to right neighbor's public key (Zi+1 - X coordinate)
Return code	
eslt_ErrorCode	ESL_ERC_PARAMETER_INVALID some parameter is NULL
	ESL_ERC_WS_STATE_INVALID work space state invalid
	ESL_ERC_WS_TOO_SMALL work space too small
	ESL_ERC_ECC_INTERNAL_ERROR calculation failed
	ESL_ERC_NO_ERROR otherwise
Functional Description	
This function calculates the intermediate value (Xi) from the left and right neighbors' public keys.	
Particularities and Limitations	
workSpace is initialized by esl_initECBD(..) both intermediate_x and .._y are sufficiently large memory areas (>= prime length of the underlying curve)	
Call context	
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces	

Table 4-341 esl_calculateIntermediateECBD

4.2.2.6.12 esl_initSharedSecretECBD

Prototype	
`eslt_ErrorCode` **`esl_initSharedSecretECBD`** `(VSECPRIM_P2VAR_PARA(eslt_WorkSpaceECBD) workSpace)`	
Parameter	
workSpace [in,out]	algorithm context buffer, initialized by esl_initECBD(..) function
Return code	
eslt_ErrorCode	ESL_ERC_PARAMETER_INVALID some parameter is NULL
	ESL_ERC_WS_STATE_INVALID work space state invalid
	ESL_ERC_WS_TOO_SMALL work space too small
	ESL_ERC_NO_ERROR otherwise
Functional Description	
This function initializes the shared secret generation for EC-B/D. It calculates the starting value from already known values (stored in the work space) Zi, ai, Xi-1 and nNodes.	
Particularities and Limitations	
workSpace is initialized by esl_initECBD(..)	
Call context	
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces	

Table 4-342 esl_initSharedSecretECBD

4.2.2.6.13 esl_updateSharedSecretECBD

Prototype	
`eslt_ErrorCode` **`esl_updateSharedSecretECBD`** `(VSECPRIM_P2VAR_PARA(eslt_WorkSpaceECBD) workSpace, const eslt_Size32 deviceID, VSECPRIM_P2CONST_PARA(eslt_Byte) intermediate_x, VSECPRIM_P2CONST_PARA(eslt_Byte) intermediate_y)`	
Parameter	
intermediate_y [in]	pointer to intermediate value (Xi - Y coordinate)
deviceID [in]	ID of the device the intermediate was received from
workSpace [in,out]	algorithm context buffer, initialized by esl_initECBD(..) function
intermediate_x [in]	pointer to intermediate value (Xi - X coordinate)
Return code	
eslt_ErrorCode	ESL_ERC_PARAMETER_INVALID some parameter is NULL
	ESL_ERC_WS_STATE_INVALID work space state invalid
	ESL_ERC_WS_TOO_SMALL work space too small
	ESL_ERC_NO_ERROR otherwise

Functional Description
This function updates the shared secret with an intermediate from another ECU. It is to be called repeatedly for all intermediate values received.
Particularities and Limitations
workSpace is initialized by esl_initECBD(..)
Call context
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces

Table 4-343 esl_updateSharedSecretECBD

4.2.2.6.14 esl_retrieveSharedSecretECBD

Prototype	
`eslt_ErrorCode` **`esl_retrieveSharedSecretECBD`** `(VSECPRIM_P2VAR_PARA(eslt_WorkSpaceECBD) workSpace, VSECPRIM_P2VAR_PARA(eslt_Byte) sharedSecret_x, VSECPRIM_P2VAR_PARA(eslt_Byte) sharedSecret_y)`	
Parameter	
sharedSecret_x [in,out]	(in) pointer to memory for shared secret (K - X coordinate) (out) shared secret (K - X coordinate)
sharedSecret_y [in,out]	(in) pointer to memory for shared secret (K - Y coordinate) (if NULL, coordinate is not returned) (out) shared secret (K - Y coordinate)
workSpace [in,out]	algorithm context buffer, initialized by esl_initECBD(..) function
Return code	
eslt_ErrorCode	ESL_ERC_PARAMETER_INVALID some parameter is NULL
	ESL_ERC_WS_STATE_INVALID work space state invalid
	ESL_ERC_WS_TOO_SMALL work space too small
	ESL_ERC_ECC_INTERNAL_ERROR calculation failed
	ESL_ERC_NO_ERROR otherwise
Functional Description	
This function retrieves the shared secret (K). When either sharedSecret_x or .._y are NULL, the respective value is DISCARDED.	
Particularities and Limitations	
workSpace is initialized by esl_initECBD(..)	
Call context	
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces	

Table 4-344 esl_retrieveSharedSecretECBD

4.2.2.6.15 esl_initSPAKE2PPreamble

Prototype	
`eslt_ErrorCode esl_initSPAKE2PPreamble (VSECPRIM_P2VAR_PARA(eslt_WorkSpaceSPAKE2PPreamble) wsSpakePre, VSECPRIM_P2ROMCONST_PARA(eslt_EccDomain) domain, VSECPRIM_P2ROMCONST_PARA(eslt_EccDomainExt) domainExt)`	
Parameter	
wsSpakePre [in,out]	(in) algorithm context buffer, initialized by esl_initSPAKE2PPreamble(..) (out) algorithm context buffer,
domain [in]	Pointer to domain parameter structure
domainExt [in]	pointer to domain parameter extension structure
Return code	
eslt_ErrorCode	ESL_ERC_NO_ERROR - success
	ESL_ERC_PARAMETER_INVALID - some parameter is NULL
	ESL_ERC_CURVE_NOT_SUPPORTED - a different curve than P256 is used
	ESL_ERC_WS_TOO_SMALL - work space too small
	ESL_ERC_ECC_DOMAIN_INVALID domain structure is invalid
	ESL_ERC_ECC_DOMAINEXT_INVALID domainExt structure is invalid
Functional Description	
This function initializes the SPAKE2+ preamble phase. In this function, the curve domain is parsed in order to find the field prime p, which is used in a later step during the preamble phase.	
Particularities and Limitations	
workSpace is initialized by esl_initWorkSpaceHeader(..) domain and domainExt have to be valid ASN1 structures for EC domain parameters The addresses of domain and domainExt have to be valid until returning from esl_calcSPAKE2PPreamble (), because the pointers to the data stored in domain and domainExt are stored in the workspace. These pointers are used as long as the workspace is used. For information about the expected data format in domain and domainExt see Chapter 4.1.	
Call context	
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces	

Table 4-345 esl_initSPAKE2PPreamble

4.2.2.6.16 esl_calcSPAKE2PPreamble

```
eslt_ErrorCode esl_calcSPAKE2PPreamble
(VSECPRIM_P2VAR_PARA(eslt_WorkSpaceSPAKE2PPreamble) wsSpakePre,
VSECPRIM_P2CONST_PARA(eslt_Byte) pbkdfOutput, const eslt_Length
pbkdfOutputLength, VSECPRIM_P2VAR_PARA(eslt_Byte) w0,
VSECPRIM_P2VAR_PARA(eslt_Length) w0Length, VSECPRIM_P2VAR_PARA(eslt_Byte) w1,
VSECPRIM_P2VAR_PARA(eslt_Length) w1Length,  VSECPRIM_P2VAR_PARA(eslt_Byte)
pointL,
```

```
  VSECPRIM_P2VAR_PARA(eslt_Length) pointLLength)
```

Parameter	
wsSpakePre [in,out]	algorithm context buffer, initialized by esl_initSPAKE2PPreamble(..)
pbkdfOutput [in]	PBKDF output, [w0s || w1s] = PBKDF(...)
pbkdfOutputLength [in]	length of the buffer referenced by pbkdfOutput
w0 [in,out]	(in) buffer to store the result w0 = w0s mod n (out) the result w0
w0Length [in,out]	(in) length of buffer referenced by w0 (out) data length (in bytes) written to w0
w1 [in,out]	(in) buffer to store the result w1 = w1s mod n (out) the result w1
w1Length [in,out]	(in) length of buffer referenced by w1 (out) data length (in bytes) written to w1
pointL	(in) buffer to store the point L = w1*P in uncompressed representation, including the encoding byte 0x04 (out) the point L = w1*P in uncompressed representation, including the encoding byte 0x04
pointLLength	(in) length of the buffer referenced by pointL (out) data length (in bytes) written to pointL
Return code	
eslt_ErrorCode	ESL_ERC_NO_ERROR - success
	ESL_ERC_PARAMETER_INVALID - some parameter is NULL
	ESL_ERC_WS_TOO_SMALL - work space too small
	ESL_ERC_WS_STATE_INVALID wrong work space state
	ESL_ERC_BUFFER_TOO_SMALL memory provided by output buffers not sufficient
	ESL_ERC_INVALID_LENGTH value of pbkdfOutputLength invalid (i.e., odd or too high)
	ESL_ERC_ERROR an error occurred during the reduction or ECC operation

Functional Description

This function calculates the values w0 and w1 as w0 = w0s mod n, w1 = w1s mod n and L = w1*P according to the SPAKE2+ preamble phase.

w0s and w1s are assumed to be concatenated in the parameter 'pbkdfOutput'. n is the curve group order and P is the curve base point.

Particularities and Limitations

workSpace is initialized by esl_initSPAKE2PPreamble(..)

The buffers referenced by w0 and w1 each must provide esl_getLengthOfEcPprivateKey() many (i.e., group order n many) bytes.

The buffer referenced by pointL must provide at least esl_getLengthOfEcPpublicKey_comp() many bytes (i.e., 2*p + 1 many bytes, where p is the field prime).

The buffer referenced by pbkdfOutput must provide at least pbkdfOutputLength many bytes.

The value of pbkdfOutputLength can be any even value between 0 and VSECPRIM_SPAKE2_P_PREAMBLE_OKMLENGTH. In most cases pbkdfOutputLength >= 2*esl_getLengthOfEcPprivateKey().

Call context
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces

Table 4-346 esl_calcSPAKE2PPreamble

4.2.2.6.17 esl_initSPAKE2P

Prototype
`eslt_ErrorCode esl_initSPAKE2P (VSECPRIM_P2VAR_PARA(eslt_WorkSpaceSPAKE2P) workSpace, VSECPRIM_P2ROMCONST_PARA(eslt_EccDomain) domain, VSECPRIM_P2ROMCONST_PARA(eslt_EccDomainExt) domainExt, VSECPRIM_P2CONST_PARA(eslt_Byte) pointM, const eslt_Length mlength, VSECPRIM_P2CONST_PARA(eslt_Byte) pointN, const eslt_Length nlength, eslt_SPAKE2PMode ciphersuite)`

Parameter	
workSpace [in,out]	algorithm context buffer, initialized by esl_initWorkSpaceHeader(..) function
domain [in]	Pointer to domain parameter structure
domainExt [in]	pointer to domain parameter extension structure
pointM [in]	Point M on the curve in uncompressed representation, including the encoding byte 0x04
mlength [in]	length of buffer referenced by pointM
pointN [in]	Point N on the curve in uncompressed representation, including the encoding byte 0x04
nlength [in]	length of buffer referenced by pointN
ciphersuite [in]	parameter specifying the ciphersuite (hash function, key derivation function, mac algorithm) and the version used by SPAKE2+ (order of keys). Currently, only the combination [SHA256 | HKDF | CMAC-AES-128] is supported

Return code	
eslt_ErrorCode	ESL_ERC_NO_ERROR - success
	ESL_ERC_PARAMETER_INVALID - some parameter is NULL or length value is invalid or point encoding is incorrect
	ESL_ERC_CURVE_NOT_SUPPORTED - a different curve than P256 is used
	ESL_ERC_WS_TOO_SMALL - work space too small
	ESL_ERC_ECC_DOMAIN_INVALID domain structure is invalid
	ESL_ERC_ECC_DOMAINEXT_INVALID domainExt structure is invalid
	ESL_ERC_MODE_INVALID value of ciphersuite is not consistent with supported ciphersuite

Functional Description
This function initializes the SPAKE2+ key-exchange algorithm.

Particularities and Limitations
workSpace is initialized by esl_initWorkSpaceHeader(..)

domain and domainExt have to be valid ASN1 structures for EC domain parameters The addresses of domain and domainExt have to be valid until returning from esl_calcSharedSecretSPAKE2P(), because the pointers to the data stored in domain and domainExt are stored in the workspace. These pointers are used as long as the workspace is used. For information about the expected data format in domain and domainExt see Chapter 4.1. The value of mlength and nlength must be (2*esl_getLengthOfEcPpublicKey_comp() + 1), (i.e., 2*p + 1, where p is the field prime). The points M and N can be found in file ESLib_SPAKE2PConstants.h.
Call context
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces

Table 4-347 esl_initSPAKE2P

4.2.2.6.18 esl_setPreambleDataPartyASPAKE2P

```
eslt_ErrorCode esl_setPreambleDataPartyASPAKE2P
(VSECPRIM_P2VAR_PARA(eslt_WorkSpaceSPAKE2P) workSpace,
VSECPRIM_P2CONST_PARA(eslt_Byte) w0, const eslt_Length w0length,
VSECPRIM_P2CONST_PARA(eslt_Byte) w1, const eslt_Length w1length)
```

Parameter	
workSpace [in,out]	algorithm context buffer, initialized by esl_initWorkSpaceHeader(..) function
w0 [in]	value w0 (w0 is the first half of the OKM of a PBKDF mod n, computed during the preamble phase)
w0length [in]	length of buffer referenced by w0
w1 [in]	value w1 (w1 is the first half of the OKM of a PBKDF mod n, computed during the preamble phase)
w1length [in]	length of buffer referenced by w1
Return code	
eslt_ErrorCode	ESL_ERC_NO_ERROR - success
	ESL_ERC_PARAMETER_INVALID - input parameter is NULL or length values invalid
	ESL_ERC_WS_TOO_SMALL - work space too small
	ESL_ERC_WS_STATE_INVALID - wrong work space state
	ESL_ERC_STATE_INVALID - esl_setPreambleDataPartyASPAKE2P() or esl_setPreambleDataPartyBSPAKE2P() was called before with the same workSpace
Functional Description	
This function provides data from the preamble phase to the SPAKE2+ algorithm for party A. The provided data will be stored in the workspace. In this function the workspace also sets a flag that the operations for party A shall be executed.	
Particularities and Limitations	
workSpace is initialized by esl_initSPAKE2P(..)	

The value of w0length and w1length must be esl_getLengthOfEcPprivateKey(), (i.e., n, where n is the elliptic curve group order).
Call context
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces

Table 4-348 esl_setPreambleDataPartyASPAKE2P

4.2.2.6.19 esl_setPreambleDataPartyBSPAKE2P

```
eslt_ErrorCode esl_setPreambleDataPartyBSPAKE2P
(VSECPRIM_P2VAR_PARA(eslt_WorkSpaceSPAKE2P) workSpace,
VSECPRIM_P2CONST_PARA(eslt_Byte) w0, const eslt_Length w0length,
VSECPRIM_P2CONST_PARA(eslt_Byte) pointL, const eslt_Length pointLLength)
```

Parameter	
workSpace [in,out]	algorithm context buffer, initialized by esl_initWorkSpaceHeader(..) function
w0 [in]	value w0 (w0 is the first half of the OKM of a PBKDF mod n, computed during the preamble phase)
w0length [in]	length of buffer referenced by w0
pointL [in]	point L in uncompressed representation, including the encoding byte 0x04 (L = w1*P, where w1 is the second half of the OKM of a PBKDF mod p, computed during the preamble phase, and P is the base point of the curve)
pointLLength [in]	length of buffer referenced by pointL
Return code	
eslt_ErrorCode	ESL_ERC_NO_ERROR - success
	ESL_ERC_PARAMETER_INVALID - input parameter is NULL or length values invalid or point encoding invalid
	ESL_ERC_WS_TOO_SMALL - work space too small
	ESL_ERC_WS_STATE_INVALID - wrong work space state
	ESL_ERC_STATE_INVALID - esl_setPreambleDataPartyASPAKE2P() or esl_setPreambleDataPartyBSPAKE2P() was called before with the same workSpace
Functional Description	
This function provides data from the preamble phase to the SPAKE2+ algorithm for party B. The provided data will be stored in the workspace. In this function the workspace also sets a flag that the operations for party B shall be executed.	
Particularities and Limitations	
workSpace is initialized by esl_initSPAKE2P(..) The value of w0length must be esl_getLengthOfEcPprivateKey(), (i.e., n, where n is the elliptic curve group order). The value of pointLLength must be (2*esl_getLengthOfEcPpublicKey_comp() + 1), (i.e., 2*p + 1, where p is the field prime).	
Call context	

> TASK > This function is Synchronous > This function is Reentrant, for different workspaces

Table 4-349 esl_setPreambleDataPartyBSPAKE2P

4.2.2.6.20 esl_calcPubValSPAKE2P

```
eslt_ErrorCode esl_calcPubValSPAKE2P
(VSECPRIM_P2VAR_PARA(eslt_WorkSpaceSPAKE2P) workSpace,
VSECPRIM_P2VAR_PARA(eslt_Byte) pubVal_x, VSECPRIM_P2VAR_PARA(eslt_Byte)
pubVal_y, VSECPRIM_P2VAR_PARA(eslt_Length) pubValLength)
```

Parameter	
workSpace [in,out]	algorithm context buffer, initialized by esl_initWorkSpaceHeader(..) function
pubVal_x [in,out]	(in) buffer to store the x-coordinate of the public value X or Y (of length getLengthOfEcPpublicKey_comp(domain)) (out) the x-coordinate of the public value X or Y
pubVal_y [in,out]	(in) buffer to store the y-coordinate of the public value X or Y (of length getLengthOfEcPpublicKey_comp(domain)) (out) the y-coordinate of the public value X or Y
pubValLength [in,out]	(in) length of the buffers referenced by pubVal_x and pubVal_y (out) length of the data in byte written to pubVal_x and pubVal_y
Return code	
eslt_ErrorCode	ESL_ERC_NO_ERROR - success
	ESL_ERC_PARAMETER_INVALID - some parameter is NULL
	ESL_ERC_STATE_INVALID - no preamble data was set before calling esl_calcPubValSPAKE2P()
	ESL_ERC_WS_TOO_SMALL - work space too small
	ESL_ERC_WS_STATE_INVALID - wrong work space state
	ESL_ERC_BUFFER_TOO_SMALL - memory provided by output buffers not sufficient
	ESL_ERC_ERROR - calculation of public values failed
	all possible error codes of esl_getBytesRNG()
	ESL_ERC_ECC_INTERNAL_ERROR - curve point operation failed
Functional Description	
This function calculates the public value for party A or party B according to the SPAKE2+ protocol. The workspace stores a flag indicating whether the operations for party A or for party B shall be executed, i.e., this function computes - X = pA = x*P + w0*M or - Y = pB = y*P + w0*N The random number x (or y) is generated using the RNG which is implemented by the call back function esl_getBytesRNG(..).	
Particularities and Limitations	
workSpace is initialized by esl_initSPAKE2P(..), esl_setPreambleDataPartyASPAKE2P() or	

esl_setPreambleDataPartyBSPAKE2P() called before The two buffers referenced by pubVal_x and pubVal_y each have to provide at least *pubValLength bytes. The value of pubValLength is expected to be esl_getLengthOfEcPpublicKey_comp(), (i.e., p, where p is the field prime)
Call context
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces

Table 4-350 esl_calcPubValSPAKE2P

4.2.2.6.21 esl_setAdditionalInformationSPAKE2P

```
eslt_ErrorCode
esl_setAdditionalInformationSPAKE2P(VSECPRIM_P2VAR_PARA(eslt_WorkSpaceSPAKE2P)
workSpace, VSECPRIM_P2CONST_PARA(eslt_Byte) infoA, const eslt_Length
infoALength, VSECPRIM_P2CONST_PARA(eslt_Byte) infoB, const eslt_Length
infoBLength, VSECPRIM_P2CONST_PARA(eslt_Byte) aad, const eslt_Length
aadLength);
```

Parameter	
workSpace [in,out]	algorithm context buffer, initialized by esl_initWorkSpaceHeader(..) function
infoA [in]	additional information concering party A (e.g. A's name,...) or NULL
infoALength [in]	length of buffer referenced by infoA, possibly 0
infoB [in]	additional information concering party B (e.g. B's name,...) or NULL
infoBLength [in]	length of buffer referenced by infoB, possibly 0
aad [in]	additional authentication data, used as input for the KDF, or NULL
aadLength [in]	length of buffer referenced by aad, possibly 0
Return code	
eslt_ErrorCode	ESL_ERC_NO_ERROR - success
	ESL_ERC_PARAMETER_INVALID - some parameter is NULL
	ESL_ERC_WS_TOO_SMALL - work space too small
	ESL_ERC_WS_STATE_INVALID - wrong work space state
	ESL_ERC_STATE_INVALID - esl_calcSharedSecretSPAKE2P() already called before
	ESL_ERC_INVALID_LENGTH - aadLength greater than configured maximal aad length
Functional Description	
This optional function provides additional input data for the derivation of the shared secret according to the SPAKE2+ protocol. The provided data will be stored in the workspace. This function does not distinguish between the two parties A and B. Calling this function is optional. The call to this function is optional. This function can be called at any time after the call to esl_initSPAKE2P() and before the call to esl_calcSharedSecretSPAKE2P().	

Particularities and Limitations
workSpace is initialized by esl_initSPAKE2P(..)
Call context
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces

Table 4-351 esl_setAdditionalInformationSPAKE2P

4.2.2.6.22 esl_calcSharedSecretSPAKE2P

```
eslt_ErrorCode esl_calcSharedSecretSPAKE2P
(VSECPRIM_P2VAR_PARA(eslt_WorkSpaceSPAKE2P) workSpace,
VSECPRIM_P2CONST_PARA(eslt_Byte) pubVal_x, VSECPRIM_P2CONST_PARA(eslt_Byte)
pubVal_y, const eslt_Length pubValLength, VSECPRIM_P2VAR_PARA(eslt_Byte)
sharedSecret, VSECPRIM_P2VAR_PARA(eslt_Length) sharedSecretLength,
VSECPRIM_P2VAR_PARA(eslt_Byte) confirmationMAC,
VSECPRIM_P2VAR_PARA(eslt_Length) confirmationMACLength)
```

Parameter	
workSpace [in,out]	(in) algorithm context buffer, initialized by esl_initSPAKE2P(..) (out) algorithm context buffer, now storing the common values Z and V, the shared secret and the MAC keys
pubVal_x [in]	x-coordinate of the other party's public value (of length getLengthOfEcPpublicKey_comp(domain))
pubVal_y [in]	y-coordinate of the other party's public value (of length getLengthOfEcPpublicKey_comp(domain))
pubValLength [in]	length of the buffers referenced by pubVal_x and pubVal_y
sharedSecret [in,out]	(in) buffer to store the secret Ke (out) secret Ke
sharedSecretLength [in,out]	(in) length of buffer referenced by sharedSecret (out) data length (in bytes) written to sharedSecret
confirmationMAC [in,out]	(in) buffer to store confirmation MAC (out) confirmation MAC
confirmationMACLength [in,out]	(in) length of buffer referenced by confirmationMAC (out) data length (in bytes) written to confirmationMAC
Return code	
eslt_ErrorCode	ESL_ERC_NO_ERROR - success
	ESL_ERC_PARAMETER_INVALID - input parameter is NULL or pubValLength is invalid
	ESL_ERC_STATE_INVALID - esl_calcPubValSPAKE2P() was not called before
	ESL_ERC_WS_TOO_SMALL - work space too small
	ESL_ERC_WS_STATE_INVALID - wrong work space state

	ESL_ERC_BUFFER_TOO_SMALL - memory provided by output buffers not sufficient
Functional Description	
This function calculates the common secret Ke, where Ka||Ke = Hash(TT), the MAC keys and the confirmation MAC according to the SPAKE2+ protocol. The MAC keys are not written to output buffers, but only stored in the workspace.	
Particularities and Limitations	
workSpace is initialized by esl_initSPAKE2P(..), esl_calcPubValSPAKE2P() called before. The buffer referenced by sharedSecret must provide at least *sharedSecretLength many bytes. The buffer referenced by confirmationMAC must provide at least *confirmationMACLength many bytes. The value of pubValLength must be esl_getLengthOfEcPpublicKey_comp(), (i.e., p, where p is the field prime). The buffers referenced by pubVal_x and pubVal_y both must provide at least pubValLength many bytes. The buffer referenced by sharedSecret must provide at least (digest length/2) bytes, where digest length is the digest length of the hash function specified by the cipher suite. The buffer referenced by confirmationMAC must provide at least MAC length bytes, where MAC length is the length of the MAC produced by the MAC algorithm specified by the cipher suite.	
Call context	
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces	

Table 4-352 esl_calcSharedSecretSPAKE2P

4.2.2.6.23 esl_confirmKeySPAKE2P

```
eslt_ErrorCode esl_confirmKeySPAKE2P
(VSECPRIM_P2VAR_PARA(eslt_WorkSpaceSPAKE2P) workSpace,
VSECPRIM_P2CONST_PARA(eslt_Byte) mac, const eslt_Length macLength)
```

Parameter	
workSpace [in,out]	algorithm context buffer, initialized by esl_initWorkSpaceHeader(..) function
mac [in]	other party's MAC to be verified
macLength [in]	length of mac in bytes
Return code	
eslt_ErrorCode	ESL_ERC_NO_ERROR - success
	ESL_ERC_PARAMETER_INVALID - some parameter is NULL
	ESL_ERC_STATE_INVALID - esl_calcSharedSecretSPAKE2P() was not called before
	ESL_ERC_WS_TOO_SMALL - work space too small
	ESL_ERC_WS_STATE_INVALID - wrong work space state
	ESL_ERC_INVALID_LENGTH - macLength does not match the expected MAC length according to the MAC algorithm specified in the cipher suite
	ESL_ERC_INCORRECT_MAC - key confirmation failed, given MAC incorrect

Functional Description
This function re-calculates the other party's confirmation MAC according to the SPAKE2+ protocol. This function performs the required key confirmation by comparing the given MAC to the recomputed one based on data stored in the workspace.
Particularities and Limitations
workSpace is initialized by esl_initSPAKE2P(..), esl_calcSharedSecretSPAKE2P() called before. The value of macLength is expected to exactly match the MAC length of the MAC algorithm, specified by the cipher suite. The buffer referenced by mac must provide at least macLength many bytes.
Call context
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces

Table 4-353 esl_confirmKeySPAKE2P

4.2.2.7 Key Derivation Functions (KDF)

4.2.2.7.1 esl_initKDFX963SHA1

Prototype	
`eslt_ErrorCode esl_initKDFX963SHA1 (VSECPRIM_P2VAR_PARA(eslt_WorkSpaceKDFX963SHA1) workSpace)`	
Parameter	
workSpace [in,out]	KDF X9.63 with SHA1 work space
Return code	
eslt_ErrorCode	ESL_ERC_NO_ERROR Operation successful
	ESL_ERC_WS_TOO_SMALL Work space too small
	ESL_ERC_PARAMETER_INVALID Input parameter is NULL
Functional Description	
Initializes KDF X9.63 with SHA1 calculation. This function initializes KDF X9.63 with SHA1 calculation.	
Particularities and Limitations	
workSpace is initialized by esl_initWorkSpaceHeader(..)	
Call context	
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces	

Table 4-354 esl_initKDFX963SHA1

4.2.2.7.2 esl_deriveKeyKDFX963SHA1

Prototype	
`eslt_ErrorCode` **`esl_deriveKeyKDFX963SHA1`** `(VSECPRIM_P2VAR_PARA(eslt_WorkSpaceKDFX963SHA1) workSpace, const eslt_Length secretLength, VSECPRIM_P2CONST_PARA(eslt_Byte) secret, const eslt_Length infoLength, VSECPRIM_P2CONST_PARA(eslt_Byte) info, const eslt_Length keyLength, VSECPRIM_P2VAR_PARA(eslt__Byte) key)`	
Parameter	
workSpace [in,out]	KDF X9.63 with SHA1 work space
secretLength [in]	length of secret in bytes
secret [in]	Pointer to the secret
infoLength [in]	length of salt in bytes (if zero, no salt is used)
info [in]	pointer to the optional salt (if NULL, no salt is used)
keyLength [in]	length of output key buffer in bytes
key [out]	pointer to the output key buffer
Return code	
eslt_ErrorCode	ESL_ERC_NO_ERROR Operation successful
	ESL_ERC_PARAMETER_INVALID Input parameter is NULL
	ESL_ERC_WS_TOO_SMALL Work space too small
	ESL_ERC_WS_STATE_INVALID Work space state invalid
	ESL_ERC_INPUT_INVALID Invalid input length
Functional Description	
Derive Key according KDF X9.63 with SHA1. This function derives a key according to KDF X9.63 with SHA1.	
Particularities and Limitations	
workSpace is initialized by esl_initKDFX963SHA1(..) key need to have at least the length of keyLength.	
Call context	
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces	

Table 4-355 esl_deriveKeyKDFX963SHA1

4.2.2.7.3 esl_initKDFX963SHA256

Prototype	
`eslt_ErrorCode` **`esl_initKDFX963SHA256`** `(VSECPRIM_P2VAR_PARA(eslt_WorkSpaceKDFX963SHA256) workSpace)`	
Parameter	
workSpace [in,out]	KDF X9.63 with SHA256 work space

Return code	
eslt_ErrorCode	ESL_ERC_NO_ERROR Operation successful
	ESL_ERC_WS_TOO_SMALL Work space too small
	ESL_ERC_PARAMETER_INVALID Input parameter is NULL
Functional Description	
Initializes KDF X9.63 with SHA256 calculation. This function initializes KDF X9.63 with SHA256 calculation.	
Particularities and Limitations	
workSpace is initialized by esl_initWorkSpaceHeader(..)	
Call context	
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces	

Table 4-356 esl_initKDFX963SHA256

4.2.2.7.4 esl_deriveKeyKDFX963SHA256

Prototype	
`eslt_ErrorCode esl_deriveKeyKDFX963SHA256 (VSECPRIM_P2VAR_PARA(eslt_WorkSpaceKDFX963SHA256) workSpace, const eslt_Length secretLength, VSECPRIM_P2CONST_PARA(eslt_Byte) secret, const eslt_Length infoLength, VSECPRIM_P2CONST_PARA(eslt_Byte) info, const eslt_Length keyLength, VSECPRIM_P2VAR_PARA(esllt_Byte) key)`	
Parameter	
workSpace [in,out]	KDF X9.63 with SHA256 work space
secretLength [in]	length of secret in bytes
secret [in]	Pointer to the secret
infoLength [in]	length of salt in bytes (if zero, no salt is used)
info [in]	pointer to the optional salt (if NULL, no salt is used)
keyLength [in]	length of output key buffer in bytes
key [out]	pointer to the output key buffer
Return code	
eslt_ErrorCode	ESL_ERC_NO_ERROR Operation successful
	ESL_ERC_PARAMETER_INVALID Input parameter is NULL
	ESL_ERC_WS_TOO_SMALL Work space too small
	ESL_ERC_WS_STATE_INVALID Work space state invalid
	ESL_ERC_INPUT_INVALID Invalid input length
Functional Description	
Derive Key according KDF X9.63 with SHA256.	

This function derives a key according to KDF X9.63 with SHA256.
Particularities and Limitations
workSpace is initialized by esl_initKDFX963SHA256(..) Length of the buffer provided by key need to have at least keyLength.
Call context
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces

Table 4-357 esl_deriveKeyKDFX963SHA256

4.2.2.7.5 esl_initKDFX963SHA512

Prototype	
`eslt_ErrorCode` **`esl_initKDFX963SHA512`** `(VSECPRIM_P2VAR_PARA(eslt_WorkSpaceKDFX963SHA512) workSpace)`	
Parameter	
workSpace [in,out]	KDF X9.63 with SHA512 work space
Return code	
eslt_ErrorCode	ESL_ERC_NO_ERROR Operation successful
	ESL_ERC_WS_TOO_SMALL Work space too small
	ESL_ERC_PARAMETER_INVALID Input parameter is NULL
Functional Description	
Initializes KDF X9.63 with SHA512 calculation. This function initializes KDF X9.63 with SHA512 calculation.	
Particularities and Limitations	
workSpace is initialized by esl_initWorkSpaceHeader(..)	
Call context	
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces	

Table 4-358 esl_initKDFX963SHA512

4.2.2.7.6 esl_deriveKeyKDFX963SHA512

Prototype
`eslt_ErrorCode` **`esl_deriveKeyKDFX963SHA512`** `(VSECPRIM_P2VAR_PARA(eslt_WorkSpaceKDFX963SHA512) workSpace, const eslt_Length secretLength, VSECPRIM_P2CONST_PARA(eslt_Byte) secret, const eslt_Length infoLength, VSECPRIM_P2CONST_PARA(eslt_Byte) info, const eslt_Length keyLength, VSECPRIM_P2VAR_PARA(esllt_Byte) key)`

Parameter	
workSpace [in,out]	KDF X9.63 with SHA512 work space
secretLength [in]	length of secret in bytes
secret [in]	Pointer to the secret
infoLength [in]	length of salt in bytes (if zero, no salt is used)
info [in]	pointer to the optional salt (if NULL, no salt is used)
keyLength [in]	length of output key buffer in bytes
key [out]	pointer to the output key buffer
Return code	
eslt_ErrorCode	ESL_ERC_NO_ERROR Operation successful
	ESL_ERC_PARAMETER_INVALID Input parameter is NULL
	ESL_ERC_WS_TOO_SMALL Work space too small
	ESL_ERC_WS_STATE_INVALID Work space state invalid
	ESL_ERC_INPUT_INVALID Invalid input length
Functional Description	
Derive Key according KDF X9.63 with SHA512. This function derives a key according to KDF X9.63 with SHA512.	
Particularities and Limitations	
workSpace is initialized by esl_initKDFX963SHA512 (..) Length of the buffer provided by key need to have at least keyLength.	
Call context	
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces	

Table 4-359 esl_deriveKeyKDFX963SHA512

4.2.2.7.7 esl_initKDF2HMACSHA1

Prototype	
`eslt_ErrorCode` **`esl_initKDF2HMACSHA1`** `(VSECPRIM_P2VAR_PARA(eslt_WorkSpaceKDF2HMACSHA1) workSpace, const eslt_Length iterationCount)`	
Parameter	
iterationCount [in]	number of (SHA-1) iterations
workSpace [in,out]	(in) algorithm context buffer, initialized by esl_initWorkSpaceHeader(..) function (out) initialized algorithm context structure
Return code	
eslt_ErrorCode	ESL_ERC_PARAMETER_INVALID input parameter is NULL
	ESL_ERC_WS_TOO_SMALL work space too small

	ESL_ERC_KDF_ITERATION_COUNT_OUT_OF_RANGE
	ESL_ERC_NO_ERROR else
Functional Description	
Initialize the key derivation.	
Particularities and Limitations	
workSpace is initialized by esl_initWorkSpaceHeader(..)	
Call context	
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces	

Table 4-360 esl_initKDF2HMACSHA1

4.2.2.7.8 esl_deriveKeyKDF2HMACSHA1

Prototype	
`eslt_ErrorCode` **`esl_deriveKeyKDF2HMACSHA1`** `(VSECPRIM_P2VAR_PARA(eslt_WorkSpaceKDF2HMACSHA1) workSpace, const eslt_Length secretLength, VSECPRIM_P2CONST_PARA(eslt_Byte) secret, const eslt_Length infoLength, VSECPRIM_P2CONST_PARA(eslt_Byte) info, const eslt_Length keyLength, VSECPRIM_P2VAR_PARA(esltt_Byte) key)`	
Parameter	
workSpace [in,out]	algorithm context buffer, initialized by esl_initWorkSpaceHeader(..) function
secretLength [in]	length of secret in bytes
secret [in]	Pointer to the secret
infoLength [in]	length of the shared info (bytes)
info [in]	pointer to the shared info
keyLength [in]	length of output key buffer in bytes
key [out]	pointer to the output key buffer
Return code	
eslt_ErrorCode	ESL_ERC_PARAMETER_INVALID input parameter is NULL
	ESL_ERC_WS_TOO_SMALL work space too small
	ESL_ERC_WS_STATE_INVALID Invalid state
	ESL_ERC_NO_ERROR else
Functional Description	
Derive Key according KDF2 with SHA1.	
Particularities and Limitations	
workSpace is initialized by esl_initKDF2HMACSHA1(..), key needs to have at least the length of keyLength.	
Call context	

> TASK > This function is Synchronous > This function is Reentrant, for different workspaces

Table 4-361 esl_deriveKeyKDF2HMACSHA1

4.2.2.7.9 esl_initKDF2HMACSHA256

Prototype	
`eslt_ErrorCode` **`esl_initKDF2HMACSHA256`** `(VSECPRIM_P2VAR_PARA(eslt_WorkSpaceKDF2HMACSHA256) workSpace, const eslt_Length iterationCount)`	
Parameter	
iterationCount [in]	number of (SHA-256) iterations
workSpace [in,out]	(in) algorithm context buffer, initialized by esl_initWorkSpaceHeader(..) function (out) initialized algorithm context structure
Return code	
eslt_ErrorCode	ESL_ERC_PARAMETER_INVALID input parameter is NULL
	ESL_ERC_WS_TOO_SMALL work space too small
	ESL_ERC_KDF_ITERATION_COUNT_OUT_OF_RANGE
	ESL_ERC_NO_ERROR else
Functional Description	
Initialize the key derivation.	
Particularities and Limitations	
workSpace is initialized by esl_initWorkSpaceHeader(..)	
Call context	
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces	

Table 4-362 esl_initKDF2HMACSHA256

4.2.2.7.10 esl_deriveKeyKDF2HMACSHA256

Prototype	
`eslt_ErrorCode` **`esl_deriveKeyKDF2HMACSHA256`** `(VSECPRIM_P2VAR_PARA(eslt_WorkSpaceKDF2HMACSHA256) workSpace, const eslt_Length secretLength, VSECPRIM_P2CONST_PARA(eslt_Byte) secret, const eslt_Length infoLength, VSECPRIM_P2CONST_PARA(eslt_Byte) info, const eslt_Length keyLength, VSECPRIM_P2VAR_PARA(esltt_Byte) key)`	
Parameter	
workSpace [in,out]	algorithm context buffer, initialized by esl_initWorkSpaceHeader(..) function

secretLength [in]	length of secret in bytes
secret [in]	Pointer to the secret
infoLength [in]	length of the shared info (bytes)
info [in]	pointer to the shared info
keyLength [in]	length of output key buffer in bytes
key [out]	pointer to the output key buffer
Return code	
eslt_ErrorCode	ESL_ERC_PARAMETER_INVALID input parameter is NULL
	ESL_ERC_WS_TOO_SMALL work space too small
	ESL_ERC_WS_STATE_INVALID Invalid state
	ESL_ERC_NO_ERROR else
Functional Description	
Derive Key according KDF2 with SHA256.	
Particularities and Limitations	
workSpace is initialized by esl_initKDF2HMACSHA256(..), key needs to have at least the length of keyLength.	
Call context	
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces	

Table 4-363 esl_deriveKeyKDF2HMACSHA256

4.2.2.7.11 esl_initHKDF_Hash_OneStep

Prototype	
`eslt_ErrorCode` **`esl_initHKDF_Hash_OneStep`** `(VSECPRIM_P2VAR_PARA(eslt_WorkSpaceHKDFHASH) workSpace, eslt_HashAlgorithm hashID)`	
Parameter	
workSpace [in,out]	HKDF hash workspace
hashID	flag which indicates which hash function shall be used; currently only ESL_HA_SHA2_256 is accepted.
Return code	
eslt_ErrorCode	ESL_ERC_NO_ERROR Operation successful
	ESL_ERC_WS_TOO_SMALL Work space too small
	ESL_ERC_PARAMETER_INVALID Input parameter is NULL
	ESL_ERC_INPUT_INVALID hashID != ESL_HA_SHA2_256
Functional Description	
Initializes hash based HKDF one step method. This function initializes the one step HKDF, based on any hash function (Option 1). Currently, only SHA2-256 is supported.	

Particularities and Limitations
workSpace is initialized by esl_initWorkSpaceHeader(..)
Call context
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces

Table 4-364 esl_initHKDF_Hash_OneStep

4.2.2.7.12 esl_deriveKeyHKDF_Hash_OneStep

Prototype
`eslt_ErrorCode esl_deriveKeyHKDF_Hash_OneStep(VSECPRIM_P2VAR_PARA(eslt_WorkSpaceHKDFHASH) workSpace, const eslt_Length secretLength, VSECPRIM_P2CONST_PARA(eslt_Byte) secret, const eslt_Length infoLength, VSECPRIM_P2CONST_PARA(eslt_Byte) info, const eslt_Length keyLength, VSECPRIM_P2VAR_PARA(eslt_Byte) key)`

Parameter	
workSpace [in,out]	HKDF hash work space
secretLength [in]	length of secret in bytes
secret [in]	Pointer to the secret
infoLength [in]	length of additional input in bytes, can be zero
info [in]	pointer to the optional additional input, can be NULL
keyLength [in]	desired length of the key to be derived in bytes
key [out]	pointer to the output key buffer

Return code	
eslt_ErrorCode	ESL_ERC_NO_ERROR Operation successful
	ESL_ERC_PARAMETER_INVALID Input parameter is NULL
	ESL_ERC_INPUT_INVALID secretLength = 0 or keyLength = 0
	ESL_ERC_WS_TOO_SMALL Work space too small
	ESL_ERC_WS_STATE_INVALID Work space state invalid
	ESL_ERC_INVALID_LENGTH Invalid input length, i.e., keyLength > 4 294 967 264 bytes for the one step method, or keyLength > 255*SHA256 digest length (in bytes) for the two step method

Functional Description
Derive Key according to hash based HKDF one step method. This function derives a key according to HKDF one step method, based on any hash function (Option 1). Currently, only SHA2-256 is supported. The hash is determined by the hashID flag, a parameter for esl_initHKDF_Hash_OneStep, which is stored in the workspace.
Particularities and Limitations
workSpace is initialized by esl_initHKDF_Hash_OneStep(..) The buffer referenced by key needs to provide at least keyLength many bytes.
Call context

> TASK > This function is Synchronous > This function is Reentrant, for different workspaces

Table 4-365 esl_deriveKeyHKDF_Hash_OneStep

4.2.2.7.13 esl_initHKDF_HMAC_SHA256

Prototype	
`eslt_ErrorCode` **`esl_initHKDF_HMAC_SHA256`** `(VSECPRIM_P2VAR_PARA(eslt_WorkSpaceHKDFHMAC) workSpace, eslt_HKDFMode mode)`	
Parameter	
workSpace [in,out]	HKDF HMAC SHA256 workspace
Mode [in]	HKDF mode, indicating whether the one-step or the two-step variant shall be used
Return code	
eslt_ErrorCode	ESL_ERC_NO_ERROR Operation successful
	ESL_ERC_WS_TOO_SMALL Work space too small
	ESL_ERC_PARAMETER_INVALID Input parameter is NULL
	ESL_ERC_MODE_INVALID mode neither ESL_HKDF_ONE_STEP_VARIANT nor ESL_HKDF_TWO_STEP_VARIANT
Functional Description	
Initializes HKDF HMAC with SHA256 calculation. This function initializes one step (Option 2) or two step HKDF HMAC with SHA256 calculation.	
Particularities and Limitations	
workSpace is initialized by esl_initWorkSpaceHeader(..)	
Call context	
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces	

Table 4-366 esl_initHKDF_HMAC_SHA256

4.2.2.7.14 esl_deriveKeyHKDF_HMAC_SHA256

Prototype	
`eslt_ErrorCode` **`esl_deriveKeyHKDF_HMAC_SHA256`** `(VSECPRIM_P2VAR_PARA(eslt_WorkSpaceHKDFHMAC) workSpace, const eslt_Length secretLength, VSECPRIM_P2CONST_PARA(eslt_Byte) secret, const eslt_Length saltLength, VSECPRIM_P2CONST_PARA(eslt_Byte) salt, const eslt_Length infoLength, VSECPRIM_P2CONST_PARA(eslt_Byte) info, const eslt_Length keyLength, VSECPRIM_P2VAR_PARA(eslt_Byte) key)`	
Parameter	
workSpace [in,out]	HKDF HMAC with SHA256 work space

secretLength [in]	length of secret in bytes
secret [in]	Pointer to the secret
saltLength[in]	Length of salt in bytes (if zero, an all-zero vector is used instead)
salt	Pointer to the optional salt (if NULL, an all-zero vector is used instead)
infoLength [in]	length of additional input in bytes, can be zero
info [in]	pointer to the optional additional input, can be NULL
keyLength [in]	desired length of the key to be derived in bytes
key [out]	pointer to the output key buffer
Return code	
eslt_ErrorCode	ESL_ERC_NO_ERROR Operation successful
	ESL_ERC_PARAMETER_INVALID Input parameter is NULL
	ESL_ERC_INPUT_INVALID secretLength = 0 or keyLength = 0
	ESL_ERC_WS_TOO_SMALL Work space too small
	ESL_ERC_WS_STATE_INVALID Work space state invalid
	ESL_ERC_INVALID_LENGTH Invalid input length, i.e., keyLength > 4 294 967 264 bytes for the one step method, or keyLength > 255*SHA256 digest length (in bytes) for the two step method
Functional Description	
Derive Key according to HKDF HMAC with SHA256 This function derives a key according to HKDF HMAC with SHA256. Either the one-step method (Option 2) or the two-step method (extraction and expansion step) is performed. The method is determined by the mode flag, a parameter for esl_initHKDF_HMAC_SHA256, which is stored in the workspace.	
Particularities and Limitations	
workSpace is initialized by esl_initHKDF_HMAC_SHA256(..) The buffer referenced by key needs to provide at least keyLength many bytes.	
Call context	
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces	

Table 4-367 esl_deriveKeyHKDF_HMAC_SHA256

4.2.2.7.15 esl_initHKDF_HMAC_SHA384

Prototype	
`eslt_ErrorCode` **`esl_initHKDF_HMAC_SHA384`** `(VSECPRIM_P2VAR_PARA(eslt_WorkSpaceHKDFHMAC) workSpace, eslt_HKDFMode mode)`	
Parameter	
workSpace [in,out]	HKDF HMAC SHA384 workspace
Mode [in]	HKDF mode, indicating whether the one-step or the two-step variant shall be used

Return code	
eslt_ErrorCode	ESL_ERC_NO_ERROR Operation successful
	ESL_ERC_WS_TOO_SMALL Work space too small
	ESL_ERC_PARAMETER_INVALID Input parameter is NULL
	ESL_ERC_MODE_INVALID mode neither ESL_HKDF_ONE_STEP_VARIANT nor ESL_HKDF_TWO_STEP_VARIANT
Functional Description	
Initializes HKDF HMAC with SHA384 calculation. This function initializes one step (Option 2) or two step HKDF HMAC with SHA384 calculation.	
Particularities and Limitations	
workSpace is initialized by esl_initWorkSpaceHeader(..)	
Call context	
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces	

Table 4-368 esl_initHDKF_HMAC_SHA384

4.2.2.7.16 esl_deriveKeyHKDF_HMAC_SHA384

Prototype
`eslt_ErrorCode` **`esl_deriveKeyHKDF_HMAC_SHA384`** `(VSECPRIM_P2VAR_PARA(eslt_WorkSpaceHKDFHMAC) workSpace, const eslt_Length secretLength, VSECPRIM_P2CONST_PARA(eslt_Byte) secret, const eslt_Length saltLength, VSECPRIM_P2CONST_PARA(eslt_Byte) salt, const eslt_Length infoLength, VSECPRIM_P2CONST_PARA(eslt_Byte) info, const eslt_Length keyLength, VSECPRIM_P2VAR_PARA(eslt_Byte) key)`

Parameter	
workSpace [in,out]	HKDF HMAC with SHA384 work space
secretLength [in]	length of secret in bytes
secret [in]	Pointer to the secret
saltLength[in]	Length of salt in bytes (if zero, an all-zero vector is used instead)
salt	Pointer to the optional salt (if NULL, an all-zero vector is used instead)
infoLength [in]	length of additional input in bytes, can be zero
info [in]	pointer to the optional additional input, can be NULL
keyLength [in]	desired length of the key to be derived in bytes
key [out]	pointer to the output key buffer
Return code	
eslt_ErrorCode	ESL_ERC_NO_ERROR Operation successful
	ESL_ERC_PARAMETER_INVALID Input parameter is NULL
	ESL_ERC_INPUT_INVALID secretLength = 0 or keyLength = 0
	ESL_ERC_WS_TOO_SMALL Work space too small
	ESL_ERC_WS_STATE_INVALID Work space state invalid

	ESL_ERC_INVALID_LENGTH Invalid input length, i.e., keyLength > 4 294 967 248 bytes for the one step method, or keyLength > 255*SHA384 digest length (in bytes) for the two step method
Functional Description	
Derive Key according to HKDF HMAC with SHA384 This function derives a key according to HKDF HMAC with SHA384. Either the one-step method (Option 2) or the two-step method (extraction and expansion step) is performed. The method is determined by the mode flag, a parameter for esl_initHKDF_HMAC_SHA384, which is stored in the workspace.	
Particularities and Limitations	
workSpace is initialized by esl_initHKDF_HMAC_SHA384(..) The buffer referenced by key needs to provide at least keyLength many bytes.	
Call context	
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces	

Table 4-369 esl_deriveKeyHKDF_HMAC_SHA384

4.2.2.8 Authenticated Encryption with Associated Data (AEAD)

4.2.2.8.1 esl_initEncryptGCM

Prototype	
`eslt_ErrorCode esl_initEncryptGCM (VSECPRIM_P2VAR_PARA(eslt_WorkSpaceGCM) workSpace, VSECPRIM_P2CONST_PARA(eslt_Byte) key, const eslt_Length keyLength, VSECPRIM_P2CONST_PARA(eslt_Byte) iv, const eslt_Length ivLength)`	
Parameter	
workSpace [in,out]	(in) algorithm context buffer, initialized by esl_initWorkSpaceHeader(..) function (out) initialized algorithm context structure
key [in]	the symmetric AES master key
keyLength [in]	the length of the AES key in bytes, must be 16, 24 or 32
iv [in]	the initialization vector
ivLength [in]	the length of the initialization vector in bytes
Return code	
eslt_ErrorCode	ESL_ERC_PARAMETER_INVALID input parameter is NULL
	ESL_ERC_WS_TOO_SMALL work space too small
	ESL_ERC_GCM_INVALID_KEY_LENGTH no valid AES keyLength
	ESL_ERC_NO_ERROR else
Functional Description	
Initializes the GCM encryption. This function initializes the GCM encryption.	

Particularities and Limitations
workSpace is initialized by esl_initWorkSpaceHeader(..)
Call context
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces

Table 4-370 esl_initEncryptGCM

4.2.2.8.2 esl_updateAuthDataEncryptGCM

Prototype	
`eslt_ErrorCode esl_updateAuthDataEncryptGCM (VSECPRIM_P2VAR_PARA(eslt_WorkSpaceGCM) workSpace, VSECPRIM_P2CONST_PARA(eslt_Byte) input, const eslt_Length inputLength)`	
Parameter	
workSpace [in,out]	(in) algorithm context buffer, initialized by esl_initWorkSpaceHeader(..) function (out) updated algorithm context structure
input [in]	additional authenticated data
inputLength [in]	the length of the input
Return code	
eslt_ErrorCode	ESL_ERC_PARAMETER_INVALID input parameter is NULL
	ESL_ERC_WS_STATE_INVALID work space state invalid
	ESL_ERC_WS_TOO_SMALL work space too small
	ESL_ERC_GCM_STATE_INVALID GCM state invalid
	ESL_ERC_GCM_TOTAL_LENGTH_OVERFLOW the input exceeded the length limit
	ESL_ERC_NO_ERROR else
Functional Description	
Hashes additional authenticated data using AES-GCM. This function hashes additional authenticated data using AES-GCM.	
Particularities and Limitations	
workSpace is initialized by esl_initWorkSpaceHeader(..)	
Call context	
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces	

Table 4-371 esl_updateAuthDataEncryptGCM

4.2.2.8.3 esl_updatePlaintextEncryptGCM

Prototype	
`eslt_ErrorCode` **`esl_updatePlaintextEncryptGCM`** `(VSECPRIM_P2VAR_PARA(eslt_WorkSpaceGCM) workSpace, VSECPRIM_P2CONST_PARA(eslt_Byte) input, const eslt_Length inputLength, VSECPRIM_P2VAR_PARA(eslt_Byte) output, VSECPRIM_P2VAR_PARA(eslt_Size32) bytes_out)`	
Parameter	
workSpace [in,out]	(in) algorithm context buffer, initialized by esl_initWorkSpaceHeader(..) function (out) updated algorithm context structure
input [in]	plaintext message
inputLength [in]	the length of the input
output [out]	a pointer to the output buffer for the encrypted message
bytes_out [in,out]	a pointer to an output counter variable, this variable is incremented by the number of bytes that were handled in this iteration of esl_updatePlaintextEncryptGCM
Return code	
eslt_ErrorCode	ESL_ERC_PARAMETER_INVALID input parameter is NULL
	ESL_ERC_WS_STATE_INVALID work space state invalid
	ESL_ERC_WS_TOO_SMALL work space too small
	ESL_ERC_GCM_TOTAL_LENGTH_OVERFLOW the input exceeded the length limit
	ESL_ERC_NO_ERROR else
Functional Description	
Encrypts the plaintext message using AES-GCM. This function encrypts the plaintext message using AES-GCM.	
Particularities and Limitations	
workSpace is initialized by esl_initWorkSpaceHeader(..), output references a valid memory location whose size is a multiple of the GCM block size (16 bytes) and greater or equal than inputLength, bytes_out is a valid pointer input and output can point to the same address, i.e., inplace encryption is possible	
Call context	
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces	

Table 4-372 esl_updatePlaintextEncryptGCM

4.2.2.8.4 esl_finalizeEncryptGCM

Prototype
`eslt_ErrorCode` **`esl_finalizeEncryptGCM`** `(VSECPRIM_P2VAR_PARA(eslt_WorkSpaceGCM) workSpace, VSECPRIM_P2VAR_PARA(eslt_Byte) output,`

`VSECPRIM_P2VAR_PARA(eslt_Size32) bytes_out, VSECPRIM_P2VAR_PARA(eslt_Byte) tag)`	
Parameter	
workSpace [in,out]	(in) algorithm context buffer, initialized by esl_initWorkSpaceHeader(..) function (out) updated algorithm context structure
output [out]	a pointer to the output buffer for a possible last block, if there is still data in the internal buffer, the remaining bytes are written here
bytes_out [out]	incremented by number of remaining bytes
tag [out]	a pointer to a buffer for the authentication tag
Return code	
eslt_ErrorCode	ESL_ERC_PARAMETER_INVALID input parameter is NULL
	ESL_ERC_WS_STATE_INVALID work space state invalid
	ESL_ERC_WS_TOO_SMALL work space too small
	ESL_ERC_NO_ERROR else
Functional Description	
Finalizes the GCM encryption. This function finalizes the GCM encryption.	
Particularities and Limitations	
workSpace is initialized by esl_initWorkSpaceHeader(..), output references a valid memory location whose size is greater or equal than the GCM block size (16 bytes), bytes_out is a valid pointer	
Call context	
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces	

Table 4-373 esl_finalizeEncryptGCM

4.2.2.8.5 esl_initDecryptGCM

Prototype	
`eslt_ErrorCode` **`esl_initDecryptGCM`** `(VSECPRIM_P2VAR_PARA(eslt_WorkSpaceGCM) workSpace, VSECPRIM_P2CONST_PARA(eslt_Byte) key, const eslt_Length keyLength, VSECPRIM_P2CONST_PARA(eslt_Byte) iv, const eslt_Length ivLength)`	
Parameter	
workSpace [in,out]	(in) algorithm context buffer, initialized by esl_initWorkSpaceHeader(..) function (out) initialized algorithm context structure
key [in]	the symmetric AES master key
keyLength [in]	the length of the AES key in bytes, must be 16, 24 or 32
iv [in]	the initialization vector
ivLength [in]	the length of the initialization vector in bytes

Return code	
eslt_ErrorCode	ESL_ERC_PARAMETER_INVALID input parameter is NULL
	ESL_ERC_WS_TOO_SMALL work space too small
	ESL_ERC_GCM_INVALID_KEY_LENGTH no valid AES keyLength
	ESL_ERC_NO_ERROR else
Functional Description	
Initializes the GCM decryption. This function initializes the GCM decryption.	
Particularities and Limitations	
workSpace is initialized by esl_initWorkSpaceHeader(..)	
Call context	
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces	

Table 4-374 esl_initDecryptGCM

4.2.2.8.6 esl_updateAuthDataDecryptGCM

Prototype	

`eslt_ErrorCode` **`esl_updateAuthDataDecryptGCM`**
`(VSECPRIM_P2VAR_PARA(eslt_WorkSpaceGCM) workSpace,`
`VSECPRIM_P2CONST_PARA(eslt_Byte) input, const eslt_Length inputLength)`	

Parameter	
workSpace [in,out]	(in) algorithm context buffer, initialized by esl_initWorkSpaceHeader(..) function (out) updated algorithm context structure
input [in]	additional authenticated data
inputLength [in]	the length of the input
Return code	
eslt_ErrorCode	ESL_ERC_PARAMETER_INVALID input parameter is NULL
	ESL_ERC_WS_STATE_INVALID work space state invalid
	ESL_ERC_WS_TOO_SMALL work space too small
	ESL_ERC_GCM_STATE_INVALID GCM state invalid
	ESL_ERC_GCM_TOTAL_LENGTH_OVERFLOW the input exceeded the length limit
	ESL_ERC_NO_ERROR else
Functional Description	
Decrypts the ciphertext using AES-GCM. This function decrypts the ciphertext using AES-GCM.	

Particularities and Limitations
workSpace is initialized by esl_initWorkSpaceHeader(..)
Call context
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces

Table 4-375 esl_updateAuthDataDecryptGCM

4.2.2.8.7 esl_updateCiphertextDecryptGCM

Prototype
`eslt_ErrorCode esl_updateCiphertextDecryptGCM (VSECPRIM_P2VAR_PARA(eslt_WorkSpaceGCM) workSpace, VSECPRIM_P2CONST_PARA(eslt_Byte) input, const eslt_Length inputLength, VSECPRIM_P2VAR_PARA(eslt_Byte) output, VSECPRIM_P2VAR_PARA(eslt_Size32) bytes_out)`

Parameter	
workSpace [in,out]	(in) algorithm context buffer, initialized by esl_initWorkSpaceHeader(..) function (out) updated algorithm context structure
input [in]	ciphertext
inputLength [in]	the length of the input
output [out]	a pointer to the output buffer for the decrypted ciphertext
bytes_out [in,out]	a pointer to an output counter variable, this variable is incremented by the number of bytes that were handled in this iteration of esl_updateCiphertextDecryptGCM
Return code	
eslt_ErrorCode	ESL_ERC_PARAMETER_INVALID input parameter is NULL
	ESL_ERC_WS_STATE_INVALID work space state invalid
	ESL_ERC_WS_TOO_SMALL work space too small
	ESL_ERC_GCM_TOTAL_LENGTH_OVERFLOW the input exceeded the length limit
	ESL_ERC_NO_ERROR else

Functional Description
Decrypts the ciphertext using AES-GCM. This function decrypts the ciphertext using AES-GCM.
Particularities and Limitations
workSpace is initialized by esl_initWorkSpaceHeader(..), output references a valid memory location whose size is a multiple of the GCM block size (16 bytes) and greater or equal than inputLength, bytes_out is a valid pointer input and output can point to the same address, i.e., inplace decryption is possible
Call context

> TASK > This function is Synchronous > This function is Reentrant, for different workspaces

Table 4-376 esl_updateCiphertextDecryptGCM

4.2.2.8.8 esl_finalizeDecryptGCM

Prototype	
`eslt_ErrorCode esl_finalizeDecryptGCM (VSECPRIM_P2VAR_PARA(eslt_WorkSpaceGCM) workSpace, VSECPRIM_P2VAR_PARA(eslt_Byte) output, VSECPRIM_P2VAR_PARA(eslt_Size32) bytes_out, VSECPRIM_P2CONST_PARA(eslt_Byte) tag)`	
Parameter	
workSpace [in,out]	(in) algorithm context buffer, initialized by esl_initWorkSpaceHeader(..) function (out) updated algorithm context structure
output [out]	a pointer to the output buffer for a possible last block, if there is still data in the internal buffer, the remaining bytes are written here
bytes_out [out]	incremented by number of remaining bytes
tag [in]	a pointer to a buffer for the authentication tag
Return code	
eslt_ErrorCode	ESL_ERC_PARAMETER_INVALID input parameter is NULL
	ESL_ERC_WS_STATE_INVALID work space state invalid
	ESL_ERC_WS_TOO_SMALL work space too small
	ESL_ERC_GCM_TAG_VERIFICATION_FAILED the provided tag does not
	match the computed one
	ESL_ERC_NO_ERROR else
Functional Description	
Finalizes the GCM operation. This function finalizes the GCM operation.	
Particularities and Limitations	
workSpace is initialized by esl_initWorkSpaceHeader(..), output references a valid memory location whose size is greater or equal than the GCM block size (16 bytes), bytes_out is a valid pointer	
Call context	
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces	

Table 4-377 esl_finalizeDecryptGCM

4.2.2.8.9 esl_initAESCCM

Prototype	
`eslt_ErrorCode esl_initAESCCM (VSECPRIM_P2VAR_PARA(eslt_WorkSpaceAESCCM) workSpace, VSECPRIM_P2CONST_PARA(eslt_Byte) key, const eslt_Length keyLength, VSECPRIM_P2CONST_PARA(eslt_Byte) nonce, const eslt_Length nonceLength, const eslt_Length aadLength, const eslt_Length msgLength, const eslt_Byte authenticationFieldSize, const eslt_Byte lengthFieldSize)`	
Parameter	
workSpace [in,out]	(in) algorithm context buffer, initialized by esl_initWorkSpaceHeader(..) function (out) initialized algorithm context structure
key [in]	the symmetric AES master key
keyLength [in]	the length of the AES key in bytes, must be 16
Nonce [in]	The nonce
nonceLength [in]	the length of the nonce in bytes
aadLength [in]	total length of the additional authentication data in bytes
msgLength [in]	total length of the message to be encrypted in bytes
authenticationFieldSize	size of the desired MAC in bytes, can be 0
lengthFieldSize	value L satisfying msgLength < 2^(8*L), see note
Return code	
eslt_ErrorCode	ESL_ERC_PARAMETER_INVALID input parameter is NULL
	ESL_ERC_WS_TOO_SMALL work space too small
	ESL_ERC_INVALID_LENGTH length of key or iv invalid or value of authenticationFieldSize
	ESL_ERC_NO_ERROR else
Functional Description	
Initializes the CCM encryption. This function initializes the CCM encryption or decryption.	
Particularities and Limitations	
workSpace is initialized by esl_initWorkSpaceHeader(..) The value L of lengthFieldSize indicates that the message to be encrypted is < 2^(8*L) bytes long. Valid values for L are 2, 3, 4, 5, 6, 7 or 8 Valid values for authenticationFieldSize are 4, 6, 8, 10, 12, 14, 16. The length of the message is limited to uint32max bytes. The nonce must be 15 - lengthFieldSize bytes long. nonceLength is expected to be 7, 8, 9, 10, 11, 12 or 13.	
Call context	
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces	

Table 4-378 esl_initAESCCM

4.2.2.8.10 esl_updateAESCCMEncrypt

Prototype
`eslt_ErrorCode esl_updateAESCCMEncrypt (VSECPRIM_P2VAR_PARA(eslt_WorkSpaceAESCCM) workSpace, VSECPRIM_P2CONST_PARA(eslt_Byte) aad, const eslt_Length aadSize, VSECPRIM_P2CONST_PARA(eslt_Byte) msg, const eslt_Length msgSize, VSECPRIM_P2VAR_PARA(eslt_Byte)out, VSECPRIM_P2VAR_PARA(eslt_Length) outSize)`

Parameter	
workSpace [in,out]	(in) algorithm context buffer, initialized by esl_initWorkSpaceHeader(..) function (out) updated algorithm context structure
aad [in]	Additional authentication data
aadLength [in]	Length of the aad buffer in bytes
Msg [in]	plaintext message to be encrypted
msgSize [in]	length of msg buffer in bytes
out [in,out]	pointer to buffer to store the encrypted message
outSize [in,out]	(in) pointer to buffer storing the number of bytes provided by out (out) pointer to buffer storing the number of bytes written to out

Return code	
eslt_ErrorCode	ESL_ERC_PARAMETER_INVALID input parameter is NULL or length value invalid
	ESL_ERC_WS_STATE_INVALID work space state invalid
	ESL_ERC_WS_TOO_SMALL work space too small
	ESL_ERC_BUFFER_TOO_SMALL memory provided by out not sufficient to store encrypted message
	ESL_ERC_CCM_TOTAL_LENGTH_OVERFLOW input length values too high
	ESL_ERC_STATE_INVALID esl_updateAESCCMEncrypt called more than once
	ESL_ERC_NO_ERROR else

Functional Description
Starts computing the CBC-MAC and encrypting the plaintext message. This function uses the additional data aad and the message msg to compute the CBC-MAC and the encrypted message. All but the last block of msg are processed. This function does not allow for a multi-update. The parameters aadSize and msgSize have to match the parameters aadLength and msgLength in the call to esl_initAESCCM().

Particularities and Limitations
workSpace is initialized by esl_initWorkSpaceHeader(..) The buffer provided by out has to provide at least as many bytes as given in outSizeLowWord and outSizeHighWord. aad can have any length between 0 and (2^32 - 16) bytes, including these values. msg can have any length between 0 and (2^32 - 1) bytes, including these values, as long as it is consistent with workSpace->wsAESCCM.lengthFieldSize, i.e., the number of bytes in msg has to be less than (2^8*(workSpace->wsAESCCM.lengthFieldSize)).

The parameters aadSize and msgSize have to match the parameters aadLength and msgLength in the call to esl_initAESCCM(). msg and out can point to the same address, i.e., in-place encryption is possible
Call context
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces

Table 4-379 esl_updateAESCCMEncrypt

4.2.2.8.11 esl_finalizeAESCCMEncrypt

Prototype	
`eslt_ErrorCode esl_finalizeAESCCMEncrypt (VSECPRIM_P2VAR_PARA(eslt_WorkSpaceAESCCM) workSpace, VSECPRIM_P2VAR_PARA(eslt_Byte) out, VSECPRIM_P2VAR_PARA(eslt_Length) outLength, VSECPRIM_P2VAR_PARA(eslt_Byte) tag, VSECPRIM_P2VAR_PARA(eslt_Length) tagLength)`	
Parameter	
workSpace [in,out]	(in) algorithm context buffer, initialized by esl_initWorkSpaceHeader(..) function (out) updated algorithm context structure
out [in,out]	(in) pointer to a buffer for the remaining bytes of the message (out) pointer to buffer storing the remaining bytes of the encrypted message
outLength [in,out]	(in) pointer to buffer storing the number of bytes provided by out (out) pointer to buffer storing the number of bytes written to out
Tag [in,out]	(in) pointer to a buffer for the authentication tag (out) pointer to buffer storing the authentication tag
tagLength [in,out]	(in) pointer to buffer storing the number of bytes provided by tag (out) pointer to buffer storing the number of bytes written to tag
Return code	
eslt_ErrorCode	ESL_ERC_PARAMETER_INVALID input parameter is NULL
	ESL_ERC_WS_STATE_INVALID work space state invalid
	ESL_ERC_WS_TOO_SMALL work space too small
	ESL_ERC_BUFFER_TOO_SMALL memory provided by out or tag not sufficient to store the remaining bytes of the encrypted message or the authentication tag
	ESL_ERC_INVALID_LENGTH value in workSpace->remainingLen invalid, e.g. esl_updateAESCCMEncrypt not called before
	ESL_ERC_NO_ERROR else
Functional Description	
Finalizes the CCM authentication and encryption. This function finalizes the AES CCM authentication and encryption by encrypting the last block of the message and by computing and encrypting the CBC-MAC.	

Particularities and Limitations
workSpace is initialized by esl_initWorkSpaceHeader(..). The buffer referenced by out has to provide at least outLength many bytes. The buffer referenced by tag has to provide at least tagLength many bytes.
Call context
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces

Table 4-380 esl_finalizeAESCCMEncrypt

4.2.2.8.12 esl_updateAESCCMDecrypt

Prototype
`eslt_ErrorCode esl_updateAESCCMDecrypt (VSECPRIM_P2VAR_PARA(eslt_WorkSpaceAESCCM) workSpace, VSECPRIM_P2CONST_PARA(eslt_Byte) aad, const eslt_Length aadSize, VSECPRIM_P2CONST_PARA(eslt_Byte) encMsg, const eslt_Length encMsgSize, VSECPRIM_P2VAR_PARA(eslt_Byte)out, VSECPRIM_P2VAR_PARA(eslt_Length) outSize)`

Parameter	
workSpace [in,out]	(in) algorithm context buffer, initialized by esl_initWorkSpaceHeader(..) function (out) updated algorithm context structure
aad [in]	Additional authentication data
aadLength [in]	Length of the aad buffer in bytes
encMsg [in]	plaintext message to be decrypted
encMsgSize [in]	length of encMsg buffer in bytes
out [in,out]	pointer to buffer to store the decrypted message
outSize [in,out]	(in) pointer to buffer storing the number of bytes provided by out (out) pointer to buffer storing the number of bytes written to out

Return code	
eslt_ErrorCode	ESL_ERC_PARAMETER_INVALID input parameter is NULL or length value invalid
	ESL_ERC_WS_STATE_INVALID work space state invalid
	ESL_ERC_WS_TOO_SMALL work space too small
	ESL_ERC_BUFFER_TOO_SMALL memory provided by out not sufficient to store encrypted message
	ESL_ERC_CCM_TOTAL_LENGTH_OVERFLOW input length values too high
	ESL_ERC_STATE_INVALID esl_updateAESCCMDecrypt called more than once
	ESL_ERC_NO_ERROR else

Functional Description
Starts re-computing the CBC-MAC and decrypting the encrypted message.

This function uses the additional data aad and the encrypted message encMsg to compute the CBC-MAC and the decrypted message. All but the last block of msg are processed. This function does not allow for a multi-update! The parameters aadSize and encMsgSize have to match the parameters aadLength and msgLength in the call to esl_initAESCCM()!
Particularities and Limitations
workSpace is initialized by esl_initWorkSpaceHeader(..) The buffer provided by out has to provide at least as many bytes as given in outsize. aad can have any length between 0 and (2^32 - 16) bytes, including these values. encMsg can have any length between 0 and (2^32 - 1) bytes, including these values, as long as it is consistent with workSpace->wsAESCCM.lengthFieldSize, i.e., the number of bytes in encMsg has to be less than (2^8*(workSpace->wsAESCCM.lengthFieldSize)). The parameters aadSize and encMsgSize have to match the parameters aadLength and msgLength in the call to esl_initAESCCM(). encMsg and out can point to the same address, i.e., in-place decryption is possible
Call context
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces

Table 4-381 esl_updateAESCCMDecrypt

4.2.2.8.13 esl_finalizeAESCCMDecrypt

Prototype	
`eslt_ErrorCode` **`esl_finalizeAESCCMDecrypt`** `(VSECPRIM_P2VAR_PARA(eslt_WorkSpaceAESCCM) workSpace, VSECPRIM_P2VAR_PARA(eslt_Byte) out, VSECPRIM_P2VAR_PARA(eslt_Length) outLength, VSECPRIM_P2VAR_PARA(eslt_Byte) tag, VSECPRIM_P2VAR_PARA(eslt_Length) tagLength)`	
Parameter	
workSpace [in,out]	(in) algorithm context buffer, initialized by esl_initWorkSpaceHeader(..) function (out) updated algorithm context structure
out [in,out]	(in) pointer to a buffer for the remaining bytes of the message (out) pointer to buffer storing the remaining bytes of the encrypted message
outLength [in,out]	(in) pointer to buffer storing the number of bytes provided by out (out) pointer to buffer storing the number of bytes written to out
Tag [in]	Tag to be verified
Return code	
eslt_ErrorCode	ESL_ERC_PARAMETER_INVALID input parameter is NULL
	ESL_ERC_WS_STATE_INVALID work space state invalid
	ESL_ERC_WS_TOO_SMALL work space too small
	ESL_ERC_BUFFER_TOO_SMALL memory provided by out not sufficient to store the remaining bytes of the encrypted message

	ESL_ERC_INVALID_LENGTH value in workSpace->remainingLen invalid, e.g. esl_updateAESCCMDecrypt not called before
	ESL_ERC_INCORRECT_TAG recomputed tag does not match the given tag
	ESL_ERC_NO_ERROR else
Functional Description	
Finalizes the CCM authentication and decryption. This function finalizes the AES CCM authentication and decryption by decrypting the last block of the message and by computing and encrypting the CBC-MAC.The recomputed authentication tag is compared to the tag given in the parameter ‘tag’.	
Particularities and Limitations	
workSpace is initialized by esl_initWorkSpaceHeader(..). The buffer referenced by out has to provide at least outLength many bytes.	
Call context	
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces	

Table 4-382 esl_finalizeAESCCMDecrypt

4.2.2.8.14 esl_initEncryptChaChaPoly

Prototype	
`eslt_ErrorCode` **`esl_initEncryptChaChaPoly`** `(VSECPRIM_P2VAR_PARA(eslt_WorkSpaceChaChaPoly) workSpace, VSECPRIM_P2CONST_PARA(eslt_Byte) key, VSECPRIM_P2CONST_PARA(eslt_Byte) nonce)`	
Parameter	
key [in]	pointer to key of length ESL_SIZEOF_ChaCha20_KEY
nonce [in]	pointer to nonce of length ESL_SIZEOF_ChaCha20_NONCE
workSpace [in,out]	(in) algorithm context buffer, initialized by esl_initWorkSpaceHeader () function (out) initialized algorithm context structure
Return code	
eslt_ErrorCode	ESL_ERC_PARAMETER_INVALID input parameter is NULL
	ESL_ERC_WS_TOO_SMALL work space too small
	ESL_ERC_NO_ERROR else
Functional Description	
This function initializes the AEAD_ChaCha20_Poly1305 (RFC 7539) stream encryption and authentication TAG creation.	
Particularities and Limitations	
workSpace is initialized by esl_initWorkSpaceHeader ()	
Call context	
> TASK > This function is Synchronous	

> This function is Reentrant, for different workspaces

Table 4-383 esl_initEncryptChaChaPoly

4.2.2.8.15 esl_updateDataChaChaPoly

Prototype	
`eslt_ErrorCode esl_updateDataChaChaPoly (VSECPRIM_P2VAR_PARA(eslt_WorkSpaceChaChaPoly) workSpace, VSECPRIM_P2CONST_PARA(eslt_Byte) in, const eslt_Length inSize, VSECPRIM_P2VAR_PARA(eslt_Byte) out, VSECPRIM_P2VAR_PARA(eslt_Length) outSize)`	
Parameter	
in [in]	pointer to input data
outSize [out]	length of encrypted data
out [in,out]	pointer to output buffer
inSize [in]	length of input data (bytes)
workSpace [in,out]	(in) algorithm context buffer, initialized by esl_initXXcryptChaChaPoly () function (out) internal state updated
outSize [in,out]	pointer to size of output buffer (bytes)
Return code	
eslt_ErrorCode	ESL_ERC_PARAMETER_INVALID input parameter is NULL
	ESL_ERC_BUFFER_TOO_SMALL output buffer too small
	ESL_ERC_WS_STATE_INVALID work space state invalid
	ESL_ERC_TOTAL_LENGTH_OVERFLOW length of data > (2^32 -1) *64
	ESL_ERC_NO_ERROR else
Functional Description	
This function passes data to be encrypted / decrypted to the AEAD_ChaCha20_Poly1305. When the workSpace has been initialized for encryption, "in" is the message and "out" is the cipher; in case of decryption, it's the other way 'round.	
Particularities and Limitations	
workSpace is initialized by esl_initWorkSpaceHeader () in and out can reference the same address, i.e., in-place encryption and decryption is possible	
Call context	
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces	

Table 4-384 esl_updateDataChaChaPoly

4.2.2.8.16 esl_updateAADChaChaPoly

Prototype	
`eslt_ErrorCode` **`esl_updateAADChaChaPoly`** `(VSECPRIM_P2VAR_PARA(eslt_WorkSpaceChaChaPoly) workSpace, VSECPRIM_P2CONST_PARA(eslt_Byte) aad, const eslt_Length aadSize)`	
Parameter	
aad [in]	pointer to AAD
workSpace [in,out]	(in) algorithm context buffer, initialized by esl_initXXcryptChaChaPoly () function (out) internal state updated
aadSize [in]	length of AAD (bytes)
Return code	
eslt_ErrorCode	ESL_ERC_PARAMETER_INVALID input parameter is NULL
	ESL_ERC_WS_STATE_INVALID work space state invalid
	ESL_ERC_TOTAL_LENGTH_OVERFLOW length of AAD > 2^64 -1
	ESL_ERC_NO_ERROR else
Functional Description	
This function passes additional authenticated data (AAD) to the AEAD_ChaCha20_Poly1305.	
Particularities and Limitations	
workSpace is initialized by esl_initWorkSpaceHeader ()	
Call context	
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces	

Table 4-385 esl_updateAADChaChaPoly

4.2.2.8.17 esl_finalizeChaChaPoly

Prototype	
`eslt_ErrorCode` **`esl_finalizeChaChaPoly`** `(VSECPRIM_P2VAR_PARA(eslt_WorkSpaceChaChaPoly) workSpace, VSECPRIM_P2VAR_PARA(eslt_Byte) tag)`	
Parameter	
workSpace [in,out]	(in) algorithm context buffer, initialized by esl_initEncryptChaChaPoly () function (out) internal state updated
tag [in,out]	pointer to TAG buffer of size ESL_SIZEOF_Poly1305_TAG
Return code	
eslt_ErrorCode	ESL_ERC_PARAMETER_INVALID input parameter is NULL
	ESL_ERC_WS_STATE_INVALID work space state invalid
	ESL_ERC_NO_ERROR else

Functional Description
This function calculates the authentication TAG of the AEAD_ChaCha20_Poly1305.
Particularities and Limitations
workSpace is initialized by esl_initWorkSpaceHeader ()
Call context
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces

Table 4-386 esl_finalizeChaChaPoly

4.2.2.8.18 esl_initDecryptChaChaPoly

Prototype	
`eslt_ErrorCode` **`esl_initDecryptChaChaPoly`** `(VSECPRIM_P2VAR_PARA(eslt_WorkSpaceChaChaPoly) workSpace, VSECPRIM_P2CONST_PARA(eslt_Byte) key, VSECPRIM_P2CONST_PARA(eslt_Byte) nonce)`	
Parameter	
key [in]	pointer to key of length ESL_SIZEOF_ChaCha20_KEY
nonce [in]	pointer to nonce of length ESL_SIZEOF_ChaCha20_NONCE
workSpace [in,out]	(in) algorithm context buffer, initialized by esl_initWorkSpaceHeader () function (out) initialized algorithm context structure
Return code	
eslt_ErrorCode	ESL_ERC_PARAMETER_INVALID input parameter is NULL
	ESL_ERC_WS_TOO_SMALL work space too small
	ESL_ERC_NO_ERROR else
Functional Description	
This function initializes the AEAD_ChaCha20_Poly1305 stream decryption and authentication TAG verification.	
Particularities and Limitations	
workSpace is initialized by esl_initWorkSpaceHeader ()	
Call context	
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces	

Table 4-387 esl_initDecryptChaChaPoly

4.2.2.8.19 esl_verifyChaChaPoly

Prototype	
`eslt_ErrorCode` **`esl_verifyChaChaPoly`** `(VSECPRIM_P2VAR_PARA(eslt_WorkSpaceChaChaPoly) workSpace, VSECPRIM_P2CONST_PARA(eslt_Byte) tag)`	
Parameter	
workSpace [in,out]	(in) algorithm context buffer, initialized by esl_initDecryptChaChaPoly () function (out) internal state updated
tag [in]	pointer to TAG of length ESL_SIZEOF_Poly1305_TAG
Return code	
eslt_ErrorCode	ESL_ERC_PARAMETER_INVALID input parameter is NULL
	ESL_ERC_WS_STATE_INVALID work space state invalid
	ESL_ERC_INCORRECT_TAG the given TAG is incorrect
	ESL_ERC_NO_ERROR else
Functional Description	
This function verifies the authentication TAG of the AEAD_ChaCha20_Poly1305.	
Particularities and Limitations	
workSpace is initialized by esl_initWorkSpaceHeader ()	
Call context	
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces	

Table 4-388 esl_verifyChaChaPoly

4.2.2.9 Random Number Generation (PRNG)

4.2.2.9.1 esl_initFIPS186

Prototype	
`eslt_ErrorCode` **`esl_initFIPS186`** `(VSECPRIM_P2VAR_PARA(eslt_WorkSpaceFIPS186) workSpace, const eslt_Length entropyLength, VSECPRIM_P2CONST_PARA(eslt_Byte) entropy, VSECPRIM_P2VAR_PARA(eslt_Byte) savedState)`	
Parameter	
workSpace [in,out]	(in) algorithm context buffer, initialized by esl_initWorkSpaceHeader(..) function (out) initialized algorithm context structure
entropyLength [in]	The length of the given entropy used as seed for the RNG.
entropy [in]	A pointer to the value that has to be used as seed.
savedState [in,out]	A pointer to a state that has been saved after the previous initialisation. Length is given by ESL_SIZEOF_RNGFIPS186_STATE. If this pointer is set to NULL, no savedState is used.

Return code	
eslt_ErrorCode	ESL_ERC_PARAMETER_INVALID input parameter is NULL
	ESL_ERC_WS_TOO_SMALL work space too small
	ESL_ERC_RNG_ENTROPY_TOO_SMALL provided entropy length is to small
	ESL_ERC_SHA1_TOTAL_LENGTH_OVERFLOW total input length overflow
	ESL_ERC_NO_ERROR else
Functional Description	
Initialize FIPS186-2. This function initializes the random number generator according to FIPS-186-2.	
Particularities and Limitations	
workSpace is initialized by esl_initWorkSpaceHeader(..)	
Call context	
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces	

Table 4-389 esl_initFIPS186

4.2.2.9.2 esl_stirFIPS186

Prototype	
`eslt_ErrorCode esl_stirFIPS186 (VSECPRIM_P2VAR_PARA(eslt_WorkSpaceFIPS186) workSpace, const eslt_Length inputLength, VSECPRIM_P2VAR_PARA(eslt_Byte) input)`	
Parameter	
workSpace [in,out]	(in) algorithm context buffer, initialized by esl_initFIPS186(..) function (out) updated algorithm context structure
inputLength [in]	the number of input bytes used to stir up therandom number generator
input [in,out]	pointer to input bytes used to stir up the random number generator
Return code	
eslt_ErrorCode	ESL_ERC_PARAMETER_INVALID input parameter is NULL
	ESL_ERC_WS_STATE_INVALID work space state invalid
	ESL_ERC_WS_TOO_SMALL work space too small
	ESL_ERC_NO_ERROR else
Functional Description	
Stir up seed state. This function stirs up the internal state of the random number generator.	
Particularities and Limitations	
workSpace is initialized by esl_initFIPS186(..)	

Call context
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces

Table 4-390 esl_stirFIPS186

4.2.2.9.3 esl_getBytesFIPS186

Prototype	
`eslt_ErrorCode esl_getBytesFIPS186 (VSECPRIM_P2VAR_PARA(eslt_WorkSpaceFIPS186) workSpace, const eslt_Length targetLength, VSECPRIM_P2VAR_PARA(eslt_Byte) target)`	
Parameter	
workSpace [in,out]	(in) algorithm context buffer, initialized by esl_initFIPS186(..) function (out) updated algorithm context structure
targetLength [in]	the number of bytes that shall be generated
target [in,out]	(in) pointer to output buffer (out) pointer to generated random bytes
Return code	
eslt_ErrorCode	ESL_ERC_PARAMETER_INVALID input parameter is NULL
	ESL_ERC_WS_STATE_INVALID work space state invalid
	ESL_ERC_WS_TOO_SMALL work space too small
	ESL_ERC_NO_ERROR else
Functional Description	
Get random byte. This function generates random numbers according to FIPS-186-2.	
Particularities and Limitations	
workSpace is initialized by esl_initFIPS186(..)	
Call context	
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces	

Table 4-391 esl_getBytesFIPS186

4.2.2.9.4 esl_initCTRDRBG

Prototype	
`eslt_ErrorCode esl_initCTRDRBG (VSECPRIM_P2VAR_PARA(eslt_WorkSpaceCTRDRBG) workSpace, eslt_DRBGMode modeDRBG)`	
Parameter	
workSpace [in,out]	(in) pointer to the DRBG workSpace (out) DRBG workspace, mode member modified

modeDRBG [in]	indication which AES size (128 or 256) the DRBG shall be based on and whether the seeding and generating functions shall use a derivation function or not
Return code	
eslt_ErrorCode	ESL_ERC_PARAMETER_INVALID input parameter is NULL
	ESL_ERC_WS_TOO_SMALL work space too small
	ESL_ERC_MODE_INVALID modeDRBG invalid
	ESL_ERC_NO_ERROR else
Functional Description	
Initializes the DRBG and sets seed status in workspace to ESL_DRBG_SEED_STATE_UNSEEDED. This function initializes the CTR DRBG AES128 or CTR DRBG AES256, depending on the value of 'modeDRBG'.	
Particularities and Limitations	
workSpace is initialized by esl_initWorkSpaceHeader(..)	
Call context	
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces	

Table 4-392 esl_initCTRDRBG

4.2.2.9.5 esl_seedCTRDRBG

Prototype	
`eslt_ErrorCode esl_seedCTRDRBG (VSECPRIM_P2VAR_PARA(eslt_WorkSpaceCTRDRBG) workSpace, VSECPRIM_P2CONST_PARA(eslt_Byte) entropyPtr, eslt_Length entropyLength, VSECPRIM_P2CONST_PARA(eslt_Byte) noncePtr, eslt_Length nonceLength, VSECPRIM_P2CONST_PARA(eslt_Byte) personalizationStringPtr, eslt_Length personalizationStringLength, VSECPRIM_P2CONST_PARA(eslt_Byte) additionalInputPtr, eslt_Length additionalInputLength)`	
Parameter	
workSpace [in,out]	(in) pointer to the DRBG workspace (out) DRBG workspace, wsAES, key and variable modified
entropyPtr [in]	pointer to entropy buffer
entropyLength [in]	length of the entropy in bytes
noncePtr [in]	pointer to nonce buffer, possibly a null pointer
nonceLength [in]	length of the nonce in bytes, possibly 0
personalizationStringPtr [in]	pointer to personalization string buffer, possibly a null pointer
personalizationStringLength [in]	length of the personalization string in bytes, possibly 0
additionalInputPtr [in]	pointer to additional input buffer, possibly a null pointer
additionalInputLength [in]	length of the additional input in bytes, possibly 0
Return code	
eslt_ErrorCode	ESL_ERC_NO_ERROR Request was successful.

	ESL_ERC_PARAMETER_INVALID an input parameter is a null pointer
	ESL_ERC_WS_TOO_SMALL workSpace too small
	ESL_ERC_MODE_INVALID incorrect DRBG mode
	ESL_ERC_WS_STATE_INVALID workSpace state invalid
	ESL_ERC_INVALID_LENGTH length of one of the input parameters invalid
	ESL_ERC_RNG_ENTROPY_TOO_SMALL entropy length too short

Functional Description

Seed function for NIST800-90A DRBG (based on AES) for initial seeding or reseeding.

This function calls esl_initialSeed_CTRDRBG() or esl_reeed_CTRDRBG, depending on the seed status.

Particularities and Limitations

workSpace is a valid pointer and initialized by esl_initWorkSpaceHeader(..). esl_initCTRDRBG() was called before.

If no derivation function is used, entropyLength must be exactly ESL_CTRDRBG_AES128_SEED_LEN (or ESL_CTRDRBG_AES256_SEED_LEN) bytes. If a derivation function is used, the minimal value of entropyLength is vSecPrim_GetCTRDRBGAES128SecurityStrengthOfGeneral() bits and the maximal value of entropyLength is (2^32 - 10) bytes.
noncePtr is a required parameter for initial seeding with derivation function. noncePtr is not required if no derivation function is used while seeding initially or if esl_seedCTRDRBG is called to reseed the DRBG. The maximal length of noncePtr is (2^32 - 10) bytes.
personalizationStringPtr is an optional parameter for initial seeding, with or without derivation function. If no derivation function is used, the maximal length of personalizationStringPtr is ESL_CTRDRBG_AES128_SEED_LEN (or ESL_CTRDRBG_AES256_SEED_LEN) bytes. If a derivation function is used, the maximal length of personalizationStringPtr is (2^32 - 10) bytes.
additionalInputPtr is an optional parameter for reseeding, with or without derivation function. If no derivation function is used, the maximal length of additionalInputPtr is ESL_CTRDRBG_AES128_SEED_LEN (or ESL_CTRDRBG_AES256_SEED_LEN) bytes. If no derivation function is used, the maximal length of additionalInputPtr is (2^32 - 10) bytes.

Call context

- TASK
- This function is Synchronous
- This function is Reentrant, for different workspaces

Table 4-393 esl_seedCTRDRBG

4.2.2.9.6 esl_getBytesCTRDRBG

Prototype

```
eslt_ErrorCode esl_getBytesCTRDRBG (VSECPRIM_P2VAR_PARA(eslt_WorkSpaceCTRDRBG)
workSpace, eslt_Length requestedLength, VSECPRIM_P2VAR_PARA(eslt_Byte)
outputBuffer, VSECPRIM_P2CONST_PARA(eslt_Byte) additionalInputPtr,
eslt_Length additionalInputLength)
```

Parameter

workSpace [in,out]	(in) pointer to the DRBG workspace (out) DRBG workspace, wsAES, key and variable modified
requestedLength [in]	number of random bytes to be generated
outputBuffer [in,out]	(in) Pointer to the output buffer (out) output buffer, containing requestedLength random bytes

additionalInputBuffer [in]	pointer to additional input buffer, possibly a null pointer
additionalInputLength [in]	length of the additional input in bytes, possibly 0
Return code	
eslt_ErrorCode	ESL_ERC_NO_ERROR Request was successful.
	ESL_ERC_PARAMETER_INVALID workSpace or outputBuffer is a null pointer
	ESL_ERC_WS_TOO_SMALL workSpace too small
	ESL_ERC_MODE_INVALID incorrect DRBG mode
	ESL_ERC_WS_STATE_INVALID invalid workspace state
	ESL_ERC_INVALID_LENGTH length of additional input invalid
	ESL_ERC_RNG_REQUESTED_LENGTH_INVALID requested length greater than maximal DRBG output size
	ESL_ERC_ENTROPY_EXHAUSTION Entropy is exhausted or DRBG not seeded. Reseed is necessary.
Functional Description	
Generates requestedLength many random bytes using the CTR DRBG based on AES.	
Particularities and Limitations	
workSpace must be initialized by calling esl_initCTRDRBG(). The DRBG has to be seeded by calling esl_seedCTRDRBG() or esl_restoreStateCTRDRBG(). The buffer referenced by outputBuffer must provide at least requestedLength bytes. additionalInputPtr is an optional parameter with and without derivation function. If no derivation function is used, the maximal length of additionalInputPtr is ESL_CTRDRBG_AES128_SEED_LEN (or ESL_CTRDRBG_AES256_SEED_LEN) bytes. If a derivation function is used, the maximal length of additionalInputPtr is (2^32 - 10) bytes.	
Call context	
> TASK	
> This function is Synchronous	
> This function is Reentrant, for different workspaces	

Table 4-394 esl_getBytesCTRDRBG

4.2.2.9.7 esl_getStateCTRDRBG

```
eslt_ErrorCode esl_getStateCTRDRBG (
VSECPRIM_P2CONST_PARA(eslt_WorkSpaceCTRDRBG) workSpace,
VSECPRIM_P2VAR_PARA(eslt_Byte) internalStateBuffer,
VSECPRIM_P2VAR_PARA(eslt_Length) internalStateBufferLength,
VSECPRIM_P2VAR_PARA(eslt_DRBGReseedCounterType) reseedCounter,
VSECPRIM_P2VAR_PARA(eslt_DRBGSeedStatusType) seedStatus,
VSECPRIM_P2VAR_PARA(eslt_DRBGMode) modeDRBG)
```

Parameter	
workSpace [in]	pointer to the DRBG workspace
internalStateBuffer [in,out]	(in) Pointer to the output buffer, where [key, V] will be stored, (out) output buffer, containing [key, V]
internalStateBufferLength [in,out]	(in) length of internalStateBuffer in bytes, (out) number of bytes written to internalStateBuffer

reseedCounter [in,out]	(in) pointer to output buffer for the reseed counter, (out) output buffer for the reseed counter, containing the value of the reseed counter
seedStatus [in,out]	(in) pointer to output buffer for the seed status, saved in the workspace header, (out) output buffer for the seed status, containing the current seed status
modeDRBG [in,out]	(in) pointer to output buffer for the DRBG mode (out) output buffer containing DRBG mode
Return code	
eslt_ErrorCode	ESL_ERC_NO_ERROR Request was successful.
	ESL_ERC_PARAMETER_INVALID input parameter is a null pointer
	ESL_ERC_BUFFER_TOO_SMALL internalStateBufferLength too short
Functional Description	
Stores key, variable, reseedCount, seed status and DRBG mode in provided output buffers. If the seed status is UNSEEDED, the values in the output buffers are set to 0. key and variable are both stored in internalStateBuffer.	
Particularities and Limitations	
workSpace must be a valid pointer and must be initialized by calling esl_initCTRDRBG(). intenalStateBuffer, reseedCounter and seedStatus need to be valid pointers. The buffer referenced by internalStateBuffer must provide at least internalStateBufferLength bytes. For CTR DRBG AES128, internalStateBuffer must provide at least ESL_CTRDRBG_AES128_SEEDLEN bytes. For CTR DRBG AES256, internalStateBuffer must provide at least ESL_CTRDRBG_AES256_SEEDLEN bytes.	
Call context	
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces	

Table 4-395 esl_getStateCTRDRBG

4.2.2.9.8 esl_restoreStateCTRDRBG

```
eslt_ErrorCode esl_restoreStateCTRDRBG (
VSECPRIM_P2VAR_PARA(eslt_WorkSpaceCTRDRBG) workSpace,
VSECPRIM_P2CONST_PARA(eslt_Byte) internalStateBuffer, eslt_Length
internalStateBufferLength, eslt_DRBGReseedCounterType reseedCounter,
eslt_DRBGSeedStatusType seedStatus, eslt_DRBGMode modeDRBG)
```

Parameter	
workSpace [in, out]	(in) pointer to the DRBG workspace, (out) DRBG workspace, key, variable, reseedCount, seed status and mode modified
internalStateBuffer [in]	Pointer to the buffer containing [key, V]
internalStateBufferLength [in]	length of internalStateBuffer in bytes
reseedCounter [in]	value of reseed counter to be restored
seedStatus [in]	current seed status to be restored
modeDRBG [in]	DRBG mode data (algorithm size, DF mode)

Return code	
eslt_ErrorCode	ESL_ERC_NO_ERROR Request was successful.
	ESL_ERC_PARAMETER_INVALID input parameter is a null pointer
	ESL_ERC_INVALID_LENGTH internalStateBufferLength too high
Functional Description	
Restores internal state of DRBG from provided input values. Stores the values from internalStateBuffer, reseedCounter, seedStatus and DRBGmode in workSpace->wsDRBG.key, workSpace->wsDRBG.variable, workSpace->wsDRBG.reseedCount, workSpace->wsDRBG.mode and workSpace->wsDRBG.seedState	
Particularities and Limitations	
workSpace must be initialized by calling esl_initCTRDRBG(). For CTR DRBG AES128, internalStateBuffer must provide exactly ESL_CTRDRBG_AES128_SEEDLEN bytes. For CTR DRBG AES256, internalStateBuffer must provide exactly ESL_CTRDRBG_AES256_SEEDLEN bytes.	
Call context	
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces	

Table 4-396 esl_restoreStateCTRDRBG

4.2.2.9.9 esl_initHASHDRBG

Prototype	
`eslt_ErrorCode esl_initHASHDRBG (VSECPRIM_P2VAR_PARA(eslt_WorkSpaceHASHDRBG) workSpace, eslt_DRBGMode modeDRBG)`	
Parameter	
workSpace [in,out]	(in) pointer to the DRBG workSpace (out) DRBG workspace, mode member modified
modeDRBG [in]	indication which SHA size the DRBG shall be based on
Return code	
eslt_ErrorCode	ESL_ERC_PARAMETER_INVALID input parameter is NULL
	ESL_ERC_WS_TOO_SMALL work space too small
	ESL_ERC_MODE_INVALID modeDRBG invalid
	ESL_ERC_NO_ERROR else
Functional Description	
Initializes the DRBG and sets seed status in workspace to ESL_DRBG_SEED_STATE_UNSEEDED. This function initializes the DRBG by setting the DRBG mode.	
Particularities and Limitations	
workSpace is initialized by esl_initWorkSpaceHeader(..)	
Call context	
> TASK	

> This function is Synchronous
> This function is Reentrant, for different workspaces

Table 4-397 esl_initHASHDRBG

4.2.2.9.10 esl_seedHASHDRBG

Prototype
`eslt_ErrorCode esl_seedHASHDRBG (VSECPRIM_P2VAR_PARA(eslt_WorkSpaceHASHDRBG) workSpace, VSECPRIM_P2CONST_PARA(eslt_Byte) entropyPtr, eslt_Length entropyLength, VSECPRIM_P2CONST_PARA(eslt_Byte) noncePtr, eslt_Length nonceLength, VSECPRIM_P2CONST_PARA(eslt_Byte) personalizationStringPtr, eslt_Length personalizationStringLength, VSECPRIM_P2CONST_PARA(eslt_Byte) additionalInputPtr, eslt_Length additionalInputLength)`

Parameter	
workSpace [in,out]	(in) pointer to the DRBG workspace (out) DRBG workspace, wsSHA512, constant, variable and reseedCounter modified
entropyPtr [in]	pointer to entropy buffer
entropyLength [in]	length of the entropy in bytes
noncePtr [in]	pointer to nonce buffer, possibly a null pointer
nonceLength [in]	length of the nonce in bytes, possibly 0
personalizationStringPtr [in]	pointer to personalization string buffer, possibly a null pointer
personalizationStringLength [in]	length of the personalization string in bytes, possibly 0
additionalInputPtr [in]	pointer to additional input buffer, possibly a null pointer
additionalInputLength [in]	length of the additional input in bytes, possibly 0

Return code	
eslt_ErrorCode	ESL_ERC_NO_ERROR Request was successful.
	ESL_ERC_PARAMETER_INVALID an input parameter is a null pointer
	ESL_ERC_WS_TOO_SMALL workSpace too small
	ESL_ERC_WS_STATE_INVALID workSpace state invalid
	ESL_ERC_MODE_INVALID incorrect DRBG mode
	ESL_ERC_RNG_ENTROPY_TOO_SMALL entropy length too short

Functional Description
Seed function for NIST800-90A Hash DRBG (based on SHA2-512) for initial seeding or reseeding. This function calls esl_initialSeed_HASHDRBG() or esl_reeed_HASHDRBG, depending on the seed status.

Particularities and Limitations
workSpace is a valid pointer and initialized by esl_initWorkSpaceHeader(..). esl_initHASHDRBG() was called before. The minimal value of entropyLength is 256 bits and the maximal value of entropyLength is (2^32 - 1) bytes. noncePtr is a required parameter for initial seeding. The maximal length of noncePtr is (2^32 - 1) bytes. personalizationStringPtr is an optional parameter for initial seeding. The maximal length of personalizationStringPtr is (2^32 - 1) bytes.

additionalInputPtr is an optional parameter for reseeding. The maximal length of additionalInputPtr is (2^32 - 1) bytes.
Call context
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces

Table 4-398 esl_seedHASHDRBG

4.2.2.9.11 esl_getBytesHASHDRBG

Prototype	
`eslt_ErrorCode esl_getBytesHASHDRBG (VSECPRIM_P2VAR_PARA(eslt_WorkSpaceHASHDRBG) workSpace, eslt_Length requestedLength, VSECPRIM_P2VAR_PARA(eslt_Byte) outputBuffer, VSECPRIM_P2CONST_PARA(eslt_Byte) additionalInputPtr, eslt_Length additionalInputLength)`	
Parameter	
workSpace [in,out]	(in) pointer to the DRBG workspace (out) DRBG workspace, wsSHA512, constant, variable and reseedCounter modified
requestedLength [in]	number of random bytes to be generated
outputBuffer [in,out]	(in) Pointer to the output buffer (out) output buffer, containing requestedLength random bytes
additionalInputBuffer [in]	pointer to additional input buffer, possibly a null pointer
additionalInputLength [in]	length of the additional input in bytes, possibly 0
Return code	
eslt_ErrorCode	ESL_ERC_NO_ERROR Request was successful.
	ESL_ERC_PARAMETER_INVALID workSpace or outputBuffer is a null pointer
	ESL_ERC_WS_TOO_SMALL workSpace too small
	ESL_ERC_WS_STATE_INVALID invalid workspace state
	ESL_ERC_MODE_INVALID incorrect DRBG mode
	ESL_ERC_RNG_REQUESTED_LENGTH_INVALID requested length greater than maximal DRBG output size
	ESL_ERC_ENTROPY_EXHAUSTION Entropy is exhausted or DRBG not seeded. Reseed is necessary.
Functional Description	
Generates requestedLength many random bytes using the Hash DRBG based on SHA2-512.	
Particularities and Limitations	
workSpace must be initialized by calling esl_initHASHDRBG(). The DRBG has to be seeded by calling esl_seedHASHDRBG() or esl_restoreStateHASHDRBG(). The buffer referenced by outputBuffer must provide at least requestedLength bytes. additionalInputPtr is an optional parameter. The maximal length of additionalInputPtr is (2^32 - 1) bytes.	
Call context	

> TASK > This function is Synchronous > This function is Reentrant, for different workspaces

Table 4-399 esl_getBytesHASHDRBG

4.2.2.9.12 esl_getStateHASHDRBG

```
eslt_ErrorCode esl_getStateHASHDRBG (
VSECPRIM_P2CONST_PARA(eslt_WorkSpaceHASHDRBG) workSpace,
VSECPRIM_P2VAR_PARA(eslt_Byte) internalStateBuffer,
VSECPRIM_P2VAR_PARA(eslt_Length) internalStateBufferLength,
VSECPRIM_P2VAR_PARA(eslt_DRBGReseedCounterType) reseedCounter,
VSECPRIM_P2VAR_PARA(eslt_DRBGSeedStatusType) seedStatus,
VSECPRIM_P2VAR_PARA(eslt_DRBGMode) modeDRBG)
```

Parameter	
workSpace [in]	pointer to the DRBG workspace
internalStateBuffer [in,out]	(in) Pointer to the output buffer, where [constant, V] will be stored, (out) output buffer, containing [constant, V]
internalStateBufferLength [in,out]	(in) length of internalStateBuffer in bytes, (out) number of bytes written to internalStateBuffer
reseedCounter [in,out]	(in) pointer to output buffer for the reseed counter, (out) output buffer for the reseed counter, containing the value of the reseed counter
seedStatus [in,out]	(in) pointer to output buffer for the seed status, saved in the workspace header, (out) output buffer for the seed status, containing the current seed status
modeDRBG [in,out]	(in) pointer to output buffer for the DRBG mode (out) output buffer containing DRBG mode
Return code	
eslt_ErrorCode	ESL_ERC_NO_ERROR Request was successful.
	ESL_ERC_PARAMETER_INVALID input parameter is a null pointer
	ESL_ERC_BUFFER_TOO_SMALL internalStateBufferLength too short
Functional Description	
Stores constant, variable, reseedCount, seed status and DRBG mode in provided output buffers. If the seed status is UNSEEDED, the values in the output buffers are set to 0. constant and variable are both stored in internalStateBuffer.	
Particularities and Limitations	
workSpace must be a valid pointer and must be initialized by calling esl_initHASHDRBG(). intenalStateBuffer, reseedCounter and seedStatus need to be valid pointers. The buffer referenced by internalStateBuffer must provide at least internalStateBufferLength bytes. For Hash DRBG SHA2-512, internalStateBuffer must provide at least 2*ESL_HASHDRBG_SHA512_SEED_LEN bytes.	
Call context	
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces	

Table 4-400 esl_getStateHASHDRBG

4.2.2.9.13 esl_restoreStateHASHDRBG

```
eslt_ErrorCode esl_restoreStateHASHDRBG (
VSECPRIM_P2VAR_PARA(eslt_WorkSpaceHASHDRBG) workSpace,
VSECPRIM_P2CONST_PARA(eslt_Byte) internalStateBuffer, eslt_Length
internalStateBufferLength, eslt_DRBGReseedCounterType reseedCounter,
eslt_DRBGSeedStatusType seedStatus, eslt_DRBGMode modeDRBG)
```

Parameter	
workSpace [in, out]	(in) pointer to the DRBG workspace, (out) DRBG workspace, constant, variable, reseedCount, seed status and mode modified
internalStateBuffer [in]	Pointer to the buffer containing [constant, V]
internalStateBufferLength [in]	length of internalStateBuffer in bytes
reseedCounter [in]	value of reseed counter to be restored
seedStatus [in]	current seed status to be restored
modeDRBG [in]	DRBG mode data (algorithm size, DF mode)
Return code	
eslt_ErrorCode	ESL_ERC_NO_ERROR Request was successful.
	ESL_ERC_PARAMETER_INVALID input parameter is a null pointer
	ESL_ERC_INVALID_LENGTH internalStateBufferLength too little
Functional Description	
Restores internal state of DRBG from provided input values. Stores the values from internalStateBuffer, reseedCounter, seedStatus and DRBGmode in workSpace->wsDRBG.constant, workSpace->wsDRBG.variable, workSpace->wsDRBG.reseedCount, workSpace->wsDRBG.mode and workSpace->wsDRBG.seedState	
Particularities and Limitations	
workSpace must be initialized by calling esl_initHASHDRBG().For Hash DRBG SHA2-512, internalStateBuffer must provide at least 2*ESL_HASHDRBG_SHA512_SEED_LEN bytes.	
Call context	
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces	

Table 4-401 esl_restoreStateHASHDRBG

4.2.2.10 Service Functions for Initialization

4.2.2.10.1 esl_initESCryptoLib

Prototype	
`eslt_ErrorCode esl_initESCryptoLib (void)`	
Parameter	
void	none

Return code	
eslt_ErrorCode	ESL_ERC_NO_ERROR Operation successful
Functional Description	
This function sets the library to a default value. sets LibStatus to some default value also initializes Lib internal vars if needed on some controllers this function initializes the library for multithreading features.	
Particularities and Limitations	
-	
Call context	
> TASK > This function is Synchronous > This function is Reentrant	

Table 4-402 esl_initESCryptoLib

4.2.2.10.2 esl_initWorkSpaceHeader

Prototype	
`eslt_ErrorCode` **`esl_initWorkSpaceHeader`** `(VSECPRIM_P2VAR_PARA(eslt_WorkSpaceHeader) workSpaceHeader, const eslt_Length workSpaceSize, esl_WatchdogFuncPtr watchdog)`	
Parameter	
workSpaceHeader [in,out]	Pointer to workspace header
workSpaceSize [in]	Size of the provided workspace
watchdog [in]	Pointer to watchdog trigger function
Return code	
eslt_ErrorCode	ESL_ERC_NO_ERROR - OK
eslt_ErrorCode	ESL_ERC_PARAMETER_INVALID - Nullpointer check failed
Functional Description	
Init workspace header. This function initializes a workSpace header. This function must be called before calling (nearly) every function from the cv act library/es to initialize the workSpace header correctly.	
Particularities and Limitations	
workspaceHeader must be a valid pointer.	
Call context	
> TASK > This function is Synchronous > This function is Reentrant, for different workspaces	

Table 4-403 esl_initWorkSpaceHeader

4.3 Services used by vSecPrim

The following table lists services provided by other components which are used by vSecPrim. For details concerning prototypes and functionalities, please refer to the documentation of the providing component.

Please note: The usage of functions provided by vStdLib is configurable, i.e., they are only used, if vSecPrimUseVstdLib is enabled. Otherwise, vSecPrim uses its own implementation of the respective functions.

Component	API
VStdLib	VStdLib_MemClr VStdLib_MemSet VStdLib_MemCpy VStdLib_MemCpy16 VStdLib_MemCpy32

Table 4-404 Services used by vSecPrim

4.4 Configurable Interfaces

4.4.1 Callout Functions

At its configurable interfaces the vSecPrim defines callout functions. The declarations of the callout functions are provided by the BSW module, i.e. the vSecPrim. It is the integrator's task to provide the corresponding function definitions. The definitions of the callouts can be adjusted to the system's needs. The vSecPrim callout function declarations are described in the following tables:

4.4.1.1 esl_getBytesRNG

Prototype	
`eslt_ErrorCode` **`esl_getBytesRNG`** `(eslt_Length targetLength, VSECPRIM_P2VAR_PARA(eslt_Byte) target)`	
Parameter	
targetLength [in]	the number of bytes that shall be generated
target [in,out]	(in) pointer to output buffer (out) pointer to generated random bytes

Return code	
eslt_ErrorCode	ESL_ERC_PARAMETER_INVALID input parameter is NULL
	ESL_ERC_WS_STATE_INVALID work space state invalid
	ESL_ERC_WS_TOO_SMALL work space too small
	ESL_ERC_NO_ERROR else
Functional Description	
Function shall generate the given number of random bytes and store these bytes in the passed array. This can be achieved by e.g. using a TRNG or PRNG of an available cryptographic hardware accelerator or using one of the provided software PRNG implementations, e.g. FIPS 186-2.	
Particularities and Limitations	
-	
Call context	
> TASK > This function is Synchronous > This function is Reentrant	

Table 4-405 esl_getBytesRNG

5 Configuration

5.1 Configuration Variants

The vSecPrim supports the configuration variants

- VARIANT-PRE-COMPILE

The configuration classes of the vSecPrim parameters depend on the supported configuration variants. For their definitions please see the descriptions of the parameters of vSecPrim_bswmd.arxml file in online help of DaVinci Configurator 5.

© 2023 Vector Informatik GmbH

6 Glossary and Abbreviations

6.1 Glossary

Term	Description
EAD	Embedded Architecture Designer; generation tool for MICROSAR Classic components

Table 6-1 Glossary

6.2 Abbreviations

Abbreviation	Description
AES	Advanced Encryption Standard
ALGOFAM	Algorithm Family
ALGOMODE	Algorithm Mode
ANSI	American National Standards Institute
API	Application Programming Interface
AUTOSAR	Automotive Open System Architecture
BSW	Basis Software
CBC	Cipher Block Chaining
CCM	Counter with CBC MAC
CMAC	Cipher-based Message Authentication Code
CRT	Chinese Remainder Theorem
CTR	Counter (mode)
DEM	Diagnostic Event Manager
DET	Development Error Tracer
DF	Derivation Function
DRBG	Deterministic Random Bit Generator
EAD	Embedded Architecture Designer
ECB	Electronic Code Book
EC(C)	Elliptic Curve (Cryptography)
ECDHE	Elliptic-curve Diffie–Hellman
ECDSA	Elliptic-curve Digital Signature Algorithm
ECU	Electronic Control Unit
EMSA	Encoding method for signature appendix
FIPS	Federal Information Processing Standards Publication
GCM	Galois Counter Mode
HIS	Hersteller Initiative Software

HKDF	HMAC key derivation function
HMAC	Keyed-Hash Message Authentication Code
ISO	International Organization for Standardization
ISR	Interrupt Service Routine
KDF	Key Derivation Function
KE	Key Element
KEA	Key Exchange Algorithm
MAC	Message Authentication Code
MICROSAR	Microcontroller Open System Architecture (the Vector AUTOSAR solution)
NIST	National Institute of Standards and Technology
PPORT	Provide Port
PRF	Pseudorandom Function
PRNG	Pseudo Random Number Generator
PSS	Probabilistic Signature Scheme
RNG	Random Number Generator
RPORT	Require Port
RSA	Rivest-Shamir-Adleman
RTE	Runtime Environment
SCC	Smart Charge Communication
SEC	Standards for Efficient Cryptography
SHA	Secure Hash Algorithm
SHE	Secure Hardware Extension
SipHash	short-input PRF Hash
SPAKE	simple password-authenticated key exchange
SRS	Software Requirement Specification
SSA	Signature Scheme with Appendix
SWC	Software Component
SWS	Software Specification
TRNG	True Random Number Generator
UID	Unique identification
FID	Functional identification

Table 6-2 Abbreviations

7 Contact

Visit our website for more information on

- News
- Products
- Demo software
- Support
- Training data
- Addresses

www.vector.com

© 2023 Vector Informatik GmbH